初稿A2（更新）

科学计算前四章 • G22、G21、G13、G18

整理日期：2026-11-06。主分支版本：`230873df074c`。第2章采用G21最新补齐分支 `1246f1a1a7bc`（尚未合入main）；第3章采用G13含3.4的完整正文。本稿保留各组原有讲解、实验说明、习题与参考解答，以及尚未完成内容。仅整理标题层级、原稿目录、公式标记及相对路径。

目录

- 第1章 • G22
- 第2章 • G21
- 第3章 • G13
- 第4章 • G18

第01章 误差的代价

> 编写组：A2（G22）；来源版本：`230873df074c`，main已合并稿。本文件按原稿收录正文、独立习题与参考解答，保留原稿待完成项。

图01-1 • 差额去哪儿了？初始化概念示意；适用条件见下文，不作为实测结果。

先看一幕：差额去哪儿了?

“只差一点，减一下就知道了。”机器人把两个大数送进计算器，却看见一个意外的零。真正丢失的是差额，还是表示差额的有效数字？

先猜，再验证： 对 $x = 10^{16}$，直接计算 $\sqrt{x+1} - \sqrt{x}$，与计算 $1/(\sqrt{x+1} + \sqrt{x})$，在常见float64实现中会得到相同结果吗？先预测，再运行本章实验。

把故事翻译成数学： 两式在实数运算中等价，但浮点运算的舍入和相消可能让实现表现不同。图中的“差额丢了”是比喻；误差大小要由实际输出与高精度参考共同判断，不能泛化成“所有相减都有问题”。

知识主题： 误差、条件数与稳定性

本章核心问题： 当计算机告诉你答案是3.141592653589793时，你凭什么相信它？

只聚焦三个核心概念： 浮点数、条件数、稳定性

先修知识： 实数运算、导数、Taylor展开与向量范数

课时： 4节 × 2学时 = 8学时；全书八章共64学时。

本章四节安排

节	内容	学时
1.1	浮点数与舍入误差	2
1.2	截断误差与灾难性抵消	2
1.3	条件数与稳定性	2
1.4	AI关联：训练中的数值问题	2

1.1 浮点数与舍入误差（2学时）

本节目标： 能解释机器数与实数的区别，按量的尺度报告误差。

概念讲解：把问题说明白

填写提示：符号位、指数、尾数的作用；机器精度与有效数字；绝对误差和相对误差。用自己的例子解释0.1为何不一定被精确表示。

实数有无限多个，而固定长度的存储只有有限种状态。因此，计算机必须用有限个可表示数来近似实数。二进制正规浮点数可以写成

$$(-1)^s(1.b_1b_2\cdots b_{p-1})_2\,2^e.$$

其中，s 是符号位，e 是指数，p 是有效二进制位数。指数决定数的尺度，有效位数决定这一尺度上的分辨率。这里的 e 是实际指数，存储时通常使用加上偏置的编码。

常用的两种 IEEE 754 二进制格式如下。

格式	NumPy 类型	符号位	指数位	显式小数位	有效二进制位数 p
binary32	`float32`	1	8	23	24
binary64	`float64`	1	11	52	53

正规数的首位总是1，因此不必单独存储。这一隐含首位使有效位数比显式小数位数多1。两种格式的精度分别约相当于7位和16位十进制有效数字。

先看一个能够精确表示的数：

$$0.15625=\frac{1}{8}+\frac{1}{32}=0.00101_2=1.01_2\times 2^{-3}.$$

它的二进制展开有限，有效位数和指数也都在上述格式的范围内，所以两种格式都能精确存储它。

但十进制的 0.1 不同：

$$0.1_{10}=0.0001100110011\cdots_2.$$

一个最简分数能写成有限二进制小数，当且仅当它的分母是2的幂。$1/10$ 的分母含有因子5，因此只能近似存储。Python 的 binary64 浮点数 `0.1` 实际等于

$$\frac{3602879701896397}{36028797018963968},$$

比 $1/10$ 略大。屏幕显示的 `0.1` 是这一存储值的简短十进制表示。Python 官方教程详细说明了这个例子。

设准确值为 y，近似值为 $\hat{y}$。定义绝对误差为

$$E_{\mathrm{a}} = |\hat{y} - y|.$$

当 $y \neq 0$ 时，定义相对误差为

$$E_{\mathrm{r}} = \frac{|\hat{y} - y|}{|y|}.$$

绝对误差衡量偏差有多大，与原量具有相同单位；相对误差衡量偏差占准确值的比例，是无量纲量。例如，误差同为 10^{-3}，对于 10^{3} 是很小的相对误差，对于 10^{-6} 却非常大。准确值为零时，应报告绝对误差；需要比较不同尺度时，可以另选有明确意义的参考尺度。

关键推导：让结论有依据

推导任务：手算一个二进制小数的表示与舍入；给出浮点舍入模型使用的条件，并说明接近零、上溢或下溢时需要注意什么。

用一个仅保留4位有效二进制位的教学格式来存储 0.1。由

$$0.1 = 1.1001100\cdots_2 \times 2^{-4},$$

可知它位于两个相邻的可表示数之间：

$$1.100_2 \times 2^{-4} = 0.09375, \qquad 1.101_2 \times 2^{-4} = 0.1015625.$$

后者更近，所以按最近舍入得到 0.1015625。其绝对误差为 0.0015625，相对误差为 0.015625。

本章采用“舍入到最近值，中点取偶数”的规则：若准确值恰在两个可表示数的中点，就选择有效数最低位为0的那个。

对有效位数为 p 的二进制格式，1与其上方相邻可表示数的间隔为

$$\varepsilon = 2^{1-p}.$$

最近舍入对应的单位舍入误差为

$$u = \frac{\varepsilon}{2} = 2^{-p}.$$

类型	1处向上的间隔 ε	单位舍入误差 u
`float32`	$2^{-23} \approx 1.19 \times 10^{-7}$	$2^{-24} \approx 5.96 \times 10^{-8}$
`float64`	$2^{-52} \approx 2.22 \times 10^{-16}$	$2^{-53} \approx 1.11 \times 10^{-16}$

“机器精度”一词在不同资料中可能指 ε，也可能指 u，阅读时应先确认定义。NumPy 的 `finfo(...).eps` 指表中的 ε。NumPy 官方文档

设 $\mathrm{fl}(x)$ 表示将实数 x 舍入成浮点数。在最近舍入下，若准确结果处于正规数范围内且未溢出，则

$$\mathrm{fl}(x) = x(1+\delta), \qquad |\delta| \le u.$$

这个模型来自浮点数的间隔：在正规数区间 $[2^e, 2^{e+1})$ 内，相邻数的间隔为 2^{e-p+1}，最近舍入的绝对误差至多为半个间隔。再除以 $|z| \geq 2^e$，便得到相对误差界 u。

对基本四则运算，z 是两个浮点输入进行准确运算后的结果。例如，

$$\mathrm{fl}(a+b) = (a+b)(1+\delta).$$

这是单次运算的模型。 一个程序可能包含很多次运算，最终误差还取决于计算顺序和误差的传播方式。

浮点格式还包含零、非正规数、无穷和 NaN。非正规数以固定间隔填补零与最小正正规数之间的区域，越接近零，可保留的相对精度越低。结果过大可能上溢为无穷，极小结果可能下溢为零，$0/0$ 等运算会产生 NaN。本章主要使用正规数范围内的舍入模型。

实验设计：先预测，再运行

实验任务：比较float32与float64的0.1+0.2、1附近可分辨间隔以及不同求和顺序；使用Decimal或手算有理数建立参考。

要填写的项目	本组内容
要回答的问题与运行前预测	两种精度是否存在小数表示误差？提高精度预计减少误差，改变求和顺序预计改变结果。
输入、参数、单位、数据来源与划分	显式float32/float64的0.1、0.2；顺序求和两组 $[10^{16}, 1, -10^{16}]$、$[10^{16}, -10^{16}, 1]$。无量纲、无数据划分。
独立参考解/基线及选择理由	Fraction给出的准确分数3/10及整数1；先将存储结果转为准确分数，再计算误差。
误差指标、容差及其依据	绝对与相对误差；1处间隔与 2^{1-p} 精确比较；真值0只报告绝对误差。
环境、种子、设备与运行预算	Python 3.12.14、NumPy 2.3.5、Matplotlib 3.9.2；Windows CPU。输入为确定性教学构造；模板固定随机种子2026，本例不抽样。每节通常数秒，实际用时由Notebook记录。

实现见A2组-实验.ipynb第1.1节，代码单元索引3、4、5（从0计）。本次已顺序运行并保存表格、数值记录及相关图。

结果解释与本节结论

应提供的证据：填写输入、dtype、参考值、绝对/相对误差表；真值为零时说明采用的误差指标。

本次float32与float64结果如下。参考值分别为准确分数 $3/10$ 和整数1。

类型	存储结果	绝对误差	相对误差
float32	0.30000001192092896	1.1920928955e-08	3.9736429850e-08
float64	0.30000000000000004	4.4408920985e-17	1.4802973662e-16

类型	顺序	结果	绝对误差	相对误差
float32	[10000000000000000, 1, -10000000000000000]	0.0	1.0	1.0
float32	[10000000000000000, -10000000000000000, 1]	1.0	0.0	0.0
float64	[10000000000000000, 1, -10000000000000000]	0.0	1.0	1.0
float64	[10000000000000000, -10000000000000000, 1]	1.0	0.0	0.0

提高精度减少了小数求和误差，但没有恢复结合律。第一种顺序先把1加到大数上，增量被舍去；第二种顺序先相消，再保留1。准确值为零时只报告绝对误差，例如 $0.5 - 0.5$ 的绝对误差为0。

易错点与讨论

待核验的风险：显示位数不等于有效精度；机器精度不等于所有数值计算的绝对误差。

显示位数只改变已存数字的呈现，不能增加有效精度。浮点间隔随指数变化：float64在1处向上的间隔约为 2.22×10^{-16}，在 10^{16} 处却为2。两种顺序求和分别得到0和1，说明单次舍入界不能直接作为整个程序的误差界。

随堂提问： 设计一道小数表示或舍入题，要求先手算再与两种精度比较。

设 $a = 0.15625$。先手算其规格化二进制表示，再判断它在float32和float64中是否精确。常见回答是“十进制小数都不精确”；正确解释是 $a = 1.01_2 \times 2^{-3}$，展开有限且位数、指数均在范围内，两种格式均精确。

1.2 截断误差与灾难性抵消（2学时）

本节目标： 能区分截断与舍入，并用等价改写缓解抵消。

概念讲解：把问题说明白

填写提示：用有限差分解释截断误差，用接近数相减解释抵消；把两者联系到稳定性，不再扩展成独立的大专题。

用有限差分近似导数，即使每一步都进行准确运算，结果通常也不等于导数。这是用有限过程替代极限过程产生的截断误差。将输入或中间结果舍入为浮点数，则会产生舍入误差。

两类误差常同时存在，而且对计算参数有不同的要求：缩小差分步长可以减少截断误差，却可能放大舍入误差。

另一种常见现象是灾难性抵消。设要计算 $a - b$，实际参与运算的近似值为

$$\hat{a} = a + e_a, \qquad \hat{b} = b + e_b.$$

暂不计减法新产生的舍入误差，则

$$\widehat{a}-\hat{b}=(a-b)+(e_a-e_b).$$

当 $a-b\neq 0$ 时，结果的相对误差为

$$\frac{|e_a-e_b|}{|a-b|}.$$

如果 a 与 b 很接近，差值很小，原来相对于 a、b 很小的误差，就可能相对于差值很大。相减并没有凭空制造这些输入误差，而是使它们在结果中变得突出。有些相近浮点数的减法本身甚至是准确的，信息已经在前面的计算中丢失。

关键推导：让结论有依据

推导任务：推导前向差分的截断项及有理化表达式；讨论差分步长减小时截断与舍入的竞争。

用前向差分近似 $f'(x)$：

$$D_hf(x)=\frac{f(x+h)-f(x)}{h},\qquad h\neq 0.$$

若 f 在包含 x 与 $x+h$ 的区间上二阶连续可微，Taylor 公式给出

$$f(x+h)=f(x)+hf'(x)+\frac{h^2}{2}f''(\xi),$$

其中 ξ 位于 x 与 $x+h$ 之间。因此

$$D_hf(x)-f'(x)=\frac{h}{2}f''(\xi).$$

若该区间内 $|f''|\le M_2$，则

$$|D_hf(x)-f'(x)|\le\frac{M_2}{2}|h|.$$

这说明前向差分的截断误差为 $O(|h|)$。

再考虑浮点计算。设两个函数值的计算误差分别为 η_1 和 η_0，则仅由这两项带来的差分误差为

$$\frac{\eta_1-\eta_0}{h},$$

其绝对值至多为 $(|\eta_1|+|\eta_0|)/|h|$。如果函数值的绝对误差约为某个尺度常数乘以 u，便得到常用的总误差估计：

$$E(h)\lesssim A|h|+\frac{Bu}{|h|},\qquad A,B>0.$$

第一项随步长减小而减小，第二项却随步长减小而增大。对正步长求右侧的极小值，得到平衡尺度

$$h_{\mathrm{bal}}\approx\sqrt{\frac{Bu}{A}}.$$

在函数与自变量尺度均为1附近、A 与 B 量级相近时，`float64` 的合适步长常在 10^{-8} 附近。这是选择步长的量级依据；具体取值还取决于函数、自变量尺度和实现。

回到开场。当 $x\ge 0$ 时，有理化得到

$$
\begin{aligned}
g(x) &= \sqrt{x+1}-\sqrt{x} \\
&= \frac{(\sqrt{x+1}-\sqrt{x})(\sqrt{x+1}+\sqrt{x})}{\sqrt{x+1}+\sqrt{x}} \\
&= \frac{1}{\sqrt{x+1}+\sqrt{x}}.
\end{aligned}
$$

新式把准确的分子1保留在表达式中，避免从两个很接近的近似平方根中重建微小差值。

在 $x = 10^{16}$ 的 `float64` 计算中，直接式的第一步就已把 $x+1$ 舍入为 x，因此两个平方根相同，结果为0。有理化式中的 $x+1$ 虽然也会舍入为 x，但这只轻微改变了量级为 2×10^{8} 的分母，分子1仍然保留。

实验设计：先预测，再运行

实验任务：保留现有大数相减示例；补充光滑函数导数的步长扫描，对照解析导数，绘制误差随步长变化的图。

要填写的项目	本组内容
要回答的问题与运行前预测	有理化预计更准确；前向差分误差预计先下降后回升。
输入、参数、单位、数据来源与划分	$x = 10^{16}$；$f(x) = e^{x}$、$x = 1$、$h = 10^{-k}$，$k = 1, \ldots, 16$。无量纲、无数据划分。
独立参考解/基线及选择理由	80位Decimal原始平方根差及差分，120位复核；解析导数为e。
误差指标、容差及其依据	平方根差相对误差，有理化小于 10^{-14}；差分用总绝对误差及截断/实现误差分解，保留全部步长。
环境、种子、设备与运行预算	Python 3.12.14、NumPy 2.3.5、Matplotlib 3.9.2；Windows CPU。输入为确定性教学构造；模板固定随机种子2026，本例不抽样。每节通常数秒，实际用时由Notebook记录。

实现见A2组-实验.ipynb第1.2节，代码单元索引11、12、13（从0计）。本次已顺序运行并保存表格、数值记录及相关图。

结果解释与本节结论

应提供的证据：给出步长、两类误差的解释及实际曲线；记录误差不再下降的区间，不能只给最佳结果。

在 $x = 10^{16}$ 处，直接平方根差为0，相对误差为1；有理化式为 5×10^{-9}，相对误差约为 4.59226×10^{-17}。高精度参考约为 $4.999999999999999875 \times 10^{-9}$。

步长	近似导数	总绝对误差	实际取样增量
1e-01	2.85884195487388	1.40560126e-01	1.00000000e-01
1e-02	2.73191865578712	1.36368273e-02	1.00000000e-02
1e-03	2.71964142253323	1.35959407e-03	1.00000000e-03
1e-04	2.71841774708292	1.35918624e-04	1.00000000e-04
1e-05	2.71829541995672	1.35914977e-05	1.00000000e-05

步长	近似导数	总绝对误差	实际取样增量
1e-06	2.71828318743061	1.35897157e-06	1.00000000e-06
1e-07	2.71828196840573	1.39946688e-07	1.00000000e-07
1e-08	2.71828182185629	6.60275094e-09	9.99999994e-09
1e-09	2.7182820439009	2.15441854e-07	1.00000008e-09
1e-10	2.71828337616853	1.54770948e-06	1.00000008e-10
1e-11	2.71831446241322	3.26339542e-05	1.00000008e-11
1e-12	2.71871414270208	4.32314243e-04	1.00008890e-12
1e-13	2.71782596428238	4.55864177e-04	9.99200722e-14
1e-14	2.70894418008538	9.33764837e-03	9.99200722e-15
1e-15	3.10862446895044	3.90342640e-01	1.11022302e-15
1e-16	0	2.71828183e+00	0.00000000e+00

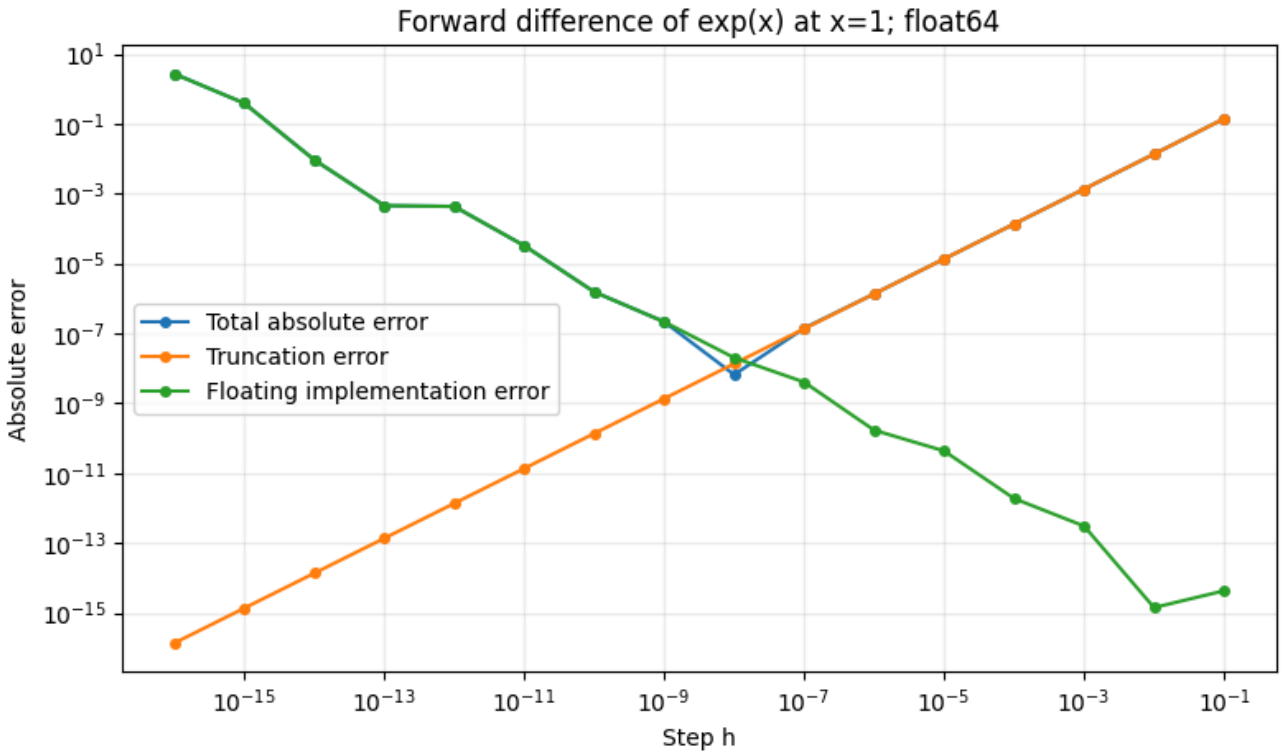


图01-2：e^x 在 $x=1$ 处的float64前向差分，双对数坐标。横轴为步长，纵轴为误差绝对值；输入及误差均无量纲。

本次最小总绝对误差约为 6.60275×10^{-9}，对应 $h=10^{-8}$。再减小步长，误差总体回升；$h=10^{-16}$ 时实际取样增量为0。设准确十进制步长下的差分为 D_h，浮点结果为 $\widehat{D}_h$，则

$$\widehat{D}_h - e = (D_h - e) + (\widehat{D}_h - D_h).$$

两项分别是截断误差和浮点实现误差，后者含步长、取样点和函数运算的误差。它们可以部分抵消，绝对值不必相加等于总绝对误差。实验覆盖表中16个步长。

易错点与讨论

待核验的风险：步长越小不一定越准；代数等价不保证浮点实现同样可靠。

本次扫描在 $h = 10^{-8}$ 附近取得较小误差，而 $h = 10^{-16}$ 时取样点重合。因此，步长需要平衡截断与舍入。有理化式显式保留分子1；若仍把分子写成浮点 `(x+1)-x`，就会再次丢失增量。

随堂提问： 设计一道同时包含有理化和差分步长选择的题，要求解释失败情形。

在float64中分别计算 $g(10^{16})$ 的直接式、有理化式，以及 e^x 在1处步长 10^{-8}、10^{-16} 的前向差分。若认为“步长更小就更准”，应解释为何后一个差分变为0。关键在于 $1 + h$ 已舍入为1，而有理化保留了准确分子1。

1.3 条件数与稳定性（2学时）

本节目标： 能分清问题敏感性与算法误差传播。

概念讲解：把问题说明白

填写提示：条件数是问题的性质，稳定性与算法有关；选择一个标量例子，避免在本章展开矩阵后向误差理论。

一个答案不准确，可能是因为输入稍有偏差就会使答案发生很大变化，也可能是算法在计算过程中放大了舍入误差。

条件性描述数学问题对输入扰动的敏感程度，属于问题本身；稳定性描述算法处理计算误差的能力，属于算法。Higham 对数值稳定性的说明给出了这一区分及其误差分析含义。

例如，

$$q(x) = \frac{1}{1 - x}$$

在 x 接近1时非常敏感：输入发生微小变化，准确函数值就可能明显改变。相反，开场中的平方根差对较大 x 的相对扰动并不敏感，直接计算却仍然失败。

稳定的算法应将计算过程中附加的误差控制在与所用精度和问题敏感性相称的水平。本节以标量扰动与两种表达式为主，不展开矩阵后向误差理论。

关键推导：让结论有依据

推导任务：在条件允许时推导标量相对条件数 $|x\, f'(x)/f(x)|$；指出x或f(x)为零时相对表述的限制。

设 f 在 x 处可微，则

$$f(x + \Delta x) - f(x) = f'(x)\Delta x + o(|\Delta x|).$$

当 $x \neq 0$ 且 $f(x) \neq 0$ 时，比较输入和输出的相对变化，得到相对条件数：

$$\boxed{\kappa_{\mathrm{rel}}(x) = \left| \frac{x f'(x)}{f(x)} \right|}.$$

对充分小的扰动，有一阶近似

$$\frac{|\Delta f|}{|f(x)|} \approx \kappa_{\mathrm{rel}}(x)\frac{|\Delta x|}{|x|}.$$

条件数越大，输入的相对误差越容易转化为较大的输出相对误差。若讨论绝对误差，相应的绝对条件数为

$$\kappa_{\mathrm{abs}}(x) = |f'(x)|.$$

当输入或输出为零、相对尺度不适用时，可以使用绝对条件数。条件数的含义始终与所选的输入、输出及误差尺度联系在一起。

对 $q(x) = 1/(1-x)$，有

$$q'(x) = \frac{1}{(1-x)^2}, \qquad \kappa_q(x) = \left|\frac{x}{1-x}\right|.$$

当 $x \to 1$ 时，条件数趋于无穷。因此，在1附近求这个函数值是一个相对病态的问题。

再看 $g(x) = \sqrt{x+1} - \sqrt{x}$。当 $x > 0$ 时，

$$\begin{aligned} g'(x) &= \frac{1}{2\sqrt{x+1}} - \frac{1}{2\sqrt{x}} \\ &= -\frac{g(x)}{2\sqrt{x}\sqrt{x+1}}, \end{aligned}$$

从而

$$\kappa_g(x) = \frac{1}{2}\sqrt{\frac{x}{x+1}} < \frac{1}{2}.$$

当 $x \to \infty$ 时，条件数趋近 $1/2$。因此，输入 x 的小相对扰动只会引起同量级的输出相对变化。开场计算失败的原因，应当从算法中寻找。

这里把 x 作为唯一输入，并把常数1视为准确常数。若将两个平方根的值看作可独立扰动的两个输入，讨论的就变成了另一个数学问题。

实验设计：先预测，再运行

实验任务：对同一问题施加不同大小的输入扰动，比较输出变化；再对同一输入比较两种等价算法。

要填写的项目	本组内容
要回答的问题与运行前预测	输入扰动放大率预计趋近9999；良态的平方根差也可能因直接式失去精度。
输入、参数、单位、数据来源与划分	Decimal输入0.9999及正扰动 $10^{-6}, 10^{-8}, 10^{-10}$；算法比较输入0、1、 10^{8}、 10^{16}。无量纲。
独立参考解/基线及选择理由	q的有限扰动闭式关系；g的120位Decimal原始差式。
误差指标、容差及其依据	相对变化放大率与闭式值的Decimal差小于 10^{-65}；有理化相对误差小于 10^{-14}。
环境、种子、设备与运行预算	Python 3.12.14、NumPy 2.3.5、Matplotlib 3.9.2；Windows CPU。输入为确定性教学构造；模板固定随机种子2026，本例不抽样。每节通常数秒，实际用时由Notebook记录。

实现见A2组-实验.ipynb第1.3节，代码单元索引17、18、19（从0计）。本次已顺序运行并保存表格、数值记录及相关图。

结果解释与本节结论

应提供的证据：分别填写输入扰动放大表和算法误差表，说明哪个证据对应条件数、哪个对应稳定性。

条件性实验在 $x = 0.9999$ 处使用高精度运算，$q(x) = 10000$，局部相对条件数为9999。

扰动	输入相对变化	输出相对变化	放大率
1e-6	1.0001000100E-6	1.0101010101E-2	10100.0000000000
1e-8	1.0001000100E-8	1.0001000100E-4	10000.0000000000
1e-10	1.0001000100E-10	1.0000010000E-6	9999.0099990100

有限扰动满足

$$\frac{q(x+d)-q(x)}{q(x)} = \frac{d}{1-x-d}, \qquad R(d) = \left|\frac{x}{1-x-d}\right|.$$

扰动减小时放大率趋近9999。$d = 10^{-4}$ 会使输入到达奇点1，函数无定义。

固定同一个 $g(x)$ 和输入，只改变表达式，得到以下算法误差表。

输入	表达式	结果	绝对误差	相对误差
0	direct	1	0.000000e+00	0.000000e+00
0	rationalized	1	0.000000e+00	0.000000e+00
1	direct	0.414213562373095	9.667293e-17	2.333891e-16
1	rationalized	0.414213562373095	4.116178e-17	9.937333e-17
1e8	direct	5.00000005558832e-05	6.808832e-13	1.361766e-08
1e8	rationalized	4.9999999875e-05	4.505701e-21	9.011403e-17
1e16	direct	0	5.000000e-09	1.000000e+00
1e16	rationalized	5e-09	2.296128e-25	4.592256e-17

前表改变输入，反映条件性；后表改变计算方式，反映算法行为。g 在大输入处的相对条件数约为 $1/2$，因此直接式的大误差应从计算过程解释。

易错点与讨论

待核验的风险：不能把某个算法很准确当作问题良态的证明，也不能把病态问题直接叫作错误算法。

条件数通过改变输入、比较对应的准确输出来研究；算法误差通过固定输入、比较计算结果与独立参考来研究。g 在大输入处的相对条件数接近 $1/2$，直接式却输出0，表明良态问题也需要合适算法。

随堂提问： 给出一组实验结果，让读者判断观察到的是问题敏感性还是算法不稳定。

甲将 $q(0.9999)$ 的输入增加 10^{-8}，观察到约一万倍的相对变化放大；乙固定 $g(10^{16})$，将表达式有理化后从0得到约 5×10^{-9}。甲对应问题敏感性，乙对应算法表现。应分别说明比较中改变的对象。

1.4 AI关联：训练中的数值问题（2学时）

本节目标： 能解释训练中上溢、下溢和不稳定损失计算的风险。

概念讲解：把问题说明白

填写提示：以softmax或log-sum-exp为主例；联系梯度尺度与精度选择，完整训练和混合精度工程作为选做。

分类模型常输出一组实数 $z_1,\dots,z_n$，称为 logits，即未经归一化的类别分数。softmax 将它们转换为概率：

$$p_i=\frac{e^{z_i}}{\sum_{j=1}^{n}e^{z_j}}.$$

对于有限的实数 logits，准确概率满足 $0<p_i<1$（$n\geq2$），且概率和为1。然而，最终结果有界，并不意味着中间计算也有界。

例如，$z=[1000,1001,1002]$ 的 softmax 概率完全正常，但直接计算 e^{1000} 就会使 `float64` 上溢。两种格式的最大有限数及其自然对数约为：

类型	最大有限数	最大有限数的自然对数
`float32`	3.40×10^{38}	88.72
`float64`	1.80×10^{308}	709.78

一旦指数上溢为 `inf`，后续的 `inf/inf` 就会得到 NaN。反过来，若 logits 全为很大的负数，直接计算的指数可能全部下溢为0，随后出现 $0/0$。

关键推导：让结论有依据

推导任务：推导减去最大值的softmax改写，写明实数等价性及它能缓解的数值问题。

取

$$m=\max_j z_j.$$

在准确算术中，分子、分母同时除以 e^m，得到

$$\boxed{p_i=\frac{e^{z_i-m}}{\sum_j e^{z_j-m}}}.$$

现在每个指数的输入都不大于0，并且至少有一项为 $e^0=1$。因此，各指数均不大于1，分母满足

$$1\leq\sum_j e^{z_j-m}\leq n.$$

在移位减法和求和未溢出的通常计算范围内，这一改写避免了指数上溢和分母因全部下溢而变成零。它也是 SciPy softmax 实现所采用的基本方法。

很小的概率仍可能下溢为零。如果后面还要取对数，就应直接计算对数概率或损失。

定义 log-sum-exp 函数

$$L(z) = \log\left(\sum_j e^{z_j}\right).$$

同样的移位给出

$$L(z) = m + \log\left(\sum_j e^{z_j - m}\right).$$

若真实类别为 y，单个样本的交叉熵损失为

$$\ell = -\log p_y = L(z) - z_y.$$

代入移位公式，并安排计算顺序，得到

$$\boxed{\ell = (m - z_y) + \log\left(\sum_j e^{z_j - m}\right)}.$$

先求 $m - z_y$，可以保留后面的较小对数项，避免先计算“大数加小量”，再减去接近的大数。

考虑

$$z = [1000, 0, -1000],$$

以第三类为真实类别。移位后得到 $[0, -1000, -2000]$。在 `float64` 中，后两项的指数都下溢为0，softmax 返回 $[1, 0, 0]$。如果先取第三类概率再计算 $-\log p_y$，结果是无穷；直接使用上面的损失公式，则得到2000。

准确的损失为

$$\ell = 2000 + \log(1 + e^{-1000} + e^{-2000}),$$

极接近2000。这说明：一个无法用当前浮点格式保留的微小概率，其对数仍可能被准确地近似。

对交叉熵的准确数学表达式求导，有

$$\frac{\partial \ell}{\partial z_i} = p_i - \mathbf{1}_{\{i=y\}},$$

其中指示函数在 $i = y$ 时为1，其余为0。这个简洁的梯度公式，也是 softmax 与交叉熵常被联合计算的重要原因。

实验设计：先预测，再运行

实验任务：使用包含大幅值的logits，对比直接指数与稳定实现；将故意溢出限制在明确标注的演示中，记录有限性和概率和。

要填写的项目	本组内容
要回答的问题与运行前预测	直接指数预计产生inf或全零，移位实现预计有限；微小概率可能为0而对数域损失有限。

要填写的项目	本组内容
输入、参数、单位、数据来源与划分	三组logits为[1000,1001,1002]、[-1000,-1001,-1002]、[1000,0,-1000]；两种精度。参数更新1减 10^{-8}；梯度核验步长 10^{-5}。
独立参考解/基线及选择理由	前两组80位Decimal原始指数归一化、120位复核；极端组500位、600位复核，以分辨1附近约 10^{-435} 的变化。
误差指标、容差及其依据	有限性、概率和、最大绝对误差；float32容差 10^{-6}、float64容差 10^{-14}；损失绝对误差小于 10^{-12}；梯度绝对差小于 10^{-8}。
环境、种子、设备与运行预算	Python 3.12.14、NumPy 2.3.5、Matplotlib 3.9.2；Windows CPU。输入为确定性教学构造；模板固定随机种子2026，本例不抽样。每节通常数秒，实际用时由Notebook记录。

实现见A2组-实验.ipynb第1.4节，代码单元索引23、24、25（从0计）。本次已顺序运行并保存表格、数值记录及相关图。

结果解释与本节结论

应提供的证据：给出输入范围、dtype、是否出现inf/NaN及输出差异；说明局部稳定实现不能保证整个训练稳定。

logits	类型	直接指数含inf	直接概率含NaN	移位概率	概率和	最大绝对误差
[1000, 1001, 1002]	float32	是	是	[0.090030573331, 0.2447284609, 0.6652409434]	1	1.234297E-8
[1000, 1001, 1002]	float64	是	是	[0.090030573117, 0.2447284711, 0.6652409558]	0.9999999999999989	1.045442E-16
[-1000, -1001, -1002]	float32	否	是	[0.6652409434, 0.2447284609, 0.09003057331]	0.9999999403953552 2	1.234297E-8
[-1000, -1001, -1002]	float64	否	是	[0.6652409558, 0.2447284711, 0.09003057317]	0.9999999999999989	1.045442E-16
[1000, 0, -1000]	float32	是	是	[1, 0, 0]	1	5.075959E-435

logits	类型	直接指数含inf	直接概率含NaN	移位概率	概率和	最大绝对误差
[1000, 0, -1000]	float64	是	是	[1, 0, 0]	1	5.075959E-435

直接实现对六种组合均出现NaN。全负数组的直接指数全部为0，其他两组有正指数上溢；移位后的结果均有限，并满足float32为 10^{-6}、float64为 10^{-14} 的概率绝对误差与归一化容差。

第三组的后两类概率下溢为0，但从logits计算第三类交叉熵得到2000。准确损失为 $2000+\log(1+e^{-1000}+e^{-2000})$，比2000大约 5.07596×10^{-435}。本例的极小误差以Decimal科学计数文本保存。

float32保存的 $1-10^{-8}$ 为1，float64保存为0.99999999。前者的更新量小于1下方浮点间隔的一半，会被舍去。对 $z=[0.2,-0.4,0.7]$、$y=1$ 的中心差分核验，最大梯度绝对差约 6.82×10^{-12}，低于 10^{-8}。以上实验针对概率、损失和单次更新，不涉及完整网络训练。

易错点与讨论

待核验的风险：把loss下降、没有NaN或AI声称稳定当作数值正确的充分证据。

第三组移位softmax返回有限的 $[1,0,0]$，但两个微小概率的相对误差均为100%。无NaN和概率和接近1仍不足以判断每个分量的准确度；损失下降还受任务与优化过程影响。应结合目标量的误差和独立参考进行判断。

随堂提问： 设计一道AI建议采用直接指数计算的辨析题，要求人工改写并提供运行证据。

教学构造的建议是：“概率不超过1，直接计算指数不会溢出。”取 $z=[1000,1001,1002]$，先判断，再运行两种实现。建议混淆了最终比值与中间指数的范围；减最大值后指数输入不大于0，可以避免本例的上溢。

回到开场：用证据回答

本章核心问题： 当计算机告诉你答案是3.141592653589793时，你凭什么相信它？

开场的运行前预测是“有理化式更可靠”。第1.1节说明float64在 10^{16} 附近的间隔为2；第1.2节Notebook给出直接式0和有理化式 5×10^{-9}，后者相对高精度参考误差约 4.59×10^{-17}；第1.3节推导出 g 的相对条件数趋近 $1/2$。因此，差额是在直接计算的表示与运算过程中丢失的，有理化保留准确分子1。

漫画只帮助识别这一机制，不能代替数值分析。面对屏幕上的3.141592653589793，可信度来自输入准确性、问题敏感性、算法误差分析及独立检验，而不是显示位数本身。

本章小结与课后习题

围绕 浮点数、条件数、稳定性 各写一段：它解决什么问题、适用条件是什么、最容易混淆什么。其他方法说明与这三个概念的联系。

浮点数。 有限存储只能表示有限个数，单位舍入误差描述正规范围内单次最近舍入的相对误差。机器间隔随尺度变化，显示位数不等于有效精度；整段程序的误差还取决于误差传播。

条件数。 条件数描述问题对输入扰动的局部敏感性。标量相对公式要求输入及输出非零；较大条件数意味着输入中的小误差可能显著影响答案。改写算法不会改变同一输入模型下的问题条件数。

稳定性。 稳定性关注算法怎样处理计算中附加的误差。有理化避免微小差值的抵消，差分步长平衡截断与舍入，移位softmax及对数域损失控制指数范围；这些方法都需要结合计算目标和误差尺度选择。

课后题按四节对应填写到A2组-习题.md，参考解答单独放在A2组-习题参考解答.md。

选读边界

后向误差分析不列为本章主线；可在第4章的残差讨论中简要联系，完整分析留作选读。

选读不计入本章8学时和最低完成要求。若添加附录，只在此处填写已有材料的名称和链接，不为了占位增加空文档。

参考资料、图片来源与AI使用

支撑的主张/公式/图片	作者、标题、版本/年份	页码/章节/链接	本组人工核对
0.1表示与显示	Python Software Foundation，Python 3.12教程	第15章 Floating-Point Arithmetic	未核对
eps、正规及非正规数范围	NumPy Developers，NumPy 2.3	numpy.finfo：Notes及Attributes	未核对
高精度转换、exp与ln	Python Software Foundation，Python 3.12标准库	decimal：Decimal对象、from_float、exp、ln	未核对
条件性与稳定性	N. J. Higham，What Is Numerical Stability?，2020	前向误差与条件数说明	未核对
移位softmax	SciPy Developers，softmax文档；Blanchard等，2019预印本	Notes、论文	未核对
图01-1	课程仓库初始化助手，概念示意	图片/A-章节导入.svg，沿用模板原图	未核对
图01-2及数值表	A2组配套Notebook，本次辅助计算	A2组-实验.ipynb第1.1—1.4节；图由1.2节代码生成	未核对

漫画为仓库初始化助手绘制的概念示意，A组需结合最终正文核对并记录修改。AI建议与人工结论分开，本章对话分别记录到A2组-AI对话记录.md、模板-B组-AI对话记录.md、模板-C组-AI对话记录.md；A/B/C按轮换角色填写。

本章习题与参考解答

第01章 误差的代价：课后习题模板

状态：待A组完善题面和解答，待B/C核验。四题分别对应四节，每题可以含推导、代码与辨析小问；不额外增加课时。

这些是出题方向，不是已完成的题目。请填写明确条件和判据，至少覆盖概念/推导、实现/数值验证、陷阱辨析三类能力。

题1：对应1.1 浮点数与舍入误差

出题方向： 设计一道小数表示或舍入题，要求先手算再与两种精度比较。

- 完整题面： 本题25分。研究0.15625与0.1的二进制表示，并比较float32、float64的精度和求和顺序。采用最近舍入，中点取偶数；4位教学格式含隐含首位、指数范围足够。
- 推导或概念小问： （1）表示与舍入（8分）。将 0.15625 写成规格化二进制形式，将 0.1 写成循环二进制形式，判断它们能否由 `float32` 和 `float64` 精确表示。

  考虑一种有效二进制位数为4（含隐含首位）、指数范围足够的教学格式。采用最近舍入，中点取偶数，将 0.1 舍入到该格式，列出相邻候选数，并求绝对误差与相对误差。

  （2）精度与小数求和（6分）。`float32` 和 `float64` 的有效二进制位数分别为24和53。求它们在1处向上的浮点间隔 ε 和单位舍入误差 u。
- 代码或结果分析小问： 第（2）问的数值验证：显式指定两种dtype计算0.1+0.2，与Fraction(3,10)比较，列出存储值及绝对/相对误差。

  （3）顺序求和（7分）。对两种类型，使用逐项、同类型加法，分别计算

$$[10^{16}, 1, -10^{16}], \qquad [10^{16}, -10^{16}, 1]$$

的和。写出准确的实数目标、浮点结果及其误差，解释两种顺序的差异。对于 `float32`，还需说明大数输入本身的表示误差是否会破坏第二种顺序中的相消。

- 陷阱辨析小问： （4）辨析（4分）。以下辨析均为教学构造。分别判断并解释：

  1. “打印20位小数，就有20位有效精度。”
  2. “准确值为零时，相对误差也等于零。”
- 验收依据： 二进制展开与4位舍入须准确；间隔等于 2^{1-p}；小数误差对准确分数计算；两种逐项求和应分别得到0和1。真值0只用绝对误差。
- 难度、预计课后用时与来源： 基础，约25分钟；依据第1.1节浮点表示与求和实验，由A2组初稿整理。

题2：对应1.2 截断误差与灾难性抵消

出题方向： 设计一道同时包含有理化和差分步长选择的题，要求解释失败情形。

- 完整题面： 本题25分。研究 $g(x)=\sqrt{x+1}-\sqrt{x}$ 在 $x\geq 0$ 时的计算，以及 e^x 在 $x=1$ 处的前向差分。平方根差测试输入为 10^{16}，差分步长为 10^{-k}，$k=1,\ldots,16$。
- 推导或概念小问： （1）等价改写（7分）。对 $x\geq 0$，推导

$$g(x)=\sqrt{x+1}-\sqrt{x}$$

的有理化式。以 `float64` 输入 $x=10^{16}$，比较两种表达式的结果，并以至少80位十进制精度计算的原始差式为参考，报告相对误差。将参考精度提高到120位，检查报告的误差数值是否稳定。指出直接式在哪一步首先丢失增量1。

（2）误差推导（7分）。取 $f(x)=e^x$、$x=1$、$h>0$。从 Taylor 公式推导前向差分

$$D_h=\frac{f(1+h)-f(1)}{h}$$

的截断误差，给出含 e^{1+h} 的上界，并在 $0<h\leq 0.1$ 上给出统一上界。

若总误差估计为 $Ah+Bu/h$，其中 $A,B>0$、u 为单位舍入误差，求使该估计式最小的正步长。解释这一结果怎样帮助选择机器计算的步长。

- 代码或结果分析小问： （3）数值实验（7分）。令 $h=10^{-k}$，$k=1,\ldots,16$。统一使用 `float64` 和 `np.exp` 计算前向差分，步长由字符串 `f"1e-{k}"` 构造。以高精度 e 为参考，提交：

  1. 全部16行“步长—近似导数—绝对误差”表；
  2. 步长与总绝对误差的双对数图；
  3. 扫描中的最小误差及对应步长；
  4. $h=10^{-16}$ 时实际取样增量 $(1+h)-1$。

  进一步用准确十进制步长在高精度下计算 D_h，将总误差分成“截断误差”与“浮点实现误差”，在同一图中比较三条误差曲线。说明为何两个误差绝对值之和不一定等于总误差的绝对值。
- 陷阱辨析小问： （4）辨析（4分）。以下辨析均为教学构造。分别判断“步长越小一定越准确”“实数中等价的表达式，其浮点计算结果必定一致”，并用本题中的结果解释。
- 验收依据： 有理化相对误差小于 10^{-14}；16行扫描与三条误差曲线完整；使用80→120位参考复核。最小网格点按本次运行报告。
- 难度、预计课后用时与来源： 中等，约35分钟；平方根差沿用模板起步例，差分扫描依据第1.2节推导。

题3：对应1.3 条件数与稳定性

出题方向： 给出一组实验结果，让读者判断观察到的是问题敏感性还是算法不稳定。

- 完整题面： 本题25分。以 $q(x)=1/(1-x)$、$x=0.9999$ 及正扰动 $d=10^{-6},10^{-8},10^{-10}$ 研究输入敏感性；以平方根差的直接式与有理化式研究同一输入下的算法表现。
- 推导或概念小问： （1）一般公式（7分）。对可微标量函数 f，推导局部相对条件数

$$\kappa_{\mathrm{rel}}(x)=\left|\frac{xf'(x)}{f(x)}\right|,$$

写明所需的非零条件及极限含义。求 $q(x)=1/(1-x)$ 的相对条件数，并计算其在 $x=0.9999$ 处的值。

（3）同一问题的不同算法（6分）。推导 $g(x)=\sqrt{x+1}-\sqrt{x}$ 在 $x>0$ 时的相对条件数，并求其在 $x\to\infty$ 时的极限。

结合题2，判断“直接式输出0，证明这个问题病态”是否成立。说明把两个平方根的值作为可独立扰动的输入，与只扰动 x 有什么区别。

（4）误差尺度（4分）。当 $x=0$ 或 $f(x)=0$ 时，相对条件数的上述推导在哪一步失效？说明可以采用的替代尺度，以及使用绝对条件数时所需的可微性条件。

- 代码或结果分析小问： （2）有限扰动（8分）。在 $x=0.9999$ 处，分别施加正扰动 $d=10^{-6},10^{-8},10^{-10}$。用高精度十进制运算，列出输入相对变化、输出相对变化及其比值。

  推导有限扰动放大率的闭式表达式，解释它与局部条件数之间的关系。说明当 $d=10^{-4}$ 时，为什么不能继续给出有限的函数值和放大率。

  第（3）问同时列出g在0、1、 10^{8}、 10^{16} 处两种算法的误差，结合条件数解释。允许NumPy、Decimal及Matplotlib。
- 陷阱辨析小问： 以下辨析均为教学构造。辨析：“直接式输出0，证明问题病态”“算法算得准，便证明问题良态”。要求根据改变输入与改变算法的区别作答，本项纳入第（3）问6分。
- 验收依据： 条件数9999；放大率与有限扰动闭式值的Decimal差小于 10^{-65}。g的条件数为 $\frac{1}{2}\sqrt{x/(x+1)}$，极限为1/2；两类证据分开解释。
- 难度、预计课后用时与来源： 中等，约30分钟；依据第1.3节条件数与有限扰动关系，由A2组初稿整理。

题4：对应1.4 AI关联：训练中的数值问题

出题方向： 设计一道AI建议采用直接指数计算的辨析题，要求人工改写并提供运行证据。

- 完整题面： 本题25分。对[1000,1001,1002]、[-1000,-1001,-1002]、[1000,0,-1000]三组logits，比较float32和float64的直接/移位softmax、第三类损失及单次参数更新。类别索引从0计；梯度核验取 $z=[0.2,-0.4,0.7]$、$y=1$。
- 推导或概念小问： （1）公式推导（7分）。对有限 logits z_i，推导减最大值的 softmax 公式及真实类别为 y 的对数域交叉熵公式。说明移位后指数与分母的范围，以及微小概率仍可能发生的变化。假设移位减法和求和均未溢出。

  从准确损失表达式推导

  $$\frac{\partial \ell}{\partial z_i}=p_i-\mathbf{1}_{\{i=y\}}.$$
- 代码或结果分析小问： （2）两种精度对照（7分）。分别使用 `float32` 和 `float64`，计算下列三组 logits 的直接 softmax 与减最大值 softmax：

  $$[1000,1001,1002],\qquad [-1000,-1001,-1002],\qquad [1000,0,-1000].$$

列出直接指数是否含 `inf`、直接概率是否含 NaN、移位概率、概率和及概率的最大绝对误差。

前两组使用80位十进制参考，并提高到120位复核。第三组使用500位参考，并提高到600位复核，观察 $1-p_0$ 的量级。极小误差以十进制科学计数文本报告；同时说明后两类概率变为零时的相对误差。

（3）损失与更新（7分）。第三组取 $y=2$，比较“先求概率再取负对数”与“从 logits 直接计算交叉熵”。写出准确损失表达式，说明机器结果2000与准确值的关系。

再比较两种类型保存的 $1-10^{-8}$，用1下方的浮点间隔解释结果。说明为什么只检查“没有 NaN”，不足以判断计算是否准确。

（4）梯度核验（4分）。取 `float64` 的 $z=[0.2,-0.4,0.7]$、$y=1$，用 $h=10^{-5}$ 的中心差分核对三个损失偏导，列出解析值、差分值及绝对差。检查解析梯度的分量和，并联系 softmax 对 logits 共同平移不变的性质作出解释。

- 陷阱辨析小问： 以下辨析均为教学构造。教学构造建议：“概率不超过1，可直接计算大正指数；只要loss下降且没有NaN，就说明数值正确。”指出中间量范围与微小概率/更新的误差问题，纳入第（1）、（3）问评分。
- 验收依据： 概率绝对误差与归一化误差：float32小于 10^{-6}，float64小于 10^{-14}；对数域损失误差小于 10^{-12}；梯度最大绝对差小于 10^{-8}。前两组参考80→120位，极端组500→600位。
- 难度、预计课后用时与来源： 中等，约35分钟；依据第1.4节移位公式与本组局部实验，不要求训练网络。

第01章 误差的代价：习题参考解答模板

状态：待填写。题号与A2组-习题.md一一对应，未推导、未运行、未核验的项目如实注明。公开仓库中的答案同样公开，本文件不纳入学生版教材目录。

题1：对应1.1 浮点数与舍入误差

- 条件与符号： 本题25分。研究0.15625与0.1的二进制表示，并比较float32、float64的精度和求和顺序。采用最近舍入，中点取偶数；4位教学格式含隐含首位、指数范围足够。 误差统一记为绝对误差 E_{a} 与非零参考下的相对误差 E_{r}。
- 手动推导或概念解释： 由 $0.15625=5/32$，得到

$$0.15625=0.00101_2=1.01_2\times 2^{-3}.$$

它只需3位有效二进制位，指数也在两种格式的范围内，因此可由 `float32` 和 `float64` 精确表示。

对 0.1，有

$$0.1=0.0001100110011\cdots_2=1.100110011\cdots_2\times 2^{-4}.$$

最简分数 $1/10$ 的分母含有因子5，二进制展开无限循环，因此两种类型都只能近似表示。

4位教学格式中，0.1 的相邻候选数为

$$a=1.100_2\times 2^{-4}=\frac{12}{128}=0.09375,\qquad b=1.101_2\times 2^{-4}=\frac{13}{128}=0.1015625.$$

到准确值 $1/10$ 的距离分别为

$$\frac{1}{10}-a=\frac{1}{160},\qquad b-\frac{1}{10}=\frac{1}{640}.$$

因此最近舍入取 b，绝对误差与相对误差分别为

$$E_{\mathrm{a}}=\frac{1}{640}=0.0015625,\qquad E_{\mathrm{r}}=\frac{1/640}{1/10}=\frac{1}{64}=0.015625.$$

评分建议： 两个二进制展开及可表示性4分；候选数与最近舍入判断2分；两种误差2分。

有效位数为 p 时，$\varepsilon=2^{1-p}$、$u=2^{-p}$。float32对应 2^{-23}、2^{-24}，float64对应 2^{-52}、2^{-53}。float64在 10^{16} 处的间隔为2，加1落在中点并舍入回原数；float32间隔更大。先使两个相反的机器数相消，再加1，则保留结果1。

- 实现位置与独立参考： Notebook第1.1节，代码单元索引3—5。参考为Fraction的准确分数3/10和整数1；二进制舍入由手算验证。
- 实际输出、误差和容差理由：

类型	存储结果	绝对误差	相对误差
float32	0.30000001192092896	1.1920928955e-08	3.9736429850e-08
float64	0.30000000000000004	4.4408920985e-17	1.4802973662e-16

类型	顺序	结果	绝对误差	相对误差
float32	[10000000000000000, 1, -10000000000000000]	0.0	1.0	1.0
float32	[10000000000000000, -10000000000000000, 1]	1.0	0.0	0.0
float64	[10000000000000000, 1, -10000000000000000]	0.0	1.0	1.0
float64	[10000000000000000, -10000000000000000, 1]	1.0	0.0	0.0

  间隔由理论值精确检查；求和比较使用准确目标1。
- 陷阱的正确解释及适用范围： 显示位数只改变已存数字的呈现，不能增加有效精度。浮点间隔随指数变化：float64在1处向上的间隔约为 2.22×10^{-16}，在 10^{16} 处却为2。两种顺序求和分别得到0和1，说明单次舍入界不能直接作为整个程序的误差界。 设 $a=0.15625$。先手算其规格化二进制表示，再判断它在float32和float64中是否精确。常见回答是“十进制小数都不精确”；正确解释是 $a=1.01_2\times2^{-3}$，展开有限且位数、指数均在范围内，两种格式均精确。 准确值为零时，相对误差公式分母为零，不能把相对误差设为零；应报告绝对误差或明确选用的非零外部尺度。
- B核验、C复核及版本： 待核验。

题2：对应1.2 截断误差与灾难性抵消

- 条件与符号： 本题25分。研究 $g(x)=\sqrt{x+1}-\sqrt{x}$ 在 $x\ge 0$ 时的计算，以及 e^x 在 $x=1$ 处的前向差分。平方根差测试输入为 10^{16}，差分步长为 10^{-k}，$k=1,\dots,16$。 误差统一记为绝对误差 E_{a} 与非零参考下的相对误差 E_{r}。
- 手动推导或概念解释： 对 $x\ge 0$，$\sqrt{x+1}+\sqrt{x}>0$，乘共轭式得 $g(x)=1/(\sqrt{x+1}+\sqrt{x})$。程序直接保留准确分子1。直接式首先在 $x+1$ 处丢失增量，再得到两个相同平方根。

  Taylor 公式给出某个 $\xi\in(1,1+h)$，使

$$e^{1+h}=e+he+\frac{h^2}{2}e^{\xi}.$$

因此

$$D_h-e=\frac{h}{2}e^{\xi}>0,\qquad |D_h-e|\le\frac{h}{2}e^{1+h}.$$

当 $0<h\le 0.1$ 时，有统一上界

$$|D_h-e|\le\frac{e^{1.1}}{2}h.$$

这是准确算术下的截断误差。令误差估计式为 $F(h)=Ah+Bu/h$，则

$$F'(h)=A-\frac{Bu}{h^2},\qquad F''(h)=\frac{2Bu}{h^3}>0.$$

唯一的正驻点为

$$h_{\mathrm{bal}}=\sqrt{\frac{Bu}{A}},\qquad F(h_{\mathrm{bal}})=2\sqrt{ABu}.$$

它给出截断误差与舍入误差平衡的尺度，可用于确定扫描区间。实际舍入随步长波动，函数计算、取样点表示和常数 A,B 都影响具体最小点。

评分建议： Taylor 公式与余项3分；上界1分；平衡步长2分；估计式与实际计算的关系1分。

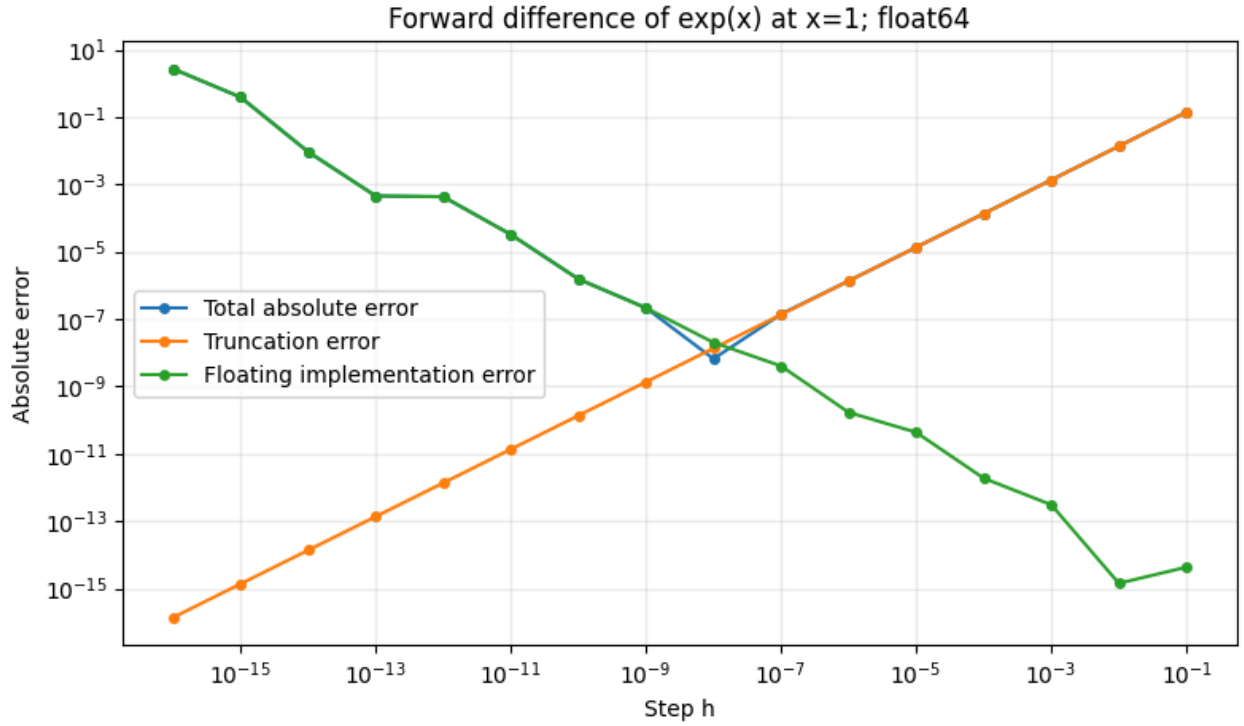
误差分解为 $\widehat{D}_h-e=(D_h-e)+(\widehat{D}_h-D_h)$。各项保留符号时严格相加，取绝对值后可以部分抵消。

- 实现位置与独立参考： Notebook第1.2节，索引10保留模板原起步代码；索引11—13补充高精度误差与完整步长扫描。原起步代码把参考转为float，只用于粗检查；最终误差在Decimal中比较。
- 实际输出、误差和容差理由：

步长	近似导数	总绝对误差	实际取样增量
1e-01	2.85884195487388	1.40560126e-01	1.00000000e-01
1e-02	2.73191865578712	1.36368273e-02	1.00000000e-02
1e-03	2.71964142253323	1.35959407e-03	1.00000000e-03
1e-04	2.71841774708292	1.35918624e-04	1.00000000e-04
1e-05	2.71829541995672	1.35914977e-05	1.00000000e-05
1e-06	2.71828318743061	1.35897157e-06	1.00000000e-06

步长	近似导数	总绝对误差	实际取样增量
1e-07	2.71828196840573	1.39946688e-07	1.00000000e-07
1e-08	2.71828182185629	6.60275094e-09	9.99999994e-09
1e-09	2.7182820439009	2.15441854e-07	1.00000008e-09
1e-10	2.71828337616853	1.54770948e-06	1.00000008e-10
1e-11	2.71831446241322	3.26339542e-05	1.00000008e-11
1e-12	2.71871414270208	4.32314243e-04	1.00008890e-12
1e-13	2.71782596428238	4.55864177e-04	9.99200722e-14
1e-14	2.70894418008538	9.33764837e-03	9.99200722e-15
1e-15	3.10862446895044	3.90342640e-01	1.11022302e-15
1e-16	0	2.71828183e+00	0.00000000e+00

平方根差直接式为0，有理化式为 5×10^{-9}，相对误差约 4.59226×10^{-17}，小于 10^{-14}；16个步长完整保留，最佳点约为 10^{-8}。



- 陷阱的正确解释及适用范围： 本次扫描在 $h = 10^{-8}$ 附近取得较小误差，而 $h = 10^{-16}$ 时取样点重合。因此，步长需要平衡截断与舍入。有理化式显式保留分子1；若仍把分子写成浮点 `(x+1)-x`，就会再次丢失增量。 在float64中分别计算 $g(10^{16})$ 的直接式、有理化式，以及 e^x 在1处步长 10^{-8}、10^{-16} 的前向差分。若认为“步长更小就更准”，应解释为何后一个差分变为0。关键在于 $1+h$ 已舍入为1，而有理化保留了准确分子1。
- B核验、C复核及版本： 待核验。

题3：对应1.3 条件数与稳定性

- 条件与符号： 本题25分。以 $q(x)=1/(1-x)$、$x=0.9999$ 及正扰动 $d=10^{-6},10^{-8},10^{-10}$ 研究输入敏感性；以平方根差的直接式与有理化式研究同一输入下的算法表现。 误差统一记为绝对误差 E_{a} 与非零参考下的相对误差 E_{r}。
- 手动推导或概念解释： 由可微性，

$$f(x+d)-f(x)=f'(x)d+o(|d|).$$

当 $x\neq 0$ 且 $f(x)\neq 0$ 时，

$$\lim_{d\to 0,\,d\neq 0}\frac{|f(x+d)-f(x)|/|f(x)|}{|d|/|x|}=\left|\frac{xf'(x)}{f(x)}\right|.$$

对 $q(x)=1/(1-x)$，定义域排除1，有

$$q'(x)=\frac{1}{(1-x)^2},\qquad \kappa_q(x)=\left|\frac{x}{1-x}\right|.$$

所以 $\kappa_q(0.9999)=9999$。

评分建议： 一阶展开2分；非零条件与极限2分；求导、条件数及数值3分。

有限扰动满足 $[q(x+d)-q(x)]/q(x)=d/(1-x-d)$，故 $R(d)=|x/(1-x-d)|$。当 $d\to 0$，放大率趋近9999；$d=10^{-4}$ 到达奇点。

当 $x>0$ 时，

$$g'(x)=\frac{1}{2\sqrt{x+1}}-\frac{1}{2\sqrt{x}}=-\frac{g(x)}{2\sqrt{x}\sqrt{x+1}}.$$

于是

$$\kappa_g(x)=\frac{1}{2}\sqrt{\frac{x}{x+1}}<\frac{1}{2},\qquad \lim_{x\to\infty}\kappa_g(x)=\frac{1}{2}.$$

对于唯一输入 x 的相对扰动，这个问题在大 x 处是良态的。直接式输出0表明计算表达式丢失了信息。实际上，对任何有限 $x\geq 0$，都有 $g(x)>0$，所以0也不能解释为某个邻近有限输入的准确函数值。

若把两个根的值作为独立输入 a,b，问题就变为计算 $a-b$。在分量相对扰动约束 $|\Delta a|\leq\eta|a|$、$|\Delta b|\leq\eta|b|$ 下，最坏的输出相对变化放大率为

$$\frac{|a|+|b|}{|a-b|}.$$

当 a,b 接近时，它可以很大。这与由同一个 x 决定两个根的输入模型不同。

评分建议： 求导与条件数3分；极限1分；问题、算法与输入模型的区分2分。

$x=0$ 时，输入相对变化 $|d|/|x|$ 无定义；$f(x)=0$ 时，输出相对变化无定义。若 f 在该点可微且导数有限，可用绝对条件数 $|f'(x)|$ 研究小的绝对扰动，或指定非零的输入、输出参考尺度。

对于 g 在0处，还应注意它没有有限的右导数：

$$\frac{g(h)-g(0)}{h}=\frac{1}{\sqrt{1+h}+1}-\frac{1}{\sqrt{h}}\longrightarrow-\infty.$$

此时绝对线性条件数也不能提供有限的局部界，应根据函数增量本身分析敏感性。

评分建议： 两种相对尺度失效的原因2分；替代尺度及其可微性条件2分。对 $g(0)$ 的说明可作为加深理解的补充。

- 实现位置与独立参考： Notebook第1.3节，索引17—19。独立参考为q的有限扰动闭式关系，以及g的120位Decimal原始差式。
- 实际输出、误差和容差理由：

扰动	输入相对变化	输出相对变化	放大率
1e-6	1.0001000100E-6	1.0101010101E-2	10100.0000000000
1e-8	1.0001000100E-8	1.0001000100E-4	10000.0000000000
1e-10	1.0001000100E-10	1.0000010000E-6	9999.0099990100

输入	表达式	结果	绝对误差	相对误差
0	direct	1	0.000000e+00	0.000000e+00
0	rationalized	1	0.000000e+00	0.000000e+00
1	direct	0.414213562373095	9.667293e-17	2.333891e-16
1	rationalized	0.414213562373095	4.116178e-17	9.937333e-17
1e8	direct	5.00000005558832e-05	6.808832e-13	1.361766e-08
1e8	rationalized	4.9999999875e-05	4.505701e-21	9.011403e-17
1e16	direct	0	5.000000e-09	1.000000e+00
1e16	rationalized	5e-09	2.296128e-25	4.592256e-17

  有限扰动结果按80位Decimal检查闭式关系；g的有理化相对误差在测试输入上小于 10^{-14}。

- 陷阱的正确解释及适用范围： 条件数通过改变输入、比较对应的准确输出来研究；算法误差通过固定输入、比较计算结果与独立参考来研究。g 在大输入处的相对条件数接近 $1/2$，直接式却输出0，表明良态问题也需要合适算法。 甲将 $q(0.9999)$ 的输入增加 10^{-8}，观察到约一万倍的相对变化放大；乙固定 $g(10^{16})$，将表达式有理化后从0得到约 5×10^{-9}。甲对应问题敏感性，乙对应算法表现。应分别说明比较中改变的对象。
- B核验、C复核及版本： 待核验。

题4：对应1.4 AI关联：训练中的数值问题

- 条件与符号： 本题25分。对[1000,1001,1002]、[-1000,-1001,-1002]、[1000,0,-1000]三组logits，比较float32和float64的直接/移位softmax、第三类损失及单次参数更新。类别索引从0计；梯度核验取 $z=[0.2,-0.4,0.7]$、$y=1$。 误差统一记为绝对误差 E_{a} 与非零参考下的相对误差 E_{r}。
- 手动推导或概念解释： 令 $m=\max_j z_j$，实数运算中分子、分母同除以 e^{m}，得到

$$p_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}, \qquad \ell = (m - z_y) + \log \sum_j e^{z_j - m}.$$

各指数输入不大于0，至少有一项等于0，因此指数值不超过1、分母介于1与 n 之间。移位避免了正指数上溢和所有指数同时下溢为零；极小的个别概率仍可能舍入为0。

从准确的光滑表达式 $\ell = \log \sum_j e^{z_j} - z_y$ 求导，得到

$$\frac{\partial \ell}{\partial z_i} = \frac{e^{z_i}}{\sum_j e^{z_j}} - \mathbf{1}_{\{i=y\}} = p_i - \mathbf{1}_{\{i=y\}}.$$

采用这一求导方式，也避免了在最大值并列时单独讨论 $\max$ 的导数。

评分建议： 两个移位公式3分；范围及下溢说明2分；梯度推导2分。

第三组准确概率为 $[1, e^{-1000}, e^{-2000}]/(1 + e^{-1000} + e^{-2000})$，后两项虽为正却会下溢为0，其分量相对误差均为100%。准确第三类损失为 $2000 + \log(1 + e^{-1000} + e^{-2000})$，机器结果2000是合理近似。

float32在1下方的间隔为 2^{-24}，更新量 10^{-8} 小于其一半，结果舍入回1。梯度以 $[\ell(z + he_i) - \ell(z - he_i)]/(2h)$ 核对；解析梯度分量和为 $\sum_i p_i - 1 = 0$，对应共同平移方向的导数为0。

- 实现位置与独立参考： Notebook第1.4节，索引23—25。前两组80→120位参考，极端组500→600位参考；梯度由解析式与中心差分交叉核对。
- 实际输出、误差和容差理由：

logits	类型	直接指数含inf	直接概率含NaN	移位概率	概率和	最大绝对误差
[1000, 1001, 1002]	float32	是	是	[0.090030573331, 0.2447284609, 0.6652409434]	1	1.234297E-8
[1000, 1001, 1002]	float64	是	是	[0.090030573117, 0.2447284711, 0.6652409558]	0.9999999999999989	1.045442E-16
[-1000, -1001, -1002]	float32	否	是	[0.6652409434, 0.2447284609, 0.09003057331]	0.999999940395355 22	1.234297E-8
[-1000, -1001, -1002]	float64	否	是	[0.6652409558, 0.2447284711, 0.09003057317]	0.9999999999999989	1.045442E-16
[1000, 0, -1000]	float32	是	是	[1, 0, 0]	1	5.075959E-435
[1000, 0, -1000]	float64	是	是	[1, 0, 0]	1	5.075959E-435

分量	解析梯度	中心差分	绝对差
0	0.312741112960930	0.312741112962200	1.27087230e-12
1	-0.828364038122046	-0.828364038119034	3.01181302e-12
2	0.515622925161116	0.515622925167936	6.81954493e-12

两种概率绝对误差与归一化误差满足题设容差；梯度最大绝对差约 6.82×10^{-12}。容差针对指定教学输入。

类型	先概率再取负对数	从logits计算损失	损失绝对误差
float32	inf	2000.0	5.075958897549E-435
float64	inf	2000.0	5.075958897549E-435

类型	更新后保存值	1下方间隔
float32	1.0	5.9604644775e-08
float64	0.99999999	1.1102230246e-16

- 陷阱的正确解释及适用范围： 第三组移位softmax返回有限的 $[1, 0, 0]$，但两个微小概率的相对误差均为100%。无NaN和概率和接近1仍不足以判断每个分量的准确度；损失下降还受任务与优化过程影响。应结合目标量的误差和独立参考进行判断。 教学构造的建议是：“概率不超过1，直接计算指数不会溢出。”取 $z = [1000, 1001, 1002]$，先判断，再运行两种实现。建议混淆了最终比值与中间指数的范围；减最大值后指数输入不大于0，可以避免本例的上溢。
- B核验、C复核及版本： 待核验。

第02章 逼近的智慧：2.1—2.4

> 编写组：A2（G21）；来源版本：`1246f1a1a7bc`，最新补齐分支，尚未合入main。本文件按原稿收录正文、独立习题与参考解答，保留原稿待完成项。

编写组别：A2组＝G21；角色：A（编写）；范围：2.1—2.4，共8学时（2.1、2.2为本次补写；2.3、2.4为原交付稿）。

本稿覆盖全章四节：2.1 插值与Runge现象、2.2 最小二乘与切比雪夫逼近（本次补写），以及 2.3 数值积分与自动微分、2.4 神经网络逼近与PINN求导（原稿）。正文与2.1—2.2实验、2.3—2.4实验、习题、参考解答配套。文中数值均来自本机执行：2.1、2.2 见实验脚本与实测摘要；2.3、2.4 见配套Notebook与 数据/。计算量均为无量纲。推导与代码由AI辅助编写，学生人工核验与B/C复核状态见编写报告。

从河岸面积到曲线斜率

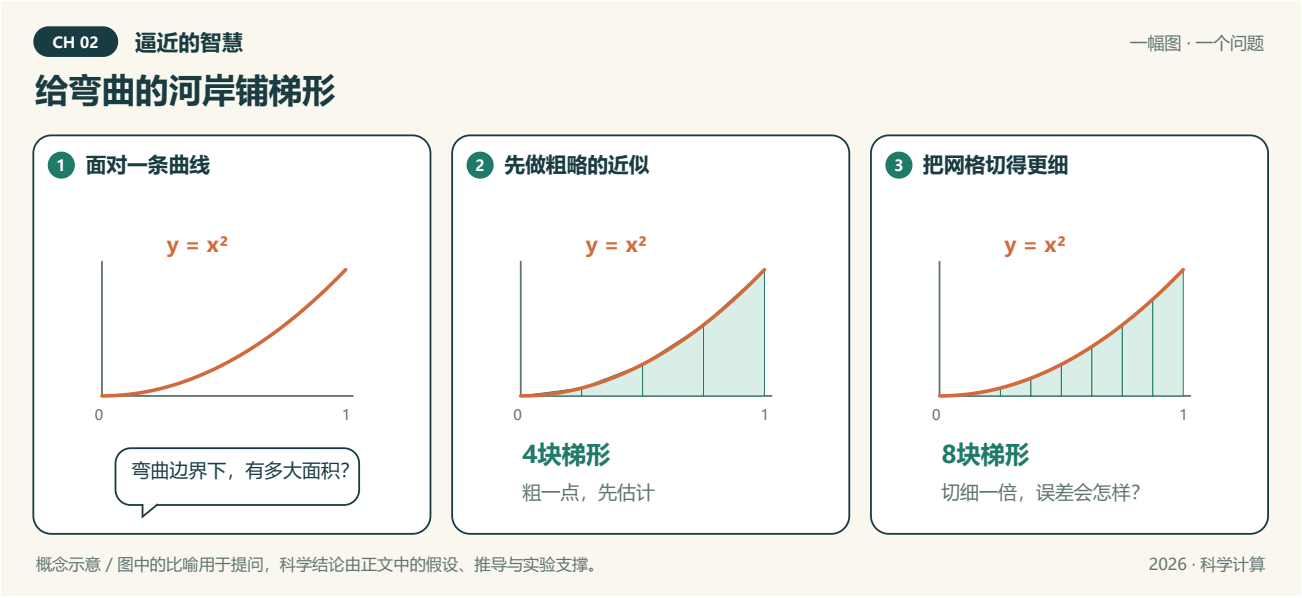


图1：沿用课程仓库的章节导入SVG，属于概念示意，不是实测图。图中“河岸”对应已知曲线，未涉及真实地形测量误差。

把一段弯曲边界下的面积切成4块梯形，再切成8块，误差能减少多少？假如我们训练一个网络把边界画得很像，它给出的斜率、曲率是否也准确？这两个问题都在讨论逼近，但需要检验不同对象：面积、函数值、一阶导数和二阶导数不能用同一个误差指标代替。

小节	学习目标	2学时建议安排
2.1	写出插值问题与Lagrange解；用余项解释等距节点为何失效	讲解20分钟、推导25分钟、实验30分钟、解释与讨论15分钟
2.2	推导正规方程与唯一性条件；比较两类基；识别过拟合	讲解20分钟、推导25分钟、实验30分钟、解释与讨论15分钟

小节	学习目标	2学时建议安排
2.3	从线性插值推出复合梯形公式；解释二阶收敛；区分差分、符号微分、AD	讲解20分钟、推导25分钟、实验30分钟、解释与讨论15分钟
2.4	写出网络函数及其输入、参数导数；构造二阶残差；识别导数逼近的限制	讲解20分钟、推导25分钟、实验30分钟、解释与讨论15分钟

先修知识：定积分、Taylor展开、链式法则、偏导数与最小二乘。下文用到的梯度和Hesse矩阵概念可与本地《数值最优化方法》第2章开头对照，PDF第22页、书内第8页。[L3]

2.1 插值与Runge现象（2学时）

本节目标： 能写出插值问题的定义和Lagrange形式的唯一解；能用余项公式说明“节点上准确”与“区间内准确”是两件不同的事；能解释等距节点为何会让误差随节点数增加而爆炸，以及Chebyshev节点改变了什么、没有改变什么。

概念讲解：把问题说明白

桌上摆着十一张卡片，每张写一个测点 $(x_0,y_0),\dots,(x_{10},y_{10})$。要求你画一条曲线，必须逐点穿过。

这就是插值。给定 $m+1$ 个互不相同的节点 $x_0<x_1<\cdots<x_m$ 和对应函数值 y_i，找一个次数不超过 m 的多项式

$$p_m(x)=a_0+a_1x+\cdots+a_mx^m,\qquad p_m(x_i)=y_i,\quad i=0,\dots,m. \tag{2.1-1}$$

把条件写成矩阵是 $Va=y$，其中 $V_{ij}=x_i^j$ 是Vandermonde矩阵。节点两两不同时 V 可逆，所以解存在且唯一。这个唯一性几乎是整节所有讨论的地基：只要有一个多项式满足全部插值条件，它就是答案，不必再找第二个。

需要先分清两个“准”：

说法	含义	谁关心
节点上准	$p_m(x_i)=y_i$	插值条件的定义，自动成立
区间内准	$\max_{x\in[a,b]}\lvert p_m(x)-f(x)\rvert$ 小	真正想用的地方

节点上满足插值条件，不能推出区间内误差小。本节要展示的，正是“前者做得完美、后者失败得很惨”的情形。

还要和下一节区分：插值要求曲线穿过每个点；最小二乘只要求靠近。数据有噪声时，硬穿等于把噪声当信号，这是2.2的主题。

关键推导：让结论有依据

先看最简单的情形。 两个节点 x_0,x_1，用直线连接：

$$p_1(x)=y_0\frac{x-x_1}{x_0-x_1}+y_1\frac{x-x_0}{x_1-x_0}. \tag{2.1-2}$$

两个系数在节点上分别取 1 和 0。若 $f\in C^2$，由Rolle定理两次可得存在 ξ 使

$$f(x)-p_1(x)=\frac{f''(\xi)}{2}(x-x_0)(x-x_1). \tag{2.1-3}$$

这就是两点插值的误差：一项关于函数，一项关于节点间距。

推广到 $m+1$ 个节点。 构造基函数

$$\ell_i(x)=\prod_{j\neq i}\frac{x-x_j}{x_i-x_j},\qquad i=0,\dots,m, \tag{2.1-4}$$

每个 ℓ_i 是 m 次多项式，且 $\ell_i(x_j)=1$（$i=j$）、$\ell_i(x_j)=0$（$i\neq j$）。于是

$$p_m(x)=\sum_{i=0}^{m}y_i\,\ell_i(x) \tag{2.1-5}$$

满足全部插值条件；由唯一性，它就是式(2.1-1)的答案。

若 $f\in C^{m+1}$，则存在 $\xi\in(a,b)$ 使得

$$f(x)-p_m(x)=\frac{f^{(m+1)}(\xi)}{(m+1)!}\,\omega(x),\qquad \omega(x)=\prod_{i=0}^{m}(x-x_i). \tag{2.1-6}$$

这个公式值得盯住看。 右边是两部分的乘积：一部分是 f 的高阶导数，另一部分是 $\omega(x)$。函数本身改不了，$\omega(x)$ 却完全由节点的位置决定——节点不是随便摆的，它是一个可以优化的变量。

节点怎么选。 以下用 n 表示节点数，插值多项式次数至多 $n-1$（前文的次数 $m=n-1$）。要在 $[-1,1]$ 上让 $\max|\omega(x)|$ 最小，取Chebyshev多项式的零点

$$x_k=\cos\frac{(2k+1)\pi}{2n},\qquad k=0,\dots,n-1,\ n\geq 1. \tag{2.1-7}$$

它们的分布是两端密、中间疏。对 n 个这样的节点，节点多项式为 $\omega(x)=2^{1-n}T_n(x)$，在所有首一 n 次多项式中具有最小的区间无穷范数 2^{1-n}。它优化的是固定节点数下的节点乘积，而不是直接保证任意函数的插值误差最小。

Runge现象说明：对某些固定函数，等距节点数增加时，插值多项式在端点附近的误差反而增长。不能把这一现象解释成“节点乘积的最大值必随节点数增长”，也不能从误差上界较大直接推出实际误差发散。需要同时考虑函数性质、节点分布和插值过程的稳定性。[R5]

实验设计：先预测，再运行

本表随本次文稿补充，预测作为课堂教学预期；不将其视为原实验运行前已登记的记录。

项目	本组内容
要回答的问题与运行前预测	① 11个等距测点的插值曲线，在节点之间是否会失控，预测在端点附近出现明显振荡；② 节点数从5增到31，等距插值误差是否单调变小，预测不会；③ 同样节点数换成Chebyshev节点，误差是否随节点数下降，预测会。
输入、参数、单位、数据来源与划分	目标函数 $f(x)=1/(1+25x^2)$、区间 $[-1,1]$；节点数 $n=5,9,11,15,21,31$，两组节点（等距、Chebyshev）各自独立；误差在4001点等距网格上评价，该网格独立于拟合过程生成，但包含部分插值节点；同时单独报告节点处误差。数据由本机生成，无量纲。

项目	本组内容
独立参考解/基线及选择理由	参考值是 f 的解析值，不取任何插值结果；基线取“节点上误差”与“网格最大误差”并列，用来区分两种“准”。
误差指标、容差及其依据	在评价网格 t_j 上记 $\max_j\|p_{n-1}(t_j)-f(t_j)\|$，并另记节点处误差。本节不给合格阈值——要展示的正是误差可以任意大；只要求同一 n 下两种节点的结果可直接比较。
环境、种子、设备与运行预算	Python 3.12.10、NumPy 1.26.4、SciPy 1.14.1、Matplotlib 3.9.2；CPU单线程、float64；插值用重心形式（`BarycentricInterpolator`），减轻直接求Vandermonde系数带来的数值问题；函数及节点无随机成分，权重计算固定random_state=2026。预算：6个节点数 × 2种节点，单次秒级。

实验代码见随附脚本 `2.1-2.2-实验.py`（并入Notebook时对应 2.1-A 与 2.1-B 两个小节），本机输出留档在 `2.1-2.2-实测摘要.txt`。运行时顺序执行，不挑选结果。

结果解释与本节结论

等距节点与Chebyshev节点在4001点评价网格上的实测对比：

节点数 n	等距节点最大误差	Chebyshev节点最大误差	等距/Cheb
5	4.384e-01	4.020e-01	1.1
9	1.045e+00	1.708e-01	6.1
11	1.916e+00	1.092e-01	17.6
15	7.195e+00	4.660e-02	154.4
21	5.982e+01	1.533e-02	3901.4
31	2.388e+03	2.062e-03	1.16e6

“多点更准”的直觉在这里失效。 等距节点从11个加到31个，误差从 1.9 涨到 2388，涨了三个数量级。这不是代码写错：在11个节点上，插值值与真值之差恰好为0——节点上完全正确。问题出在节点之间。

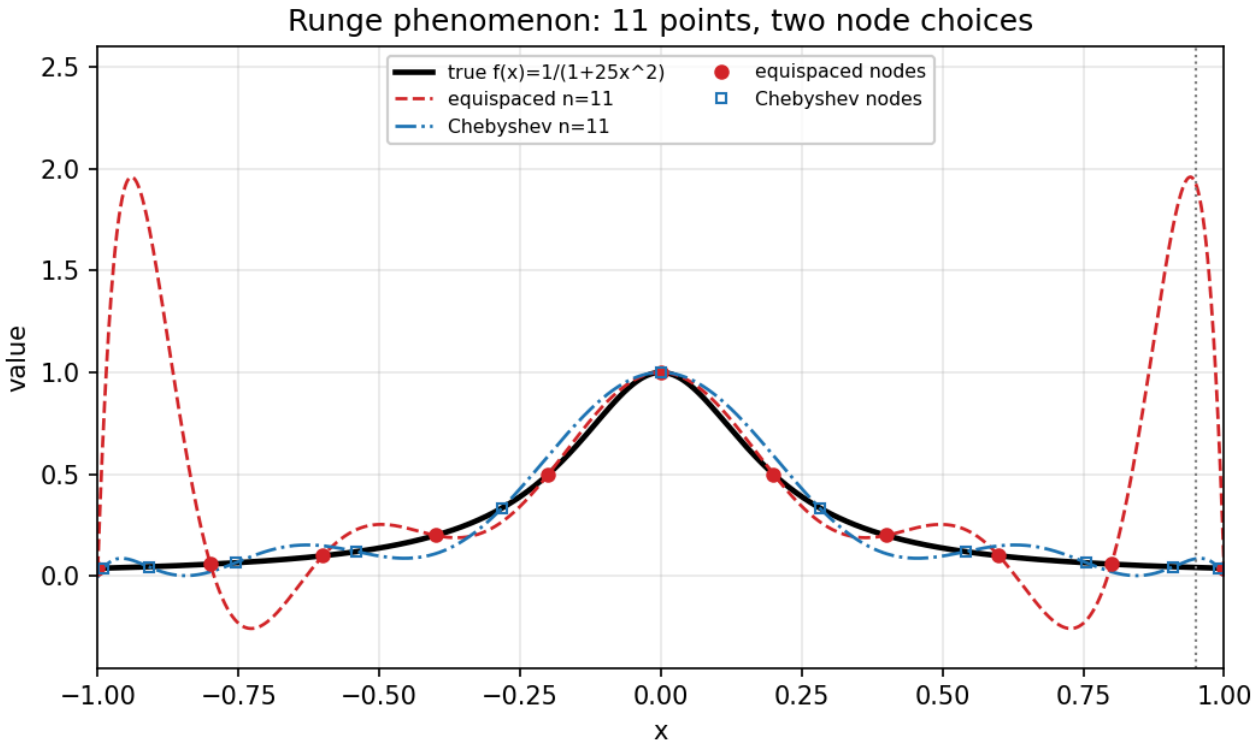


图2：本机实测。同一函数、同样11个节点，等距插值（虚线）在两端剧烈振荡，Chebyshev节点（点划线）贴着曲线。竖点线标出 $x = 0.95$：等距插值给出 1.9236，真值为 0.0424。

在 $x = 0.95$ 处，真值 0.0424，等距插值给出 1.9236——一个几乎不存在的峰。而 $x = 0.95$ 离最近的节点并不远，这正是Runge现象最刺人的地方：错误不在天边，就在你最想信任的地方。

为什么会这样。 余项公式提示节点分布与函数高阶导数共同影响误差，但它本身不是Runge发散的充分证明。特别是 ω 是乘积，正负因子不会像加法那样互相抵消；取绝对值后，各节点距离的绝对值相乘。这里用数值实验展示该固定函数的等距插值在端点附近失效，并用另一组节点作对照。

换节点之后。 表第三列：$n = 11$ 时约0.109，$n = 31$ 时约0.00206，在所列节点数与同一评价网格上误差下降。同样的函数与节点数，改变节点位置即可明显改善本例结果。

适用边界。 Chebyshev零点优化固定节点数的节点乘积，并改善插值的稳定性；是否收敛及其速度仍需函数正则性等条件。不能保证任意连续函数都一致收敛，也不能反过来断言所有不光滑函数都无法收敛。采样网格的最大值仍不是连续全区间误差的严格上界。[R5]

易错点与讨论

1. “曲线穿过所有点，所以它是对的”。 节点误差恒为0，与区间误差无关：$n = 11$ 时节点误差为0，区间最大误差是1.92。
2. “点越多越准”。 对本例的等距节点是错的：n 从11到31，误差从1.92涨到2388。一般收敛结论需要节点分布与函数正则性的具体条件，不能仅凭点数作保证。
3. 把条件数问题当成Runge现象。 Vandermonde矩阵病态会让算法不稳；Runge现象是插值多项式本身发散。两者都会让曲线变乱，来源不同，处理方式也不同。
4. Chebyshev节点等于万能药。 它只优化 ω；函数的高阶导数、区间端点、计算格式仍要单独检查。

随堂提问： 有人看到11个点算出的曲线“完美穿过”所有测点，就把中间某处的读数当作可信值上报。你会用本节哪个量、哪一条数据劝阻他？如果他说“那就再加10个点”，你要先补什么条件？

2.2 最小二乘与切比雪夫逼近（2学时）

本节目标： 能从一个过定方程组写出最小二乘目标，推导正规方程并说明唯一性条件；能解释“基函数不改变拟合空间、只改变数值条件”；能用条件数和独立测试误差说明为什么残差小不等于拟合好。

概念讲解：把问题说明白

2.1 要求曲线穿过每一点。现实中这往往做不到，也不该做：测量有噪声，硬穿等于把噪声当成真值。

于是问题改成：用 m 个基函数 $\phi_0,\dots,\phi_{m-1}$ 的线性组合去尽量靠近数据，

$$p(x)=\sum_{k=0}^{m-1} c_k\phi_k(x),\qquad \min_{c}\ \|Ac-y\|_2^2,\quad A_{ik}=\phi_k(x_i). \tag{2.2-1}$$

A 是 $N\times m$ 的设计矩阵。样本数 N 大于未知数 m 时方程组过定，一般无解；最小二乘的意思就是“找不到精确解，就找残差最短的那个”。

为什么用2-范数？因为平方和可导、有解析解、几何上对应正交投影。这不是唯一选择：1-范数更抗离群点，无穷范数对应一致逼近。本节只走2-范数这条主线。

名字要先分清。 “切比雪夫逼近”有两种常见含义：一是用Chebyshev多项式作基做最小二乘（本节做的事）；二是极小极大逼近，让 $\max|p-f|$ 最小，Chebyshev级数截断在适当正则性条件下可作近似，但一般不等于极小极大解。前者最小化残差平方和，后者最小化最大偏差。两者解一般不同。

关键推导：让结论有依据

把目标展开：

$$J(c)=(Ac-y)^{\mathsf T}(Ac-y)=c^{\mathsf T}A^{\mathsf T}Ac-2c^{\mathsf T}A^{\mathsf T}y+y^{\mathsf T}y. \tag{2.2-2}$$

对 c 求梯度：

$$\nabla J(c)=2A^{\mathsf T}Ac-2A^{\mathsf T}y. \tag{2.2-3}$$

令其为零，得到正规方程

$$A^{\mathsf T}Ac=A^{\mathsf T}y. \tag{2.2-4}$$

唯一性。 若 A 列满秩——m 个基函数在 N 个点上线性无关——则 $A^{\mathsf T}A$ 正定、可逆，解唯一：

$$c=(A^{\mathsf T}A)^{-1}A^{\mathsf T}y. \tag{2.2-5}$$

几何解释：残差 $r=y-Ac$ 与 A 的每一列正交，即 $A^{\mathsf T}r=0$。最短的残差必然垂直于张成的子空间。

但式(2.2-5)不该直接拿去编程。 条件数满足

$$\kappa(A^{\mathsf T}A)=\kappa(A)^2, \tag{2.2-6}$$

这里的等式指精确算术下列满秩矩阵的2-范数条件数；显式形成 $A^{\mathsf T}A$ 后还会引入舍入误差，不能把它解释成“有效位数总是少一半”。下面的实验把这句话变成具体数字：$\kappa(A)=4.63\times10^{6}$ 时 $\kappa(A^{\mathsf T}A)=2.14\times10^{13}$，两种解法得到的系数最大相差 4.152。系数差反映数值敏感性，预测值还需另外比较；以下实测值随当前环境记录，不能要求跨平台精确复制。

实验设计：先预测，再运行

本表随本次文稿补充，预测作为课堂教学预期；不将其视为原实验运行前已登记的记录。

项目	本组内容
要回答的问题与运行前预测	① 同一组数据、同次数，用幂基与Chebyshev基做拟合，结果是否不同，预测解几乎相同、条件数差很多；② 次数从3升到12，训练残差与测试误差是否同向变化，预测训练残差降、测试误差未必降；③ 14次多项式能否穿过全部15个点，预测能，且测试误差更差。
输入、参数、单位、数据来源与划分	训练数据：$y = e^x$ 在 $[0,1]$ 上15个等距点加独立噪声 $\mathcal{N}(0, 0.03^2)$，种子2026。测试：同区间400点上的解析值（绘图用600点），网格不参与拟合；它与15个训练点共有8个重合点（含两个端点和6个内部点），其余392点不重合。次数 $d = 3, 6, 9, 12$；两组基：幂基 $1, x, \ldots, x^d$ 与Chebyshev基 $T_0, \ldots, T_d$（自变量线性映射到 $[-1,1]$）。
独立参考解/基线及选择理由	参考是解析函数 e^x，不取任何拟合结果；两组基互为对照——精确算术下它们张成同一个多项式空间，理论上给出同一个解，差异只能来自数值条件。
误差指标、容差及其依据	记录设计矩阵条件数 $\kappa(A)$、训练集最大残差、测试集最大绝对误差，以及正规方程法与 `lstsq` 的系数差。不给合格阈值，只做同次数下两种基的对比、同一种基下不同次数的对比。
环境、种子、设备与运行预算	Python 3.12.10、NumPy 1.26.4、SciPy 1.14.1；CPU单线程、float64、种子2026；求解统一用 `numpy.linalg.lstsq`（最小二乘求解），正规方程只作对照，不用来做最终结果。预算：4个次数 × 2种基，单次秒级。

实验代码见随附脚本 `2.1-2.2-实验.py`（并入Notebook时对应 2.2-A 与 2.2-B），输出留档在 `2.1-2.2-实测摘要.txt`。

结果解释与本节结论

固定数据、固定次数，只换基函数。理论预测两者给出同一个解，实测如下：

次数	幂基 $\kappa(A)$	幂基训练残差	幂基测试误差	Cheb $\kappa(A)$	Cheb 训练残差	Cheb 测试误差
3	1.05e+02	5.168e-02	2.045e-02	1.82e+00	5.168e-02	2.045e-02
6	1.82e+04	5.203e-02	2.530e-02	1.85e+00	5.203e-02	2.530e-02
9	4.63e+06	3.419e-02	3.150e-02	2.99e+00	3.419e-02	3.150e-02
12	2.41e+09	5.639e-03	1.764e-01	1.65e+01	5.639e-03	1.764e-01

两件事，一件比一件值得记：

- 表中残差与误差在所列有效数字下相同。 两类基张成同一个多项式空间，精确算术下的最小二乘拟合函数相同；有限精度结果通常只有近似相同，不能宣称逐位一致。

- 条件数相差约八个数量级。 次数12时，幂基 2.41×10^{9}，Chebyshev基只有 16.5。更小的条件数通常有助于减轻舍入和数据扰动的放大，但不能保证任意更高次数仍稳定。

再看次数本身的影响：

次数	训练集最大残差	独立测试最大误差
1	1.386e-01	1.338e-01
2	5.238e-02	1.366e-02
3	5.168e-02	2.045e-02
6	5.203e-02	2.530e-02
9	3.419e-02	3.150e-02
12	5.639e-03	1.764e-01
14	1.028e-07	9.688e-01

次数14时，15个数据点、15个未知系数，方程恰好可解，多项式穿过每一个点：训练残差 1.03×10^{-7}，测试误差却涨到 0.9688，约为函数幅值（约2.7）的三分之一。这不是算法失灵，是过拟合——模型把噪声也当成了信号。

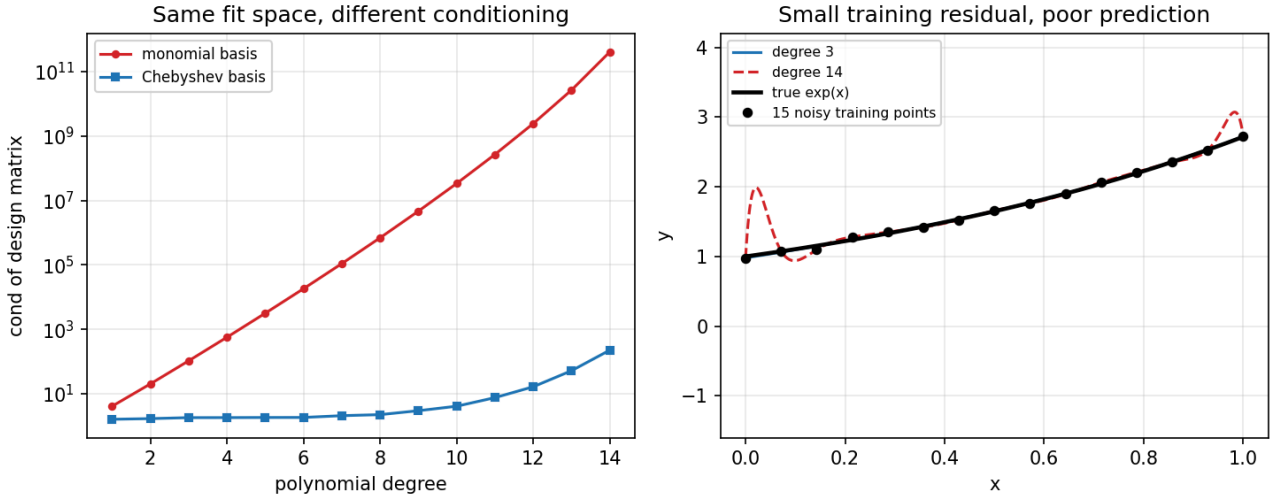


图3：左为设计矩阵条件数随次数的变化（对数坐标），右为3次与14次多项式的拟合效果。同样15个训练点，3次曲线贴着 e^{x}，14次曲线两端被噪声带偏。

这张表还给出一个诊断方法：训练误差与测试误差必须分开看。只看训练残差，14次是“最好的”；看测试误差，它是最差的。

本节结论。 最小二乘将可能不相容的方程组转为残差最小化问题，列满秩时解唯一，代价是接受残差不为零。正规方程给出闭式解，也埋下条件数平方的雷，工程上因此更愿意用QR或SVD。在相同次数下更换这两类基不改变拟合空间，但会影响数值条件。次数不是越高越好：14次把15个点全穿过，测试误差 0.9688——残差小只说明你拟合了这15个点，不说明你逼近了 e^{x}。

易错点与讨论

1. “残差更小，所以模型更好”。 见表：14次训练残差最小（1.03e-07），测试误差最大（0.9688）。残差是在训练集上算的，泛化要看没参与拟合的点。
2. “用了Chebyshev基就是切比雪夫逼近”。 本节做的是以Chebyshev多项式为基的最小二乘，目标是最小化2-范数残差；极小极大逼近最小化无穷范数。两者解一般不同，不能混称。
3. “式(2.2-5)可以直接编程序”。 $\kappa(A^{\mathrm{T}}A)=\kappa(A)^2$：本例中 4.63×10^{6} 被平方成 2.14×10^{13}，系数差达 4.152。应优先采用QR/SVD等较稳定的最小二乘求解途径，并检查预测值误差。
4. “基函数只是写法，不影响结果”。 数学上对，数值上不对：条件数相差约八个数量级，决定了高次时算不算得出来。

随堂提问： 同学交来14次多项式那条曲线，说“训练残差只有 10^{-7}，比3次的好一万倍”。你手上有哪张表、哪个数可以立刻反驳？如果他反问“那你怎么知道3次更好”，你还需补做哪一件事才能回答？

2.3 数值积分与自动微分（2学时）

本节目标： 能从线性插值推出复合梯形公式，写出带符号误差与二阶上界；能说明“阶数为2”与“每加密一次误差恰好变为四分之一”的差别；能区分有限差分、符号微分与自动微分的误差来源，并为同一复合函数手写计算图、用独立解析式核验AD。

实验设计：先预测，再运行

本表于本次修订中依据已有实验补写，用于后续课堂“先预测、再运行”；表中预测是教学预期，不作为原实验运行前已登记的记录。

项目	本组内容
要回答的问题与运行前预测	① $f(x)=x^2$ 上 n 加倍时绝对误差是否变为 $1/4$，预测实测阶 $p\approx2$；② 同一复合函数上解析式、中心差分与AD的差距，预测AD接近机器精度、中心差分随 h 先降后升；③ 换成 e^x 后是否仍呈二阶，用来检验结论不依赖单个特例。
输入、参数、单位、数据来源与划分	积分：$f(x)=x^2$、$[a,b]=[0,1]$、$n=4,8,16,32,64,128$，另用 e^x 跑同一组 n。求导：$F(x)=\sin(x^2)+e^x$、$x_0=0.7$、$h=10^{-1},\dots,10^{-16}$ 共16档；AD在 $[-1,1]$ 的201点等距网格上复核。数据全部由本机生成，无量纲。
独立参考解/基线及选择理由	积分取解析值 $1/3$ 与精确式 $T_n=1/3+1/(6n^2)$（后文式2.3-3），不拿“更细网格的同一程序”当精确值，避免循环论证；求导取独立写出的 $F'(x)=2x\cos(x^2)+e^x$（后文式2.3-5），不拿AD自身当参考。
误差指标、容差及其依据	积分记带符号误差 $I-T_n$ 与绝对误差，并报相邻误差比与实测阶 $p_n=\log_2(E_n/E_{2n})$；求导记绝对误差。容差：T_n 与精确式之差 $<10^{-14}$，用于本规模代数比较；实测阶与2之差 $<10^{-7}$（x^2）、<0.01（e^x，受高阶项影响）；AD与解析式在201点上 $<10^{-12}$。以上只适用于本题规模与float64。

项目	本组内容
环境、种子、设备与运行预算	Python 3.12.10、NumPy 2.5.4、PyTorch 2.14.1+cpu、Matplotlib 3.11.2；Windows AMD64、CPU单线程、float64。本组实验无随机成分，种子2026只作环境记录。预算：6个积分档位、16个差分步长、201点网格，单次运行秒级；未按结果回调参数或挑选步长。

2.3.1 复合梯形公式与收敛性

讲解：对插值函数积分。 设要计算

$$I=\int_a^b f(x)\,\mathrm{d}x,\qquad a<b.$$

将区间等分为 n 个小区间，$h=(b-a)/n$，$x_i=a+ih$。在每个小区间用端点之间的直线替代原曲线：

$$p_i(x)=\frac{x_{i+1}-x}{h}f(x_i)+\frac{x-x_i}{h}f(x_{i+1}).$$

这正是本地插值课件中的两节点Lagrange插值；本节新增的是对这条直线积分。[L2] 因为两个权函数在小区间内的积分都是 $h/2$，

$$\int_{x_i}^{x_{i+1}}p_i(x)\,\mathrm{d}x=\frac{h}{2}\bigl[f(x_i)+f(x_{i+1})\bigr].$$

求和得到复合梯形公式：

$$T_n=h\left[\frac{f(a)}{2}+\sum_{i=1}^{n-1}f(x_i)+\frac{f(b)}{2}\right]. \tag{2.3-1}$$

公式只需 $n+1$ 个函数值。本节只讨论一维等距情形；不把高维积分或Monte Carlo纳入必修范围。对一次多项式，分段直线与函数本身一致，因而求积精确；对其他函数，误差取决于曲线的弯曲程度及步长。

推导一：一般误差的条件与符号。 若 $f\in C^2[a,b]$，线性插值余项给出

$$f(x)-p_i(x)=\frac{f''(\xi_x)}{2}(x-x_i)(x-x_{i+1}).$$

区间内 $(x-x_i)(x-x_{i+1})\le 0$，且其积分为 $-h^3/6$。利用 f'' 的上下界与连续性，可选取小区间内的 ξ_i，使

$$\int_{x_i}^{x_{i+1}}(f-p_i)\,\mathrm{d}x=-\frac{h^3}{12}f''(\xi_i).$$

再求和，并用连续函数的介值性质，得到某个 $\xi\in(a,b)$ 满足

$$I-T_n=-\frac{b-a}{12}h^2f''(\xi),\qquad |I-T_n|\le\frac{b-a}{12}h^2\max_{[a,b]}|f''|. \tag{2.3-2}$$

这里始终用 $I-T_n$ 表示带符号误差，以免与绝对误差混淆。若 $f''\ge 0$，梯形位于曲线之上，$T_n\ge I$。一般结论是误差上界为 $O(h^2)$；只有当非零主项主导、尚未受舍入误差影响时，加密误差比才趋近4。若主项恰好抵消，或函数不满足上述光滑性，不能机械套用“永远缩小四倍”。

推导二：x^2 的精确误差。 取 $f(x)=x^2$、$[a,b]=[0,1]$，$I=1/3$。直接使用平方和公式：

$$T_n=\frac{1}{n}\left[\sum_{i=1}^{n-1}\frac{i^2}{n^2}+\frac{1}{2}\right]=\frac{1}{3}+\frac{1}{6n^2}.$$

因此

$$E_n = |T_n - I| = \frac{1}{6n^2}, \qquad \frac{E_n}{E_{2n}} = 4, \qquad p_n = \log_2 \frac{E_n}{E_{2n}} = 2. \tag{2.3-3}$$

这既给出独立参考值，也回答开场问题。它是该二次函数的精确恒等式，比一般的渐近二阶结论更强。梯形起步例来自仓库，本稿补充了两条推导、e^x 的独立验证和结果解释。

实验与解释。 Notebook“2.3-A”自行实现式(2.3-1)，不调用同一公式的库函数作为唯一参考。理论值来自 $1/3$ 与式(2.3-3)。另检查一次函数的精确性及 $\int_0^1 e^x dx = e - 1$，以避免只对一个特例有效。浮点核对容差为 10^{-14}，只用于本例的小规模代数比较。

区间数 n	T_n	绝对误差	前一层误差/本层误差	实测阶数
4	0.343750000000	1.04166667e-02	—	—
8	0.335937500000	2.60416667e-03	4.000000	2.000000
16	0.333984375000	6.51041667e-04	4.000000	2.000000
32	0.333496093750	1.62760417e-04	4.000000	2.000000
64	0.333374023438	4.06901042e-05	4.000000	2.000000
128	0.333343505859	1.01725260e-05	4.000000	2.000000

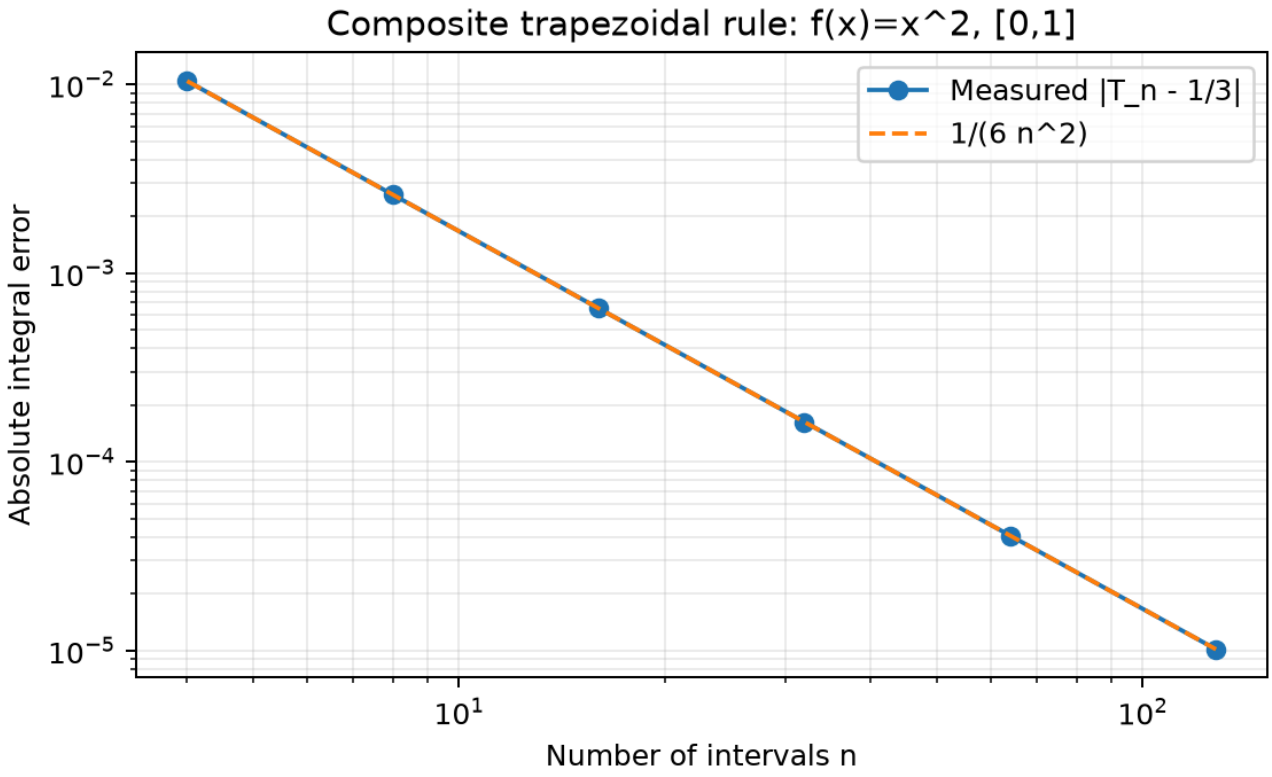


图4：本机实测，$f(x) = x^2$，float64，横轴为区间数，纵轴为绝对积分误差。数值与精确误差式相符；e^x 的阶数检查见Notebook。

讨论： 积分误差不是函数在节点上的误差。梯形公式使用的端点值可以完全正确，但区间内部直线替代曲线仍有面积误差。加密节点减少的是求积离散误差，不能自动消除函数值本身的观测误差或浮点误差。

2.3.2 有限差分、符号微分与自动微分

下面改问“怎样求导”。数值积分从函数值估计面积；三种求导途径则从不同的信息获得导数：

方法	输入与操作	输出与主要限制
有限差分	在相邻点评价函数并组成差商	数值近似；有步长截断、相减舍入及可能的噪声放大
符号微分	对表达式按求导规则变换	导数表达式；需正确处理定义域、分支、表达式复杂性，代入后仍有浮点误差
自动微分（AD）	将数值程序拆成基本运算，沿计算图使用局部导数与链式法则	给定点的导数、Jacobian乘积等；依赖计算图、求导规则和浮点精度

AD的对象是计算过程，不要求先生成整条导数表达式，也不需要构造差分步长。上述区分与Baydin等的综述一致。[R1] AD通常没有有限差分的步长截断误差，但不等于“数学上无误差”：溢出、下溢、不稳定计算、不可导点、计算图断开及不支持求导的操作都需要另行判断。

以中心差分为例，若函数在邻域内足够光滑，

$$D_hF(x)=\frac{F(x+h)-F(x-h)}{2h}=F'(x)+\frac{h^2}{6}F'''(x)+O(h^4).$$

其截断主项为 $O(h^2)$。在通常的浮点误差模型下，相减后除以 h 的舍入影响可粗略写为 $O(\varepsilon_{\mathrm{mach}}/h)$，因此总误差近似呈现

$$|D_hF-F'|\lesssim C_1h^2+C_2\frac{\varepsilon_{\mathrm{mach}}}{h}. \tag{2.3-4}$$

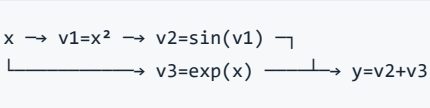
常数依赖函数、输入与实现；这不是在所有程序上的严格统一上界。它解释了“步长越小越准”的说法为什么不成立，也衔接本地引论课件中截断误差与舍入误差共同作用的讨论。[L1]

2.3.3 计算图与链式法则

取同一个复合函数

$$F(x)=\sin(x^2)+e^x.$$

按运算顺序拆成 $v_1=x^2$、$v_2=\sin v_1$、$v_3=e^x$、$y=v_2+v_3$。计算图的两条路径都依赖 x，求导必须把两条贡献相加。



前向模式： 设输入切向量 $\dot{x}=1$，逐步传播：

$$\dot{v}_1=2x,\qquad \dot{v}_2=\cos(v_1)\dot{v}_1,\qquad \dot{v}_3=e^x,\qquad \dot{y}=\dot{v}_2+\dot{v}_3.$$

反向模式： 设输出伴随量 $\bar{y}=1$，从后向前传播：

$$\bar{v}_2=1,\quad \bar{v}_3=1,\quad \bar{v}_1=\cos(v_1),\quad \bar{x}=2x\bar{v}_1+e^x\bar{v}_3.$$

两种模式均得到

$$F'(x) = 2x\cos(x^2) + e^x. \tag{2.3-5}$$

对单个输入、单个输出，两种模式都很直接；对很多参数、一个标量损失的训练问题，反向模式可一次回传获得全部参数梯度，代价是保存或重建所需中间量。

实验设计。 Notebook“2.3-B”用PyTorch `torch.autograd.grad` 实现反向AD；以独立写出的式(2.3-5)为参考。在 $x = 0.7$ 评价中心差分，$h = 10^{-1}, \ldots, 10^{-16}$；另在201点网格检查AD。AD与解析式的核对容差为 10^{-12}。差分曲线允许在过小步长处变差，其结果不应被删去。

在 $x = 0.7$，解析式与AD均输出 3.2490187095246466；本点浮点差为0，201点检查网格的最大差为 4.441e-16。本点的0表示本次浮点结果相同。

差分步长 h	中心差分绝对误差
1e-01	7.302029e-03
1e-02	7.268056e-05
1e-03	7.267713e-07
1e-05	7.437828e-11
1e-08	3.340995e-08
1e-12	6.205026e-05
1e-16	1.028573e+00

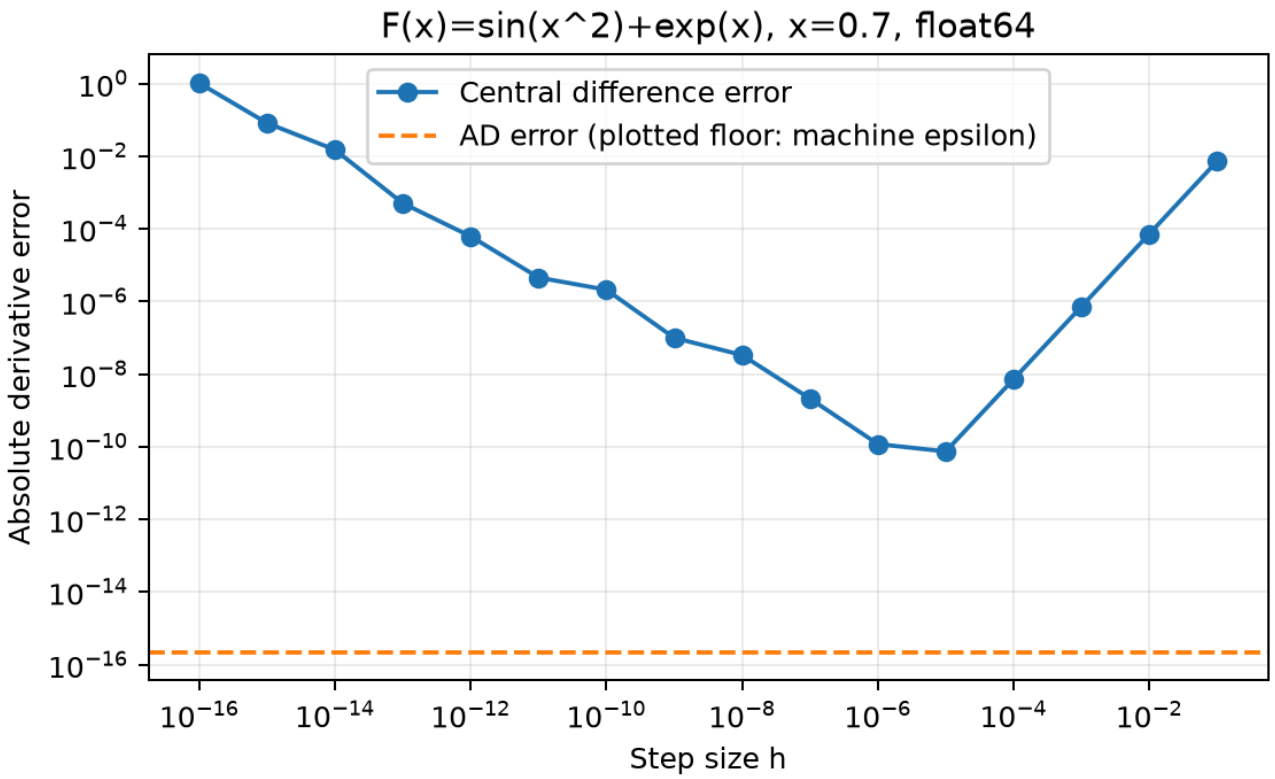


图5：同一函数、同一点、float64的实测误差。为在对数坐标显示，AD参考线的绘图下限设为机器epsilon；真实数值差异另列于正文与数据文件。曲线低谷位置不是通用的最佳步长。

结果解释与本节结论

x^2 积分将 n 从4改为8，绝对误差由 $1/96$ 变成 $1/384$；这是一项求积结果。式(2.3-5)则是求导的独立参考。若有人用“积分误差缩小四倍”证明AD准确，其证据并不对应所需主张。完整练习见题3。

易错点与讨论

1. 节点值对，积分不一定对。 函数值全部精确时，区间内部用直线替曲线仍有 $O(h^2)$ 的面积误差。加密减少的是求积离散误差，不能消除观测误差与浮点误差。
2. “二阶”不等于“每次恰好四分之一”。 式(2.3-3)是 x^2 的精确恒等式；一般函数只有主项主导、且未进入舍入区间时，误差比才趋近4。e^x 的实测阶为 1.998874、1.999718、1.999930、1.999982、1.999996，正是渐近结论而非逐次恒等。
3. 差商不是AD，积分误差也不是求导证据。 中心差分误差按 $C_1h^2 + C_2\varepsilon_{\text{mach}}/h$ 呈V形（式2.3-4），$h = 10^{-16}$ 时已达 1.028573；AD在 $x = 0.7$ 与解析式的本次浮点差为0，201点检查网格的最大差为4.441e-16。差分与AD是不同的求导途径；积分误差描述面积逼近，不能用于证明求导准确。
4. AD“无截断误差”只针对当前计算图。 4.441e-16 的吻合说明实现对当前程序与解析式一致，不说明导数逼近目标函数的准确度；后者是2.4的评价对象。

随堂提问： 若有人把 x^2 的结果推广成“任意函数每次加密，误差都恰好降为四分之一”，应补充哪些光滑性和主项条件？请对照 e^x 的实测阶，说明“渐近趋近4”与“每次恰好为4”的区别。完整练习见题3。

2.4 AI关联：神经网络逼近与PINN求导（2学时）

本节目标： 能把单隐藏层网络写成函数逼近器，并区分输入导数 $\partial u/\partial x$ 与参数导数 $\partial u/\partial\theta$；能说明PINN强形式残差所需的二阶输入导数从何而来、`create_graph=True` 起什么作用；能在与训练点不重合的测试集上分别报告函数值、一阶与二阶导数误差，并解释为什么函数拟合好不保证导数准确。

实验设计：先预测，再运行

本表于本次修订中依据已有实验补写，用于后续课堂“先预测、再运行”；表中预测是教学预期，不作为原实验运行前已登记的记录。

项目	本组内容
要回答的问题与运行前预测	① 只用函数值MSE训练的两层tanh网络能否达到教学精度，预测测试集最大函数误差 $< 10^{-3}$；② 一阶、二阶导数误差是否显著大于函数误差，预测逐阶放大；③ 是否存在“函数误差趋于0、导数误差却发散”的构造，预测存在且能给出见证点；④ 把tanh换成ReLU会怎样，预测经典二阶导数在折点处取不到。
输入、参数、单位、数据来源与划分	目标 $g(x) = \sin(\pi x)$、$x \in [-1, 1]$；41个等距训练点，400个均匀小区间中点作独立测试点（与训练点最小距离 $> 10^{-12}$，代码内断言）。网络1－32－32－1、tanh、1153个参数。优化：Adam 2000步、学习率 10^{-3}，随后L-BFGS最多400次迭代（max_eval 500、strong Wolfe线搜索、history 50）。训练只用函数值MSE，不使用导数标签或PDE残差。

项目	本组内容
独立参考解/基线及选择理由	函数值与一、二阶导数均以解析式 $\sin(\pi x)$、$\pi\cos(\pi x)$、$-\pi^2\sin(\pi x)$ 为参考；另把训练后的权重转成NumPy，按层独立传播值、一阶与二阶导数，用于核对AD实现——它只检验求导实现，不充当目标函数参考。高频反例 v_k 与ReLU诊断是解析构造，不由训练得到。
误差指标、容差及其依据	在400个测试点上分别报告最大采样绝对误差、RMSE与离散相对二范数；测试集在训练前固定，不用于选超参数或挑种子。容差：函数值最大误差 $<10^{-2}$（幅值1的教学目标，题面给定）；NumPy链式法则与AD之差 $<10^{-10}$；导数与残差没有预设“准确”阈值，实测值如实报告，不因未达标而更换测试点。
环境、种子、设备与运行预算	Python 3.12.10、NumPy 2.5.4、PyTorch 2.14.1+cpu、Matplotlib 3.11.2；CPU单线程、float64、种子2026。预算：1153个参数、Adam 2000步及L-BFGS最多400次迭代，本次训练2.201秒，L-BFGS闭包调用469次（含线搜索评价，不等于迭代次数）；不需要GPU、外部数据或在线API。

2.4.1 神经网络作为函数逼近器

将单输入、单输出的单隐藏层网络写为

$$u_\theta(x)=c+\sum_{j=1}^{m}a_j\sigma(w_jx+b_j),\qquad \theta=(c,a_1,\dots,a_m,w_1,\dots,w_m,b_1,\dots,b_m). \tag{2.4-1}$$

网络把基函数 $\sigma(w_jx+b_j)$ 的形状和线性组合系数一起学习。给定训练点 (x_i,y_i)，函数值拟合采用

$$L_{\text{data}}(\theta)=\frac{1}{N}\sum_{i=1}^{N}\left[u_\theta(x_i)-y_i\right]^2.$$

训练寻找使样本误差较小的参数；它是关于参数的优化问题。有限训练数据、固定网络大小和有限训练预算都不会自动保证区间一致逼近，更不会自动保证导数准确。即便援引某类通用逼近结论，也须区分“存在合适参数”与“本次训练找到了这些参数”。

本节采用光滑的 $\sigma(z)=\tanh z$。实际实验扩展为两隐藏层网络1－32－32－1，共1153个参数；单隐藏层表达式用于清楚展示各类偏导数，两隐藏层则按同样的链式法则逐层传播。

2.4.2 参数导数与输入导数

令 $z_j=w_jx+b_j$。在参数固定时，输入导数描述曲线在空间中的变化：

$$\frac{\partial u_\theta}{\partial x}=\sum_j a_jw_j\sigma'(z_j),\qquad \frac{\partial^2u_\theta}{\partial x^2}=\sum_j a_jw_j^2\sigma''(z_j). \tag{2.4-2}$$

在输入固定时，参数导数描述改变权重会怎样改变网络输出：

$$\frac{\partial u_\theta}{\partial a_j}=\sigma(z_j),\quad \frac{\partial u_\theta}{\partial w_j}=a_jx\sigma'(z_j),\quad \frac{\partial u_\theta}{\partial b_j}=a_j\sigma'(z_j),\quad \frac{\partial u_\theta}{\partial c}=1. \tag{2.4-3}$$

于是训练所用的损失梯度是

$$\frac{\partial L_{\text{data}}}{\partial\theta_k}=\frac{2}{N}\sum_i\left[u_\theta(x_i)-y_i\right]\frac{\partial u_\theta(x_i)}{\partial\theta_k}.$$

它与 $\partial u_\theta/\partial x$ 的求导对象不同，不能把 `loss.backward()` 得到的权重梯度直接称作物理场的空间导数。对于tanh，

$$\sigma'(z) = 1 - \tanh^2 z, \qquad \sigma''(z) = -2\tanh z\big[1 - \tanh^2 z\big].$$

从输入导数到PINN残差。 考察一维Poisson边值问题：

$$-u''(x) = \pi^2 \sin(\pi x), \quad x \in (-1, 1), \qquad u(-1) = u(1) = 0. \tag{2.4-4}$$

解析解为 $u_*(x) = \sin(\pi x)$。用网络构造残差

$$r_\theta(x) = -\frac{\partial^2 u_\theta(x)}{\partial x^2} - \pi^2 \sin(\pi x).$$

若继续训练PINN，可在内部配点 ξ_i 上定义

$$L_{\text{PINN}} = \frac{1}{N_r}\sum_i r_\theta(\xi_i)^2 + \lambda_b\big[u_\theta(-1)^2 + u_\theta(1)^2\big], \qquad \lambda_b > 0.$$

这展示了两个求导层次：先对输入求二阶导数组成残差，再对参数求导来优化残差损失。以微分方程约束网络的思路可参看Raissi等原始工作。[R3] 本节仅计算残差并检查梯度链路；完整的PDE训练、采样和验证留在第6章。

```
x = x.detach().clone().requires_grad_(True)
u = model(x)
ux = torch.autograd.grad(u, x, torch.ones_like(u), create_graph=True)[0]
uxx = torch.autograd.grad(ux, x, torch.ones_like(ux), create_graph=True)[0]
r = -uxx - torch.pi**2 * torch.sin(torch.pi * x)
```

这里保留导数计算图，才能继续求高阶导数或对残差损失求参数梯度；`create_graph=True` 的含义见官方接口文档。[R2] 当前MLP逐个样本独立，Jacobian具有所需的逐点结构，传入全1向量可获得每个样本的一阶输入导数；若模型存在跨样本耦合，这段写法不能直接当作逐点导数。

2.4.3 激活函数光滑性与高阶导数误差

光滑性为何重要。 有限ReLU网络是分段仿射函数，在每个仿射区域内二阶导数为0，折点处经典导数可能不存在。因此直接用其经典二阶导数去拟合式(2.4-4)中一般非零的源项，会遇到结构性困难。PyTorch在折点返回约定值，并不证明经典导数在那里存在。本节选用tanh以满足二阶输入求导的光滑性需要；这并不排除ReLU在弱形式等其他方法中的用途，也不意味着tanh自动避免优化困难。

拟合好不等于导数好：一个解析反例。 在 $[-1, 1]$ 上，令

$$v_k(x) = \sin(\pi x) + \frac{1}{k}\sin(kx), \qquad k = 10, 100, 1000.$$

由于 $x = \pi/(2k)$ 位于区间内，函数误差的上界可以取到：

$$\|v_k - u_*\|_\infty = \frac{1}{k} \longrightarrow 0.$$

然而

$$v_k' - u_*' = \cos(kx), \qquad v_k'' - u_*'' = -k\sin(kx),$$

从而

$$\|v_k' - u_*'\|_\infty = 1, \qquad \|v_k'' - u_*''\|_\infty = k. \tag{2.4-5}$$

小振幅、高频率的扰动可以在函数值上几乎不可见，却被微分放大。这里是AI辅助设计、具有精确见证点的教学反例，不是伪造的网络训练结果，也不记作“已发现的AI错误”。

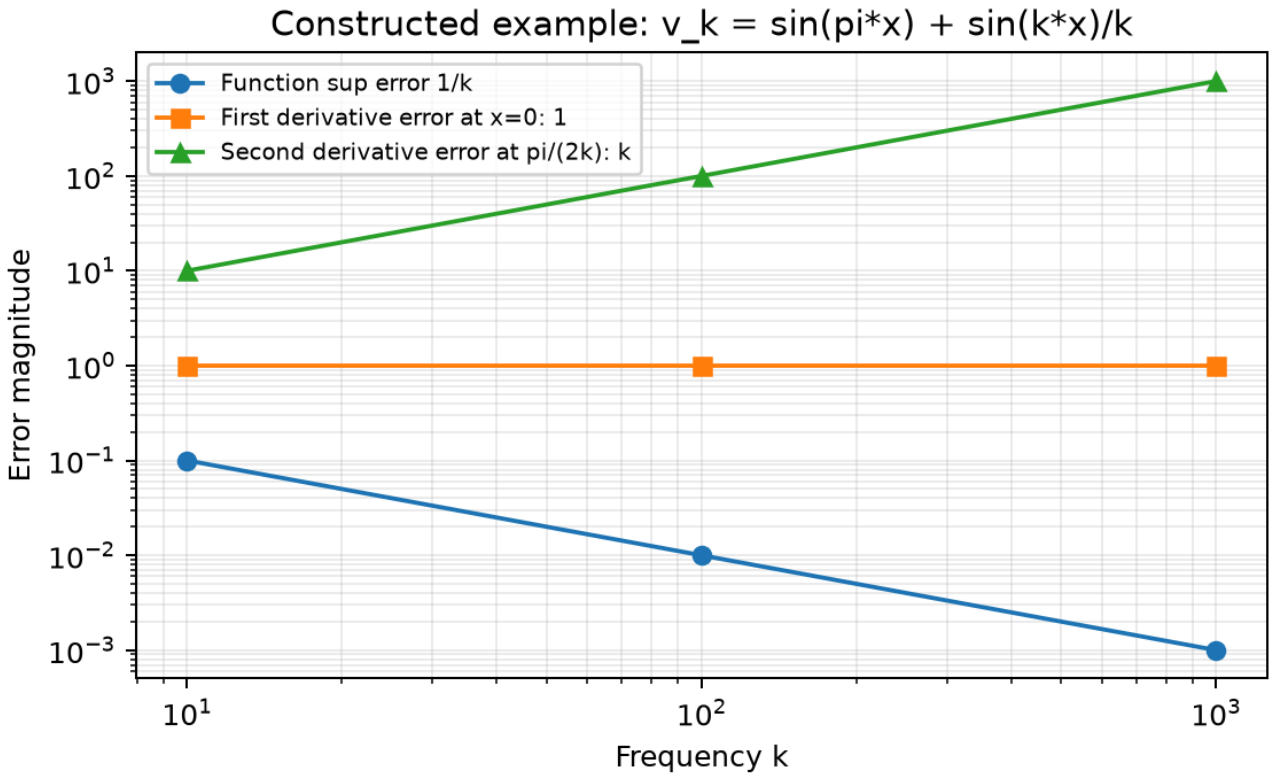


图6：按解析见证点实际计算；函数误差为 $1/k$，一阶导数差在0点为1，二阶导数差在 $\pi/(2k)$ 为 k。横轴为扰动频率，纵轴为误差大小。

实验：训练小网络，再分别检查函数和导数。 Notebook“2.4-A”拟合 $g(x)=\sin(\pi x)$。固定41个等距训练点；独立测试集为400个均匀小区间中点，与训练点无重合。使用CPU、float64、种子2026、Adam 2000步（学习率 10^{-3}）和L-BFGS最多400次迭代（强Wolfe线搜索）。训练只使用函数值MSE，不使用导数标签，也不使用PDE残差。测试网格不用于选择超参数或挑选最佳模型。

函数值、一阶导数、二阶导数各自报告最大采样绝对误差、RMSE和离散相对二范数：

$$E_{\max}=\max_j |q_\theta(x_j)-q_*(x_j)|,\quad \mathrm{RMSE}=\sqrt{\frac{1}{M}\sum_j |q_\theta(x_j)-q_*(x_j)|^2},\quad E_{2,\mathrm{rel}}=\frac{\|q_\theta-q_*\|_2}{\|q_*\|_2}.$$

分母在本实验的三个目标向量上均非零；指标是测试点上的数值指标，不是整个连续区间的严格误差上界。各阶导数有不同的尺度，不能只比较未归一化的绝对误差。

最终训练MSE为 4.531542e-08；本次CPU训练用时约 2.20 秒，L-BFGS闭包调用 469 次（其中包括线搜索评价，不等于迭代次数）。

评价量	最大采样绝对误差	RMSE	离散相对二范数
函数值 u	4.135050e-04	2.140816e-04	3.027571e-04
一阶导数 u_x	1.522785e-02	4.264197e-03	1.919563e-03
二阶导数 u_{xx}	3.562128e-01	9.239099e-02	1.323869e-02

NumPy逐层链式法则与AD的最大差异（值、一阶、二阶）分别为 4.441e-16、1.332e-15、7.105e-15。Poisson残差RMSE为 9.239099e-02，两端点预测分别为 -1.485380e-04、4.515525e-05。

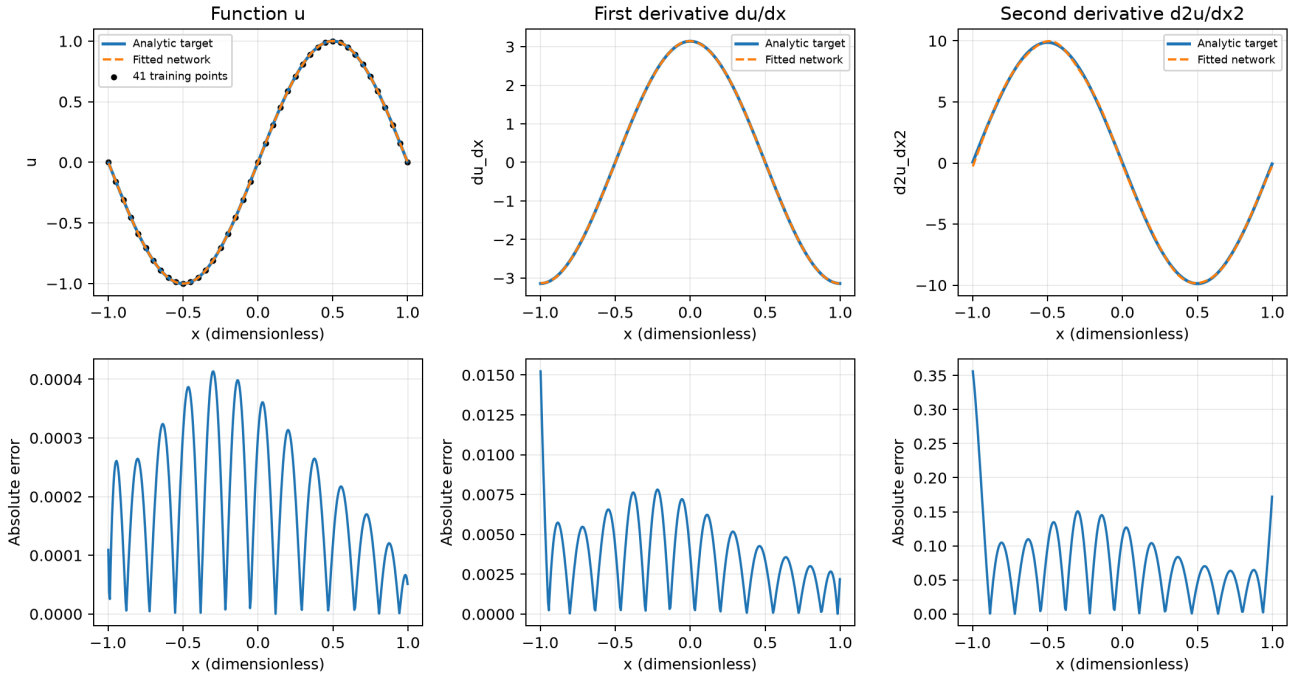


图7：上排为独立测试点上网络与解析函数及其导数的比较，下排为对应绝对误差；左上同时标出训练点。网络结构、种子与训练预算在测试前固定。

为了分清“AD实现正确”和“导数逼近准确”，实验把训练后的权重转为NumPy，独立传播值、一阶导数和二阶导数。对每个仿射层 $a = Wz + b$，传播 $a' = Wz'$、$a'' = Wz''$；对tanh层传播

$$z' = (1 - z^2)a', \qquad z'' = (1 - z^2)a'' - 2z(1 - z^2)(a')^2.$$

这里的逐元素乘法按分量理解。NumPy与AD的吻合表明求导实现对当前网络一致；网络与目标解析导数的差异则表明另一个逼近问题。即使前者接近浮点精度，后者仍可能明显较大。

结果解释与本节结论

本实验的Poisson残差没有参与训练，不能因已计算残差或其参数梯度就称为“已完成PINN求解”。即便另一次训练在有限配点上残差很小，还要单独检查边界条件、独立点残差、解误差和优化是否可靠。由式(2.4-5)可知，仅有函数值误差下降，无法推出高阶导数收敛。

易错点与讨论

1. 训练误差不是测试误差。 本轮训练MSE为 4.531542e-08，对应训练RMSE约 2.128742e-04；400个独立测试点上的函数RMSE为 2.140816e-04，最大函数误差为 4.135050e-04。MSE与最大绝对误差的定义和尺度不同，不能直接用二者的比值判断泛化差距。训练点上的小残差只说明拟合了区间内41个采样，不说明整个区间上一致逼近。
2. 函数值接近不能推出导数接近。 式(2.4-5)的 v_k 给出反例：函数误差 $1/k$，一阶导数误差恒为1，二阶为 k。本组实测在 $k = 1000$ 时为 10^{-3}、1、1000，量级与解析结论一致。
3. 算出残差不等于解了PDE。 本实验的残差RMSE为 9.239099e-02，且网络未经残差训练；`loss.backward()` 得到的是参数梯度，不是场量的空间导数，两者求导对象不同。

4. ReLU折点处的“导数”是框架约定。 实测 $x=-0.5,0,0.5$ 处一阶导为0、0、1，二阶全为0；折点处经典二阶导数不存在，框架返回值不能当作数学结论，这也是本节改用tanh的原因。

随堂提问： 若同学看到 $k=1000$ 时函数误差仅 10^{-3}，认为二阶导数误差也应小于 10^{-3}，应如何回答？给出 $x=\pi/2000$，二阶导数差的绝对值恰为1000；问题出在把函数值一致接近错误地推成导数接近。完整练习见题4。

回到开场与全章小结

插值使多项式满足节点条件，但本章Runge例说明节点上精确不能保证区间内精确。最小二乘允许残差存在；相同次数下，两类基的拟合空间相同，而条件数可能相差很大。次数增加也可能过拟合噪声，因此要同时报告训练残差和未参与拟合的网格误差。

对明确给定的 x^2 曲线，4块改成8块梯形，面积误差严格变为四分之一。对一般 C^2 函数，可以给出二阶误差上界，但实测阶数还受主项、函数性质和舍入影响。用更多信息提高逼近质量，需要同时说明误差针对什么量、评价在哪些点以及依据什么独立参考。

AD把链式法则应用于数值计算图，为网络训练与输入求导提供工具。网络函数值逼近、网络导数的实现正确性、目标导数的逼近质量以及PDE求解可靠性是依次需要检查的不同主张。本节给出了连接它们的公式与最小实验，完整PINN训练仍由第6章展开。

参考资料、材料对应与图片来源

本地文件均来自用户指定的上课材料目录。课件中提及的李庆扬、王能超、易大义《数值分析》第5版是课件所列教材；本次没有把它与 `book2.pdf` 误认为同一本书，也没有声称直接查阅未提供的原版教材。

编号	来源与定位	本稿用途与核对状态
L1	《第一章-数值分析与科学计算引论.pdf》，PDF第2—3页（误差来源、Taylor截断与舍入）	支撑误差分类；已提取并核对课件文字，见材料核对表
L2	《第二章-插值法-线条漫画版.pdf》，PDF第6页（两节点线性插值）、第9—10页（插值余项及证明思路）、第32页（分段低次插值导入）、第35页（分段线性插值）	衔接从插值到梯形求积；页面号按PDF物理页计；不将课件二手页码冒充原书直接核查
L3	高立，《数值最优化方法》，北京大学出版社，2014年8月第1版，ISBN 978-7-301-24645-0；本地 `book2.pdf`，PDF第4页版权、第22页（书内第8页）梯度及Hesse矩阵	衔接参数优化与导数概念；已图像查看原页。本书不是本节数值积分或PINN的直接出处
L4	`ch01a Lagrange插值.ppt`、`ch01b Newton插值.ppt`、`ch01c 埃尔米特插值.ppt`、`ch01d 三次样条插值.ppt`	本地课程背景资料；本稿只借用分段线性插值的衔接，不把Newton、Hermite与样条扩为任务范围

编号	来源与定位	本稿用途与核对状态
R1	Baydin, Pearlmutter, Radul, Siskind. Automatic Differentiation in Machine Learning: a Survey. JMLR 18(153):1—43, 2018；第2节求导方法比较、第3节AD模式	论文与PDF入口；AD概念来源，网页原文已读取
R2	PyTorch 2.14文档，`torch.autograd.grad`，参数`grad_outputs`、`create_graph`	官方接口文档；已读取接口，实装版本见运行记录
R3	Raissi, Perdikaris, Karniadakis. Physics Informed Deep Learning (Part I): Data-driven Solutions of Nonlinear Partial Differential Equations. arXiv:1711.10561, v1, 2017；第2节连续时间模型	原始预印本；用于说明PINN以微分方程残差约束网络的思路
R5	Driscoll 与 Braun，Fundamentals of Numerical Computation，第9.3节；SciPy 1.14.1 `BarycentricInterpolator`；NumPy 1.26 `lstsq`	教材、SciPy接口、NumPy接口；用于本次插值稳定性与实现说明，已读取
R4	课程仓库第02章README、A组模板、教材详细目录及协作规则；基准完整SHA `0ee58465e43fb061db90cbf7509fb58e0f206fea`	任务入口；确认A2＝G21与六项交付、2.4范围

图1来自该仓库模板，保留文件名 `A-章节导入.svg`；图2—3由新增2.1—2.2实验实际计算绘制，图4—7沿用2.3—2.4配套Notebook的既有实测图。高频反例、网络实验与习题均为本次AI辅助设计，未声称学生独立设计。AI使用与真实人工参与情况分别记录在A2组-AI对话记录.md。

本章习题与参考解答

A2组（G21）课后习题：第2.1—2.4节

本章四节各配一题：题1对应2.1、题2对应2.2、题3对应2.3、题4对应2.4，各50分。配套正文、2.1—2.2实验、2.3—2.4实验、参考解答。所有“错误说法”均为教学构造，不是真实AI原话。每题50分，兼顾概念推导、实现验证和陷阱辨析。

题1：节点选择与Runge现象（对应2.1，50分）

对象：$f(x)=1/(1+25x^2)$，区间 $[-1,1]$。允许使用NumPy、SciPy、Matplotlib；全部计算使用float64。

1. 推导（15分）。 由两点插值公式写出线性插值的误差项；取 $n=3$ 的等距节点 $-1,0,1$，写出 $\omega(x)$ 并求 $\max_{[-1,1]}|\omega|$；再取同数的Chebyshev节点，给出同一个量的值，说明差在哪里。
2. 实现（15分）。 对等距与Chebyshev两组节点分别做插值，节点数 $n=5,9,11,15,21,31$，在4001点等距网格上报告最大绝对误差；另单独报告节点处误差。提交表格与双对数图，横轴为 n。不得用4001点网格上的最大值冒充连续区间上的严格上界。

3. 诊断（10分）。 指出等距 $n=11$ 时最大误差出现的位置，说明为什么不在区间中部；给出 $x=0.95$ 处两种插值的数值与真值，并解释该位置离最近节点并不远，为什么仍然错得厉害。
4. 辨析（10分）。 判断并说明条件：“节点上误差为0，所以插值曲线可靠”“节点数增加，误差必然减小”“只要换成Chebyshev节点，任何函数都能保证收敛”。说明插值误差能否用“穿过所有测点”来验收。

验收依据： 等距 $n=11$ 区间最大误差 1.916（允许±1e-3），Chebyshev 为 0.1092；$x=0.95$ 处等距插值 1.9236、真值 0.0424；等距 $n=31$ 为 2.388e3。容差只适用于本题区间与float64。

难度与用时： 中等，预计课后60—75分钟，为教学估计，不是已发生的人工作时。来源：仓库2.1节模板的实验任务；新增 $n=3$ 的手算 ω 与端点诊断。

题2：最小二乘与基函数（对应2.2，50分）

对象：$y=e^x$、$x\in[0,1]$；训练集为15个等距点，加独立噪声 $\mathcal{N}(0,0.03^2)$，种子2026；评价集为同区间400个点的解析值，不参与拟合；其中8点与训练点重合（两个端点和6个内部点），其余392点不重合。

1. 推导（15分）。 从 $J(c)=\|Ac-y\|_2^2$ 展开并求梯度，推出正规方程；说明解唯一当且仅当 A 列满秩；在列满秩、2-范数条件数与精确算术下证明 $\kappa(A^{\mathsf{T}}A)=\kappa(A)^2$，据此解释为什么实际计算应使用QR或SVD而不是 $(A^{\mathsf{T}}A)^{-1}$。
2. 实现（15分）。 对 $d=3,6,9,12$ 分别用幂基 $1,x,\dots,x^d$ 与Chebyshev基 $T_0,\dots,T_d$ 拟合，报告设计矩阵条件数、训练集最大残差、测试集最大误差；求解统一用 `lstsq`，正规方程只作对照，并报告两者系数之差。
3. 诊断（10分）。 拟合 $d=14$，报告训练残差与测试误差，作图说明过拟合；回答该多项式是否穿过全部15个训练点，以及“穿过”在这里意味着什么。
4. 辨析（10分）。 判断并说明条件：“训练残差越小，模型越好”“以Chebyshev多项式为基的最小二乘就是切比雪夫逼近”“正规方程形式简单，可以直接用于高次拟合”。说明残差最小化与最大偏差最小化是不是同一件事。

验收依据： $d=12$ 时幂基 $\kappa(A)=2.41e9$、Chebyshev基为 16.5，两组拟合函数在精确算术下相同，有限精度误差只要求在合理容差内一致；正规方程与 `lstsq` 的系数差用于诊断病态，不要求跨平台复现固定数值；当前环境实测值见摘要；$d=14$ 时训练残差 1.03e-7、测试误差 0.9688。跨平台浮点末位不要求一致，但量级与结论必须复现。

难度与用时： 中等偏上，预计课后70—85分钟。来源：仓库2.2节模板的实验任务；新增条件数平方的显式对照与 $d=14$ 过拟合诊断。

题3：积分加密与复合函数求导（对应2.3，50分）

对象A为 $I=\int_0^1 x^2\,\mathrm{d}x$；对象B为 $F(x)=\sin(x^2)+e^x$。 允许使用NumPy、Matplotlib及PyTorch；所有数值计算使用float64。

1. 推导（15分）。 从相邻节点上的线性插值出发写出等距复合梯形公式。使用平方和公式推导 $T_n=1/3+1/(6n^2)$。分别写出带符号误差 $I-T_n$ 与绝对误差；说明 n 加倍时误差如何变化。

2. 积分实现（10分）。 自行实现复合梯形公式，计算 $n = 4, 8, 16, 32, 64, 128$ 的积分、绝对误差和相邻加密阶数。提交表与双对数图。不能用同一积分程序更细网格的结果代替精确参考 $1/3$。另用 $\int_0^1 e^x \mathrm{d}x = e - 1$ 检查二阶收敛。
3. 计算图与求导（15分）。 画出B的计算图，逐步用链式法则求 F'；在 $x = 0.7$ 比较解析式、AD与中心差分。差分步长取 $10^{-1}, \ldots, 10^{-16}$，提交误差图。指出图中截断与舍入误差各占主导的大致区域，并解释为何不能只给出一个步长。
4. 辨析（10分）。 判断并说明条件：“梯形积分始终二阶且加密后误差一定恰好缩小四倍”“机器自动计算的差商就是自动微分”“AD结果等于解析式，因此没有任何误差”。说明积分误差能否充当求导准确性的证据。

验收依据： 小问1推导及符号正确；积分误差与 $1/(6n^2)$ 的绝对差小于 10^{-14}，x^2 的实测阶数距2小于 10^{-7}，e^x 的阶数距2小于0.01。解析式与AD在本题的201点检查网格上最大差异小于 10^{-12}。差分小步长恶化是应解释的结果，不以全部步长达标为要求。容差只适用于本题规模及float64。

难度与用时： 中等，预计课后60—75分钟，为教学估计，不是已发生的人工作时。来源：仓库2.3起步梯形例的实质扩展；新增计算图、双参考求导与第二个积分函数。

题4：函数拟合、导数误差与PINN残差（对应2.4，50分）

目标为 $g(x) = \sin(\pi x)$，$x \in [-1, 1]$。 使用两隐藏层tanh网络1－32－32－1；41个等距训练点；400个均匀小区间中点作为独立测试点。 固定种子2026、CPU单线程、float64、Adam 2000步（学习率0.001）和L-BFGS最多400次迭代，具体设置见Notebook。 训练仅使用函数值MSE；不得用测试集选择参数或以导数标签替换本题任务。

1. 两种导数（10分）。 先以单隐藏层 $u_\theta = c + \sum_j a_j \tanh(w_j x + b_j)$ 为例，推导 u_x, u_{xx} 和对 a_j, w_j, b_j, c 的偏导数。解释它们分别在函数拟合和物理残差中承担什么作用。
2. 实现与对照（15分）。 按指定配置拟合目标；在不与训练点重合的测试集上分别计算函数值、一阶及二阶输入导数的最大绝对误差、RMSE、相对二范数。用解析式作为独立目标参考，并另用手写链式法则验证当前网络的AD导数。提交图表、种子与环境，不把训练MSE当作测试误差。
3. 高频反例（10分）。 对 $v_k(x) = g(x) + \sin(kx)/k$，$k = 10, 100, 1000$，严格求出函数值和一二阶导数差的无穷范数，并给出能取到相关值的区间内见证点。说明它反驳了哪一种推理。不能仅靠有限网格搜索充当严格证明。
4. Poisson残差与光滑性（15分）。 对 $-u'' = \pi^2 \sin(\pi x)$、$u(-1) = u(1) = 0$，写出残差和含边界项的PINN损失；用已拟合网络计算独立点残差RMSE与端点误差。解释 `create_graph=True`、输入导数与参数梯度的关系；讨论ReLU为何不适宜直接用于本题经典二阶强形式。最后判断：“只用函数值MSE训练后，既然已算出残差，就已完成PINN求解”。

验收依据： 推导符号、阶数和求导对象正确；训练、测试点不重合；图表与解析导数匹配。函数最大测试误差 $< 10^{-2}$ 作为幅值1的教学拟合目标；若未达到，应保留结果并诊断，不得改测试点掩盖。导数和残差无预设“准确”阈值，必须如实报告；对同一网络，NumPy链式法则与AD最大差异应小于 10^{-10}。跨平台训练不要求逐位复制给定误差，固定权重的求导核对与重新训练的复现须分开说明。

难度与用时： 中等偏上，预计课后75—90分钟。来源：仓库2.4任务的AI辅助具体化；新增独立链式法则、见证点反例与残差的诊断性评价。完整PDE训练不在本题范围内。

A2组（G21）习题参考解答：题1—题4

配套题面与2.1—2.2实验、2.3—2.4实验。题1、题2 的数值与 `2.1-2.2-实测摘要.txt` 一致；题3、题4 的数值与 `数据/` 内记录一致。以下数值为本次环境的参考输出；训练数值可能随平台与版本变化，公式与误差判据以题面为准。

题1参考解答（50分）

1.1 线性插值余项（小问1的一部分，5分）

设 p_1 是过 $(x_0,f(x_0))$、$(x_1,f(x_1))$ 的直线。固定 $x\notin\{x_0,x_1\}$，令

$$g(t)=f(t)-p_1(t)-K\,(t-x_0)(t-x_1),\qquad K=\frac{f(x)-p_1(x)}{(x-x_0)(x-x_1)}.$$

则 g 在 x_0,x_1,x 三点为零。对 g 用两次Rolle定理，存在 ξ 使 $g''(\xi)=0$；而 $p_1''=0$、$[(t-x_0)(t-x_1)]''=2$，故

$$0=g''(\xi)=f''(\xi)-2K\;\Longrightarrow\;\;f(x)-p_1(x)=\frac{f''(\xi)}{2}(x-x_0)(x-x_1).$$

这就是两点插值的余项。评分：辅助函数构造、两次Rolle与结论合计5分；与下节节点乘积推导合计为小问1的15分。

1.2 $n=3$ 的 ω（小问1的其余10分）

等距。 节点 $-1,0,1$，于是

$$\omega(x)=(x+1)x(x-1)=x^3-x,\qquad \omega'(x)=3x^2-1.$$

临界点 $x=\pm1/\sqrt{3}$，代入得

$$\max_{[-1,1]}|\omega|=\left|\frac{1}{3\sqrt{3}}-\frac{1}{\sqrt{3}}\right|=\frac{2}{3\sqrt{3}}\approx0.3849.$$

Chebyshev。 $n=3$ 时节点为 $\cos\frac{\pi}{6},\cos\frac{\pi}{2},\cos\frac{5\pi}{6}$，即 $\pm\frac{\sqrt{3}}{2},0$，于是

$$\omega(x)=x\left(x^2-\frac{3}{4}\right),\qquad \max_{[-1,1]}|\omega|=\frac{1}{4}=0.25.$$

差在哪里。 同样是3个点，等距把 $\max|\omega|$ 放到 0.3849，Chebyshev 压到 0.25，比值 1.54。这个比值会随 n 迅速放大：一般地，Chebyshev节点的 $\max|\omega|=2^{1-n}$，而等距节点不具有这个最优性质；不能把两者之比增大混同为节点乘积本身必然增大。评分：等距表达式与极值5分，Chebyshev节点与0.25共3分，解释2分；与1.1合计15分。

1.3 区间误差实测（15分）

节点数 n	等距最大误差	Chebyshev最大误差
5	4.384e-01	4.020e-01
9	1.045e+00	1.708e-01
11	1.916e+00	1.092e-01

节点数	等距最大误差	Chebyshev最大误差
15	7.195e+00	4.660e-02
21	5.982e+01	1.533e-02
31	2.388e+03	2.062e-03

节点处误差两列均为 0（机器精度内），与“插值条件成立”一致。图见正文图2与 `图片/A2组-Runge现象.png`，原始表见 `2.1-2.2-实测摘要.txt`。评分：实现与网格划分6分，两列数据完整4分，节点误差单独报告3分，图2分。

1.4 诊断（10分）

等距 $n=11$ 时最大误差出现在 $x\approx -0.94$，对应对称位置的振荡峰；本例等距高次插值在端点附近产生显著振荡。节点乘积的因子是相乘而非相加，不能以“异号相消”解释；实际误差还取决于函数和节点分布。

$x=0.95$ 处：等距插值 1.9236，Chebyshev插值约 0.085535，真值 0.0424，等距插值绝对误差约 1.88。该点最近的等距节点是 $x=1$，距离0.05；离节点近仍不能保证插值误差小，需要实际评价插值多项式。评分：位置与理由6分，两个数值与解释4分。

1.5 辨析（10分）

说法	判断	依据
节点上误差为0，所以插值可靠	假	节点误差恒为0是定义，与区间误差无关；$n=11$ 区间误差 1.92
节点数增加，误差必然减小	假	等距 $n=11\to 31$：1.92 → 2388
Chebyshev节点保证任意函数收敛	假	需函数正则性等条件；不能保证任意连续函数都一致收敛
用“穿过所有测点”验收插值	假	穿过是必要条件，不是质量指标；须报区间误差或独立网格误差

评分：每条判断1分，依据各1.5分，共10分。

题2参考解答（50分）

2.1 正规方程与条件数（15分）

展开 J：

$$J(c)=c^{\mathsf T}A^{\mathsf T}Ac-2c^{\mathsf T}A^{\mathsf T}y+y^{\mathsf T}y,\qquad \nabla J=2A^{\mathsf T}Ac-2A^{\mathsf T}y.$$

令梯度为零得 $A^{\mathsf T}Ac=A^{\mathsf T}y$。

唯一性。 若 A 列满秩，则对任意 $z\neq 0$ 有 $Az\neq 0$，于是 $z^{\mathsf T}A^{\mathsf T}Az=\|Az\|_2^2>0$，$A^{\mathsf T}A$ 正定、可逆，解唯一。反之若列相关，存在 $z\neq 0$ 使 $Az=0$，$A^{\mathsf T}A$ 奇异，解不唯一。

条件数平方。 以下假设 A 列满秩，使用2-范数条件数，推导在精确算术下成立。设 $A=U\Sigma V^{\mathsf T}$ 为奇异值分解，$\sigma_{\max},\sigma_{\min}$ 为非零奇异值的最大与最小。则 $A^{\mathsf T}A=V\Sigma^{\mathsf T}\Sigma V^{\mathsf T}$，其特征值为 σ_i^2，故

$$\kappa(A^{\mathsf{T}}A) = \frac{\sigma_{\max}^2}{\sigma_{\min}^2} = \kappa(A)^2.$$

平方的意思是：$\kappa(A)$ 已经损失的那部分有效位数，在正规方程里再损失一遍。评分：展开与求导5分，唯一性5分，条件数证明5分。

2.2 两类基的对比（15分）

次数	幂基 $\kappa(A)$	幂基训练残差	幂基测试误差	Cheb $\kappa(A)$	Cheb 训练残差	Cheb 测试误差
3	1.05e+02	5.168e-02	2.045e-02	1.82e+00	5.168e-02	2.045e-02
6	1.82e+04	5.203e-02	2.530e-02	1.85e+00	5.203e-02	2.530e-02
9	4.63e+06	3.419e-02	3.150e-02	2.99e+00	3.419e-02	3.150e-02
12	2.41e+09	5.639e-03	1.764e-01	1.65e+01	5.639e-03	1.764e-01

两组指标在表中所列有效数字下相同，两者张成同一个 d 次多项式空间，精确算术下拟合函数相同；有限精度结果仍会有差异。条件数方面，$d = 12$ 时 2.41e9 对 16.5。

正规方程对照（$d = 9$）：$\kappa(A^{\mathsf{T}}A) = 2.14 \times 10^{13}$，$\kappa(A) = 4.63 \times 10^{6}$，两种解法的系数最大差 4.152。评分：实现6分，表6分，条件数与结论3分。

2.3 过拟合诊断（10分）

$d = 14$ 时未知系数15个、数据点15个，Vandermonde矩阵可逆，多项式穿过全部训练点：训练最大残差 1.028e-07（受病态和浮点舍入共同影响），测试最大误差 0.9688，约为函数幅值（$e^1 - e^0 \approx 1.7$ 到 2.7 的量级）的三分之一以上。

“穿过”在这里只意味着15个约束被15个自由度精确满足，与“逼近 e^x”无关；噪声也被当成信号拟合进去，两端出现大幅摆动（见正文图3右）。评分：两个数值4分，解释4分，指出自由度数等同于数据点数2分。

2.4 辨析（10分）

说法	判断	依据
训练残差越小越好	假	$d = 14$ 训练残差最小（1.03e-07），测试误差最大（0.9688）
Chebyshev基最小二乘＝切比雪夫逼近	假	前者最小化2-范数残差，后者最小化无穷范数；解一般不同
正规方程可直接用于高次拟合	假	$\kappa(A^{\mathsf{T}}A) = \kappa(A)^2$，$d = 9$ 已达 2.14e13，系数差 4.152
残差最小化与最大偏差最小化是同一件事	假	2-范数与无穷范数目标不同，最优点一般不同

评分：每条判断1分，依据各1.5分，共10分。

题3参考解答（50分）

3.1 从插值到求积（15分）

令 $h=1/n$、$x_i=i/n$。局部插值为

$$p_i(x)=\frac{x_{i+1}-x}{h}x_i^2+\frac{x-x_i}{h}x_{i+1}^2.$$

积分为 $h(x_i^2+x_{i+1}^2)/2$。累加时内部节点各出现两次、端点各出现一次，因此

$$T_n=\frac{1}{n}\left[\sum_{i=1}^{n-1}\frac{i^2}{n^2}+\frac{1}{2}\right]=\frac{(n-1)n(2n-1)}{6n^3}+\frac{1}{2n}=\frac{1}{3}+\frac{1}{6n^2}.$$

所以 $I-T_n=-1/(6n^2)$，绝对误差 $E_n=1/(6n^2)$，$E_{2n}=E_n/4$。凸函数的梯形位于曲线上方，可用几何直觉再核对符号。评分：局部插值及积分5分，平方和与整理7分，误差符号和加密关系3分。

3.2 数值验证（10分）

区间数 n	T_n	绝对误差	前一层误差/本层误差	实测阶数
4	0.343750000000	1.04166667e-02	—	—
8	0.335937500000	2.60416667e-03	4.000000	2.000000
16	0.333984375000	6.51041667e-04	4.000000	2.000000
32	0.333496093750	1.62760417e-04	4.000000	2.000000
64	0.333374023438	4.06901042e-05	4.000000	2.000000
128	0.333343505859	1.01725260e-05	4.000000	2.000000

本次 e^x 的五个相邻阶数约为1.998874、1.999718、1.999930、1.999982、1.999996，符合渐近二阶而非每次恰好2。实现、表与图见Notebook“2.3-A”，原始记录见**数据/A2组-积分收敛.csv**。评分：实现4分，独立参考及表3分，阶数图与解释3分。

3.3 计算图及求导（15分）

定义 $v_1=x^2$、$v_2=\sin v_1$、$v_3=e^x$、$y=v_2+v_3$。对两条输入路径累加：

$$\frac{\mathrm{d}y}{\mathrm{d}x}=\frac{\partial y}{\partial v_2}\frac{\mathrm{d}v_2}{\mathrm{d}v_1}\frac{\mathrm{d}v_1}{\mathrm{d}x}+\frac{\partial y}{\partial v_3}\frac{\mathrm{d}v_3}{\mathrm{d}x}=2x\cos(x^2)+e^x.$$

在 $x=0.7$，解析式与AD均输出 3.2490187095246466；本点浮点差为0，201点检查网格的最大差为 4.441e-16。本点的0表示本次浮点结果相同。

差分步长 h	中心差分绝对误差
1e-01	7.302029e-03
1e-02	7.268056e-05
1e-03	7.267713e-07
1e-05	7.437828e-11
1e-08	3.340995e-08

差分步长	中心差分绝对误差
1e-12	6.205026e-05
1e-16	1.028573e+00

中等步长时误差近似正比于 h^2；很小步长时相近函数值相减损失有效数字，并在除以 h 后放大。不能把某次扫描得到的最小误差步长直接推广到其他函数或精度。评分：图与链式法则6分，独立参考及AD4分，步长扫描与解释5分。

3.4 辨析（10分）

1. “始终二阶且恰好缩小四倍”不成立。C^2 条件支持二阶上界，非零主项支配时通常渐近接近4；一次函数误差为0，误差比甚至无定义。对本题 x^2，精确误差式才保证恰为4。
2. “自动计算差商就是AD”不成立。差商由相邻函数值近似导数；AD在计算图上累积局部导数，机制不同。
3. “AD无任何误差”不成立。这里输出恰巧与浮点解析式一致，不排除其他输入或计算过程的舍入、溢出、不可导及实现问题。
4. 积分误差不能证明求导准确。两个任务的输出、误差来源和独立参考都不同。

评分：前三点各3分，最后一点1分；合理同义解释可得分。

题4参考解答（50分）

4.1 输入导数与参数偏导（10分）

记 $t_j=\tanh(w_jx+b_j)$，则

$$u_x=\sum_j a_jw_j(1-t_j^2),\qquad u_{xx}=-2\sum_j a_jw_j^2t_j(1-t_j^2).$$

参数偏导为

$$u_{a_j}=t_j,\qquad u_{w_j}=a_jx(1-t_j^2),\qquad u_{b_j}=a_j(1-t_j^2),\qquad u_c=1.$$

空间导数在参数固定时改变输入，描述斜率、曲率；参数导数在输入固定时改变权重，用来构造损失梯度。PINN残差训练还需把输入导数的运算图对参数继续求导。评分：输入两式4分，参数四式4分，求导对象2分。

4.2 实测误差与独立实现（15分）

解析目标为

$$g=\sin(\pi x),\qquad g'=\pi\cos(\pi x),\qquad g''=-\pi^2\sin(\pi x).$$

最终训练MSE为 4.531542e-08；本次CPU训练用时约 2.20 秒，L-BFGS闭包调用 469 次（其中包括线搜索评价，不等于迭代次数）。

评价量	最大采样绝对误差	RMSE	离散相对二范数
函数值 u	4.135050e-04	2.140816e-04	3.027571e-04

评价量	最大采样绝对误差	RMSE	离散相对二范数
一阶导数 u_x	1.522785e-02	4.264197e-03	1.919563e-03
二阶导数 u_{xx}	3.562128e-01	9.239099e-02	1.323869e-02

NumPy逐层链式法则与AD的最大差异（值、一阶、二阶）分别为 4.441e-16、1.332e-15、7.105e-15。Poisson残差RMSE为 9.239099e-02，两端点预测分别为 -1.485380e-04、4.515525e-05。

手写链式法则采用“仿射层传播导数，tanh层加乘积项”的逐层规则，见正文与Notebook。它只验证当前网络的AD导数，不能代替与 g', g'' 的比较。评分：固定配置与独立点4分，三个指标及图5分，手写链式对照4分，结果解释2分。

4.3 高频反例（10分）

函数差为 $\sin(kx)/k$，其绝对值不超过 $1/k$，在 $x=\pi/(2k)$ 取到；因此无穷范数恰为 $1/k$。 一阶导数差为 $\cos(kx)$，绝对值不超过1，在 $x=0$ 取到；二阶差为 $-k\sin(kx)$，绝对值不超过 k，在 $x=\pi/(2k)$ 取到。

k	函数误差无穷范数	一阶导数差无穷范数	二阶导数差无穷范数
10	0.1	1	10
100	0.01	1	100
1000	0.001	1	1000

见证点都在 $[-1,1]$ 内，因此结论不依赖网格是否恰好捕捉振荡峰值。这反驳“函数一致收敛就必有导数一致收敛”；不反驳在附加条件下成立的导数逼近定理。评分：三阶误差6分，见证点与定义域2分，逻辑判断2分。

4.4 残差、边界和光滑性（15分）

残差与损失分别是

$$r_\theta(x) = -u_{\theta,xx}(x) - \pi^2 \sin(\pi x),$$

$$L = \frac{1}{N_r}\sum_i r_\theta(\xi_i)^2 + \lambda_b\left[u_\theta(-1)^2 + u_\theta(1)^2\right], \qquad \lambda_b > 0.$$

由于 $g'' = -\pi^2\sin(\pi x)$，残差 $r_\theta = -(u_{\theta,xx} - g'')$，因此在同一测试集上残差RMSE恰等于二阶导数RMSE。本次为约0.092391；两端点绝对误差约为0.00014854和0.00004516。

`create_graph=True` 会构建导数计算的图，使下一阶输入求导和残差对参数回传成为可能。若只为最终报告二阶导数而不继续对其求导，最后一次求导可以不保留图；若要优化二阶残差，通常需要保留。常数输出偏置不影响二阶输入导数，所以某些参数可能在纯残差损失中没有梯度依赖；本实验用 `allow_unused=True` 显式处理，并非把缺失梯度默认为实现成功。

ReLU在仿射区域内二阶导数为0，折点的经典导数可能不存在；直接用于本题的经典二阶强形式时难以表示一般非零源项。框架在折点返回的约定值不能作为可导性证明。弱形式或其他表示方式需另行分析。

只用函数值MSE拟合、然后评价残差，是PINN求导准备，不是以残差训练完成PDE求解。还没有通过物理残差更新权重，也没有完整PDE训练的误差与边界验证过程。评分：残差与损失4分，实际残差与边界3分，计算图3分，ReLU3分，求解状态2分。

第03章 离散的世界

> 编写组：A2（G13）；来源版本：`230873df074c`，main已合并稿。本文件按原稿收录正文、独立习题与参考解答，保留原稿待完成项。

3.0 章首导入：一次快充，计算机需要回答多少个问题？

在高速服务区，一辆新能源汽车接入快充桩。驾驶员看着屏幕上的充电进度，关心的是：“还要等多久才能充好电？”而负责电池热管理的工程师，却需要思考另一个看不见的问题：

电池内部的热量正在向哪里传递？继续充电后，哪个位置的温度可能最高？

高功率充电不仅涉及电能的输入，也伴随着电池内部的产热与散热。电池包中的温度并不一定处处相同：靠近冷却通道的位置可能较凉，散热条件不同的位置则可能出现更高的温度。

要设计合理的温控策略，工程师不能只知道“现在有多热”，还需要预测：接下来会怎样变化？

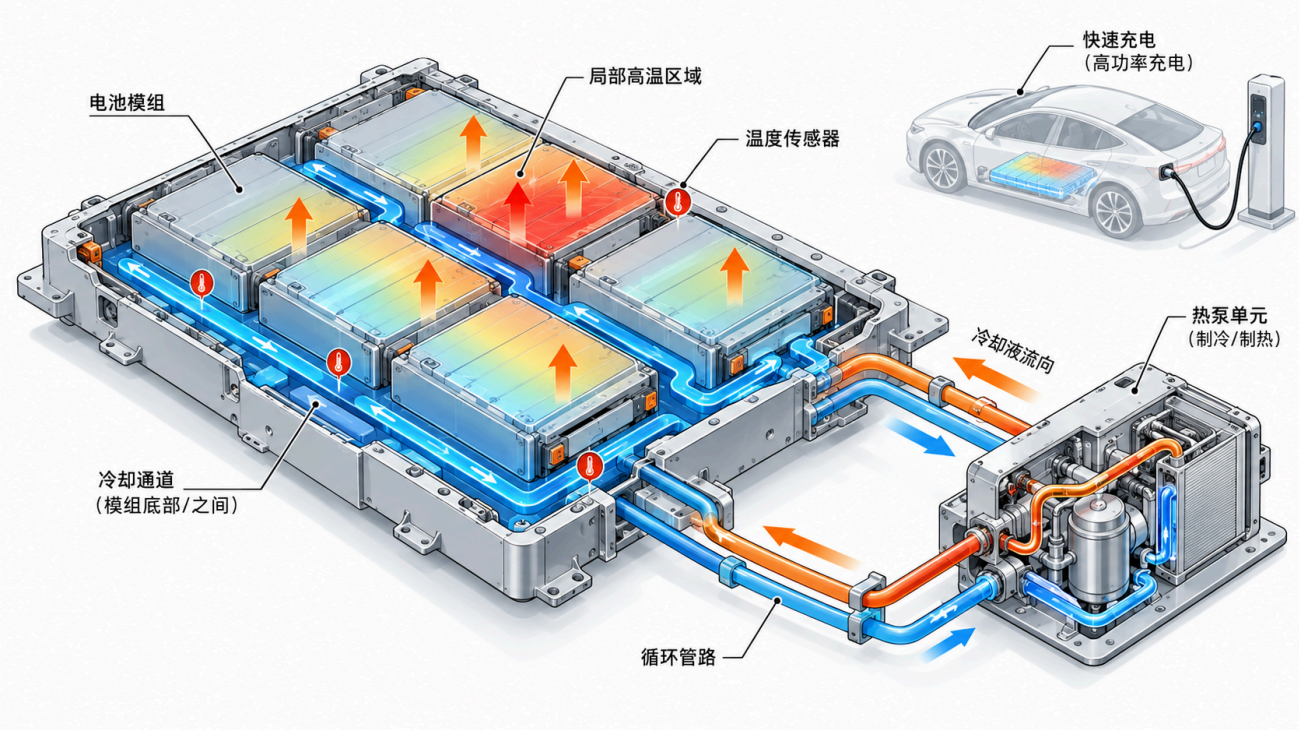


图3-1 新能源汽车快充时的电池热管理及其数值建模示意图。图中展示了快充场景、电池冷却结构、内部温度分布及空间离散化过程。

这幅图揭示了一个重要事实：现实中的温度变化是连续发生的，但计算机却只能通过有限次运算进行预测。那么，计算机究竟怎样描述这样一个连续变化的物理过程？

3.0.1 问题一：怎样预测下一时刻的温度？

如果先把电池近似看成温度均匀的整体，就可以用常微分方程（ordinary differential equation, ODE）描述其状态随时间的变化。例如，一般的一阶初值问题可以写为

$$\begin{cases} y'(t) = f(t, y(t)), \\ y(t_0) = y_0. \end{cases} \tag{3.0.1}$$

其中，$y(t)$ 表示随时间变化的状态量，$f(t,y)$ 描述其瞬时变化率。然而，计算机通常不能直接获得整个连续时间区间上的解析解，而需要把时间划分成一个个离散节点，再从已知状态逐步计算未来状态。Euler方法（Euler method）和四阶Runge–Kutta方法（fourth-order Runge–Kutta method, RK4）提供了不同的时间推进策略。即使求解同一个微分方程、采用相同的初始条件，它们也可能给出不同的数值结果。

同一个物理过程，为什么不同算法会预测出不同的结果？我们又该相信哪一个？

这正是 3.1 节“ODE初值问题：Euler与RK4”要回答的问题。

3.0.2 问题二：怎样发现电池内部的局部高温？

只预测电池的平均温度显然不够。即使平均温度处于合理范围，局部区域也可能存在温度较高的情况。为了描述热量在空间中的传递，就需要从常微分方程进一步走向偏微分方程（partial differential equation, PDE）。在适当的简化假设下，一维热传导过程可以用热方程（heat equation）描述：

$$\frac{\partial u}{\partial t} = \kappa \frac{\partial^2 u}{\partial x^2}. \tag{3.0.2}$$

其中，$u(x,t)$ 表示位置 x、时刻 t 的温度，κ 为热扩散系数。为了利用计算机求解，我们可以沿空间方向设置网格节点，并使用有限差分方法（finite difference method）近似空间导数。但这里存在一个出人意料的现象：即使真实温度正在逐渐趋于平稳，数值计算得到的温度也可能发生剧烈振荡，甚至不断增大。这并不意味着真实电池突然违反了热传导规律，而可能是离散方法的时间步长与空间网格不匹配，导致计算失去了稳定性（stability）。

为什么一个看似合理的计算公式，会产生不符合物理规律的结果？

3.2 节“PDE有限差分：热传导与CFL条件”将帮助我们理解数值稳定性及其与网格、时间步长之间的关系。

3.0.3 问题三：面对复杂结构，能否换一种计算思路？

实际电池包并不是一根简单的直杆。电芯、冷却通道和结构连接件具有不同的几何形状。如果希望进一步考虑复杂结构中的物理场分布，仅依靠规则网格上的简单差分格式，可能不够方便。有没有办法把复杂区域拆分成许多容易处理的小单元，再将它们组合起来？有限元方法（finite element method）提供了另一种思路。它把连续区域划分为有限个单元，通过简单的局部函数构造近似解，再利用弱形式（weak form）与Galerkin方法（Galerkin method）建立可以求解的代数方程组。更令人意外的是：即使使用分片直线近似一条光滑曲线，且这些直线不能在每一点上满足原微分方程，也可能在某些节点上得到完全准确的数值。这究竟是巧合，还是背后存在更深刻的数学原理？

在 3.3 节“有限元：弱形式与Galerkin方法”中，我们将从一维边值问题出发，理解有限元如何通过改变方程的检验方式，将连续问题转化为有限维计算问题。

3.0.4 问题四：人工智能能否参与预测？

随着人工智能技术的发展，神经网络也开始被用于微分方程建模与求解。传统数值方法通常从已知的数学模型出发，通过离散化计算系统状态；而在某些数据驱动任务中，我们还希望利用观测数据学习系统的变化规律。例如，Neural ODE（神经常微分方程）利用神经网络表示微分方程的右端函数，并通过数值积分获得

连续时间系统的演化轨迹；而 PINN（物理信息神经网络）则将微分方程残差、初始条件或边界条件等信息纳入训练目标，使神经网络的预测受到物理规律约束。

这些方法为物理建模与数据驱动计算的结合提供了新的思路。但是，一个新的问题也随之出现：如果神经网络能够很好地拟合已有数据，是否意味着它一定能够准确预测未来？如果改变初始条件、时间步长或数值求解器，预测结果又是否仍然可靠？在 3.4 节“AI关联：Neural ODE与PINN”中，我们将带领读者初步认识数值计算与人工智能之间的联系。

为了突出这些方法的基本数学原理，本章将采用理想化的衰减模型、一维热传导模型及一维边值问题进行推导和实验，而不直接求解完整的新能源汽车电池热管理系统。尽管这些方法采用了不同的离散思想和计算策略，却共同指向本章最重要的问题：当连续的现实世界被转化为有限的时间步长、空间网格和数值变量时，我们凭什么相信计算机得到的结果是准确、稳定且可信的？

3.0.5 章首思考

思考 1：平均温度下降，就意味着处处都在冷却吗？

如果某个电池包的平均温度正在下降，是否可以认定电池内部每一个位置的温度都在下降？为什么？

思考 2：网格越细，计算结果就一定越可靠吗？

对于热传导问题，如果不断缩小空间网格间距，而保持时间步长不变，计算结果是否一定更加准确？

思考 3：人工智能的预测一定比传统数值方法更好吗？

如果一个神经网络对已有实验数据拟合得非常准确，是否就说明它在不同初始条件下也能够可靠预测系统的变化？

请读者先形成自己的判断，再带着这些问题进入本章。在后续的数学推导与数值实验中，我们将逐步寻找答案。

3.1 常微分方程初值问题：Euler 方法与经典四阶 Runge–Kutta 方法

导览：一杯热茶，为什么会被“算”得比室温还冷？

一个看似不可能的预测

一杯刚泡好的热茶放在桌上，周围室温基本保持不变。随着时间推移，热茶逐渐冷却，温度越来越接近室温。若用“茶水温度与室温之差”描述冷却过程，那么在理想化的牛顿冷却模型中，只要初始时茶水高于室温，这个温差就会逐渐趋于零，而不会突然变成负值。

假设把温差和时间都作适当的无量纲化处理，初始温差记为 1，变化率恰好等于当前温差的相反数。现在不求完整的温度变化曲线，只想预测一段时间之后的温差。

一种直观办法是：测出此刻的降温速度，并假设接下来的一小段时间内，茶水都按这个速度降温。可是，如果把这“一小段时间”取得太长，例如一步跨过 2.5 个无量纲时间单位，计算竟会给出温差 -1.5；而相应连续模型的精确温差约为 0.0821，仍然为正。

同一个冷却规律，为什么连续模型的温差始终为正，简单的数值预测却可能越过零点？

30 秒思考：问题出在哪里?

先不急着计算更多步骤。对于上述反常预测，下面哪一种解释更合理？

- A． 微分方程描述的冷却过程本身必然会让温差在某一时刻变成负值。
- B． 用当前斜率代表整个时间步的变化可能过于粗糙，尤其在步长较大时，数值结果可能偏离真实解。
- C． 只要使用高阶算法，不论时间步取得多大，都一定能得到稳定且准确的结果。

可以先记下判断，读完本节再用更新公式和稳定性分析检验。

从这个问题出发

这杯热茶实际上提出了三个依次深入的数学问题：如何根据当前状态计算下一步？能否利用更多斜率信息提高精度？怎样判断计算结果是否可信？

为回答这些问题，本节先建立常微分方程初值问题的离散形式，介绍向前 Euler 与向后 Euler 方法；再由低阶 Runge－Kutta 方法过渡到经典四阶 Runge－Kutta 方法；最后讨论局部误差、全局误差、稳定性以及计算成本。热茶冷却的情境只用于帮助理解，后文将以统一的无量纲线性衰减方程进行严格计算。

本节研究常微分方程初值问题的固定步长数值求解。首先介绍时间离散和向前、向后 Euler 方法，继而由一阶、二阶 Runge－Kutta 方法引出经典四阶 Runge－Kutta 方法，最后从误差、稳定性与计算成本三个方面比较这些算法。为使推导与计算相互印证，全文采用统一的符号约定，并以线性衰减方程作为贯穿算例。

3.1.1 初值问题与 Euler 方法

（一）初值问题及时间离散

考虑一阶常微分方程初值问题（initial value problem，简称 IVP）

$$\begin{cases} y'(t) = f\big(t, y(t)\big), \quad t \in [t_0, T], \\ y(t_0) = y_0. \end{cases} \tag{3.1.1}$$

其中，t 表示自变量（通常为时间），$y(t)$ 表示待求状态函数，$f(t,y)$ 为已知的右端函数；t_0、y_0 和 T 分别为初始时刻、初始状态和终止时刻。以下主要讨论标量方程；对于向量值状态，相关公式可按分量或向量形式推广。

初值问题的求解依赖于微分方程与初始条件两方面的信息。若 f 在初值附近连续，且关于状态变量 y 满足局部 Lipschitz 条件，则可在一定时间区间内保证解的存在唯一性。解能否延拓至整个 $[t_0, T]$，还取决于所研究问题的具体条件。即使解存在唯一，也未必能得到便于计算的解析表达式，因此需要构造离散近似。

将区间 $[t_0, T]$ 均匀划分为 N 个时间步，设

$$\begin{cases} h = \dfrac{T - t_0}{N} > 0, \\ t_n = t_0 + nh, \qquad n = 0, 1, \dots, N. \end{cases} \tag{3.1.2}$$

记 $y(t_n)$ 为精确解在节点 t_n 处的值，y_n 为它的数值近似，并取 $y_0 = y(t_0)$ 作为计算初值。此后，$y(t_n)$ 始终表示精确值，y_n 始终表示数值近似值。本节所讨论的方法均属于单步方法，即由当前状态 y_n 和本步内的函数信息计算下一状态 y_{n+1}。

根据微积分基本定理，精确解满足

$$y(t_{n+1}) = y(t_n) + \int_{t_n}^{t_{n+1}} f\big(t, y(t)\big)\,\mathrm{d}t. \tag{3.1.3}$$

因此，设计数值方法的一个基本思路是：用可计算的近似表达式代替式（3.3）中的积分，再用数值状态代替精确状态。

（二）向前 Euler 方法

若采用积分区间左端点的函数值近似式（3.3）的积分，即取

$$\int_{t_n}^{t_{n+1}} f\big(t, y(t)\big)\,\mathrm{d}t \approx h f\big(t_n, y(t_n)\big), \tag{3.1.4}$$

便得到向前 Euler 方法（forward Euler method），亦称显式 Euler 方法：

$$\begin{cases} t_{n+1} = t_n + h, \\ y_{n+1} = y_n + h f(t_n, y_n). \end{cases} \tag{3.1.5}$$

由式（3.5）可见，向前 Euler 方法使用当前点 (t_n, y_n) 处的斜率预测下一步的状态。在几何上，它相当于以当前点的切线近似解曲线在一个时间步内的变化。由于右端只含已知量，下一节点的近似值可以直接计算，故称为显式方法。

这一公式也可由 Taylor 展开得到。若精确解具有连续的二阶导数，则存在 $\xi_n \in (t_n, t_{n+1})$，使

$$y(t_{n+1}) = y(t_n) + h y'(t_n) + \frac{h^2}{2} y''(\xi_n). \tag{3.1.6}$$

代入 $y' = f(t, y)$ 并略去二阶余项，即得到式（3.5）。这同时表明：若从精确状态出发推进一步，向前 Euler 的一步局部差值为 $O(h^2)$。有关其多步误差将在 3.1.3 节进一步讨论。

（三）向后 Euler 方法

向前 Euler 方法使用左端点斜率。如果改用积分区间右端点的函数值近似式（3.3）的积分，则得到向后 Euler 方法（backward Euler method），亦称隐式 Euler 方法：

$$\begin{cases} t_{n+1} = t_n + h, \\ y_{n+1} = y_n + h f(t_{n+1}, y_{n+1}). \end{cases} \tag{3.1.7}$$

与式（3.5）不同，式（3.7）右端出现了未知的 y_{n+1}。因此，向后 Euler 一般不能通过一次直接代入完成更新，而需要求解关于下一时刻状态的代数方程。令

$$F_n(u) = u - y_n - h f(t_{n+1}, u), \qquad F_n(u) = 0, \tag{3.1.8}$$

则 F_n 的适当零点即为所求的 y_{n+1}。对于非线性右端函数，如果 f 对 y 可微，且迭代初值合适、分母不为零，可采用 Newton 法求解：

$$u^{(m+1)} = u^{(m)} - \frac{F_n\big(u^{(m)}\big)}{1 - h f_y\big(t_{n+1}, u^{(m)}\big)}. \tag{3.1.9}$$

其中，m 为非线性迭代次数，f_y 表示 f 对第二个变量的偏导数。若 y 为向量，式（3.9）相应地转化为利用雅可比矩阵求解线性方程组的 Newton 步。应当区分时间离散误差与代数方程求解误差：只有隐式方程被求解到足够精度时，才能观察到向后 Euler 方法本身的理论精度。

向前 Euler 与向后 Euler 的一步局部差值均为 $O(h^2)$，在通常的光滑性及误差传播条件下均为全局一阶方法；但两者的稳定性有显著区别。向后 Euler 虽然每一步可能需要额外的代数求解工作，却具有更好的衰减稳定性。这说明显式与隐式主要区别于更新方程的求解方式，而非精度阶数的高低。

（四）Euler 方法的一个基本算例

例 3.1　考虑无量纲线性衰减模型

$$\begin{cases} y'(t) = -y(t), & t \geqslant 0, \\ y(0) = 1. \end{cases} \tag{3.1.10}$$

这里的 t 与 y 均为无量纲变量。利用分离变量法可得解析解

$$y(t) = e^{-t}, \qquad y(0.5) = e^{-0.5} \approx 0.606530659713. \tag{3.1.11}$$

取步长 $h = 0.5$。由式（3.5），向前 Euler 第一步得到 $y_1^{\mathrm{FE}} = 1 + 0.5(-1) = 0.5$；由式（3.7），向后 Euler 第一步满足 $y_1^{\mathrm{BE}} = 1 - 0.5 y_1^{\mathrm{BE}}$，从而

$$\begin{cases} y_1^{\mathrm{FE}} = 0.5, \\ y_1^{\mathrm{BE}} = \dfrac{1}{1+h} = \dfrac{2}{3} \approx 0.666666666667. \end{cases} \tag{3.1.12}$$

在这一算例中，向前 Euler 低估真实解，向后 Euler 高估真实解。两种方法分别采用步长起点和终点的变化率，但都只利用了单次斜率近似。若希望在相同步长下进一步提高精度，就需要更充分地利用一个时间步内的斜率信息。

3.1.2 Runge–Kutta 方法：从一阶到四阶

（一）从单阶段方法到两阶段方法

向前 Euler 方法也可以看作最简单的一阶显式 Runge－Kutta 方法。记

$$\begin{cases} k_1 = f(t_n, y_n), \\ y_{n+1} = y_n + h k_1, \end{cases} \tag{3.1.13}$$

其中，k_1 表示当前节点的斜率。当一个时间步内变化率变化较快时，仅用 k_1 近似整个区间上的平均变化率，可能产生较大的截断误差。因此，可以通过增加中间阶段获得更充分的信息。

一种典型的二阶显式 Runge－Kutta 方法先用起点斜率预测终点状态，再把起点斜率和预测终点的斜率取平均：

$$\begin{cases} k_1 = f(t_n, y_n), \\ k_2 = f(t_n + h, y_n + h k_1), \\ y_{n+1} = y_n + \dfrac{h}{2}(k_1 + k_2). \end{cases} \tag{3.1.14}$$

该形式通常称为显式梯形型二阶 Runge－Kutta 方法（亦称 Heun 方法）；这里仅用它说明从一阶段到多阶段的构造思路，不将其列为 Euler 方法的独立分支。特别要注意，式（3.14）使用的是预测终点状态，因而仍为显式方法，不能与需要求解未知终点状态的隐式梯形法混为一谈。

二阶精度可由 Taylor 展开作简要解释。对于足够光滑的标量初值问题，沿精确解有 $y'' = f_t + f_y f$，因此

$$y(t_n + h) = y(t_n) + h f + \frac{h^2}{2}(f_t + f_y f) + O(h^3), \tag{3.1.15}$$

式中右端的 f、f_t 与 f_y 均在 $(t_n, y(t_n))$ 处取值。从精确起点出发，将式（3.14）中的 k_2 展开，可以得到与式（3.15）相同的常数项、一次项和二次项，因此式（3.14）的一步局部差值为 $O(h^3)$，在通常条件下全局误差为 $O(h^2)$。

由此可见，Runge－Kutta 方法通过合理选取阶段位置、预测状态和组合权重提高精度，而并非只要增加函数求值次数就必然获得更高阶数。

（二）经典四阶 Runge–Kutta 方法

经典四阶 Runge－Kutta 方法（classical fourth-order Runge－Kutta method，简称 RK4）在一个时间步内计算四个阶段斜率。其阶段公式为

$$\begin{cases} k_1 = f(t_n, y_n), \\ k_2 = f\left(t_n + \dfrac{h}{2},\ y_n + \dfrac{h}{2}k_1\right), \\ k_3 = f\left(t_n + \dfrac{h}{2},\ y_n + \dfrac{h}{2}k_2\right), \\ k_4 = f(t_n + h,\ y_n + hk_3). \end{cases} \tag{3.1.16}$$

其更新公式为

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \tag{3.1.17}$$

这里，k_1 表示起点斜率，k_2 和 k_3 分别使用两次不同的中点预测状态，k_4 表示由第三阶段预测得到的终点斜率。由于四个 k_i 在本节均定义为斜率，最终增量由式（3.17）统一乘以步长 h；这一约定在下文保持不变。

Runge－Kutta 方法通常用系数表（Butcher tableau）简明表示阶段之间的依赖关系。经典 RK4 的系数表为

$$\begin{array}{c|cccc} 0 & 0 & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array} \tag{3.1.18}$$

表中第一列为阶段时间系数 c_i，中间四列为阶段系数 a_{ij}，最后一行为更新权重 b_i。RK4 的四个阶段及 $1:2:2:1$ 的权重共同满足四阶 Runge－Kutta 方法的阶条件；当精确解与右端函数足够光滑时，其一步局部差值为 $O(h^5)$，固定有限区间上的全局误差为 $O(h^4)$。仅对一个测试方程验证 Taylor 级数的匹配，还不能代替对一般非线性问题的四阶阶条件的验证。

（三）经典 RK4 的单步计算

继续考虑例 3.1 的方程 $y' = -y$，初值为 $y_0 = 1$，步长为 $h = 0.5$。根据式（3.16），有

$$\begin{cases} k_1 = -1, \\ k_2 = -\left(1 + \dfrac{0.5}{2}k_1\right) = -0.75, \\ k_3 = -\left(1 + \dfrac{0.5}{2}k_2\right) = -0.8125, \\ k_4 = -(1 + 0.5k_3) = -0.59375. \end{cases} \tag{3.1.19}$$

将四个阶段代入式（3.17），得到

$$\begin{aligned} y_1^{\mathrm{RK4}} &= 1 + \frac{0.5}{6}\left[-1 + 2(-0.75) + 2(-0.8125) - 0.59375\right] \\ &= 0.606770833333\ldots. \end{aligned} \tag{3.1.20}$$

与解析值 $y(0.5) \approx 0.606530659713$ 比较，三种方法的单步结果见表 3.1-1。

表 3.1-1　线性衰减方程在 $h = 0.5$ 下的一步计算结果

方法	$t=0.5$ 处的数值解	与解析解的绝对误差
向前 Euler	0.500000000000	0.106530659713
向后 Euler	0.666666666667	0.060136006954
经典 RK4	0.606770833333	0.000240173621

在该算例及该步长下，经典 RK4 给出了更接近解析解的近似值。不过，这一数值比较不能直接推广为对任意微分方程、任意步长的精度排序。要评价一个方法，还需要考虑多步误差的传播和步长对稳定性的影响。

3.1.3 误差、稳定性与计算成本

（一）局部误差与全局误差

将一个数值方法的单步更新抽象地记作 $y_{n+1}=\Phi_h(t_n,y_n)$。对于隐式方法，Φ_h 表示在相关代数方程存在适当唯一解的前提下由此定义的单步映射。局部截断误差考察从精确状态出发的一次推进误差，而全局误差考察实际计算经过多步后产生的总误差。定义

$$\begin{cases}\delta_{n+1}=y(t_{n+1})-\Phi_h\big(t_n,y(t_n)\big),\\ e_n=y(t_n)-y_n.\end{cases}\tag{3.1.21}$$

本节采用局部截断差值不除以步长 h的约定。方法称为 p 阶，通常是指在足够光滑且误差传播可控的条件下，其局部差值为 $O(h^{p+1})$，固定有限区间的全局误差为 $O(h^p)$。两种误差不能仅根据阶数形式相同而混淆。

以向前 Euler 为例，假设 f 对 y 满足 Lipschitz 条件，常数为 $L\geqslant 0$，且存在与步长无关的常数 $M>0$，使计算区间内有 $|\delta_{n+1}|\leqslant Mh^2$。由递推式可得

$$|e_{n+1}|\leqslant(1+hL)|e_n|+Mh^2.\tag{3.1.22}$$

当 $L>0$ 且 $e_0=0$ 时，反复应用上述不等式，得到

$$|e_n|\leqslant\frac{Mh}{L}\big[(1+hL)^n-1\big]\leqslant\frac{Mh}{L}\big[e^{L(T-t_0)}-1\big].\tag{3.1.23}$$

因此，在 $T-t_0$ 固定时，向前 Euler 的全局误差为 $O(h)$。当 $L=0$ 时，直接求和仍得到 $|e_n|\leqslant M(T-t_0)h$。这说明一步局部差值为二阶量，并不意味着多步全局误差也为二阶量。向后 Euler 与 RK4 也需在各自适用的光滑性和稳定性假设下分析误差传播。

三种主要方法的典型精度阶数见表 3.1-2。

表 3.1-2　三种单步方法的典型精度与计算特点

方法	一步局部差值	固定区间全局误差	每步主要计算
向前 Euler	$O(h^2)$	$O(h)$	1 次右端函数求值
向后 Euler	$O(h^2)$	$O(h)$	求解 1 个隐式代数方程
经典 RK4	$O(h^5)$	$O(h^4)$	4 次右端函数求值

表中的误差阶数均为在相应条件下、当 $h\to 0$ 时的渐近结论，并不表示任取较大的步长都能达到对应的精度。

（二）线性稳定性与稳定函数

为分析一个方法如何传播衰减模态，考虑标准线性测试方程

$$y'(t)=\lambda y(t),\qquad \mathrm{Re}(\lambda)<0. \tag{3.1.24}$$

令 $z=h\lambda$。对于该方程，许多单步方法的更新可以表示为

$$y_{n+1}=R(z)y_n, \tag{3.1.25}$$

其中 $R(z)$ 称为方法的稳定函数。对于负实部的衰减模态，条件 $|R(z)|<1$ 表示数值解幅值在每一步严格减小；满足此条件的 z 构成相应的严格绝对稳定区域。

分别将式（3.24）代入向前 Euler、向后 Euler 和经典 RK4 公式，得到

$$\begin{cases} R_{\mathrm{FE}}(z)=1+z,\\ R_{\mathrm{BE}}(z)=\dfrac{1}{1-z},\\ R_{\mathrm{RK4}}(z)=1+z+\dfrac{z^2}{2}+\dfrac{z^3}{6}+\dfrac{z^4}{24}. \end{cases} \tag{3.1.26}$$

对于前述衰减模型 $\lambda=-1$，有 $z=-h$。向前 Euler 的衰减条件为 $|1-h|<1$，即 $0<h<2$。向后 Euler 满足 $R_{\mathrm{BE}}(-h)=1/(1+h)$，故对任意 $h>0$ 都有 $0<R_{\mathrm{BE}}(-h)<1$。经典 RK4 在负实轴上的严格稳定区间约为 $-2.7853<z<0$；因此，在该测试方程上要求 $0<h<2.7853$（近似值）。

向后 Euler 的稳定性质还可以扩展到整个左半复平面。若 $\mathrm{Re}(z)\leqslant 0$，则

$$|1-z|^2=(1-\mathrm{Re}\,z)^2+(\mathrm{Im}\,z)^2\geqslant 1,\qquad |R_{\mathrm{BE}}(z)|\leqslant 1. \tag{3.1.27}$$

这表明向后 Euler 是 A-稳定的，而向前 Euler 与经典显式 RK4 的绝对稳定区域均有界。因此，向后 Euler 虽然仅为一阶，却可能更适于处理具有快速衰减模态的刚性问题；其代价是每步通常需解隐式方程。

例 3.2　仍取 $y'=-y$、$y(0)=1$，但将步长增大为 $h=2.5$，在区间 $[0,10]$ 上推进四步。由式（3.26），三种方法的放大因子分别为

$$\begin{cases} R_{\mathrm{FE}}(-2.5)=-1.5,\\ R_{\mathrm{BE}}(-2.5)=\dfrac{2}{7},\\ R_{\mathrm{RK4}}(-2.5)=0.6484375. \end{cases} \tag{3.1.28}$$

由于 $y_0=1$，四步后有 $y_4=[R(-2.5)]^4$。向前 Euler 得到 $y_4^{\mathrm{FE}}=5.0625$，数值解在过程中交替变号且幅值增长，与精确解 $y(10)=e^{-10}\approx 4.539993\times 10^{-5}$ 的衰减行为相反。向后 Euler 和 RK4 的放大因子绝对值虽小于 1，其计算值也未必足够接近这一极小的精确值。这一算例表明，稳定性反映误差和模态的传播性质，不能替代实际精度检验。

（三）步长、精度与计算成本

固定计算区间 $[t_0,T]$ 且采用等距网格时，步数 $N=(T-t_0)/h$。若只按右端函数 f 的求值次数衡量显式方法的基本成本，则

$$\begin{cases} N_f^{\mathrm{FE}}=N,\\ N_f^{\mathrm{RK4}}=4N. \end{cases} \tag{3.29}$$

其中，N_f^{FE}、N_f^{RK4} 分别表示向前 Euler 与经典 RK4 的右端函数求值次数。向后 Euler 每步还要进行代数方程求解，其成本依赖方程结构、所用迭代方法和停止准则，不能简单用一个固定的函数求值次数表示。

对于同一平滑且非刚性的初值问题，减小步长通常能够降低离散误差，但也会增加步数；在相同误差要求下，高阶 RK4 可能用较少的总函数求值次数达到目标。另一方面，当稳定性要求迫使显式方法使用很小的步长时，即使精度阶数较低，隐式方法仍可能更具优势。因此，选择数值方法应当综合考虑误差容限、稳定性限制、单步求解成本与问题本身的性质。

3.1.4 数值实验：线性衰减问题的精度、稳定性与效率

前文分别给出了向前 Euler、向后 Euler 和经典 RK4 的更新公式，并从理论上讨论了它们的误差阶数与稳定性。下面仍以例 3.1 的线性衰减方程为对象，通过数值计算检验这些结论。实验安排为两个相互补充的部分：首先在固定终止时刻逐步缩小步长，研究数值解的收敛规律；然后在较大的步长下观察数值稳定性。最后，在同一误差要求下比较计算成本。

（一）实验模型与计算方案

考虑式（3.10）给出的初值问题 $y'(t)=-y(t)$、$y(0)=1$，其解析解由式（3.11）给出，即 $y(t)=e^{-t}$。主实验固定终止时刻为 $T=2$，取步数 $N=4,8,16,32,64$，并令 $h=T/N$；稳定性实验改用区间 $[0,10]$ 和步长 $h=2.5$。所有变量均经过无量纲化，不依赖外部采样数据。

为保证比较公平，三种方法使用相同的初值、网格节点和终止时刻。其中，向前 Euler 与 RK4 直接按式（3.5）和式（3.16）—（3.17）推进；对于本例的线性方程，向后 Euler 的隐式方程可在每一步精确化为 $y_{n+1}=y_n/(1+h)$，因而不必引入 Newton 迭代误差。这一化简仅适用于当前模型，不能据此认为一般隐式 Euler 方法都无需代数求解。

以终点的绝对误差作为主要精度指标：

$$E_T(h)=|y_N-y(T)|=\left|y_N-e^{-T}\right|. \tag{3.30}$$

当步长减半时，采用相邻两组终点误差估计观测收敛阶：

$$p_{\mathrm{obs}}(h)=\log_2\left(\frac{E_T(h)}{E_T(h/2)}\right). \tag{3.31}$$

若计算处于渐近收敛区间且舍入误差尚未占主导，则预期向前、向后 Euler 的 p_{obs} 趋近于 1，而经典 RK4 趋近于 4。这只是运行前的理论预测，需要通过计算结果检验。

对于本例，还可把三种更新写成统一的线性递推形式，以独立核查程序输出：

$$\begin{cases} y_{n+1}^{\mathrm{FE}}=(1-h)\,y_n^{\mathrm{FE}},\\[2mm] y_{n+1}^{\mathrm{BE}}=\dfrac{1}{1+h}\,y_n^{\mathrm{BE}},\\[2mm] y_{n+1}^{\mathrm{RK4}}=\left(1-h+\dfrac{h^2}{2}-\dfrac{h^3}{6}+\dfrac{h^4}{24}\right)y_n^{\mathrm{RK4}}, \end{cases} \qquad y_0^{\mathrm{FE}}=y_0^{\mathrm{BE}}=y_0^{\mathrm{RK4}}=1. \tag{3.32}$$

在每组实验中，将逐步计算结果与式（3.32）的递推值以及解析解 e^{-t} 对照，以检查实现与误差计算是否一致。

（二）数值解与终点误差

主实验的精确参考值为 $y(2)=e^{-2}\approx 0.135335283237$。三种方法的终点近似值见表 3.1-3，终点绝对误差见表 3.1-4。表内数值由双精度浮点计算得到，并按表头对应的小数位数显示；误差由未舍入的计算结果求得。

表 3.1-3　不同步长下的终点数值解（$T=2$）

步数 N	步长 h	向前 Euler y_N^{FE}	向后 Euler y_N^{BE}	经典 RK4 y_N^{RK4}
4	0.5	0.0625000000	0.1975308642	0.1355497705
8	0.25	0.1001129150	0.1677721600	0.1353461420
16	0.125	0.1180670870	0.1519006530	0.1353358945
32	0.0625	0.1267887864	0.1437056889	0.1353353195
64	0.03125	0.1310840325	0.1395424874	0.1353352854

表 3.1-4　不同步长下的终点绝对误差（$T=2$）

步数 N	步长 h	向前 Euler	向后 Euler	经典 RK4
4	0.5	7.2835×10^{-2}	6.2196×10^{-2}	2.1449×10^{-4}
8	0.25	3.5222×10^{-2}	3.2437×10^{-2}	1.0859×10^{-5}
16	0.125	1.7268×10^{-2}	1.6565×10^{-2}	6.1123×10^{-7}
32	0.0625	8.5465×10^{-3}	8.3704×10^{-3}	3.6259×10^{-8}
64	0.03125	4.2513×10^{-3}	4.2072×10^{-3}	2.2079×10^{-9}

以 $h=0.5$ 为例，三种方法在区间 $[0,2]$ 上的数值轨迹与解析曲线如图 3.1-1 所示。

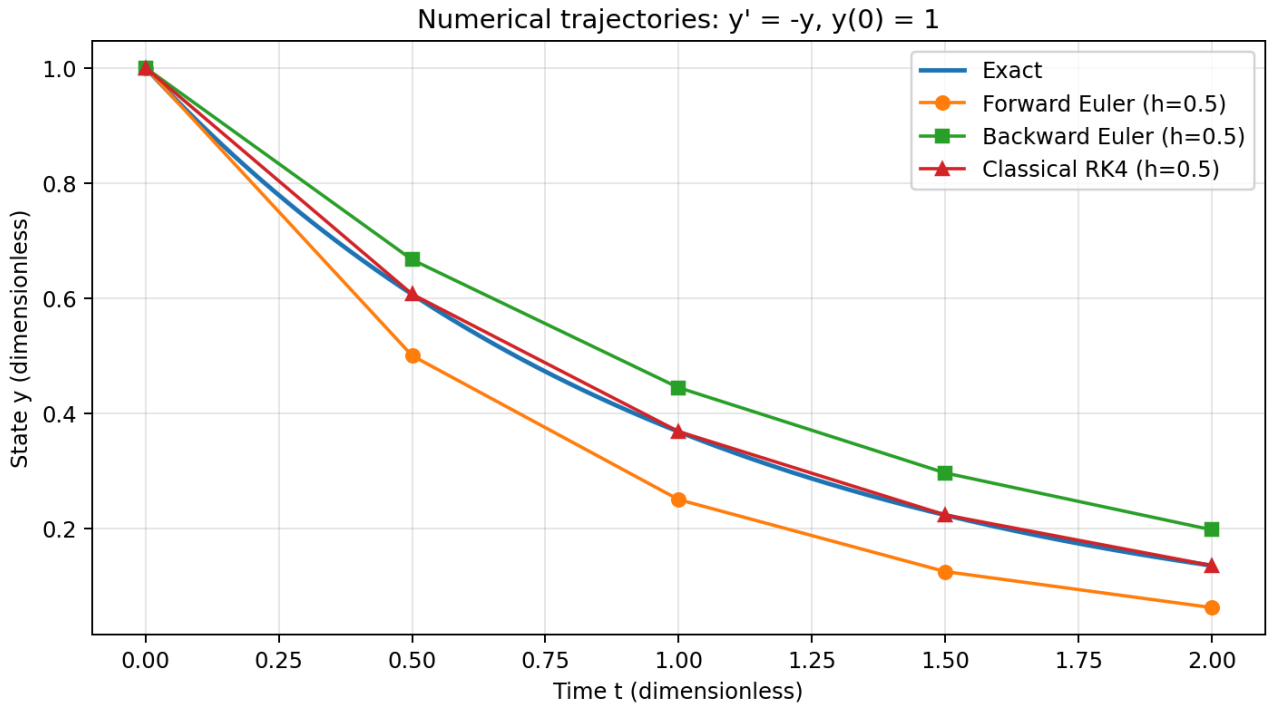


图 3.1-1　向前 Euler、向后 Euler 与经典 RK4 的数值轨迹。模型为 $y'=-y$、$y(0)=1$，终止时刻 $T=2$，步长 $h=0.5$；横、纵坐标均为无量纲量。

图 3.1-1 表明，在这一衰减模型中，向前 Euler 的离散值位于精确曲线下方，向后 Euler 的离散值位于其上方，而 RK4 的节点值与精确解较为接近。随着步长减小，表 3.1-4 中三种方法的终点误差均逐渐下降，说明对该问题及所选网格，步长加密确实提高了计算精度。

（三）步长加密与收敛阶

利用式（3.31）计算相邻两组步长对应的观测收敛阶，得到表 3.1-5。

表 3.1-5　步长减半时的观测收敛阶

步数变化	向前 Euler	向后 Euler	经典 RK4
4 → 8	1.0481	0.9392	4.3040
8 → 16	1.0284	0.9695	4.1510
16 → 32	1.0147	0.9848	4.0753
32 → 64	1.0074	0.9924	4.0376

由表可见，向前 Euler 的观测收敛阶由约 **1.048** 逐渐接近 **1**，向后 Euler 的观测收敛阶从约 **0.939** 逐渐接近 **1**；经典 RK4 的观测收敛阶则由约 **4.304** 逐渐接近 **4**。这些现象与表 3.1-2 给出的理论阶数一致。由于有限步长下仍有高阶误差项，观测阶数不必恰好等于方法的理论阶数。

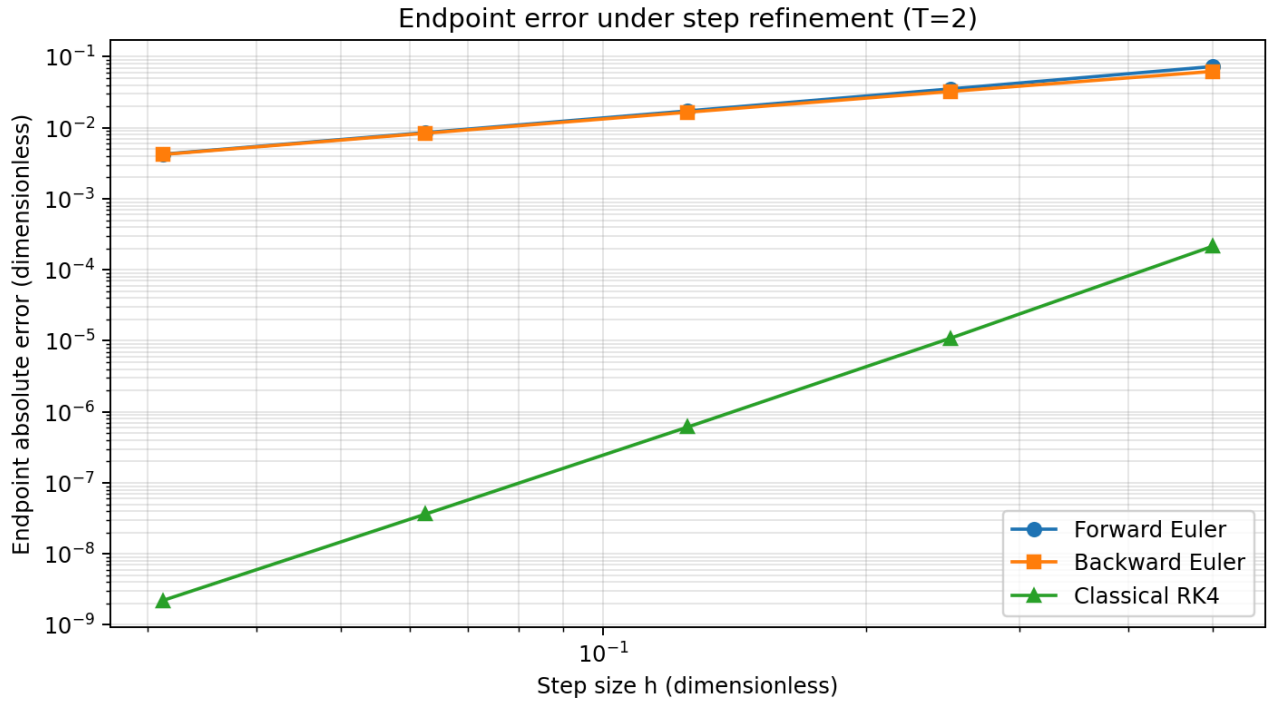


图 3.1-2　终点绝对误差随步长变化的双对数图。$T=2$，$N=4,8,16,32,64$；横轴为步长 h，纵轴为 $E_T(h)$，二者均为无量纲量。

双对数坐标下，误差曲线的斜率近似反映误差随步长衰减的阶数。图 3.1-2 中两种 Euler 方法的曲线彼此接近，具有近似一阶的变化规律；RK4 的误差随 h 减小而明显更快地下降，表现出接近四阶的收敛规律。需要注意，图中结果只反映所考察的有限步长范围；当步长极小时，浮点舍入误差可能使继续加密不再产生同样的误差下降比例。

（四）较大步长下的稳定性

为了区别“数值解是否呈现衰减趋势”与“数值结果是否足够准确”，改取 $T=10$、$h=2.5$，共推进 $N=4$ 步。由式（3.28）可知，三种方法的单步放大因子分别为 -1.5、$2/7$ 和 0.6484375。此时精确终点值为 $e^{-10}\approx 4.539993\times 10^{-5}$。

表 3.1-6　较大步长下的稳定性比较（$T=10$，$h=2.5$）

方法	放大因子 $R(-2.5)$	终点数值解 y_4	终点绝对误差
向前 Euler	-1.5	5.062500000	5.062454600
向后 Euler	0.285714286	0.006663890	0.006618490
经典 RK4	0.648437500	0.176796023	0.176750623

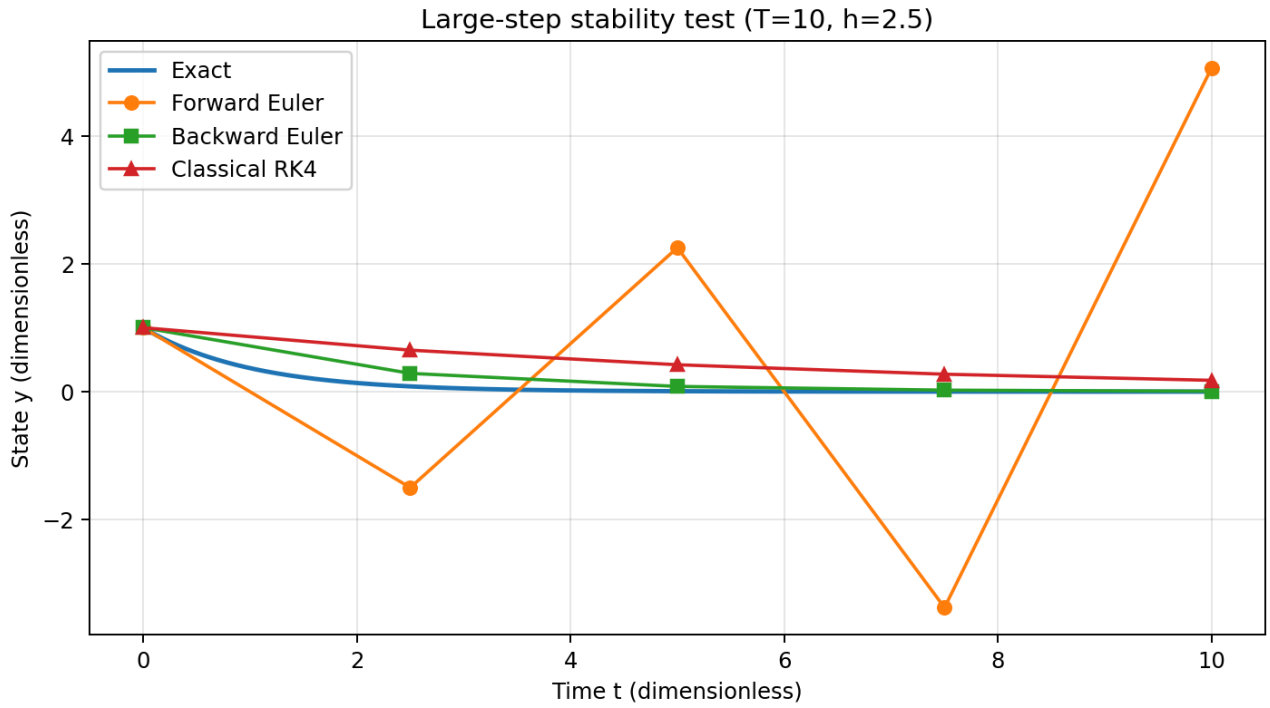


图 3.1-3　大步长稳定性实验。模型为 $y'=-y$、$y(0)=1$，$T=10$，$h=2.5$；横、纵坐标均为无量纲量。

向前 Euler 的放大因子满足 $|R_{\mathrm{FE}}(-2.5)|>1$，因此数值序列交替变号，且其幅值逐步增大；导览中出现的负温差预测正是第一步 $y_1=-1.5$ 的结果。相比之下，向后 Euler 与 RK4 在该步长下的放大因子绝对值均小于 1，数值序列仍表现为衰减，但表 3.1-6 表明它们的终点误差仍不能忽略。

这说明，绝对稳定性是关于误差传播和衰减模态的性质，并不等价于数值解已满足某一精度要求。向后 Euler 对此测试方程的大步长稳定性更好，也不能据此推断它在任意步长下都比 RK4 精确。

（五）相同误差要求下的计算成本

比较数值方法的效率时，不宜仅比较同一步长下每一步的函数求值次数。这里规定终点绝对误差应满足 $E_T(h)\leqslant 10^{-4}$，并只在 $N=4,8,16,32,\dots$ 的二分加密候选网格中寻找首次达标的计算结果。该误差阈值是为本次教学比较自行规定的指标，并非适用于所有初值问题的通用标准。

表 3.1-7　相同终点误差要求下的计算量（$T=2$）

方法	首次达标步数 N	步长 h	终点绝对误差	本例主要计算成本
向前 Euler	4096	0.00048828125	6.6087×10^{-5}	4096 次右端函数求值
向后 Euler	4096	0.00048828125	6.6076×10^{-5}	4096 次线性隐式更新（本例可直接求解）
经典 RK4	8	0.25	1.0859×10^{-5}	32 次右端函数求值

在本次候选网格和精度目标下，RK4 只需 32 次右端函数求值即可达标；向前 Euler 则需 4096 次，两者之比为 128。这说明对于当前平滑、非刚性的模型，提高单步计算精度可以显著减少达到给定精度所需的总函数求值次数。

向后 Euler 的成本不能直接与显式方法按“每步一次函数求值”比较。本例因方程线性，隐式更新可直接化为除法；对于一般非线性初值问题，则需考虑 Newton 迭代、雅可比矩阵、线性方程组求解以及迭代停止准则等额外成本。故表 3.1-7 不构成对所有问题的普遍性能排名。

（六）结果讨论与复现说明

本实验得到三个相互关联但不能混同的结论。首先，步长加密实验支持向前、向后 Euler 为一阶方法和经典 RK4 为四阶方法的理论判断。其次，大步长实验说明高阶方法也有有限的绝对稳定区域，而保持数值衰减并不代表误差必然很小。最后，在满足相同精度目标的前提下比较计算成本，往往比只比较“每步多少次函数求值”更有意义。

实验在双精度浮点环境中进行，所有输入均来自确定性的解析模型，不涉及随机种子或数据集划分。配套 Jupyter Notebook 提供三种更新过程、解析递推核查、逐步加密的数值表、三幅图像及计算成本统计，能够独立重新执行。图 3.1-1 至图 3.1-3 均由该 Notebook 生成，图中的坐标、单位、图例和参数与本节一致。

本实验并未检验非线性、强刚性、解不够光滑、不同函数求值成本或自适应步长的情形。因此，本节结论应理解为标准平滑线性衰减问题上的教学验证，而不应直接外推到所有常微分方程初值问题。

3.1.5 本节小结

常微分方程初值问题的数值求解以离散节点上的近似值代替连续解。向前 Euler 和向后 Euler 分别使用时间步起点和终点的斜率，二者同为一阶方法，但显式、隐式计算机制及稳定性明显不同。Runge－Kutta 方法通过在单步内引入多个精心设计的阶段提高精度；经典 RK4 利用四个阶段斜率的加权组合，实现通常条件下的四阶收敛。

分析数值结果时，应分别回答三个问题：随着步长减小，误差如何变化；给定步长能否正确控制衰减模态；为达到既定误差要求需要付出多少计算代价。只有把这三方面结合起来，才能合理评价不同数值积分方法的适用性。

通过本节的线性衰减数值实验，可以进一步看到：理论误差阶数需通过步长加密与独立参考解检验，稳定性需通过相应稳定函数分析，而计算效率应在相同精度目标下比较。这一认识也是继续学习刚性方程求解与自适应步长算法的基础。

参考文献J. C. Butcher, Numerical Methods for Ordinary Differential Equations, 2nd ed., John Wiley & Sons, 2008，参见第 2 章中关于 Euler 方法（§20）、Euler 方法分析（§21）与 Runge–Kutta 方法（§23）的相关章节。

3.2 PDE有限差分：热传导与CFL条件（2学时）

本节目标： 能把一维热方程、初始条件、边界条件以及显式有限差分格式联系起来，并理解空间离散与时间推进的基本构造方式。

本节依次展开 3.2.1 一维热方程与初边值条件、3.2.2 空间差分与显式时间推进 和 3.2.3 稳定性限制与网格实验。其中，3.2.2 建立 FTCS 显式格式，推导局部截断误差及其 CFL 时间步长限制；3.2.3 再通过网格实验比较满足与违反稳定性条件时的数值行为。

导入：从“人群散开”理解热扩散

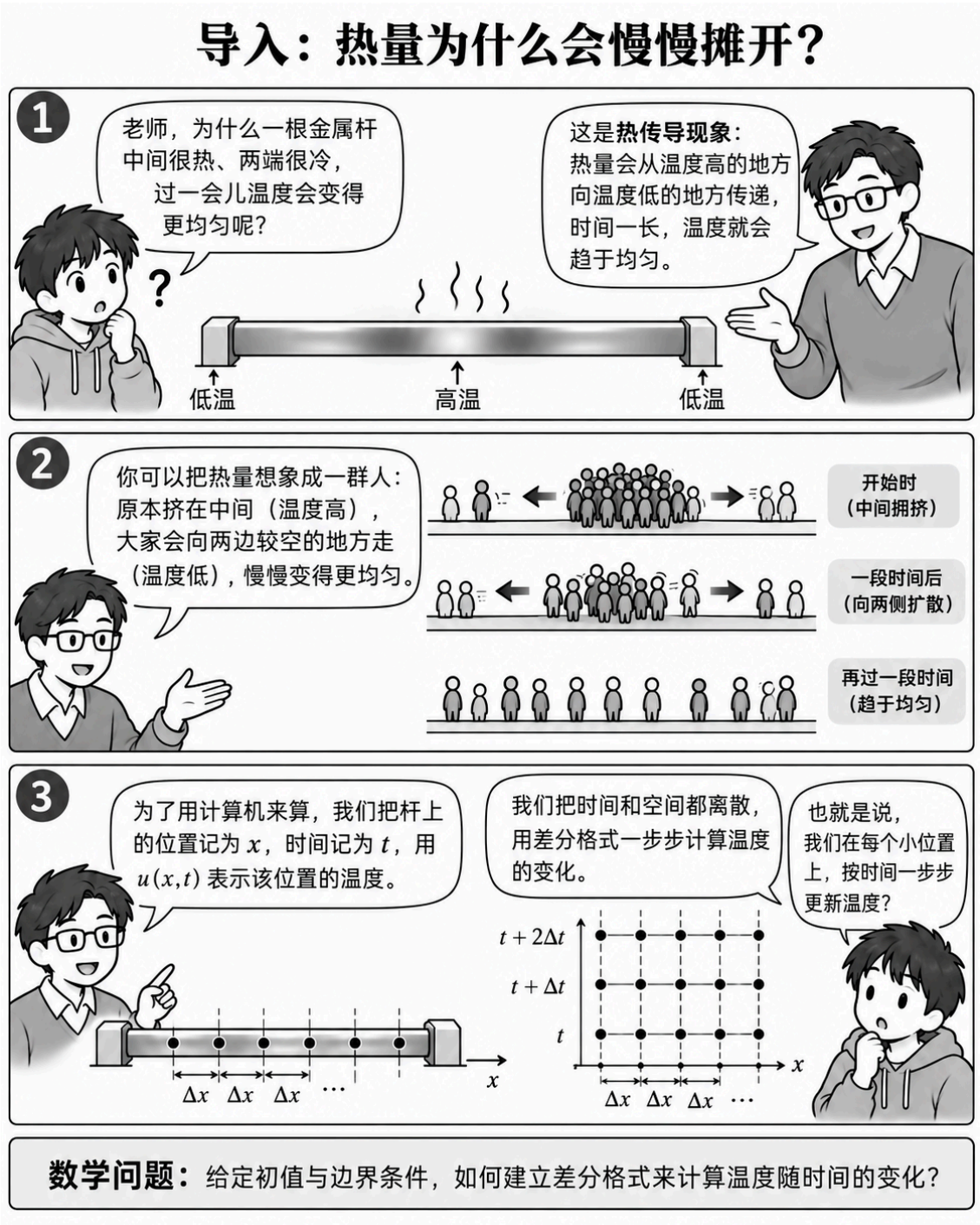


图 3.2-1　热量扩散的类比。金属杆中部温度较高、两端温度较低；漫画把温度的不均匀类比为“人群在中间过于拥挤”，帮助我们直观理解扩散过程会不断削弱局部差异。

设想一根金属杆：开始时中间很热、两端较冷。如果把它静置一段时间，我们会观察到中间逐渐降温，而两侧逐渐升温，原本明显的温度差被一点点“摊平”。最终，在没有持续加热且边界条件允许的情况下，温度分布会朝着更加均匀的状态演化。

漫画给出了一个便于理解的比喻：把“温度分布不均”想象成“人群分布不均”。开始时，大量的人挤在中间，而两边比较空；随着人群向较空的位置散开，中间不再那么拥挤，两侧的人逐渐增多，整体分布趋于平缓。热传导也表现出类似的宏观趋势——局部过高的温度会下降，局部较低的温度会受到周围影响而上升，空间上的温度起伏逐渐减弱。

这个比喻只用于帮助理解“扩散会抹平差异”这一数学特征，并不意味着热量真的像人一样移动。真正要研究的是一个连续的温度场：在不同位置、不同时间，温度分别是多少。

因此，我们把杆上的位置记为 x，把时间记为 t，并用

$$u(x,t)$$

表示位置 x 在时刻 t 的温度。漫画中的“中间热、两边冷”便可以理解为：在初始时刻，函数 $u(x,0)$ 在杆的中部较大、靠近两端较小；随着时间增加，曲线的峰值逐渐降低，空间分布越来越平缓。

到这里，一个生活中的热传导现象就转化成了几个具体的数学问题：

1. 温度为什么会这样变化？
   应当用怎样的偏微分方程描述 $u(x,t)$ 随时间的演化？
2. 只知道方程够不够？
   为了确定一根具体金属杆的温度变化，还需要给出怎样的初始温度和边界条件？
3. 计算机怎样求这个连续问题？
   计算机只能处理有限个数值，因此需要把空间划分成间距为 Δx 的网格，把时间划分成步长为 Δt 的时间层，并用差分近似偏导数。
4. 时间步长可以任意大吗？
   如果一次时间推进跨得太远，离散算法可能不再像真实热传导那样“抹平差异”，反而产生越来越强的数值振荡。因此，显式格式还必须满足一个把 Δt、Δx 与热扩散系数 κ 联系起来的稳定性限制。

本节正沿着这条路线展开：

```
热传导现象
    ↓
温度函数 u(x,t)
    ↓
热方程 + 初始条件 + 边界条件
    ↓
空间网格 Δx + 时间步长 Δt
    ↓
中心差分 + 前向 Euler
    ↓
FTCS 显式格式
    ↓
CFL 时间步长限制
```

下面先解决前两个问题。在 3.2.1 中建立一维热方程及其初边值条件；随后在 3.2.2 中把连续模型放到时空网格上，推导显式有限差分格式，并说明为什么时间步长必须受到 CFL 条件的约束。

3.2.1 一维热方程与初边值条件

导入中的漫画表明，热传导过程会逐渐削弱空间温度差。要定量描述这一现象，需要建立连续的数学模型。本小节从一维常系数热方程出发，依次讨论控制方程、初始条件、边界条件和相容性，并给出一个可供后续数值验证的解析算例。

3.2.1.1 一维热方程的物理意义

为回答导入中“温度为什么会逐渐变平？”这一问题，首先考虑连续模型。暂不涉及数值求解，将实际金属杆理想化为一根长度为 L 的细长均匀导热杆。沿杆长方向取空间坐标 x，用

$$u(x,t)$$

表示位置 x 在时刻 t 的温度。若忽略横截面方向的温度变化，并假设材料均匀、热物性参数不随温度变化、杆内没有额外热源，那么温度的演化可由一维常系数热方程描述：

$$\frac{\partial u}{\partial t} = \kappa \frac{\partial^2 u}{\partial x^2}, \qquad 0 < x < L, \quad t > 0, \tag{3.2.1}$$

其中 $\kappa > 0$ 称为热扩散系数（thermal diffusivity），单位为

$$[\kappa] = \mathrm{m}^2/\mathrm{s}.$$

参数 κ 衡量温度差在材料内部被“摊平”的快慢。相同的初始温度分布下，κ 越大，热量重新分布得越快，温度场趋于均匀的速度也越快。

热方程中出现空间二阶导数并不是偶然的。可以把 u_{xx} 理解为一点的温度相对于邻域平均温度的“弯曲程度”。如果某处温度低于两侧邻域，温度曲线通常在该处向上弯曲，即 $u_{xx} > 0$，于是

$$u_t = \kappa u_{xx} > 0,$$

该处温度会上升；反之，如果某处形成局部温度峰值，则通常 $u_{xx} < 0$，从而 $u_t < 0$，温度开始下降。因此，热方程天然具有一种平滑作用：尖锐的空间起伏不断衰减，温度分布逐渐变得平缓。

这种连续模型中的耗散性质十分重要。后面建立有限差分格式时，我们不仅要求数值格式“近似了微分方程”，还希望离散解保留类似的平滑行为。稳定性条件正是在回答这个问题。

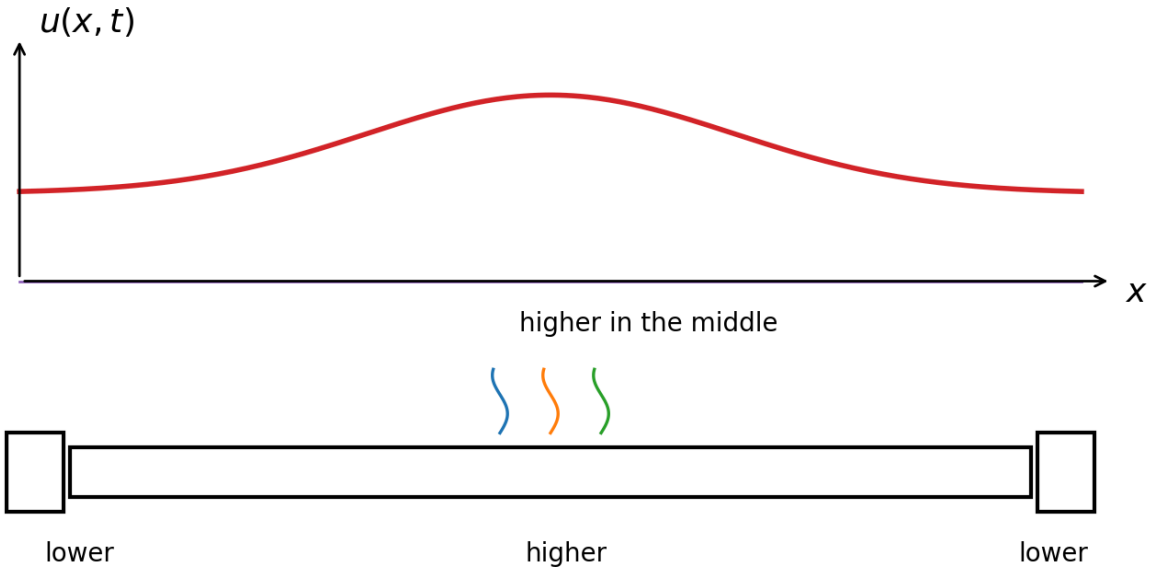


图 3.2-2　一维热传导问题示意图。下方是一根中部较热、两端较冷的金属杆；上方曲线表示同一时刻的温度分布 $u(x,t)$。这个图把“物理对象”和“数学函数”对应起来，有助于理解热方程研究的是温度曲线如何随时间逐渐变平。

3.2.1.2 初始条件与边界条件

方程（3.2.1）说明了杆内部温度如何演化，却没有告诉我们两个关键信息：初始时刻杆内的温度是多少，以及杆的两个端点如何与外界交换热量。

由于热方程对时间是一阶的，需要给出一个初始状态。本节设

$$u(x,0)=u_0(x),\qquad 0\le x\le L, \tag{3.2.2}$$

其中 $u_0(x)$ 是给定的初始温度分布。

同时，由于方程对空间是二阶的，还需要在空间区间的两个端点给出边界条件。为了突出有限差分方法本身，本节采用最简单的齐次 Dirichlet 边界条件

$$u(0,t)=u(L,t)=0,\qquad t>0. \tag{3.2.3}$$

其物理意义是：杆的两端始终与温度为 $0^\circ\mathrm{C}$ 的恒温环境保持理想接触，因此端点温度被固定。这里的“0”主要是为了简化表达；若端点维持在其他恒定温度，经过适当变量平移后，也可转化为类似问题。

于是，本节研究的标准初边值问题为

$$\begin{cases} u_t=\kappa u_{xx}, & 0<x<L,\quad t>0,\\ u(0,t)=u(L,t)=0, & t>0,\\ u(x,0)=u_0(x), & 0\le x\le L. \end{cases} \tag{3.2.4}$$

微分方程、初始条件和边界条件三者共同决定温度随时间的演化。数值计算时也必须同时离散和实现这三部分，而不能只离散方程本身。

3.2.1.3 初边值数据的相容性

若希望解在 $t=0$ 附近保持足够光滑，初始数据还应当与边界数据在两个“角点”

$$(0,0),\qquad (L,0)$$

处一致。对于本节的零 Dirichlet 边界，这意味着

$$u_0(0)=u_0(L)=0. \tag{3.2.5}$$

这称为初边值数据的相容性条件。

例如，当杆长取 $L=1\,\mathrm{m}$ 时，若令 $u_0(x)=1+\sin(\pi x)$，则初值给出

$$u_0(0)=u_0(1)=1,$$

而边界条件要求 $t>0$ 时两个端点立即保持为零。这样会在 $t=0$ 的端点附近形成初始边界层，解的正则性也会受到影响。

需要特别区分：初边值数据不相容与数值格式不稳定不是同一件事。 前者属于连续问题的数据正则性问题；后者则是离散误差是否会随着时间步进被不断放大的问题。本节为避免这两种现象相互干扰，统一假设 u_0 足够光滑并满足（3.2.5）。

3.2.1.4 模型算例及其解析解

取

$$L=1\,\mathrm{m},\qquad \kappa=0.1\,\mathrm{m}^2/\mathrm{s},$$

初边值问题为

$$\begin{cases} u_t = 0.1u_{xx}, & 0 < x < 1, \quad t > 0, \\ u(0,t) = u(1,t) = 0, & t > 0, \\ u(x,0) = \sin(\pi x), & 0 \le x \le 1. \end{cases} \tag{3.2.6}$$

初值 $\sin(\pi x)$ 在两端恰好为零，因此与边界条件相容。它在杆的中央 $x = 1/2$ 处取得最大值，表示初始时刻杆的中部温度最高，而两端保持低温。

这个问题还有一个重要优点：解析解可以直接写出，

$$u(x,t) = \sin(\pi x) e^{-0.1\pi^2 t}. \tag{3.2.7}$$

验证也很直接。由

$$u_{xx} = -\pi^2 \sin(\pi x) e^{-0.1\pi^2 t},$$

以及

$$u_t = -0.1\pi^2 \sin(\pi x) e^{-0.1\pi^2 t},$$

可见确有 $u_t = 0.1u_{xx}$。

式（3.2.7）还把热方程的耗散性质表现得十分清楚：空间形状始终保持为 $\sin(\pi x)$，但振幅按照

$$e^{-0.1\pi^2 t}$$

不断衰减，并最终趋近于零。后面的数值实验将利用这一解析解作为独立参考，从而判断有限差分结果究竟是“稳定但存在离散误差”，还是已经出现了由时间步长选择不当导致的数值不稳定。

3.2.1.5 由连续模型到离散计算

计算机不能直接存储整个连续函数 $u(x,t)$。有限差分方法的基本思路是：只在一组离散的空间点和时间点上近似 u，再用相邻网格点函数值的差商近似偏导数。

因此，接下来需要解决两个问题：

空间二阶导数 u_{xx} 应怎样用相邻网格点近似？时间导数 u_t 又应该怎样推进到下一时间层？

本节采用最基本的一组组合：

空间：二阶中心差分

和

时间：前向 **Euler**

由此得到经典的 FTCS（Forward-Time, Centred-Space）显式格式。需要强调的是：一个差分公式即使在 $\Delta x, \Delta t \to 0$ 时能够逼近微分方程，也未必能够稳定地进行长时间计算。对于热方程，空间网格与时间步长之间必须满足特定比例关系，这就是本节所讨论的扩散型 CFL 限制。

至此，导入中的第一个问题已经明确：我们已经有了连续的控制方程、初始条件和边界条件。下面转向后两个问题——如何把连续模型变成计算机能够执行的递推公式，以及为什么时间步长不能任意取值。 这两件事将分别通过差分近似和稳定性分析来回答。

3.2.2 空间差分与显式时间推进

3.2.1 已经用热方程描述温度演化，并通过初始条件和边界条件确定了具体的初边值问题。现在进一步回答导入漫画提出的计算问题：怎样将连续温度场转化为计算机能够处理的有限个数值？为此，先建立时空网格，再分别离散 u_{xx} 与 u_t，进而构造 FTCS 显式格式，并讨论时间步长受到 CFL 条件约束的原因。

3.2.2.1 均匀时空网格及离散记号

在空间区间 $[0, L]$ 上取均匀网格

$$x_j = j\Delta x, \qquad j = 0, 1, \dots, J,$$

其中

$$\Delta x = \frac{L}{J}.$$

时间方向同样取等距网格

$$t_n = n\Delta t, \qquad n = 0, 1, 2, \dots.$$

记

$$U_j^n \approx u(x_j, t_n)$$

为精确温度 $u(x_j, t_n)$ 的数值近似。下文约定：小写 u 表示连续问题的精确解，大写 U 表示离散数值解；j 和 n 分别为空间节点下标与时间层下标。

对于零 Dirichlet 边界，网格端点的数值不需要通过差分方程计算，而是直接由边界条件给出：

$$U_0^n = U_J^n = 0. \tag{3.2.8}$$

初始层则由初值采样得到

$$U_j^0 = u_0(x_j). \tag{3.2.9}$$

因此，真正需要递推计算的是内部节点

$$j = 1, 2, \dots, J-1.$$

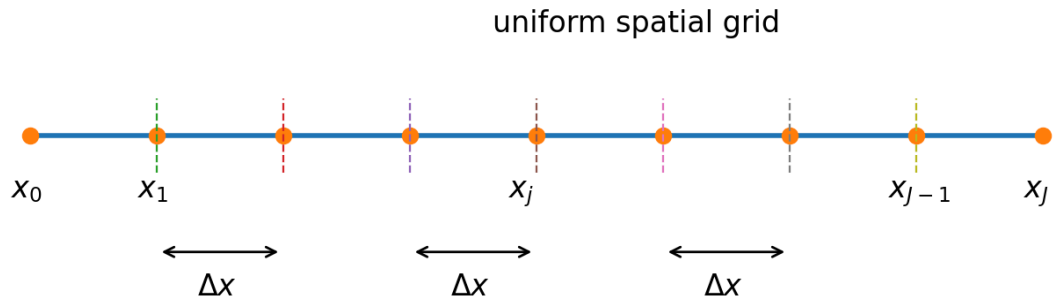


图 3.2-3　一维均匀空间网格示意图。连续区间 $[0, L]$ 被离散为节点 $x_0, x_1, \dots, x_J$，相邻节点之间的距离都等于 Δx。有限差分方法正是在这些节点上存储和更新温度近似值。

3.2.2.2 二阶中心空间差分

热方程右端包含空间二阶偏导数 u_{xx}。有限差分的思路是：不直接计算连续函数的导数，而是使用相邻节点的温度近似这一导数。下面从 Taylor 公式出发，逐步推导二阶中心差分，并确定它的误差阶数。

在固定时间 t_n 上，仍用 Δx 表示均匀空间网格间距。为使书写简洁，以下暂用 $u, u_x, u_{xx}, \dots$ 表示它们在同一展开中心 (x_j, t_n) 处的取值。假设精确解关于空间足够光滑。

（1）向右展开。 因为 $x_{j+1} = x_j + \Delta x$，根据 Taylor 公式，

$$u(x_{j+1}, t_n) = u + \Delta x u_x + \frac{(\Delta x)^2}{2} u_{xx} + \frac{(\Delta x)^3}{6} u_{xxx} + \frac{(\Delta x)^4}{24} u_{xxxx} + \frac{(\Delta x)^5}{120} u_{xxxxx} + \mathcal{O}((\Delta x)^6).$$

（2）向左展开。 因为 $x_{j-1} = x_j - \Delta x$，只需将上式中的 Δx 替换成 $-\Delta x$：

$$u(x_{j-1}, t_n) = u - \Delta x u_x + \frac{(\Delta x)^2}{2} u_{xx} - \frac{(\Delta x)^3}{6} u_{xxx} + \frac{(\Delta x)^4}{24} u_{xxxx} - \frac{(\Delta x)^5}{120} u_{xxxxx} + \mathcal{O}((\Delta x)^6).$$

（3）将左右两式相加。 一阶、三阶和五阶导数项符号相反，因而相互抵消：

$$u(x_{j+1}, t_n) + u(x_{j-1}, t_n) = 2u + (\Delta x)^2 u_{xx} + \frac{(\Delta x)^4}{12} u_{xxxx} + \mathcal{O}((\Delta x)^6).$$

（4）移项并除以 $(\Delta x)^2$。 从两端减去 $2u = 2u(x_j, t_n)$，得到

$$u(x_{j+1}, t_n) - 2u(x_j, t_n) + u(x_{j-1}, t_n) = (\Delta x)^2 u_{xx} + \frac{(\Delta x)^4}{12} u_{xxxx} + \mathcal{O}((\Delta x)^6).$$

因此，

$$\frac{u(x_{j+1}, t_n) - 2u(x_j, t_n) + u(x_{j-1}, t_n)}{(\Delta x)^2} = u_{xx}(x_j, t_n) + \frac{(\Delta x)^2}{12} u_{xxxx}(x_j, t_n) + \mathcal{O}((\Delta x)^4).$$

将 u_{xx} 单独放在左侧，得到

$$u_{xx}(x_j, t_n) = \frac{u(x_{j+1}, t_n) - 2u(x_j, t_n) + u(x_{j-1}, t_n)}{(\Delta x)^2} - \frac{(\Delta x)^2}{12} u_{xxxx}(x_j, t_n) + \mathcal{O}((\Delta x)^4). \tag{3.2.10}$$

式（3.2.10）表明：二阶中心差分的误差主项与 $(\Delta x)^2$ 成正比。若忽略余项，即用

$$D_{xx} u(x_j, t_n) := \frac{u(x_{j+1}, t_n) - 2u(x_j, t_n) + u(x_{j-1}, t_n)}{(\Delta x)^2}$$

近似 $u_{xx}(x_j, t_n)$，则其空间精度为二阶。这里的二阶精度来自左右取点的对称性，而不是仅仅因为差分公式使用了三个节点。

在实际计算中，用未知的精确温度 $u(x_j, t_n)$ 换成可存储的近似值 U_j^n，便得到用于后续递推的空间离散算子

$$D_{xx} U_j^n = \frac{U_{j+1}^n - 2U_j^n + U_{j-1}^n}{(\Delta x)^2}.$$

3.2.2.3 前向 Euler 时间离散

热方程左端是时间一阶导数 u_t。与空间差分的推导类似，在固定空间位置 x_j 的条件下，对时间作 Taylor 展开；不同的是，显式时间推进只使用已知的当前时间层 t_n 来计算下一层 t_{n+1}。

（1）在 t_n 附近向前展开。 由 $t_{n+1} = t_n + \Delta t$，

$$u(x_j, t_{n+1}) = u(x_j, t_n) + \Delta t\, u_t(x_j, t_n) + \frac{(\Delta t)^2}{2} u_{tt}(x_j, t_n) + \frac{(\Delta t)^3}{6} u_{ttt}(x_j, t_n) + \mathcal{O}((\Delta t)^4).$$

（2）减去当前时刻的温度，再除以 Δt。 于是

$$\frac{u(x_j, t_{n+1}) - u(x_j, t_n)}{\Delta t} = u_t(x_j, t_n) + \frac{\Delta t}{2} u_{tt}(x_j, t_n) + \frac{(\Delta t)^2}{6} u_{ttt}(x_j, t_n) + \mathcal{O}((\Delta t)^3).$$

（3）解出时间导数并识别误差主项。 得到

$$u_t(x_j, t_n) = \frac{u(x_j, t_{n+1}) - u(x_j, t_n)}{\Delta t} - \frac{\Delta t}{2} u_{tt}(x_j, t_n) + \mathcal{O}((\Delta t)^2). \tag{3.2.11}$$

因此，前向时间差分

$$D_t^+ u(x_j, t_n) := \frac{u(x_j, t_{n+1}) - u(x_j, t_n)}{\Delta t}$$

对 u_t 具有一阶时间精度。式 (3.2.11) 同时提醒我们，时间差分与空间中心差分的精度不同：前者的主误差是 $\mathcal{O}(\Delta t)$，后者的主误差是 $\mathcal{O}((\Delta x)^2)$。

3.2.2.4 FTCS 显式差分格式

现在把两种差分近似放回热方程 $u_t = \kappa u_{xx}$。在内部网格点 (x_j, t_n) 处，$j = 1, \dots, J-1$，依次完成以下步骤。

（1）用差商代替偏导数。 使用 3.2.2.3 中的前向时间差分近似 u_t，并使用 3.2.2.2 中的中心空间差分近似 u_{xx}。将网格点上的精确函数值替换为数值近似值 U_j^n，忽略截断误差，得

$$\frac{U_j^{n+1} - U_j^n}{\Delta t} = \kappa \frac{U_{j+1}^n - 2U_j^n + U_{j-1}^n}{(\Delta x)^2}, \qquad j = 1, \dots, J-1. \tag{3.2.12}$$

上式左端含有下一时间层的一个未知量 U_j^{n+1}，右端仅含当前时间层的三个数值。

（2）两端乘以 Δt。 得到

$$U_j^{n+1} - U_j^n = \frac{\kappa \Delta t}{(\Delta x)^2} \left(U_{j+1}^n - 2U_j^n + U_{j-1}^n \right).$$

（3）将 U_j^n 移至右端。 于是

$$U_j^{n+1} = U_j^n + \frac{\kappa \Delta t}{(\Delta x)^2} \left(U_{j+1}^n - 2U_j^n + U_{j-1}^n \right). \tag{3.2.13}$$

（4）引入无量纲参数并合并同类项。 记

$$r = \frac{\kappa \Delta t}{(\Delta x)^2}. \tag{3.2.14}$$

则式 (3.2.13) 的右端可依次整理为

$$
\begin{aligned}
U_j^{n+1} &= U_j^n + r\left(U_{j+1}^n - 2U_j^n + U_{j-1}^n\right) \\
&= U_j^n + rU_{j+1}^n - 2rU_j^n + rU_{j-1}^n \\
&= rU_{j-1}^n + (1-2r)U_j^n + rU_{j+1}^n.
\end{aligned}
$$

因此，最终得到 FTCS（Forward-Time, Centred-Space）显式格式

$$
U_j^{n+1} = rU_{j-1}^n + (1-2r)U_j^n + rU_{j+1}^n, \qquad j = 1, \ldots, J-1. \tag{3.2.15}
$$

这里“显式”的含义是：给定第 n 层数据后，不需要求解新的联立方程组，便能直接计算第 $n+1$ 层的每个内部节点。算法的完整规定还包括初始层 $U_j^0 = u_0(x_j)$ 和每层边界值 $U_0^n = U_J^n = 0$；缺少初值或边界赋值，单靠式 (3.2.15) 不能形成完整的计算程序。

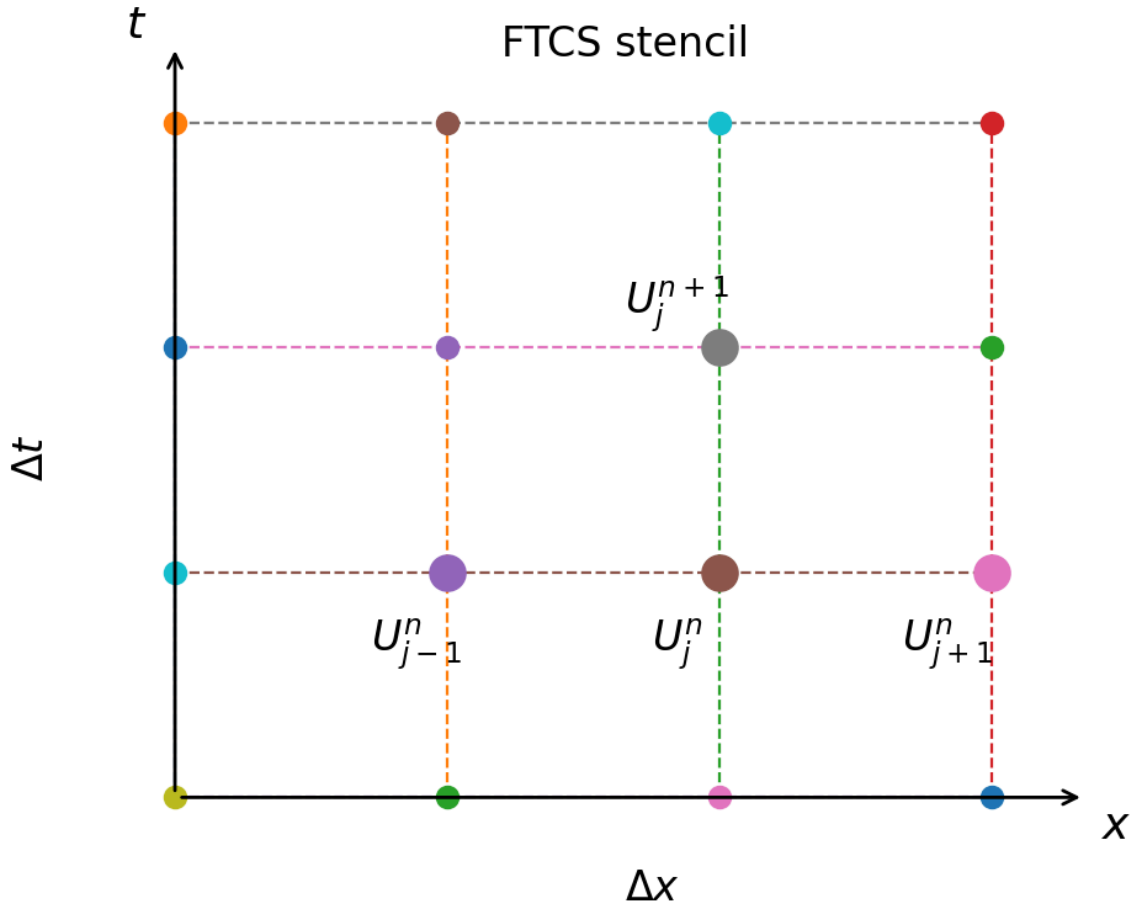


图 3.2-4　FTCS 显式格式的四点模板。计算新的温度值 U_j^{n+1} 时，只需要上一时间层的三个相邻节点 U_{j-1}^n、U_j^n 和 U_{j+1}^n。图中清楚展示了“空间中心差分 + 时间前向推进”的结构。

图 3.2-4 中，新时间层的一个节点与旧时间层三个相邻节点相连，正是式 (3.2.15) 的几何表示。这也解释了为什么显式格式虽然更新方便，仍必须对时间步长加以限制：每一步都在重复使用上一层的近似值，误差也会随之进入下一次计算。

3.2.2.5 无量纲扩散参数 r

参数

$$r = \frac{\kappa \Delta t}{(\Delta x)^2}$$

同时包含材料性质、空间网格和时间步长，是决定显式格式行为的核心参数。在传热计算中，它也常被称为网格尺度上的 Fourier 数。

由于扩散过程在时间 Δt 内对应的典型扩散长度为

$$\ell_d \sim \sqrt{\kappa \Delta t},$$

于是

$$r = \left(\frac{\sqrt{\kappa \Delta t}}{\Delta x} \right)^2.$$

因此，r 可以理解为“一个时间步内的扩散尺度相对于空间网格尺度的平方”。

若 Δt 太大，则一次时间推进试图让温度信息跨越过多空间网格，而显式格式只利用当前位置及左右各一个邻点的信息。在这种情况下，离散递推可能无法保持真实热扩散所具有的耗散性质，于是稳定性限制自然出现。

3.2.2.6 局部截断误差与相容性

建立差分格式以后，还必须回答一个基本问题：当网格逐渐加密时，差分方程能否正确逼近原来的热方程？ 为此，我们研究局部截断误差。下面不仅给出误差的阶数，还推导它的具体主项，并说明误差如何进入每一步的数值更新。

（1）定义局部截断误差。 在内部节点 (x_j, t_n)，把精确解 u（而不是数值近似 U）代入 FTCS 差分方程左、右两端，定义

$$\tau_j^n = \frac{u(x_j, t_{n+1}) - u(x_j, t_n)}{\Delta t} - \kappa \frac{u(x_{j+1}, t_n) - 2u(x_j, t_n) + u(x_{j-1}, t_n)}{(\Delta x)^2}. \tag{3.2.16}$$

若差分公式能够完全重现微分方程，那么 τ_j^n 应等于零。实际计算时 τ_j^n 一般不为零，它量化了差分算子作用于精确解时产生的残差。本书采用除以 Δt 之后的残差作为局部截断误差；相应的单步更新缺陷是 $\Delta t\, \tau_j^n$。

为获得具体误差主项，暂假定 u 在所讨论的时空区域内具有足够多的连续导数，使以下 Taylor 展开及余项估计成立。以下所有未注明取值点的导数均在 (x_j, t_n) 处计算。

（2）计算时间差分产生的误差。 由 3.2.2.3 的 Taylor 展开，

$$\begin{aligned} \frac{u(x_j, t_{n+1}) - u(x_j, t_n)}{\Delta t} &= \frac{\Delta t\, u_t + \frac{(\Delta t)^2}{2} u_{tt} + \frac{(\Delta t)^3}{6} u_{ttt} + \mathcal{O}((\Delta t)^4)}{\Delta t} \\ &= u_t + \frac{\Delta t}{2} u_{tt} + \frac{(\Delta t)^2}{6} u_{ttt} + \mathcal{O}((\Delta t)^3). \end{aligned}$$

因此，前向时间差分相对于精确时间导数的误差为

$$\underbrace{\frac{u(x_j, t_{n+1}) - u(x_j, t_n)}{\Delta t} - u_t}_{\text{时间离散误差}} = \frac{\Delta t}{2} u_{tt} + \frac{(\Delta t)^2}{6} u_{ttt} + \mathcal{O}((\Delta t)^3).$$

它的首项与 Δt 成正比。

（3）计算空间差分产生的误差。 由 3.2.2.2 的左右对称 Taylor 展开，

$$
\begin{aligned}
&u(x_{j+1},t_n)-2u(x_j,t_n)+u(x_{j-1},t_n)\\
&\quad=(\Delta x)^2u_{xx}+\frac{(\Delta x)^4}{12}u_{xxxx}+\mathcal{O}((\Delta x)^6).
\end{aligned}
$$

两端除以 $(\Delta x)^2$，得到

$$
\begin{aligned}
&\frac{u(x_{j+1},t_n)-2u(x_j,t_n)+u(x_{j-1},t_n)}{(\Delta x)^2}\\
&\quad=u_{xx}+\frac{(\Delta x)^2}{12}u_{xxxx}+\mathcal{O}((\Delta x)^4).
\end{aligned}
$$

所以，空间中心差分相对于精确二阶空间导数的误差为

$$
\underbrace{\frac{u(x_{j+1},t_n)-2u(x_j,t_n)+u(x_{j-1},t_n)}{(\Delta x)^2}-u_{xx}}_{\text{空间离散误差}}=\frac{(\Delta x)^2}{12}u_{xxxx}+\mathcal{O}((\Delta x)^4).
$$

它的首项与 $(\Delta x)^2$ 成正比。

（4）代回残差定义并利用控制方程。 将步骤（2）、（3）的表达式逐项代入式 (3.2.16)，注意空间项前面带有系数 $-\kappa$：

$$
\begin{aligned}
\tau_j^n&=\left[u_t+\frac{\Delta t}{2}u_{tt}+\frac{(\Delta t)^2}{6}u_{ttt}+\mathcal{O}((\Delta t)^3)\right]\\
&\quad-\kappa\left[u_{xx}+\frac{(\Delta x)^2}{12}u_{xxxx}+\mathcal{O}((\Delta x)^4)\right]\\
&=\underbrace{(u_t-\kappa u_{xx})}_{=0}\\
&\quad+\frac{\Delta t}{2}u_{tt}-\frac{\kappa(\Delta x)^2}{12}u_{xxxx}+\frac{(\Delta t)^2}{6}u_{ttt}+\mathcal{O}((\Delta t)^3+(\Delta x)^4).
\end{aligned}
$$

因为精确解满足 $u_t=\kappa u_{xx}$，第一项严格等于零。因此，

$$
\tau_j^n=\frac{\Delta t}{2}u_{tt}(x_j,t_n)-\frac{\kappa(\Delta x)^2}{12}u_{xxxx}(x_j,t_n)+\mathcal{O}((\Delta t)^2+(\Delta x)^4).
$$

若进一步利用 κ 为常数 这一假设，则

$$
\begin{aligned}
u_{tt}&=\frac{\partial}{\partial t}(u_t)=\frac{\partial}{\partial t}(\kappa u_{xx})\\
&=\kappa\frac{\partial^2}{\partial x^2}(u_t)=\kappa\frac{\partial^2}{\partial x^2}(\kappa u_{xx})=\kappa^2u_{xxxx}.
\end{aligned}
$$

代入上一式后，局部截断误差可以写成

$$
\begin{aligned}
\tau_j^n&=\left(\frac{\kappa^2\Delta t}{2}-\frac{\kappa(\Delta x)^2}{12}\right)u_{xxxx}(x_j,t_n)+\mathcal{O}((\Delta t)^2+(\Delta x)^4)\\
&=\mathcal{O}(\Delta t+(\Delta x)^2).
\end{aligned}
\tag{3.2.17}
$$

这一结果给出了 FTCS 格式“时间一阶、空间二阶”的准确含义：将精确解代入离散方程后，时间离散误差的主量级为 Δt，空间离散误差的主量级为 $(\Delta x)^2$。当 $\Delta t\to0$、$\Delta x\to0$ 时，$\tau_j^n\to0$，因此该格式与热方程相容。

（5）给出不依赖高阶渐近展开的误差上界。 为了后面严格证明收敛性，考虑固定终止时间 $T>0$，并假设 u_{tt} 与 u_{xxxx} 在闭区域 $[0,L]\times[0,T]$ 上连续。记

$$
M_t=\max_{[0,L]\times[0,T]}|u_{tt}|,\qquad M_x=\max_{[0,L]\times[0,T]}|u_{xxxx}|.
$$

按带余项的 Taylor 定理，存在 $\eta\in(t_n,t_{n+1})$，使

$$\frac{u(x_j,t_{n+1})-u(x_j,t_n)}{\Delta t}-u_t(x_j,t_n)=\frac{\Delta t}{2}u_{tt}(x_j,\eta).$$

同样，在 x_{j-1} 与 x_{j+1} 两侧分别作到三阶的 Taylor 展开，其四阶余项表明存在 $\xi_-\in(x_{j-1},x_j)$、$\xi_+\in(x_j,x_{j+1})$，使

$$\begin{aligned}&\frac{u(x_{j+1},t_n)-2u(x_j,t_n)+u(x_{j-1},t_n)}{(\Delta x)^2}-u_{xx}(x_j,t_n)\\&=\frac{(\Delta x)^2}{24}\left[u_{xxxx}(\xi_-,t_n)+u_{xxxx}(\xi_+,t_n)\right].\end{aligned}$$

将上述两式代入 (3.2.16)，再用三角不等式，便得到适用于所有内部节点及 $t_{n+1}\le T$ 的统一上界

$$|\tau_j^n|\le\frac{\Delta t}{2}M_t+\frac{\kappa(\Delta x)^2}{12}M_x.$$

该估计只要求二阶时间导数和四阶空间导数有界，并不需要假设高阶余项的具体展开。它也说明式 (3.2.17) 的大 $\mathcal{O}$ 常数可以在给定时间区间内不依赖网格参数地控制。

（6）区别局部残差与单步更新误差。 将 (3.2.16) 两边乘以 Δt，并把精确解的下一时间层单独写在左端，有

$$\begin{aligned}u(x_j,t_{n+1})=u(x_j,t_n)&\\+r\left[u(x_{j+1},t_n)-2u(x_j,t_n)+u(x_{j-1},t_n)\right]&\\+\Delta t\,\tau_j^n.&\end{aligned}$$

因此，一次时间更新造成的缺陷 为 $\Delta t\,\tau_j^n=\mathcal{O}((\Delta t)^2+\Delta t(\Delta x)^2)$，而式 (3.2.16) 中按 Δt 归一化后的局部截断误差是 $\mathcal{O}(\Delta t+(\Delta x)^2)$。两者不能混淆。

至此，我们只证明了“差分算子在局部逼近热方程”。这些小误差在许多时间步中是否会积累成大误差，还取决于递推格式的稳定性。下一小节将用定理和证明给出严格结论。

3.2.2.7 CFL 稳定性条件与收敛性

上一小节证明 FTCS 格式与热方程相容，但相容性只研究一次局部差分与微分方程的接近程度。数值求解需要重复推进许多时间步；如果某种误差在递推中被不断放大，算法即使相容，也不一定能得到可信的结果。下面首先给出稳定性的严格定理，然后用 Fourier 模态解释 CFL 限制的来源。

回顾式 (3.2.15)，对每个内部节点有

$$U_j^{n+1}=rU_{j-1}^n+(1-2r)U_j^n+rU_{j+1}^n,\qquad r=\frac{\kappa\Delta t}{(\Delta x)^2}.\tag{3.2.18}$$

注意右端的三个系数分别为 r、$1-2r$、r。若希望新温度是上一时间层相邻温度的非负加权平均，就必须要求

$$r\ge0,\qquad 1-2r\ge0,\qquad r+r+(1-2r)=1.$$

因为 $\kappa>0$、$\Delta t>0$、$\Delta x>0$，所以 $r>0$；上面的非负性要求只剩下 $r\le1/2$。因此，当前 FTCS 格式的标准扩散稳定性限制为

$$0\le r=\frac{\kappa\Delta t}{(\Delta x)^2}\le\frac{1}{2}.\tag{3.2.19}$$

移项后，可直接得到可供计算时使用的时间步长上界：

$$\Delta t\le\frac{(\Delta x)^2}{2\kappa}.\tag{3.2.20}$$

这两个等价表达式就是本节所称的扩散型 CFL 条件。首先证明它为什么能够确保数值解不发生非物理放大。

定理 3.2.1（FTCS 格式的离散最大值原理、最大范数稳定性与收敛性）

设 $L>0$、$\kappa>0$，在空间区间 $[0,L]$ 上取 $J\geq 2$ 个等长小区间，网格间距为 $\Delta x=L/J$；时间步长为 $\Delta t>0$。在内部节点采用式 (3.2.18) 的 FTCS 格式，边界节点满足 $U_0^n=U_J^n=0$。若 $0\leq r\leq 1/2$，则有：

（1）离散最大值原理与稳定性。 对任意初始数据及所有整数 $n\geq 0$，

$$\min_{0\leq j\leq J} U_j^n \leq U_i^{n+1} \leq \max_{0\leq j\leq J} U_j^n, \qquad 0\leq i\leq J.$$

特别地，若定义离散最大范数

$$\|U^n\|_\infty := \max_{0\leq j\leq J}|U_j^n|,$$

则

$$\|U^{n+1}\|_\infty \leq \|U^n\|_\infty \leq \|U^0\|_\infty. \tag{3.2.21}$$

并且，对同一差分格式、同一零边界条件下的任意两组数值解 U^n, V^n，有

$$\|U^n - V^n\|_\infty \leq \|U^0 - V^0\|_\infty.$$

这表明初始数据的扰动不会被时间推进放大，因而格式在最大范数意义下稳定。

（2）收敛性及误差估计。 设精确解 $u(x,t)$ 满足热方程及零 Dirichlet 边界条件，并且在 $[0,L]\times[0,T]$ 上具有连续且有界的 u_{tt}、u_{xxxx}；假设初值按 $U_j^0=u(x_j,0)$ 精确采样。记

$$u^n := \big(u(x_0,t_n), u(x_1,t_n), \dots, u(x_J,t_n)\big),$$

以及 3.2.2.6 中定义的导数上界 M_t、M_x。则对每个 $0\leq t_n=n\Delta t\leq T$，

$$\|u^n - U^n\|_\infty \leq t_n\left(\frac{\Delta t}{2}M_t + \frac{\kappa(\Delta x)^2}{12}M_x\right) \leq C_T\big(\Delta t + (\Delta x)^2\big), \tag{3.2.22}$$

其中 C_T 可依赖于 T,κ 及精确解的相应导数上界，但与 $\Delta x,\Delta t$ 无关。因此，在 CFL 条件成立并且网格加密时，数值解以时间一阶、空间二阶收敛到精确解在网格上的取值。

证明

（1）离散最大值原理。 固定时间层 n 和任意内部节点 j。由式 (3.2.18)，

$$U_j^{n+1} = rU_{j-1}^n + (1-2r)U_j^n + rU_{j+1}^n.$$

在 $0\leq r\leq 1/2$ 时，三个系数非负，而且它们的和等于 1，所以右端是三个旧温度值的凸组合。设

$$m_n = \min_{0\leq i\leq J} U_i^n, \qquad M_n = \max_{0\leq i\leq J} U_i^n.$$

由于 $m_n\leq U_{j-1}^n, U_j^n, U_{j+1}^n\leq M_n$，将三个不等式分别乘以非负系数并相加，得到

$$\begin{aligned} U_j^{n+1} &\geq r\,m_n + (1-2r)m_n + r\,m_n = m_n,\\ U_j^{n+1} &\leq r\,M_n + (1-2r)M_n + r\,M_n = M_n. \end{aligned}$$

对于边界节点 $j=0$ 和 $j=J$，由于 $U_0^{n+1}=U_J^{n+1}=0$，且上一时间层也有零边界值，所以 $m_n\le 0\le M_n$，同样满足上述范围限制。由此得证离散最大值原理。

（2）最大范数稳定性。 对任意内部节点使用三角不等式：

$$\begin{aligned}|U_j^{n+1}| &\le r|U_{j-1}^n|+(1-2r)|U_j^n|+r|U_{j+1}^n|\\ &\le \big[r+(1-2r)+r\big]\|U^n\|_\infty\\ &= \|U^n\|_\infty.\end{aligned}$$

边界值为零，因此对所有节点取最大值可得 $\|U^{n+1}\|_\infty\le\|U^n\|_\infty$。反复应用该不等式，得到式 (3.2.21)。

为说明小扰动也不会增长，取同一格式的两组解，记 $W_j^n=U_j^n-V_j^n$。由于格式线性，两个递推式相减后仍有

$$W_j^{n+1}=rW_{j-1}^n+(1-2r)W_j^n+rW_{j+1}^n.$$

而 $W_0^n=W_J^n=0$。把刚证明的最大范数估计应用于 W^n，得到

$$\|U^n-V^n\|_\infty=\|W^n\|_\infty\le\|W^0\|_\infty=\|U^0-V^0\|_\infty.$$

这便证明了稳定性。特别地，若初始温度非负，则凸组合及零边界还保证全部后续数值温度非负。

（3）收敛性估计。 将精确解在网格上的取值记为 $u_j^n=u(x_j,t_n)$，并定义误差 $e_j^n=u_j^n-U_j^n$。由局部截断误差定义式 (3.2.16)，精确解满足含残差的离散关系

$$u_j^{n+1}=u_j^n+r(u_{j+1}^n-2u_j^n+u_{j-1}^n)+\Delta t\,\tau_j^n.$$

从这条关系中减去 FTCS 数值递推式 (3.2.18)，按 $e_j^n=u_j^n-U_j^n$ 合并项，得到误差递推方程

$$e_j^{n+1}=re_{j-1}^n+(1-2r)e_j^n+re_{j+1}^n+\Delta t\,\tau_j^n. \tag{3.2.23}$$

这里与前面的齐次递推相比，多出了局部截断误差驱动项 $\Delta t\,\tau_j^n$。由于数值初值精确采样，$e_j^0=0$；数值边界与精确边界一致，故对所有 n 均有 $e_0^n=e_J^n=0$。

记

$$E_n:=\max_{0\le j\le J}|e_j^n|,\qquad \Theta_n:=\max_{1\le j\le J-1}|\tau_j^n|.$$

对式 (3.2.23) 两端取绝对值，再利用 $r\ge 0$、$1-2r\ge 0$ 与三角不等式，依次得到

$$\begin{aligned}|e_j^{n+1}| &\le r|e_{j-1}^n|+(1-2r)|e_j^n|+r|e_{j+1}^n|+\Delta t|\tau_j^n|\\ &\le \big[r+(1-2r)+r\big]E_n+\Delta t\,\Theta_n\\ &= E_n+\Delta t\,\Theta_n.\end{aligned}$$

对节点取最大值可得 $E_{n+1}\le E_n+\Delta t\,\Theta_n$。从第 0 层依次递推到第 n 层，且使用 $E_0=0$，得到

$$\begin{aligned}E_n &\le E_0+\Delta t\sum_{m=0}^{n-1}\Theta_m\\ &= \Delta t\sum_{m=0}^{n-1}\Theta_m\\ &\le n\Delta t\max_{0\le m\le n-1}\Theta_m\\ &= t_n\max_{0\le m\le n-1}\Theta_m.\end{aligned}$$

再代入 3.2.2.6 已经推导的统一截断误差上界

$$\Theta_m \le \frac{\Delta t}{2} M_t + \frac{\kappa (\Delta x)^2}{12} M_x,$$

便有

$$E_n \le t_n \left(\frac{\Delta t}{2} M_t + \frac{\kappa (\Delta x)^2}{12} M_x \right).$$

由于 $t_n \le T$，若取 $C_T = T \max\{M_t/2, \kappa M_x/12\}$，则

$$E_n \le C_T \big(\Delta t + (\Delta x)^2 \big),$$

这正是式 (3.2.22)。定理得证。$\square$

上面的证明说明：稳定性控制了已有误差的传播，截断误差决定每一步新引入多少误差。 两者合在一起，才能保证全局误差随着网格加密而趋于零。这里直接建立了误差上界，不需要仅凭“相容性加稳定性意味着收敛”作口头推断。

定理 3.2.2（FTCS 的 Fourier 模态稳定性判据）

对于一维均匀无限网格上的 FTCS 齐次误差递推，在 $\kappa > 0$、$\Delta t > 0$ 下，所有可分辨的 Fourier 模态均不随时间步进放大的充要条件为 $0 < r \le 1/2$。若 $r > 1/2$，则存在逐步增长的高频误差模态。

证明

第一步：写出齐次误差递推。 取同一个 FTCS 格式所得的两组数值解之差 w_j^n，它满足

$$w_j^{n+1} = w_j^n + r(w_{j+1}^n - 2w_j^n + w_{j-1}^n).$$

这是线性常系数递推，可以考察一个空间频率单独演化的情形。设

$$w_j^n = A_n e^{\mathrm{i} j\theta}, \qquad -\pi \le \theta \le \pi,$$

其中 θ 为无量纲波数，$\mathrm{i}^2 = -1$，A_n 表示该频率在第 n 个时间层的振幅。

第二步：将 Fourier 模态代入递推式。 注意相邻节点分别满足

$$w_{j+1}^n = A_n e^{\mathrm{i} j\theta} e^{\mathrm{i}\theta}, \qquad w_{j-1}^n = A_n e^{\mathrm{i} j\theta} e^{-\mathrm{i}\theta}.$$

因此，

$$\begin{aligned} A_{n+1} e^{\mathrm{i} j\theta} = {} & A_n e^{\mathrm{i} j\theta} \\ & + r A_n e^{\mathrm{i} j\theta} \left(e^{\mathrm{i}\theta} - 2 + e^{-\mathrm{i}\theta} \right). \end{aligned}$$

消去公共的非零因子 $e^{\mathrm{i} j\theta}$，即可得到振幅的递推关系

$$A_{n+1} = G(\theta) A_n, \qquad G(\theta) = 1 + r(e^{\mathrm{i}\theta} - 2 + e^{-\mathrm{i}\theta}).$$

其中 $G(\theta)$ 称为放大因子。

第三步：化简放大因子。 利用恒等式

$$e^{\mathrm{i}\theta} + e^{-\mathrm{i}\theta} = 2\cos\theta, \qquad 1 - \cos\theta = 2\sin^2\frac{\theta}{2},$$

可逐步整理：

$$
\begin{aligned}
G(\theta) &= 1 + r(2\cos\theta - 2) \\
&= 1 - 2r(1 - \cos\theta) \\
&= 1 - 4r\sin^2\frac{\theta}{2}.
\end{aligned}
$$

即

$$
G(\theta) = 1 - 4r\sin^2\left(\frac{\theta}{2}\right). \tag{3.2.24}
$$

第四步：要求所有频率的振幅不增长。 由 $A_n = G(\theta)^n A_0$ 可知，模态不随时间增长等价于 $|G(\theta)| \le 1$。因 $G(\theta)$ 是实数，这又等价于

$$
-1 \le 1 - 4r\sin^2\left(\frac{\theta}{2}\right) \le 1.
$$

对 $r > 0$ 而言，右侧不等式自动成立。左侧不等式等价于

$$
4r\sin^2\left(\frac{\theta}{2}\right) \le 2.
$$

由于 $0 \le \sin^2(\theta/2) \le 1$，当 $\theta = \pi$ 时该值达到 1，于是上式对所有波数成立的充要条件是

$$
4r \le 2 \quad\Longleftrightarrow\quad r \le \frac{1}{2}.
$$

反过来，如果 $r > 1/2$，取 $\theta = \pi$，就有

$$
G(\pi) = 1 - 4r < -1, \qquad |G(\pi)| > 1.
$$

对应的网格模态为 $e^{ij\pi} = (-1)^j$，在相邻网格点上正负交替。它的振幅逐步乘以 $|G(\pi)| > 1$，因而发生增长。充分性与必要性均已证明。□

严格性说明：有限零 Dirichlet 网格与无限网格的区别。 本节实际计算的区间为 $[0, L]$，且具有零 Dirichlet 边界。其空间特征模态是

$$
\phi_j^{(m)} = \sin\left(\frac{m\pi j}{J}\right), \qquad m = 1, 2, \dots, J-1.
$$

利用正弦加法公式可得

$$
\phi_{j-1}^{(m)} + \phi_{j+1}^{(m)} = 2\cos\left(\frac{m\pi}{J}\right)\phi_j^{(m)}.
$$

将其代入齐次递推，得到第 m 个模态的放大因子

$$
G_m = 1 - 2r + 2r\cos\left(\frac{m\pi}{J}\right) = 1 - 4r\sin^2\left(\frac{m\pi}{2J}\right).
$$

因为 $m = J-1$ 时 $\sin^2(m\pi/(2J)) = \cos^2(\pi/(2J))$ 最大，所以对固定的有限 J，全部离散正弦模态均不增长的精确条件为

$$
r \le \frac{1}{2\cos^2\left(\frac{\pi}{2J}\right)}. \tag{3.2.25}
$$

当 J 固定时，这个谱界可略大于 $1/2$；然而随着网格加密 $J \to \infty$，右端趋于 $1/2$。因此，$r \le 1/2$ 是保证离散最大值原理、最大范数稳定性且与网格点数无关的标准 CFL 限制。不能不加说明地把它称为每一个固定有限 Dirichlet 网格的精确谱稳定性充要条件。

时间步长限制的实际意义。 由式 (3.2.20) 可知，稳定时间步长的上限与空间网格间距的平方成正比。例如把 Δx 减小为原来的 $1/2$，在扩散系数 κ 不变时，为保持同样的 r，就必须把 Δt 减小为原来的 $1/4$。

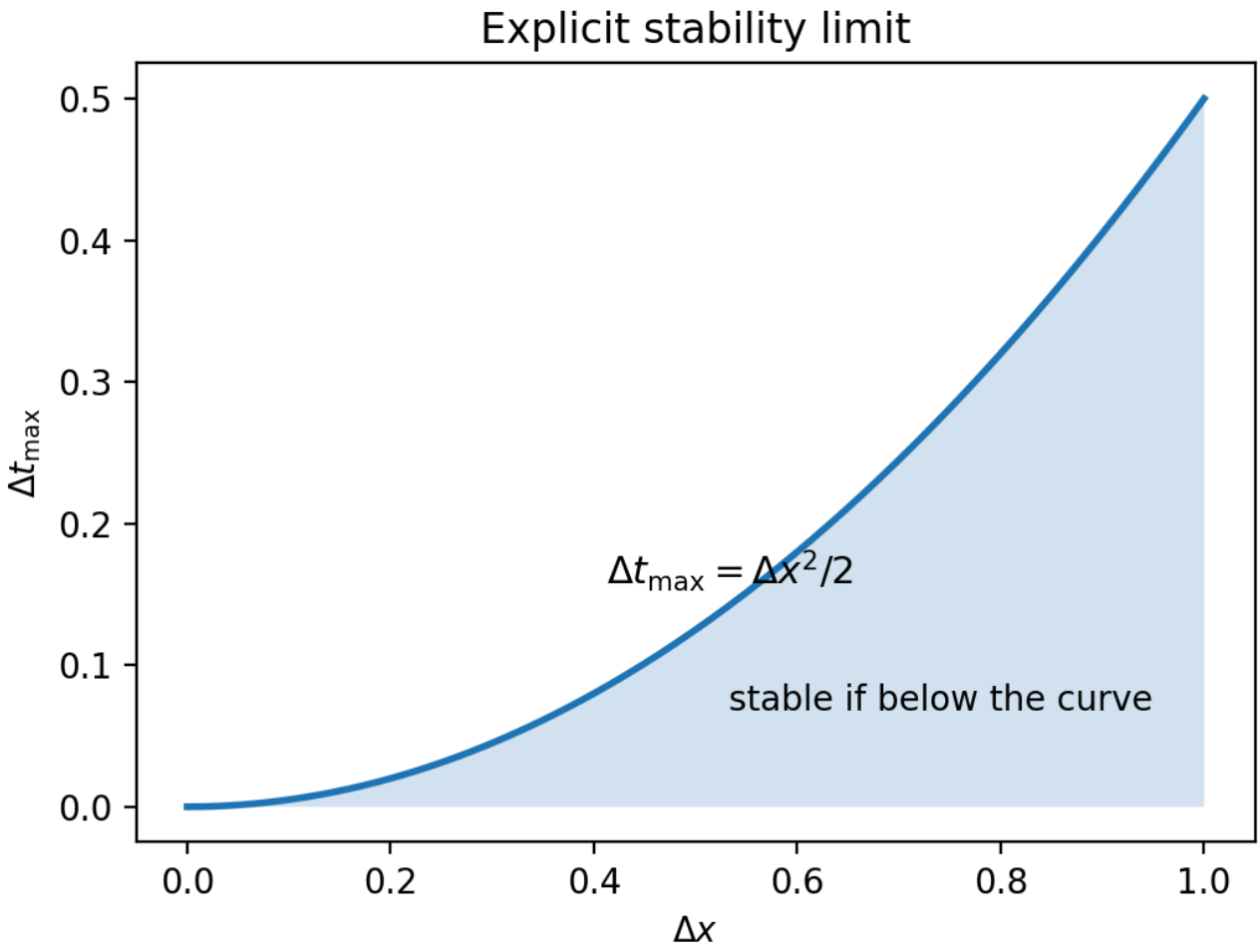


图 3.2-5　显式稳定性限制的示意图。曲线表示 $\Delta t_{\max} \propto (\Delta x)^2$ 的二次关系（图中取无量纲示意 $\kappa = 1$）。它强调：当空间网格加密时，显式方法允许的最大时间步长会更快减小。

空间网格越细，显式格式在同一物理时间区间内通常需要越多次时间推进，因而计算成本随之增加。对于保持 r 在固定正范围内的网格加密序列，可以把这种关系概括为

$$\Delta t = \mathcal{O}((\Delta x)^2).$$

稳定性与精度是两个不同的问题。 满足式 (3.2.19) 能控制离散解和扰动的增长，但并不意味着网格已经足够精细。即使 $r < 1/2$，若 Δx 或 Δt 较大，误差也可能明显；反之，当 $r > 1/2$ 时，失去非负加权平均性质，而 Fourier 分析表明在充分细的网格上存在被放大的高频模态。故数值方法的合理选择应同时考虑：

$$\begin{cases} \text{稳定性：} & \dfrac{\kappa \Delta t}{(\Delta x)^2} \le \dfrac{1}{2}, \\ \text{精度：} & \Delta t, \Delta x \text{ 足够小，使误差满足计算要求。} \end{cases}$$

还须强调，式 (3.2.19) 并非所有偏微分方程通用的 CFL 公式，而是针对一维常系数热方程、均匀空间网格、二阶中心空间差分与前向 Euler 时间推进这一具体组合推导出来的。改变维数、离散格式、扩散系数或边界处理时，应重新建立对应的稳定性判据。

至此，我们完成了“连续热方程 $\to$ 时空离散 $\to$ FTCS 递推 $\to$ 截断误差 $\to$ 稳定性与收敛性”的推导链条。在 3.2.3 中，将通过网格实验检验这些理论结论，并比较稳定时间步与超出 CFL 限制的时间步所产生的误差和振荡现象。

3.2.3 稳定性限制与网格实验

上一小节从 Taylor 展开建立了 FTCS 显式格式，并通过离散最大值原理和 Fourier 模态分析得到了扩散型 CFL 限制。理论告诉我们：差分格式能够逼近微分方程，并不意味着它在任意时间步长下都能可靠地计算。 本小节不再重复稳定性定理的证明，而是把结论转化为可观察、可测量、可重复的数值现象。

以下沿用 3.2.1 的金属杆模型，并以解析解作为独立参考，考察三个问题：满足稳定性限制时，数值温度是否按预期衰减？违反限制时，微小误差如何被放大？在保持稳定的前提下，网格加密是否能够提高精度？

3.2.3.1 模型与稳定性预测

考虑长度为 L、热扩散系数为 $\kappa > 0$ 的均匀导热杆，其两端保持零温度。作为主要参考问题，仍取光滑初值 $u(x,0) = A\sin(\pi x/L)$，其中 A 为初始温度振幅。相应的精确解为

$$u(x,t) = A\sin\left(\frac{\pi x}{L}\right)\exp\left[-\kappa\left(\frac{\pi}{L}\right)^2 t\right]. \tag{3.2.26}$$

式 (3.2.26) 给出了我们希望数值方法重现的物理趋势：温度曲线保持光滑，其振幅随时间衰减，不会无缘无故出现越来越大的正负振荡。

在均匀网格 $x_j = j\Delta x$、$t_n = n\Delta t$ 上，3.2.2 中推导的 FTCS 格式为

$$U_j^{n+1} = U_j^n + r\left(U_{j+1}^n - 2U_j^n + U_{j-1}^n\right), \qquad r = \frac{\kappa\Delta t}{(\Delta x)^2}. \tag{3.2.27}$$

在零 Dirichlet 边界 $U_0^n = U_J^n = 0$ 下，内部离散正弦模态

$$\phi_j^{(m)} = \sin\left(\frac{m\pi j}{J}\right), \qquad m = 1, 2, \ldots, J-1,$$

经过一次 FTCS 时间推进后，振幅乘以

$$G_m(r) = 1 - 4r\sin^2\left(\frac{m\pi}{2J}\right). \tag{3.2.28}$$

这个放大因子使稳定性判据具有直接的实验含义：当 $|G_m| \le 1$ 时，该模态不会逐步放大；当某个 $|G_m| > 1$ 时，即使它的初始振幅极小，也可能在多次迭代后变得十分明显。对于本节的均匀网格 FTCS 格式，标准的、与网格加密相容的稳定性条件是

$$0 < r \le \frac{1}{2} \quad \Longleftrightarrow \quad \Delta t \le \frac{(\Delta x)^2}{2\kappa}.$$

这里有一个容易忽视的细节：仅以单个低频正弦函数为初值，不一定能在短时间实验中暴露违反 CFL 条件后的不稳定性。 因为初始的 $\sin(\pi x/L)$ 恰好对应第一个离散正弦模态，纵使 $r > 1/2$，它本身的放大因子 $G_1(r)$ 仍可能满足 $|G_1| < 1$。为检验格式对微小高频扰动的反应，我们在稳定性对比实验中使用

$$u_\varepsilon(x,0;J) = A\sin\left(\frac{\pi x}{L}\right) + \varepsilon\sin\left(\frac{(J-1)\pi x}{L}\right), \tag{3.2.29}$$

其中 $\varepsilon = 10^{-12}\,^\circ\mathrm{C}$。两项均在 $x = 0, L$ 处为零，且对固定 J 都是光滑函数，因此初值与边界条件相容。第二项是一种人为加入且可控的高频测试扰动，而不是声称真实金属杆内必然存在这一特定波形。

由于热方程是线性的，式（3.2.29）对应的解析解可以直接由两个正弦模态叠加得到：

$$u_\varepsilon(x,t;J)=A\sin\left(\frac{\pi x}{L}\right)e^{-\kappa(\pi/L)^2t} +\varepsilon\sin\left(\frac{(J-1)\pi x}{L}\right)e^{-\kappa((J-1)\pi/L)^2t}. \tag{3.2.30}$$

高频解析模态衰减得比低频模态快得多，而不稳定的离散高频模态却可能反向增长。这种“解析解快速衰减、数值解反而爆炸”的对照，正是本实验要捕捉的现象。

3.2.3.2 实验方案与参数设置

本实验不使用外部观测数据，而是构造具有解析解的初边值问题，因此模型假设、数值误差和稳定性效应可以相互区分。为了使比较公平，稳定组与不稳定组采用相同的物理模型、空间网格、终止时间和初值；唯一有意改变的主要参数是时间步长所决定的 r。

表 3.2-1　实验设计与复现信息

实验要素	方案与记录
要回答的问题与运行前预测	① $r=0.4$ 时温度应保持有界并接近解析解；② $r=0.6$ 时，加入的高频扰动应被放大并导致数值振荡；③ 保持 $r=0.4$、空间网格减半时，误差预计约缩小至原来的四分之一。
输入、参数、单位、数据来源与划分	$L=1\,\mathrm{m}$，$\kappa=0.1\,\mathrm{m^2/s}$，$A=1\,^\circ\mathrm{C}$，$T=0.6\,\mathrm{s}$；零 Dirichlet 边界。稳定性对比使用 $J=40$、$\varepsilon=10^{-12}\,^\circ\mathrm{C}$。网格实验采用 $J=20,40,80,160$ 及无扰动初值。所有输入均为人为构造的教学算例，没有训练集或测试集划分。
独立参考解 / 基线及选择理由	稳定性对比使用式 (3.2.30)，网格收敛实验使用式 (3.2.26)。它们通过连续热方程直接得到，不需要由 FTCS 数值解生成，因此能够独立检验算法。
误差指标、容差及其依据	记录最大节点误差 E_∞、均方根节点误差 E_{RMS}、温度最大模值 M^n 和高频模态系数。教学验收阈值预先设为：稳定组终点 $E_\infty<5\times10^{-4}\,^\circ\mathrm{C}$；不稳定组终点 $E_\infty>10^{6}\,^\circ\mathrm{C}$；网格加密的相邻观测阶在 $[1.8,2.2]$ 内。这些数值阈值仅针对本算例，不是普适稳定性定理。
环境、种子、设备与运行预算	本次参考运行采用 Python 3.12.10、NumPy 1.26.4、Matplotlib 3.9.2，使用 CPU 和 `float64` 浮点数；不涉及随机抽样，无需设置随机种子。最细网格有 161 个节点、3840 次时间推进；参考运行中数值计算约耗时 0.14 s（不含图像保存，机器不同会有差异）。

实验 3.2-1（稳定与不稳定时间步长的对照）。 固定 $J=40$，因此 $\Delta x=L/J=0.025\,\mathrm{m}$。由稳定性条件计算得

$$\Delta t_{\max}=\frac{0.025^2}{2\times0.1}=0.003125\,\mathrm{s}. \tag{3.2.31}$$

分别选择以下两组时间步长：

组别	r	Δx (m)	Δt (s)	到达 $T=0.6\ \mathrm{s}$ 的步数 N	事先预测
A：满足 CFL 条件	0.4	0.025	0.002500	240	高频误差衰减，温度近似解析解
B：违反 CFL 条件	0.6	0.025	0.003750	160	高频误差被放大，后期出现交替振荡和爆炸

两组都采用式 (3.2.29) 的相同初值。为了避免仅比较终点而遗漏误差的发展过程，程序逐步记录误差、最大温度模值和高频模态振幅，并在 $t=0,0.15,0.30,0.45,0.60\,\mathrm{s}$ 检查其变化。

实验 3.2-2（稳定条件下的网格收敛性）。 取 $r=0.4$ 不变，令 $J=20,40,80,160$。这一实验采用无扰动初值 $u(x,0)=A\sin(\pi x/L)$，使各个网格逼近同一个连续问题。因为

$$\Delta t=\frac{r(\Delta x)^2}{\kappa}, \tag{3.2.32}$$

每次把空间网格间距减半时，时间步长也减为原来的四分之一。3.2.2 已证明，对于足够光滑的解，稳定 FTCS 格式的总体误差典型地满足 $E_\infty=\mathcal{O}(\Delta t+(\Delta x)^2)$。因此，在这里保持 r 不变，应观察到 $E_\infty=\mathcal{O}((\Delta x)^2)$。

3.2.3.3 误差度量与实验记录

设 $u_{\mathrm{ref}}(x_j,t_n)$ 表示相应实验使用的解析解。定义网格节点上的最大绝对误差为

$$E_\infty^n=\max_{1\le j\le J-1}\left|U_j^n-u_{\mathrm{ref}}(x_j,t_n)\right|, \tag{3.2.33}$$

内部节点上的均方根误差为

$$E_{\mathrm{RMS}}^n=\left[\frac{1}{J-1}\sum_{j=1}^{J-1}\left(U_j^n-u_{\mathrm{ref}}(x_j,t_n)\right)^2\right]^{1/2}. \tag{3.2.34}$$

两者单位均为 $^\circ\mathrm{C}$。另外用 $M^n=\max_{0\le j\le J}|U_j^n|$ 描述数值温度的整体幅值；为观察高频分量，可将离散解投影到 $\phi_j^{(J-1)}$ 上：

$$a_{\mathrm{hi}}^n=\frac{\sum_{j=1}^{J-1}U_j^n\sin((J-1)\pi j/J)}{\sum_{j=1}^{J-1}\sin^2((J-1)\pi j/J)}. \tag{3.2.35}$$

在精确算术中，这一高频模态系数满足 $a_{\mathrm{hi}}^n=\varepsilon G_{J-1}(r)^n$。在双精度浮点计算中，当它衰减到机器舍入误差量级之后，不能再期望观测值继续严格按照这个公式下降。

数值实验按照式 (3.2.27) 对内部节点进行逐层更新，并在每一时间层直接赋值零 Dirichlet 边界 $U_0^n=U_J^n=0$。在计算过程中记录最大绝对误差、均方根误差、温度最大模值及高频模态系数，用于检验稳定性和网格收敛性。后文直接呈现运行得到的实验图像，并结合数据表解释其数学含义。

误差增长曲线和高频模态曲线采用对数纵轴。对初始零误差或接近机器精度的数值，绘图时使用 10^{-16} 作为显示下限，以保证曲线可以在对数坐标上呈现。这一处理仅涉及图形显示，不改变表中的原始误差结果。

3.2.3.4 数值结果与分析

以下数值来自对上述模型和 FTCS 更新的实际运行，而不是依据理论公式预先填写的模拟结果。由于采用 `float64`，不同 Python 或 NumPy 版本、不同处理器上的最后若干位可能略有差别，但不影响稳定与不稳定行为的判断。

（1）时间步长与数值稳定性。 时间步长是否满足 CFL 条件，直接改变了数值解的命运。

表 3.2-2　相同网格、不同时间步长的终点结果（$J = 40$，$T = 0.6\,\mathrm{s}$）

指标	A：$r = 0.4$，满足 CFL 条件	B：$r = 0.6$，违反 CFL 条件
时间步长 Δt（s）	0.002500	0.003750
时间步数 N	240	160
最大绝对误差 E_∞（°C）	2.35955×10^{-4}	1.57265×10^{11}
均方根误差 E_{RMS}（°C）	1.68971×10^{-4}	1.12620×10^{11}
终点最大温度模值 M^N（°C）	0.552886	1.57265×10^{11}

由式（3.2.26）计算，未扰动主模态在 $T = 0.6\,\mathrm{s}$ 的最大温度约为 $0.553122\,^\circ\mathrm{C}$。稳定组的结果 $0.552886\,^\circ\mathrm{C}$ 与其非常接近；不稳定组则出现超过 $10^{11}\,^\circ\mathrm{C}$ 的非物理数值。这不是金属杆真的被加热到了如此高的温度，而是有限差分迭代失去了稳定性。

只看终点有时会掩盖爆炸前的短暂“正常”阶段。因此还应检查误差的增长过程。

表 3.2-3　不同时间的最大节点误差 E_∞^n（单位：°C）

时间 t（s）	A：$r = 0.4$	B：$r = 0.6$
0.00	0	0
0.15	9.19863×10^{-5}	1.71602×10^{-4}
0.30	1.58648×10^{-4}	3.96858×10^{-1}
0.45	2.05215×10^{-4}	2.49731×10^{5}
0.60	2.35955×10^{-4}	1.57265×10^{11}

在前 $0.15\,\mathrm{s}$，不稳定组看起来仍可能与解析解接近；到 $0.30\,\mathrm{s}$，它的误差已经接近初始主温度振幅；此后误差迅速跨越多个数量级。图 3.2-6 使用对数纵轴显示这一过程，图 3.2-7 则将同一终止时刻的稳定组与不稳定组空间温度剖面并列对照：左图中 $r = 0.4$ 的数值解与解析解几乎重合，右图中 $r = 0.6$ 的数值解出现剧烈的正负交替振荡。由于两组幅值相差极大，两幅子图使用相同的空间横轴、但各自独立的温度纵轴刻度；比较时应注意纵轴数量级，而不能只比较图上线条的高度。故意保留这些异常值，是因为它们本身就是需要解释的实验现象。

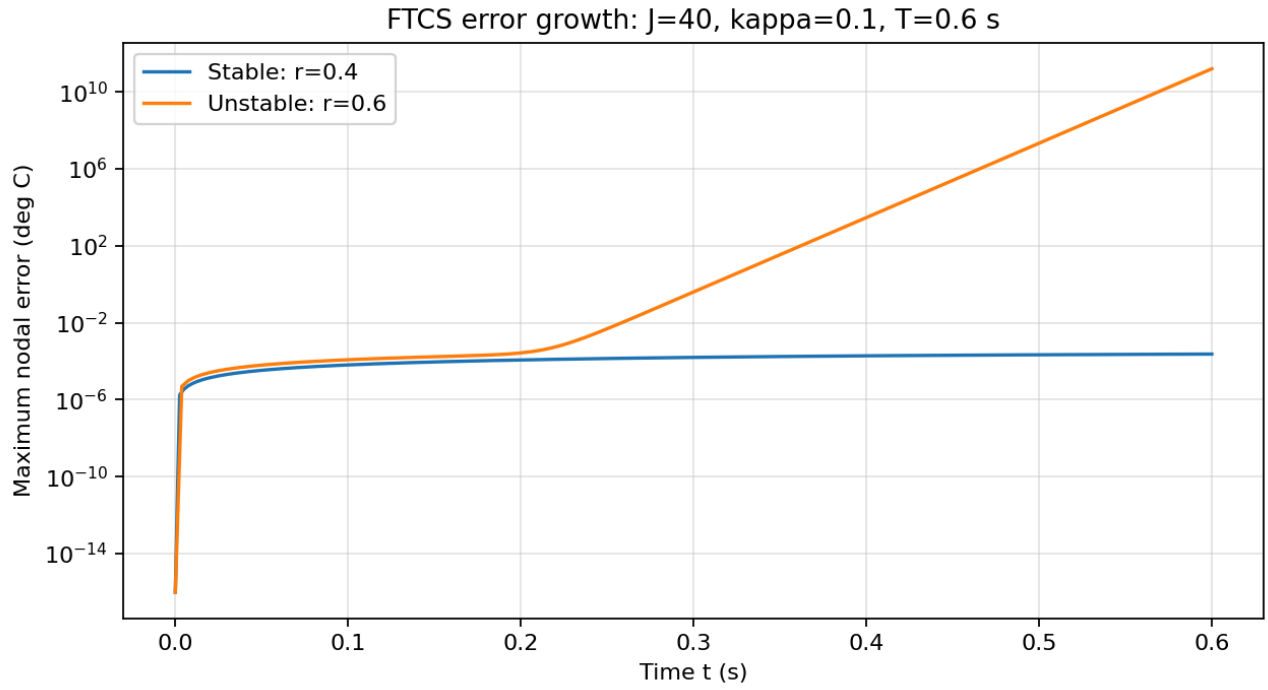


图 3.2-6　FTCS 最大节点误差随时间的变化。横轴为时间 t（s），纵轴为 E_∞（°C，对数刻度）；图例区分稳定组 $r=0.4$ 与不稳定组 $r=0.6$，参数为 $J=40$、$\kappa=0.1\,\mathrm{m^2/s}$。

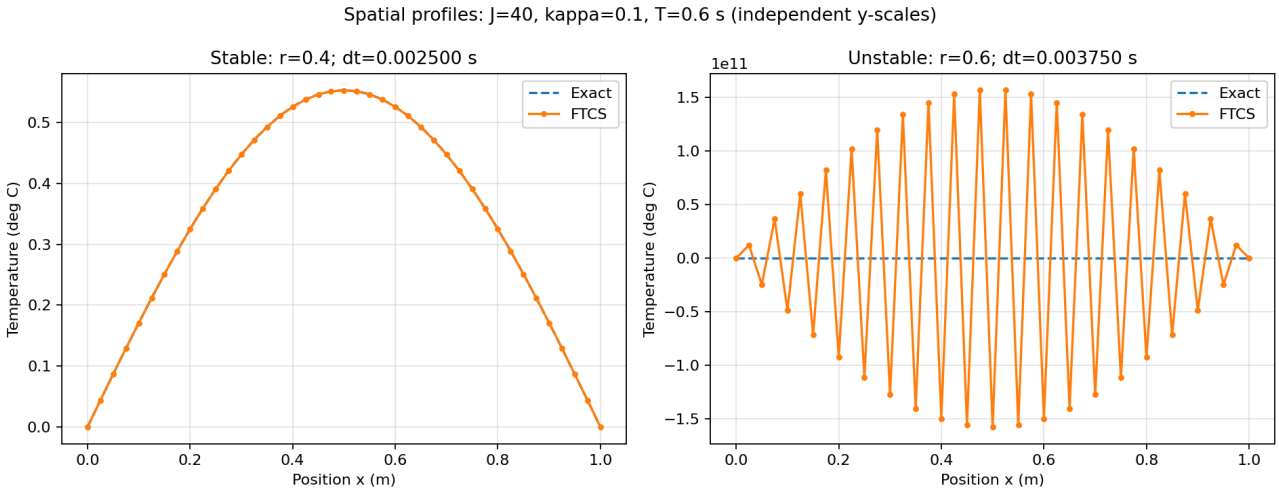


图 3.2-7　稳定组与不稳定组的空间温度剖面对比（$T=0.6\,\mathrm{s}$，$J=40$，$\kappa=0.1\,\mathrm{m^2/s}$）。左：满足 CFL 条件的 $r=0.4$；右：违反 CFL 条件的 $r=0.6$。两组均绘制 FTCS 数值解与解析解；横轴为位置 x（m），纵轴为温度（°C）。两幅子图的纵轴范围不同，以完整呈现稳定组的真实剖面与不稳定组的数值振荡。

（2）高频扰动与误差放大。 为什么微小的高频扰动可以引发如此大的误差？

对于 $J=40$，最高阶离散正弦模态的放大因子为

$$G_{39}(r)=1-4r\cos^2\left(\frac{\pi}{80}\right). \tag{3.2.36}$$

计算可得：$r=0.4$ 时，$G_{39}\approx-0.597534$；$r=0.6$ 时，$G_{39}\approx-1.396301$。负号表示这一高频模态在相邻时间层之间改变符号；真正决定增长或衰减的是模长 $|G_{39}|$。

在 $r=0.6$ 的不稳定组中，经过 $N=160$ 步，忽略舍入误差时，初始振幅仅为 $10^{-12}\,^\circ\mathrm{C}$ 的高频扰动理论上会被放大到大约

$$10^{-12}\times|-1.396301|^{160}\approx 1.57\times10^{11}\,^\circ\mathrm{C}. \tag{3.2.37}$$

这个预测数量级与实际计算吻合。图 3.2-8 通过式（3.2.35）的高频投影进一步说明：失控增长主要发生在高频模态上，而不是解析解本身要求温度增长。

这里也揭示了为什么实验要刻意加入微小的高频扰动。如果只使用低频初值 $A\sin(\pi x/L)$，那么在精确算术下只激发第一个离散模态，即使设置了 $r=0.6$，该模态仍可能衰减。一次“没有看到爆炸”的运行不能反证不稳定性定理；真正的稳定性要求控制所有允许的误差模态，而不仅是一个特殊初值所激发的模态。

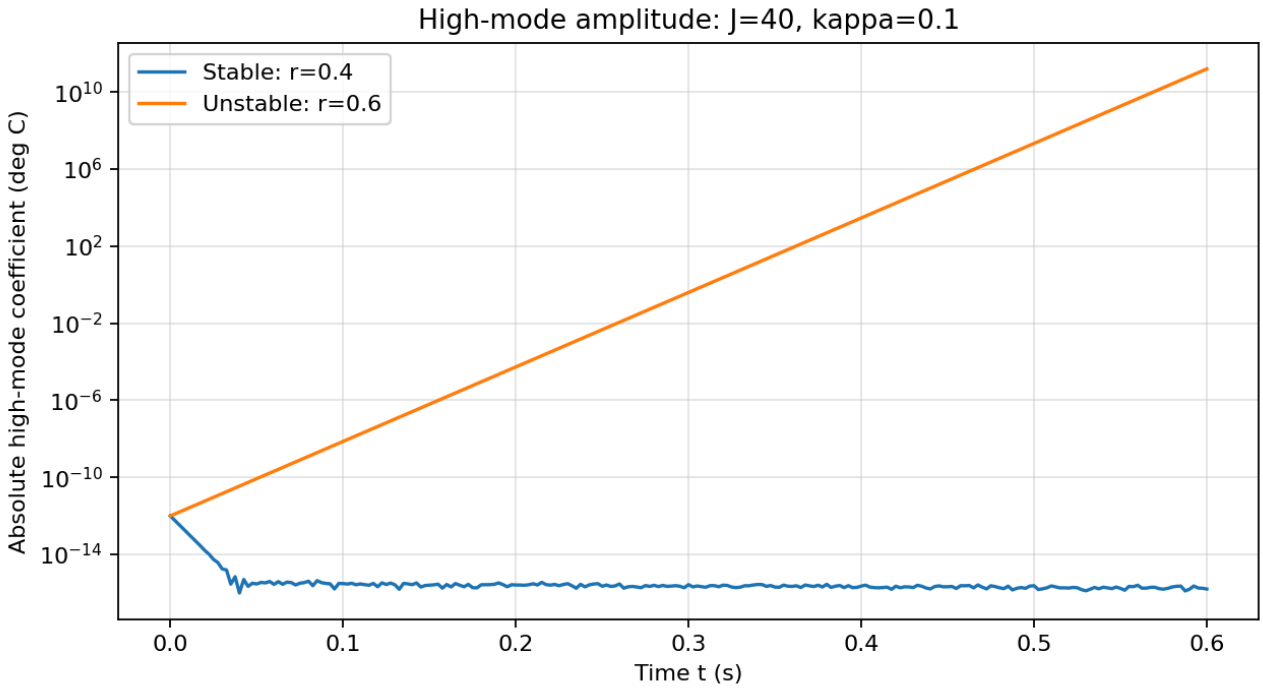


图 3.2-8　最高阶离散正弦模态的系数绝对值随时间变化。横轴为时间 t（s），纵轴为模态振幅（$^\circ\mathrm{C}$，对数刻度）；图例区分 $r=0.4$ 与 $r=0.6$，参数为 $J=40$。

（3）网格加密与精度。 稳定之后，网格加密是否会提高精度？

表 3.2-4　保持 $r=0.4$ 的网格收敛实验（$T=0.6\,\mathrm{s}$）

空间分段数 J	Δx (m)	Δt (s)	时间步数 N	E_∞ (°C)	观测阶 p
20	0.05000	0.01000000	60	9.46641×10^{-4}	—
40	0.02500	0.00250000	240	2.35955×10^{-4}	2.0043
80	0.01250	0.00062500	960	5.89449×10^{-5}	2.0011
160	0.00625	0.00015625	3840	1.47335×10^{-5}	2.0003

其中相邻两组网格的观测收敛阶定义为

$$p=\frac{\log\left(E_\infty(\Delta x)/E_\infty(\Delta x/2)\right)}{\log 2}. \tag{3.2.38}$$

例如，由 $J=20$ 加密到 $J=40$ 时，最大误差约缩小为原来的 $1/4.012$，从而得到 $p\approx 2.0043$。后两次加密也表现出近似二阶收敛。这与前一小节的总体误差估计

$$E_\infty = \mathcal{O}\left(\Delta t + (\Delta x)^2\right)$$

相吻合：本实验始终保持 r 为常数，因此 Δt 与 $(\Delta x)^2$ 同阶，两部分误差共同表现为二阶空间收敛。图 3.2-9 将观测误差曲线与 $\mathcal{O}((\Delta x)^2)$ 参考斜率进行对照。

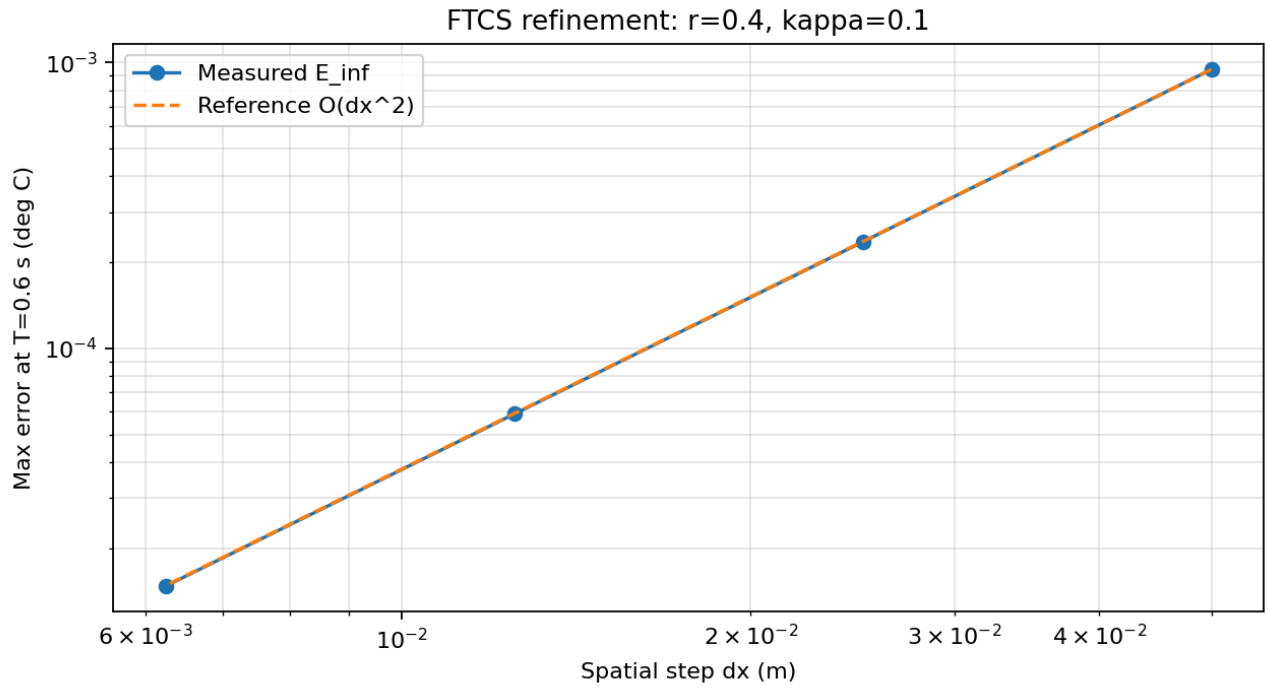


图 3.2-9　空间网格加密时的最大节点误差。横轴为 Δx（m，对数刻度），纵轴为终点误差 E_∞（°C，对数刻度）；图例区分实测误差与 $\mathcal{O}((\Delta x)^2)$ 参考曲线，参数为 $r=0.4$、$T=0.6\,\mathrm{s}$。

需要注意，本实验同时缩小了 Δx 和 Δt，所以它验证的是保持 r 不变的联合网格收敛趋势，不能单凭这一组实验把时间误差和空间误差完全分离。若要分别测量时间阶与空间阶，还需设计不同的参数控制实验。

本节结论。 对于这里的一维常系数热方程，FTCS 格式在 $r=0.4$ 时能够稳定推进，并在网格加密下表现出预期的二阶误差收敛；取 $r=0.6$ 时，虽然只增加了一个极小的光滑高频扰动，数值解仍因高频误差逐步放大而失控。由此可见，稳定性决定误差能否受控，精度决定受控后的误差有多小；二者必须分别检查。

以上结论只在本节测试的常系数、均匀网格、零 Dirichlet 边界、前向 Euler 时间推进和双精度计算设置下得到了数值验证。其他边界条件、非均匀网格、变扩散系数、二维或三维扩散，以及不同时间积分方法尚未进行本节实验，不能由这些数值直接推断其行为。

3.2.3.5 CFL 条件的适用范围

有一种容易出现的错误表述：“求解 PDE 时只要满足 $r=\kappa\Delta t/(\Delta x)^2\le 1/2$ 就稳定。”这一说法遗漏了方程类型与离散格式，因此并不成立。本节的 $r\le 1/2$ 只针对一维常系数热方程、均匀网格、二阶中心空间差分和前向 Euler 时间推进这一具体组合的标准网格稳定性要求。

首先，即便仍求解同一个热方程，只要把时间推进改为后向 Euler，就得到隐式格式

$$\frac{U_j^{n+1}-U_j^n}{\Delta t}=\kappa\frac{U_{j+1}^{n+1}-2U_j^{n+1}+U_{j-1}^{n+1}}{(\Delta x)^2}. \tag{3.2.39}$$

将单个离散正弦模态代入，可得其放大因子

$$G_m^{\mathrm{BE}}=\frac{1}{1+4r\sin^2(m\pi/(2J))}. \tag{3.2.40}$$

由于 $r\ge 0$ 时 $0<G_m^{\mathrm{BE}}\le 1$，因此这一隐式格式在该模型的模态稳定性意义下不要求 $r\le 1/2$。它需要在每一步求解线性方程组，并且即使稳定也仍有离散误差。由此可见，同一 PDE，改变时间离散方法，稳定性限制就可能改变。

其次，对一维线性平流方程 $u_t+au_x=0$（$a>0$），使用前向 Euler 与一阶迎风空间差分时，常见稳定性限制是 $0\le a\Delta t/\Delta x\le 1$。它所涉及的是与 Δx 成一次比例的时间步长，而不是本节扩散格式中的平方关系。

还需要区分“标准网格稳定性条件”和“固定有限网格的精确谱界限”。对于本节的 J 个均匀分段、零 Dirichlet 边界，式 (3.2.28) 在所有 $m=1,\dots,J-1$ 上满足 $|G_m|\le 1$ 的精确条件为

$$0\le r\le\frac{1}{2\cos^2(\pi/(2J))}. \tag{3.2.41}$$

对 $J=40$，上界约为 0.500772，略高于 $1/2$；当 $J\to\infty$ 时，它趋向 $1/2$。所以不能笼统声称“在每一个固定有限零边界网格上，任何 $r>1/2$ 都必然导致增长”。本实验选择的 $r=0.6$ 明显超过固定网格的精确界限，因而确实会激发增长模态。

最后，稳定不等于精确，观察到暂时不爆炸也不等于已经证明稳定。 一个计算结果能否可靠，需要同时核对初边值条件、离散格式、时间步长、误差与网格加密趋势。

3.2.3.6 思考题及解析

思考题。 对一根长度 $L=1\,\mathrm{m}$ 的金属杆，热扩散系数取 $\kappa=0.1\,\mathrm{m^2/s}$，两端采用零 Dirichlet 边界。将空间区间均匀分成 $J=40$ 段，用 FTCS 显式格式求解。现在有三个候选时间步长：

$$\Delta t_1=0.0025\,\mathrm{s},\qquad \Delta t_2=0.003125\,\mathrm{s},\qquad \Delta t_3=0.00375\,\mathrm{s}.$$

哪几个满足标准 CFL 条件？若进一步将 Δx 减半，允许的最大稳定时间步长是多少？是否只要选了稳定步长，就说明数值解一定足够精确？

常见回答及其问题。 有的同学会认为 Δt_3 也可以选，因为初始温度曲线足够光滑；另一些同学会认为 Δt_1 比 Δt_2 小，因此必然得到精确解。这两种看法都混淆了特定初值下的数值表现、格式稳定性与离散精度。

解答与分析。 当前网格间距为 $\Delta x=1/40=0.025\,\mathrm{m}$，因此标准稳定性上界为

$$\Delta t_{\max}=\frac{(\Delta x)^2}{2\kappa}=0.003125\,\mathrm{s}.$$

于是 Δt_1 对应 $r=0.4$，Δt_2 对应 $r=0.5$，二者满足条件；Δt_3 对应 $r=0.6$，不满足条件。边界值 $r=0.5$ 在标准线性稳定判据下是允许的，但实际计算常选择略小于上界的步长，留出一定的余量。

若空间步长减半为 $\Delta x'=0.0125\,\mathrm{m}$，则

$$\Delta t'_{\max} = \frac{(0.0125)^2}{2 \times 0.1} = 0.00078125\,\mathrm{s}.$$

在保持 $r = 0.4$ 时，应取 $\Delta t' = 0.000625\,\mathrm{s}$。这说明网格加密不仅增加了空间节点数，还会显著增加显式时间推进的步数。

至于精度问题，$r \le 1/2$ 只是防止格式在标准稳定性意义下放大误差的要求；误差大小还取决于空间步长、时间步长、解的光滑程度及所需的计算容差。若希望判断结果是否足够精确，应该进一步与解析解对比，并进行网格收敛检验，而不能仅凭 CFL 条件下结论。

通过这一实验与讨论，我们把 3.2.2 的理论分析落实到了可重复的计算证据上：先确定模型与初边值条件，再选择离散格式；先检查稳定性，再通过独立参考解和网格加密检验精度。

3.3 一维有限元方法：弱形式与 Galerkin 离散

有限元方法将连续求解区域划分为若干单元，在每个单元上选取简单的近似函数，再通过变分形式将局部信息组合为整体代数方程组。本节以一维 Poisson 边值问题为模型，依次讨论弱形式的建立、连续分片线性有限元空间、Galerkin 离散、单元装配、边界条件处理与误差验证。

3.3.1 模型问题与有限元基本思想

有限元方法的发展与工程结构分析密切相关。对于形状复杂的机翼等结构，可先将其划分为许多简单单元，再根据单元之间的连接关系建立整体计算模型。这一“局部近似—整体组装”的思想也适用于微分方程的数值求解。

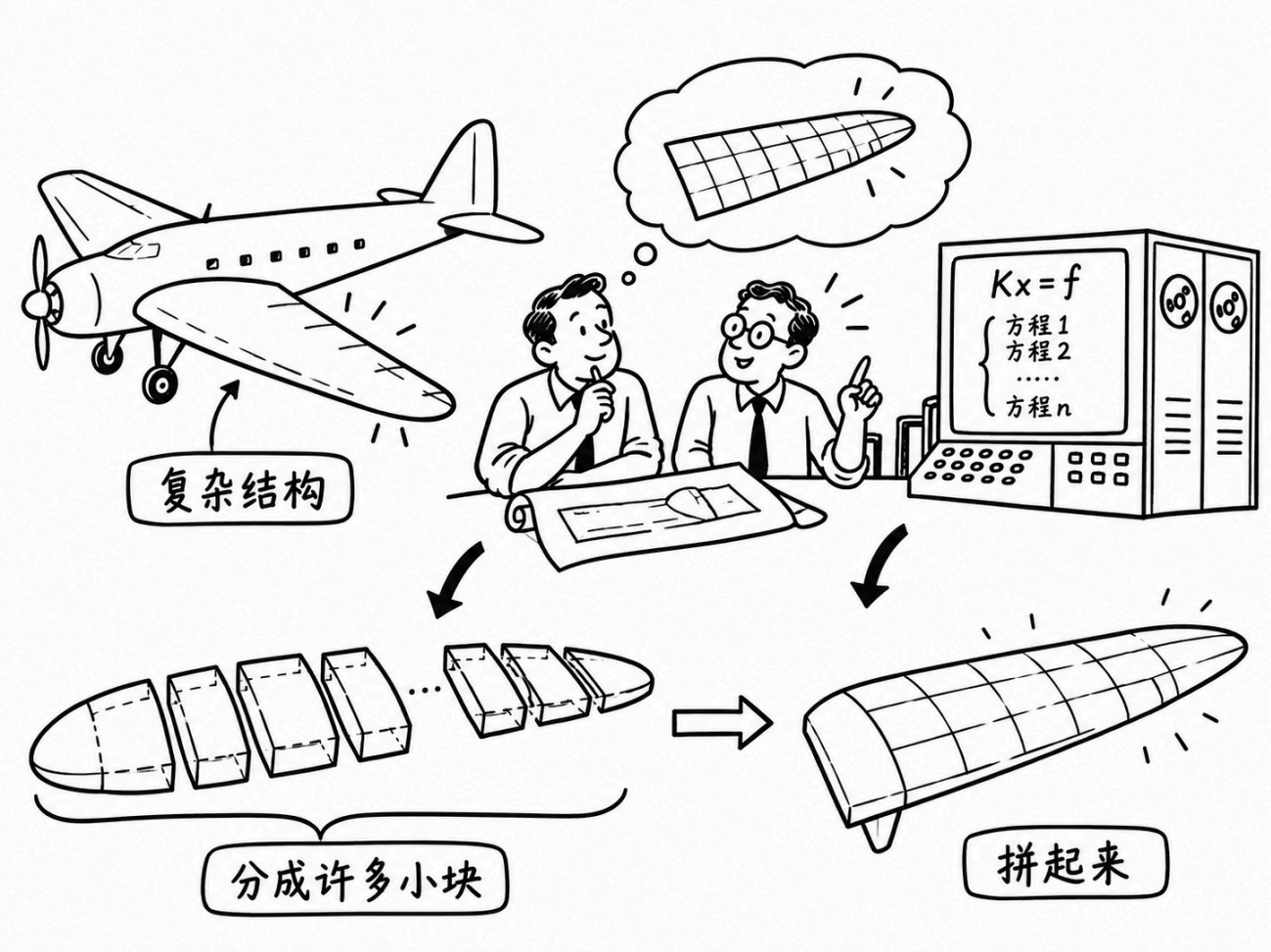


图 3.3-1　复杂工程结构的单元离散化示意

为了说明基本原理，考虑下列一维边值问题：

$$\begin{cases} -u''(x) = 1, & 0 < x < 1, \\ u(0) = u(1) = 0. \end{cases} \tag{3.3.1}$$

将区间 $[0,1]$ 等分为两个单元，并以连续分片线性函数 u_h 近似精确解 u。由于两端节点值已知且为零，有限元近似只包含一个待定的内部节点值 $U_1 = u_h(1/2)$。

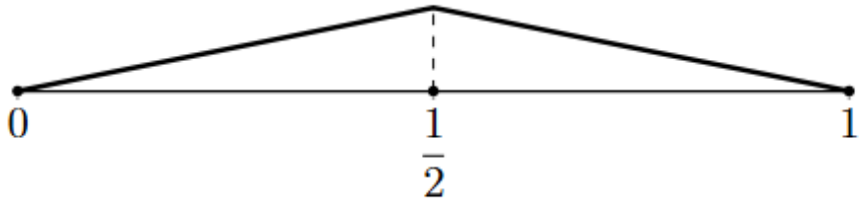


图 3.3-2　一维两单元网格及分片线性近似

在每个单元内部，折线的二阶导数为零，即

$$u_h''(x) = 0.$$

但原方程要求

$$-u''(x) = 1.$$

而且在折点处，经典意义下的二阶导数甚至不存在。

因此，分片线性近似通常不能在经典意义下逐点满足原微分方程。

另一方面，直接积分并结合边界条件，可得本例的精确解

$$u(x) = \frac{x(1-x)}{2},$$

其中点值为

$$u\left(\frac{1}{2}\right) = \frac{1}{8}.$$

后文将通过有限元装配计算得到相同的中间节点值：

$$U_1 = \frac{1}{8}.$$

中间节点的精确一致并不表示折线在单元内部与精确解重合。这种现象需要通过弱形式及 Galerkin 离散来解释。

有限元方法并不改变原微分方程，而是采用适用于分片函数的积分检验条件：先构造弱形式，再在有限维函数空间中寻找近似解。

下面从一般的源项函数出发，建立这一计算过程的数学基础。

3.3.2 弱形式与函数空间

以微分方程和边界条件直接描述问题、并要求方程逐点成立的表述称为强形式（strong form）。连续分片线性函数在单元内部的一阶导数为常数，在单元交界处可能出现斜率跳跃，因而一般无法满足强形式的经典可微性要求。通过检验函数与分部积分得到的弱形式（weak form），仅需处理一阶弱导数，为有限元近似提供了适当的数学框架。在精确解具有充分光滑性的情形下，弱形式可由强形式推导得到；在适当条件下，它为原边值问题提供一致的广义表述。

3.3.2.1 从强形式到弱形式

考虑前述一维 Poisson 边值问题

$$\begin{cases} -u''(x) = f(x), & 0 < x < 1, \\ u(0) = u(1) = 0. \end{cases} \tag{3.3.2}$$

为说明弱形式的来源，暂时假定精确解 u 足够光滑。取任意一个足够光滑、且满足 $v(0) = v(1) = 0$ 的函数 $v(x)$，称其为检验函数（test function）。由于待求解的端点值已经给定，检验函数可以理解为不改变这些端点值的扰动，因此在两端取零。这里的“任意”十分重要：接下来推导的积分关系必须对每一个符合要求的检验函数成立。

将方程 $-u''(x) = f(x)$ 两边乘以 $v(x)$，并在整个区间上积分，得到

$$-\int_0^1 u''(x)v(x)\,\mathrm{d}x = \int_0^1 f(x)v(x)\,\mathrm{d}x. \tag{3.3.3}$$

对左端进行分部积分，便有

$$\int_0^1 u'(x)v'(x)\,\mathrm{d}x - \big[u'(x)v(x)\big]_0^1 = \int_0^1 f(x)v(x)\,\mathrm{d}x. \tag{3.3.4}$$

由于 $v(0) = v(1) = 0$，边界项满足

$$\big[u'(x)v(x)\big]_0^1 = u'(1)v(1) - u'(0)v(0) = 0. \tag{3.3.5}$$

因此得到

$$\int_0^1 u'(x)v'(x)\,\mathrm{d}x = \int_0^1 f(x)v(x)\,\mathrm{d}x. \tag{3.3.6}$$

这就是该边值问题的弱形式所依据的积分关系：寻找满足原边界条件的函数 u，使上式对所有允许的检验函数 v 成立。需要注意，边界项之所以消失，是因为检验函数在端点取零，并不是因为 $u(0) = u(1) = 0$ 就能推出 $u'(0) = u'(1) = 0$。

与强形式相比，上式仅含 u 的一阶导数。连续分片线性函数在各单元内具有分段常数的一阶导数，即使在单元交界处发生斜率跳跃，仍然可以计算上述积分。因此，弱形式为采用分片线性函数进行有限元近似提供了依据。

至于允许选取哪些试探函数和检验函数，以及如何把弱形式转化为有限个代数方程，将在后面继续讨论。

3.3.2.2 试探空间与检验空间

上文通过积分分部得到了只含一阶导数的积分关系。要将这一关系作为完整的数学问题，还需要说明：未知函数 u 和检验函数 v 分别允许从哪些函数中选取？

对于本节的齐次 Dirichlet 边界条件，取函数空间

$$V = H_0^1(0,1) = \{w \in H^1(0,1) : w(0) = w(1) = 0\}. \tag{3.3.7}$$

其中，$H^1(0,1)$ 表示函数本身及其一阶弱导数均平方可积的函数空间；$H_0^1(0,1)$ 则进一步要求函数在两个端点取零。在一维区间上，端点值可以按照函数的连续代表来理解。这里还假定已知源项 f 平方可积，以保证下面的积分有意义。

因此，该边值问题的弱形式可表述为：求 $u \in V$，使得对任意 $v \in V$，均有

$$\int_0^1 u'(x)v'(x)\,\mathrm{d}x = \int_0^1 f(x)v(x)\,\mathrm{d}x. \tag{3.3.8}$$

这里，未知函数 u 所属的空间称为试探空间，检验函数 v 所属的空间称为检验空间。在当前齐次 Dirichlet 问题中，两者取同一个空间 V，但作用不同：u 是需要确定的函数，v 用来检验积分关系是否成立。

需要强调，弱形式并没有取消对函数的要求，而是以一阶弱导数的可积性代替了对经典二阶导数的要求。例如，满足端点零值条件的连续分片线性函数属于 V：它在单元内部的一阶导数为常数，节点处即使发生斜率跳跃，也不影响相应积分的计算。

3.3.2.3 从无限维问题到有限维近似

至此，我们已经给出了弱问题的完整表述，但还没有得到可以直接求解的有限个代数方程。原因在于：V 是一个无限维函数空间，未知量仍然是函数 u，而且积分关系必须对 V 中所有检验函数成立。

有限元方法的下一步，是先将区间划分为若干小单元，再从 V 中选取由连续分片线性函数构成的有限维子空间 $V_h \subset V$。我们将用 $u_h \in V_h$ 近似 u，并相应地从 V_h 中选择检验函数。这样，未知函数就可以用有限个系数表示，积分关系也将转化为有限个代数方程。需要注意，离散后只要求积分关系对 V_h 中的检验函数成立，因此得到的是原弱问题的近似，而不是原问题在所有检验函数下的精确解。

这一过程可以概括为

$$\underbrace{\text{在 } V \text{ 中求 } u}_{\text{无限维弱问题}} \quad \longrightarrow \quad \underbrace{\text{在 } V_h \subset V \text{ 中求 } u_h}_{\text{有限维近似问题}}. \tag{3.3.9}$$

至于如何建立网格、构造 V_h，以及如何把有限维问题写成线性方程组，将在下面依次说明。

3.3.3 网格与分片线性有限元空间

3.3.3.1 网格划分

上一小节提出，要用一个有限维空间近似原来的无限维空间。为了构造这样的空间，首先将求解区间 $[0,1]$ 划分为 N 个小区间，取节点

$$0 = x_0 < x_1 < \cdots < x_N = 1. \tag{3.3.10}$$

其中，x_i 称为节点（node），相邻两个节点之间的区间

$$K_e = [x_{e-1}, x_e], \qquad e = 1, 2, \ldots, N \tag{3.3.11}$$

称为一个单元（element）。这些单元共同构成区间 $[0,1]$ 上的网格（mesh）。有限元方法中的“元”，指的就是这样的单元。

记第 e 个单元的长度为 h_e，并定义网格尺寸 h：

$$h_e = x_e - x_{e-1}, \qquad h = \max_{1 \le e \le N} h_e. \tag{3.3.12}$$

网格不要求各单元等长；当所有单元等长时，$h_e = h = 1/N$。一般来说，h 越小，表示网格划分越细。

建立网格只是完成了区间的划分。接下来还需要规定：在每个单元上，用什么样的函数近似未知解？

3.3.3.2 连续分片线性有限元空间

为使近似函数既易于计算，又能够用于前面建立的弱形式，我们在每个单元上选取次数不超过一次的多项式，并要求它们在相邻节点处连续，同时满足两端的齐次 Dirichlet 边界条件。由此定义连续分片线性有限元空间

$$V_h = \left\{ v_h \in C^0([0,1]) \;\middle|\; \begin{array}{l} v_h|_{K_e} \in P_1(K_e), \quad e = 1,2,\ldots,N, \\ v_h(0) = v_h(1) = 0 \end{array} \right\}. \tag{3.3.13}$$

这里，$C^0([0,1])$ 表示在整个区间上连续的函数，$P_1(K_e)$ 表示单元 K_e 上次数不超过 1 的多项式构成的空间，$v_h|_{K_e}$ 表示函数 v_h 在该单元上的限制。下标 h 表示这一空间依赖于所选网格。

因此，V_h 中的函数在每个单元上是一条直线段，整体上则是一条连续折线。虽然它们在单元交界处可能存在折角，但其一阶弱导数在各单元内部为常数，因而平方可积。又因为这些函数在两端取零，所以它们属于前面定义的空间 $V = H_0^1(0,1)$，即

$$V_h \subset V. \tag{3.3.14}$$

更重要的是，连续分片线性函数由各节点处的函数值唯一确定。由于两端节点值已经固定为零，只剩下 $N-1$ 个内部节点值可以独立选取。因此，V_h 是一个维数为 $N-1$ 的有限维空间。

至此，我们在原来的无限维空间 V 中构造了一个可由有限个节点值描述的近似子空间 V_h。下一步将构造这个空间的基函数，并由此把近似函数写成有限个未知系数的线性组合。

3.3.3.3 帽函数与有限元近似解

上一小节构造了连续分片线性有限元空间 V_h，并说明其中的函数由 $N-1$ 个内部节点值唯一确定。为了将求解未知函数的问题转化为确定有限个系数的问题，需要为 V_h 选取一组便于计算的基函数。

对每个内部节点 x_i（$i = 1,2,\ldots,N-1$），定义函数 $\phi_i \in V_h$，使它在网格节点上满足

$$\phi_i(x_j) = \begin{cases} 1, & j = i, \\ 0, & j \neq i, \end{cases} \qquad j = 0,1,\ldots,N. \tag{3.3.15}$$

由于 ϕ_i 在每个单元上都是次数不超过一次的多项式，以上节点取值条件唯一确定了它。利用相邻节点之间的线性插值，可得

$$\phi_i(x) = \begin{cases} \dfrac{x - x_{i-1}}{x_i - x_{i-1}}, & x_{i-1} \le x \le x_i, \\[2ex] \dfrac{x_{i+1} - x}{x_{i+1} - x_i}, & x_i \le x \le x_{i+1}, \\[2ex] 0, & \text{其他位置}. \end{cases} \tag{3.3.16}$$

函数 ϕ_i 在节点 x_i 处取值为 1，向左右相邻节点线性下降至 0，在其他位置为 0。由于图形形似帽子，这类基函数称为帽函数（hat function）。它只在相邻的两个单元 K_i 和 K_{i+1} 上可能非零，这一性质称为局部支撑。

下图给出一维网格上帽函数的形状，可以看到每个基函数只在相邻两个单元上非零：

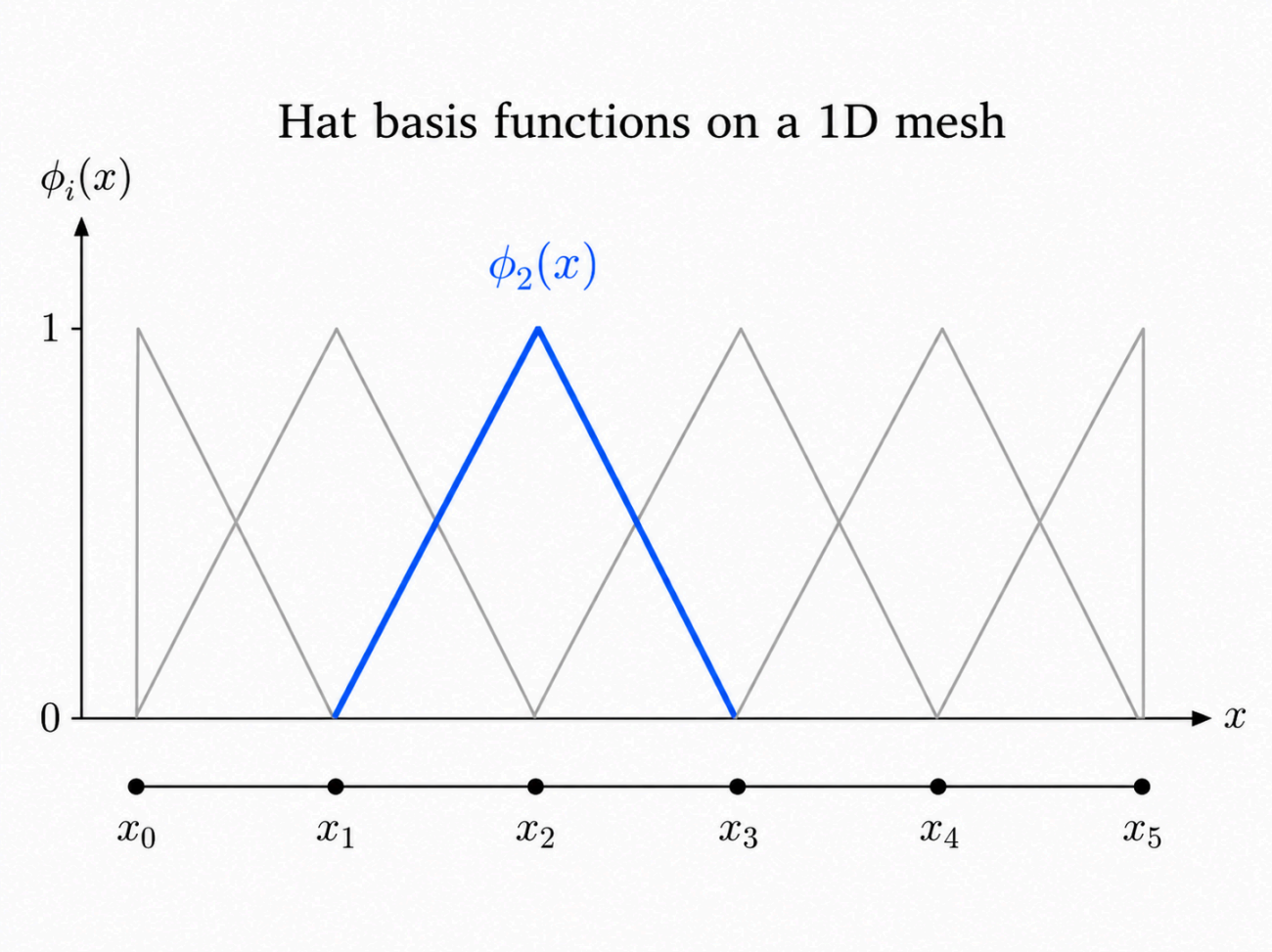


图 3.3-3　一维连续分片线性帽函数

从图中可以看出，帽函数在对应节点处取值为 1，在相邻节点处降为 0，其余位置恒为零。这正是它具有局部支撑的原因。 需要注意，图中两端的灰色半帽函数用于示意一般节点形函数。在本节的齐次 Dirichlet 问题中，边界节点值已经固定为零，因此 V_h 的基函数只取内部节点对应的 $\phi_1,\dots,\phi_{N-1}$。

帽函数的节点取值性质使任意 $u_h \in V_h$ 都可以唯一表示为

$$u_h(x) = \sum_{j=1}^{N-1} U_j \phi_j(x). \tag{3.3.17}$$

事实上，令 $U_j = u_h(x_j)$，则上式右端与 u_h 在全部节点上的取值相同；又因为二者在每个单元上都是一次多项式，所以它们在整个区间上相同。

反过来，令 $x = x_i$，利用 $\phi_j(x_i) = 1$（当 $j = i$）和 $\phi_j(x_i) = 0$（当 $j \neq i$），得到

$$u_h(x_i) = \sum_{j=1}^{N-1} U_j \phi_j(x_i) = U_i, \qquad i = 1,\dots,N-1. \tag{3.3.18}$$

因此，U_i 就是有限元近似解在节点 x_i 处的函数值。结合前面的唯一性结论，$\phi_1,\dots,\phi_{N-1}$ 构成了 V_h 的一组基。

回到3.3.1 节引例中的两单元网格，它只有一个内部节点 $x_1 = 1/2$，因而 $u_h(x) = U_1 \phi_1(x)$。这里的 U_1 正是折线中点尚待确定的高度。一般情况下，有限元节点值 U_i 是未知量，不能预先认为它等于精确解的节点值 $u(x_i)$。

3.3.4 Galerkin 离散与线性方程组

3.3.4.1 Galerkin 有限维弱形式

通过帽函数展开，近似函数 u_h 已经由 $N-1$ 个未知系数 $U_1,\dots,U_{N-1}$ 表示。但是，仅有这一表示还不能确定这些系数。接下来需要利用前面建立的弱形式，给出确定它们的方程。

原弱问题要求：求 $u\in V$，使积分关系对任意 $v\in V$ 成立。有限元方法将试探函数和检验函数都限制在子空间 $V_h\subset V$ 中，得到以下 Galerkin 有限元问题：求 $u_h\in V_h$，使得

$$\int_0^1 u_h'(x)v_h'(x)\,\mathrm{d}x=\int_0^1 f(x)v_h(x)\,\mathrm{d}x,\qquad \forall v_h\in V_h. \tag{3.3.19}$$

这里，u_h 所在的试探空间和 v_h 所在的检验空间均为 V_h，这就是本节所采用的 Galerkin 方法。与原弱问题相比，离散后只要求积分关系对 V_h 中的全部检验函数成立，而不要求对整个 V 中的全部函数成立。

虽然上式包含“任意检验函数”，但 V_h 已经是有限维空间。由于帽函数 $\phi_1,\dots,\phi_{N-1}$ 构成它的一组基，且积分关系对检验函数是线性的，只需依次取 $v_h=\phi_i$（$i=1,\dots,N-1$），即可得到 $N-1$ 个方程，用来确定 $N-1$ 个节点未知量。

至此，我们不仅用有限个系数表示了有限元近似解，还得到了确定这些系数的有限个方程。下一部分将把积分方程整理成线性方程组，并介绍逐单元计算与整体装配。

3.3.4.2 刚度矩阵与载荷向量

上一部分已经说明，$\phi_1,\dots,\phi_{N-1}$ 构成有限元空间 V_h 的一组基。由于 Galerkin 积分关系对检验函数是线性的，只需依次取 $v_h=\phi_i$（$i=1,\dots,N-1$）。于是有限维弱形式可写为

$$\int_0^1 u_h'(x)\phi_i'(x)\,\mathrm{d}x=\int_0^1 f(x)\phi_i(x)\,\mathrm{d}x,\qquad i=1,\dots,N-1. \tag{3.3.20}$$

由上一小节的帽函数展开式，有限元近似解为

$$u_h(x)=\sum_{j=1}^{N-1}U_j\phi_j(x). \tag{3.3.21}$$

在每个单元内部求导，得到

$$u_h'(x)=\sum_{j=1}^{N-1}U_j\phi_j'(x). \tag{3.3.22}$$

这里的导数可按分段导数理解：虽然帽函数在个别节点处可能没有经典导数，但这不影响上述积分的计算。

将展开式代入 Galerkin 积分关系，并交换有限求和与积分的顺序，得

$$\sum_{j=1}^{N-1}U_j\int_0^1\phi_j'(x)\phi_i'(x)\,\mathrm{d}x=\int_0^1 f(x)\phi_i(x)\,\mathrm{d}x,\qquad i=1,\dots,N-1. \tag{3.3.23}$$

定义刚度矩阵（stiffness matrix）K 和载荷向量（load vector）F 的分量为

$$K_{ij}=\int_0^1\phi_j'(x)\phi_i'(x)\,\mathrm{d}x,\qquad F_i=\int_0^1 f(x)\phi_i(x)\,\mathrm{d}x,\quad i,j=1,\dots,N-1. \tag{3.3.24}$$

这样，上面的积分方程就成为

$$\sum_{j=1}^{N-1} K_{ij}U_j = F_i, \qquad i = 1, \dots, N-1. \tag{3.3.25}$$

将各个方程组合起来，便得到线性方程组

$$KU = F, \qquad U = [U_1 \quad U_2 \quad \cdots \quad U_{N-1}]^{\mathsf{T}}. \tag{3.3.26}$$

其中，$K \in \mathbb{R}^{(N-1)\times(N-1)}$，$F \in \mathbb{R}^{N-1}$，$U$ 是由内部节点未知值组成的向量。求得 U 后，便可由帽函数展开式重构近似解 u_h。

需要注意：这里的 K 和 F 只对应内部节点自由度，因为两端节点值已由齐次 Dirichlet 边界条件给定。后面介绍逐单元装配时，为了统一处理每个单元的两个端点，将先形成包含全部节点的装配矩阵和向量，再处理边界自由度；两种矩阵应使用不同记号。

3.3.4.3 逐单元积分与矩阵稀疏性

虽然刚度矩阵和载荷向量已经有了明确的积分表达式，但如何实际计算这些积分，还需要结合网格和帽函数的局部结构。

前面把区间 $[0,1]$ 划分为 N 个单元 $K_e = [x_{e-1}, x_e]$。由于整个区间上的积分等于各单元积分之和，因此

$$\begin{aligned} K_{ij} &= \sum_{e=1}^{N} \int_{K_e} \phi_j'(x)\phi_i'(x)\,\mathrm{d}x, \\ F_i &= \sum_{e=1}^{N} \int_{K_e} f(x)\phi_i(x)\,\mathrm{d}x. \end{aligned} \tag{3.3.27}$$

这表明，整体矩阵和向量的各个分量，都可以由单元上的积分贡献累加得到。

进一步，帽函数 ϕ_i 只在节点 x_i 左右的两个相邻单元上可能非零。因此，当 $|i-j|>1$ 时，ϕ_i' 和 ϕ_j' 不会在同一个单元内部同时非零，从而有

$$K_{ij} = 0, \qquad |i-j| > 1. \tag{3.3.28}$$

也就是说，对本节的一维连续分片线性有限元，刚度矩阵 K 只有主对角线及其相邻两条对角线上可能出现非零元素，即它是一个三对角稀疏矩阵。

在每个单元内部，最多只有与该单元两个端点对应的节点形函数非零。因此，可以先计算这个单元产生的小矩阵和小向量，再按照局部节点与全局节点的对应关系，将各单元贡献累加到整体矩阵和向量的相应位置。这一过程称为装配（assembly）。

为了清楚展示单元计算与装配过程，后文将首先使用包含全部节点的全局量 K^{all}、F^{all}。对本节的齐次 Dirichlet 边界条件，消去边界自由度后，其内部节点子系统恰好对应前面得到的 $KU = F$。下面先计算一个线性单元的局部刚度矩阵。

3.3.5 单元计算、整体装配与边界条件

3.3.5.1 线性单元形函数与局部刚度矩阵

上一小节已经说明，整体刚度矩阵中的积分可以分解为各单元上的积分。为了计算这些单元贡献，考虑第 e 个单元

$$K_e = [x_{e-1}, x_e], \qquad h_e = x_e - x_{e-1} > 0, \qquad e = 1, 2, \dots, N. \tag{3.3.29}$$

在 K_e 上，线性函数由两个端点值唯一确定。为此，定义两个局部形函数：

$$N_1(x)=\frac{x_e-x}{h_e},\qquad N_2(x)=\frac{x-x_{e-1}}{h_e},\qquad x\in K_e. \tag{3.3.30}$$

其中，N_1 在左端点取 1、右端点取 0；N_2 则恰好相反。它们分别对应左、右端点的节点值。设 $U_i=u_h(x_i)$，则有限元近似函数在该单元上的表达式为

$$u_h(x)=U_{e-1}N_1(x)+U_eN_2(x),\qquad x\in K_e. \tag{3.3.31}$$

这里省略了局部形函数对单元编号 e 的依赖；对于内部节点，它们正是前面全局帽函数在该单元上的限制。局部计算同时保留单元两端的节点，但全局有限元空间 V_h 的独立未知量只对应内部节点。对于本节的齐次 Dirichlet 问题，边界节点值已知，即 $U_0=U_N=0$。

对两个局部形函数求导，得到

$$(N_1)'(x)=-\frac{1}{h_e},\qquad (N_2)'(x)=\frac{1}{h_e}. \tag{3.3.32}$$

与整体刚度矩阵的定义相对应，第 e 个单元的局部刚度矩阵定义为

$$K^{(e)}=\begin{bmatrix}\displaystyle\int_{K_e}\big((N_1)'\big)^2\,\mathrm{d}x & \displaystyle\int_{K_e}(N_1)'(N_2)'\,\mathrm{d}x\\[2ex] \displaystyle\int_{K_e}(N_2)'(N_1)'\,\mathrm{d}x & \displaystyle\int_{K_e}\big((N_2)'\big)^2\,\mathrm{d}x\end{bmatrix}. \tag{3.3.33}$$

由于形函数的导数在单元内均为常数，例如

$$\begin{aligned}\int_{K_e}\big((N_1)'\big)^2\,\mathrm{d}x&=\frac{1}{h_e^2}(x_e-x_{e-1})=\frac{1}{h_e},\\ \int_{K_e}(N_1)'(N_2)'\,\mathrm{d}x&=-\frac{1}{h_e^2}(x_e-x_{e-1})=-\frac{1}{h_e}.\end{aligned} \tag{3.3.34}$$

其余两项同理，于是得到

$$K^{(e)}=\frac{1}{h_e}\begin{bmatrix}1 & -1\\ -1 & 1\end{bmatrix}. \tag{3.3.35}$$

这个结果表明，对当前的一维 Poisson 问题，线性单元的局部刚度矩阵只由单元长度 h_e 决定。它描述的是该单元对整体矩阵的贡献，而不是一个需要单独求解的方程组。

3.3.5.2 局部载荷向量

除了刚度矩阵，还需要计算源项 $f(x)$ 对各节点的贡献。与整体载荷向量的定义对应，第 e 个单元的局部载荷向量为

$$F^{(e)}=\begin{bmatrix}F_1^{(e)}\\ F_2^{(e)}\end{bmatrix}=\begin{bmatrix}\displaystyle\int_{K_e}f(x)N_1(x)\,\mathrm{d}x\\[2ex] \displaystyle\int_{K_e}f(x)N_2(x)\,\mathrm{d}x\end{bmatrix}. \tag{3.3.36}$$

它的两个分量分别表示该单元对左、右端点载荷的贡献。若源项在单元上为常数 $f(x)=c$，由于两个局部形函数在单元上的积分均为 $h_e/2$，便有

$$F^{(e)}=\frac{ch_e}{2}\begin{bmatrix}1\\1\end{bmatrix}. \tag{3.3.37}$$

例如，3.3.1 节引例中的模型取 $f(x)=1$，因此每个单元的局部载荷向量为 $\frac{h_e}{2}[1,1]^{\mathsf T}$。对于一般的 $f(x)$，若积分可以解析计算，就直接求积分；否则可采用 Gauss 求积等数值积分方法。

至此，每个单元都产生了一对局部计算结果：刚度矩阵 $\boldsymbol{K}^{(e)}$ 与载荷向量 $\boldsymbol{F}^{(e)}$。下一步需要将它们组合成描述整个区间的矩阵和向量。

3.3.5.3 整体矩阵与载荷向量的装配

所谓装配（assembly），就是根据局部节点与全局节点的对应关系，将各单元的贡献累加到整体矩阵和载荷向量中。为了与前面只包含内部自由度的 $\boldsymbol{K}$、$\boldsymbol{F}$ 区分，这里将尚未处理边界条件、包含全部节点的装配结果记为 $\boldsymbol{K}^{\mathrm{all}}$ 和 $\boldsymbol{F}^{\mathrm{all}}$。

回到3.3.1 节引例中的两单元网格，取

$$x_0 = 0, \qquad x_1 = \frac{1}{2}, \qquad x_2 = 1. \tag{3.3.38}$$

两个单元分别为 $\boldsymbol{K}_1 = [x_0, x_1]$ 和 $\boldsymbol{K}_2 = [x_1, x_2]$，长度均为 $h_1 = h_2 = 1/2$。根据上一小节的结果，两个局部刚度矩阵相同：

$$K^{(1)} = K^{(2)} = 2\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & -2 \\ -2 & 2 \end{bmatrix}. \tag{3.3.39}$$

第一个单元的局部节点对应全局节点 $(0,1)$，因此对全部节点组成的矩阵，其贡献位于第 $0,1$ 行及第 $0,1$ 列：

$$\begin{bmatrix} 2 & -2 & 0 \\ -2 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix}. \tag{3.3.40}$$

第二个单元对应全局节点 $(1,2)$，其贡献位于第 $1,2$ 行及第 $1,2$ 列：

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & -2 \\ 0 & -2 & 2 \end{bmatrix}. \tag{3.3.41}$$

将这两部分逐项相加，得到未经边界条件处理的完整装配矩阵

$$K^{\mathrm{all}} = \begin{bmatrix} 2 & -2 & 0 \\ -2 & 4 & -2 \\ 0 & -2 & 2 \end{bmatrix}. \tag{3.3.42}$$

下面的示意图直观地展示了两个单元的局部刚度矩阵如何按共享节点累加装配成整体刚度矩阵：

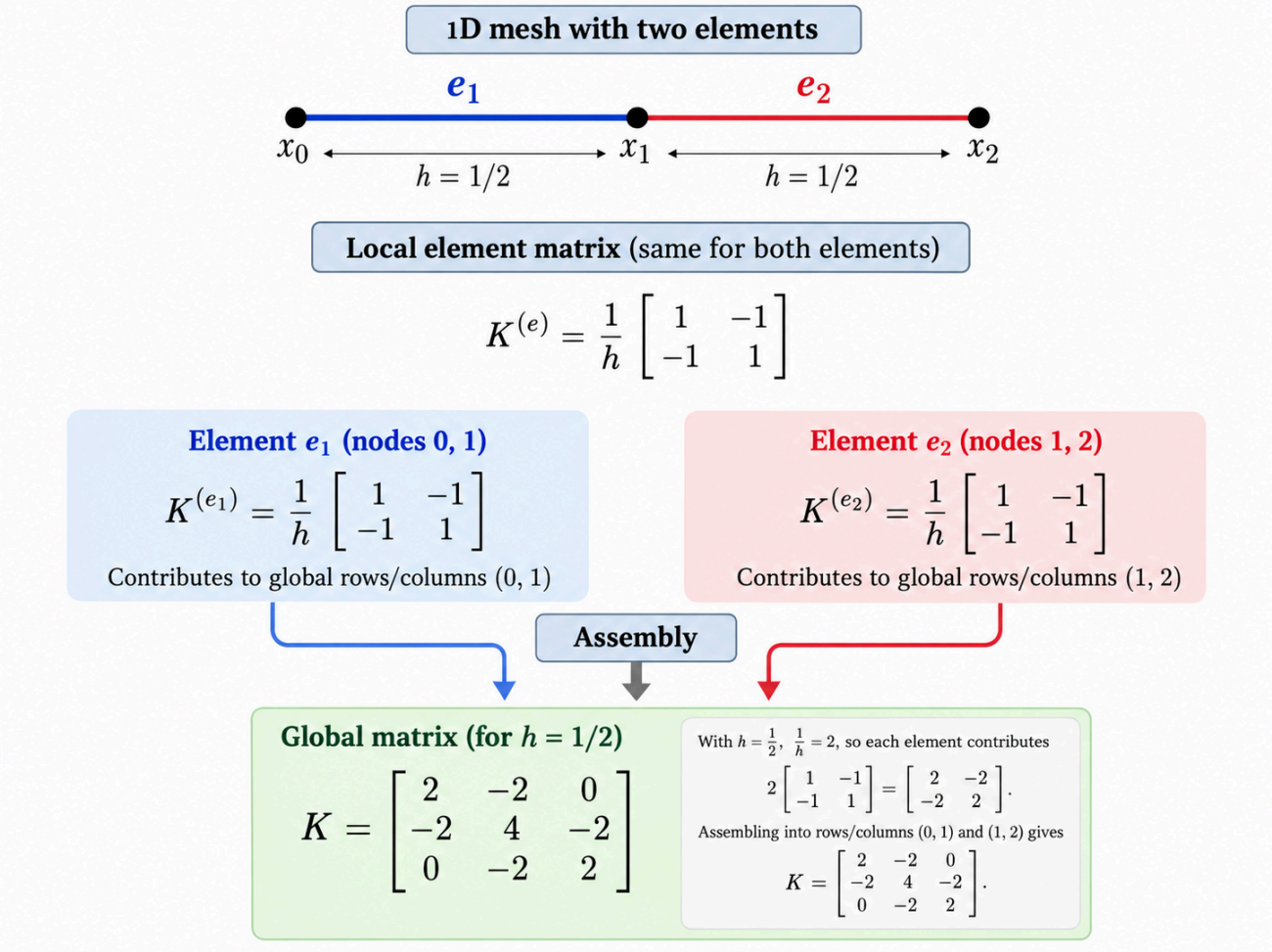


图 3.3-4　局部刚度矩阵的整体装配示意

如上图所示，中间节点 x_1 同时属于左右两个单元，因此两个单元对该节点的刚度贡献相加，使完整装配矩阵对应的对角元素成为 $2+2=4$。这正体现了共享节点处累加贡献的装配规则。

载荷向量也按照相同的节点对应关系装配。若继续取3.3.1 节引例中的 $f(x)=1$，由于两个单元长度均为 $1/2$，其局部载荷向量都是

$$F^{(1)} = F^{(2)} = \begin{bmatrix} 1/4 \\ 1/4 \end{bmatrix}. \tag{3.3.43}$$

因此，完整装配载荷向量为

$$F^{\text{all}} = \begin{bmatrix} 1/4 \\ 1/4 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 1/4 \\ 1/4 \end{bmatrix} = \begin{bmatrix} 1/4 \\ 1/2 \\ 1/4 \end{bmatrix}. \tag{3.3.44}$$

应当注意，K^{all} 与 F^{all} 只是包含全部节点的装配结果，不能直接当作前文内部自由度方程 $KU=F$ 中的 K 和 F。本节的齐次 Dirichlet 边界条件已给定 $U_0=U_2=0$，所以真正需要求解的只有内部节点值 U_1。如何从完整装配结果得到最终的求解系统，将在下一小节说明。

3.3.5.4 Dirichlet 边界条件的施加

上一小节通过逐单元装配，得到了包含全部节点的刚度矩阵 K^{all} 和载荷向量 F^{all}。但这还不是可以直接求解的线性系统，因为本节的齐次 Dirichlet 边界条件已经规定

$$u(0)=u(1)=0, \qquad U_0=U_N=0. \tag{3.3.45}$$

也就是说，两端节点值是已知的，真正需要确定的只有内部节点值。更重要的是，前面建立的弱形式仅要求积分关系对端点为零的检验函数成立，因此不能直接要求完整装配矩阵的每一行都满足 $K^{\mathrm{all}}U^{\mathrm{all}}=F^{\mathrm{all}}$。

将内部节点集合记为 $I=\{1,\dots,N-1\}$，边界节点集合记为 $B=\{0,N\}$。为了书写方便，按内部节点和边界节点的顺序排列相应的矩阵与向量：

$$K^{\mathrm{all}}=\begin{bmatrix}K_{II}^{\mathrm{all}} & K_{IB}^{\mathrm{all}}\\ K_{BI}^{\mathrm{all}} & K_{BB}^{\mathrm{all}}\end{bmatrix},\qquad F^{\mathrm{all}}=\begin{bmatrix}F_I^{\mathrm{all}}\\ F_B^{\mathrm{all}}\end{bmatrix},\qquad U^{\mathrm{all}}=\begin{bmatrix}U_I\\ U_B\end{bmatrix}. \tag{3.3.46}$$

这里，U_I 是内部节点未知值组成的向量，U_B 是已知的边界节点值向量。由于检验函数在两端为零，有限元弱形式所对应的是内部节点方程

$$K_{II}^{\mathrm{all}}U_I+K_{IB}^{\mathrm{all}}U_B=F_I^{\mathrm{all}}. \tag{3.3.47}$$

将已知的边界节点值移到右端，得到真正需要求解的系统：

$$K_{II}^{\mathrm{all}}U_I=F_I^{\mathrm{all}}-K_{IB}^{\mathrm{all}}U_B. \tag{3.3.48}$$

对于本节的齐次 Dirichlet 边界条件，$U_B=0$，因此上式简化为

$$K_{II}^{\mathrm{all}}U_I=F_I^{\mathrm{all}}. \tag{3.3.49}$$

这恰好是前面由内部节点帽函数直接推导的 $KU=F$，其中 $K=K_{II}^{\mathrm{all}}$、$F=F_I^{\mathrm{all}}$、$U=U_I$。

现在完成3.3.1 节引例中的两单元计算。上一小节已经得到

$$K^{\mathrm{all}}=\begin{bmatrix}2 & -2 & 0\\ -2 & 4 & -2\\ 0 & -2 & 2\end{bmatrix},\qquad F^{\mathrm{all}}=\begin{bmatrix}\frac{1}{4}\\ \frac{1}{2}\\ \frac{1}{4}\end{bmatrix}. \tag{3.3.50}$$

因为只有中间节点 $x_1=1/2$ 是内部节点，所以只需取矩阵的中间一行，并代入 $U_0=U_2=0$：

$$-2U_0+4U_1-2U_2=\frac{1}{2}\quad\Longrightarrow\quad 4U_1=\frac{1}{2}\quad\Longrightarrow\quad U_1=\frac{1}{8}. \tag{3.3.51}$$

于是有限元近似解为 $u_h(x)=\frac{1}{8}\phi_1(x)$。这个中点值与3.3.1 节引例中精确解 $u(x)=x(1-x)/2$ 在 $x=1/2$ 处的取值一致，但两单元折线在单元内部仍不是精确解。

由此可见，施加 Dirichlet 边界条件不是求解结束后才将两端节点值“补成零”，而是在确定未知自由度和建立最终求解系统时就必须考虑。完整装配矩阵的边界节点行并不需要与 F_B^{all} 逐项相等；这些位置由给定的边界条件约束。

3.3.6 数值实现与收敛性验证

为统一记号，下文以 $U_i=u_h(x_i)$ 表示有限元解的节点值，以 W_i 表示中心差分法的节点值；$K^{(e)}$、$F^{(e)}$ 表示单元刚度矩阵和单元载荷向量，K^{all}、F^{all} 表示尚未施加边界条件的完整装配结果，K、F 则仅表示内部自由度系统中的矩阵和向量。

前面已经通过弱形式、连续分片线性有限元空间及 Galerkin 方法，将一维 Poisson 边值问题转化为线性方程组，并完成了单元计算、整体装配与 Dirichlet 边界处理。但是，能够求出节点值并不意味着数值方法已经得到验证：除了检查计算过程是否正确，还要考察近似解在整个区间上的误差。以下采用制造解进行数值验证，并在同一网格上与二阶中心差分法比较，考察节点误差、积分误差以及网格加密时的收敛规律。

3.3.6.1 实验目的与预期误差阶

在相同的一维均匀网格上实现连续分片线性 Galerkin 有限元（FEM）和二阶中心差分法（FDM），重点回答三个问题：

1. 单元矩阵、局部载荷向量、整体装配和边界条件处理是否与前面的理论推导一致？

2. FEM 与 FDM 的节点值能否逼近精确解？FEM 在节点处准确，是否意味着其在单元内部也同样准确？

3. 网格加密后，各种误差以怎样的速度减小？数值结果是否符合理论预期？

理论预期。 对本节的光滑一维 Poisson 模型，若单元积分准确计算、精确解满足 $u \in H^2(0,1) \cap H_0^1(0,1)$，则连续分片线性有限元满足相应的典型误差估计：

$$|u-u_h|_{H^1(0,1)} \le Ch\|u''\|_{L^2(0,1)}, \qquad \|u-u_h\|_{L^2(0,1)} \le Ch^2\|u''\|_{L^2(0,1)}.$$

这里 C 与网格尺寸 h 无关。因而预期 FEM 的 H^1 半范数误差约为一阶、L^2 误差约为二阶；在本实验的光滑模型上，中心差分法的节点最大误差预期约为二阶。上述误差指标的定义不同，不能仅凭其数值大小判断某一种方法普遍更优。

3.3.6.2 模型问题与计算设置

采用制造解（manufactured solution）方法，先指定一个已知的光滑函数

$$u(x)=\sin(\pi x), \qquad 0 \le x \le 1,$$

再由 $u''(x)=-\pi^2\sin(\pi x)$ 得到待求解的边值问题

$$\begin{cases} -u''(x)=f(x)=\pi^2\sin(\pi x), & 0<x<1, \\ u(0)=u(1)=0. \end{cases}$$

这样便有已知的精确解，可用于评价数值误差。计算 FEM 和 FDM 时，只使用源项 $f(x)$ 与边界数据；已知函数 $u(x)$ 仅用于计算误差，不直接提供内部节点的数值解。

将区间 $[0,1]$ 均匀划分为 N 个单元，取 $x_i=i/N$、$h=1/N$，依次使用 $N=8,16,32,64,128$。FEM 与 FDM 采用相同节点。程序使用 Python 3、NumPy 和 Matplotlib，局部刚度矩阵由解析公式给出；FEM 载荷向量采用每单元四点 Gauss–Legendre 求积，计算积分误差时使用每单元十二点 Gauss–Legendre 求积，以减小误差评估时的数值积分偏差。本实验不涉及随机过程。

3.3.6.3 两种数值离散方法

（1）Galerkin 有限元。 在单元 $K_e=[x_{e-1},x_e]$ 上，$h_e=x_e-x_{e-1}$，利用两个局部形函数 N_1,N_2，计算

$$K^{(e)}=\frac{1}{h_e}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \qquad F^{(e)}=\begin{bmatrix} \displaystyle\int_{K_e} f(x)N_1(x)\,\mathrm{d}x \\ \displaystyle\int_{K_e} f(x)N_2(x)\,\mathrm{d}x \end{bmatrix}.$$

先将各单元贡献累加为完整装配矩阵 K^{all} 和载荷向量 F^{all}，再结合 $U_0=U_N=0$，求解内部节点方程

$$K^{\mathrm{all}}_{II}U_I=F^{\mathrm{all}}_I.$$

最后恢复全部节点值，利用局部线性插值重构连续分片线性函数 u_h。这里的 K_{II}^{all} 是施加边界条件后真正参与求解的内部自由度矩阵，不应与 K^{all} 混淆。

（2）中心差分法。 在相同的内部节点上，采用二阶中心差分近似

$$-u''(x_i) \approx \frac{-W_{i-1}+2W_i-W_{i+1}}{h^2}, \qquad i=1,\dots,N-1.$$

结合 $W_0=W_N=0$ 构造三对角方程组，求得差分节点值 W_i。FDM 首先产生节点数值，而 FEM 还给出单元之间的连续分片线性近似。比较节点值时使用同一类节点误差，评价 FEM 整个区间上的精度时还应使用积分误差。

3.3.6.4 误差指标与经验收敛阶

在共同网格节点上，分别记录

$$E_\infty^{\mathrm{FEM}}=\max_{0\le i\le N}|u(x_i)-U_i|, \qquad E_\infty^{\mathrm{FDM}}=\max_{0\le i\le N}|u(x_i)-W_i|.$$

对于 FEM，还计算整个区间上的两种误差：

$$E_{L^2}^{\mathrm{FEM}}=\left(\sum_{e=1}^{N}\int_{K_e}(u-u_h)^2\,\mathrm{d}x\right)^{1/2},$$

$$E_{H^1}^{\mathrm{FEM}}=\left(\sum_{e=1}^{N}\int_{K_e}(u'-u_h')^2\,\mathrm{d}x\right)^{1/2}.$$

其中 $E_{H^1}^{\mathrm{FEM}}$ 表示 H^1 半范数误差，并非完整的 H^1 范数误差。两个积分误差采用单元上的高阶数值求积近似计算。

若某种误差满足 $E_h\approx Ch^p$，则当网格尺寸从 h 减半为 $h/2$ 时，经验收敛阶为

$$p_h=\frac{\log(E_h/E_{h/2})}{\log 2}.$$

这一指标只有在误差明显大于数值舍入误差时才具有可靠的解释意义。

3.3.6.5 计算步骤与程序核验

算法 3.3-1　一维 Galerkin 有限元与中心差分的对比计算流程

可按照如下步骤编写程序：

1. 建立均匀网格并确定局部节点和全局节点的对应关系；
2. 计算各单元 $K^{(e)}$ 与 $F^{(e)}$，完成 K^{all}、F^{all} 的装配；
3. 施加齐次 Dirichlet 边界条件，求解内部自由度并重构 u_h；
4. 在相同节点上实现中心差分法，得到 W_i；
5. 计算节点误差、FEM 积分误差和内部线性系统残差；
6. 对各级网格重复计算，整理误差表、收敛阶表和数值图像。

正式实验之前，先用前文的两单元问题 $-u''=1$、$u(0)=u(1)=0$ 检查程序。应得到

$$K^{\mathrm{all}}=\begin{bmatrix}2&-2&0\\-2&4&-2\\0&-2&2\end{bmatrix},\qquad F^{\mathrm{all}}=\begin{bmatrix}\frac{1}{4}\\\frac{1}{2}\\\frac{1}{4}\end{bmatrix},\qquad U_1=\frac{1}{8}.$$

同时检查完整刚度矩阵的对称性、内部自由度矩阵的正定性、端点边界值以及求解残差。若上述基本检查不通过，应先修正代码，再进行网格加密实验。根据算法 3.3-1 可编写 `fem_fdm_experiment.py`；本节给出了程序应满足的校验基准与数值结果。

3.3.6.6 数值结果与收敛阶分析

（1）数值解的图像比较

先取 $N=8$，比较精确解、连续分片线性有限元解和中心差分节点值。

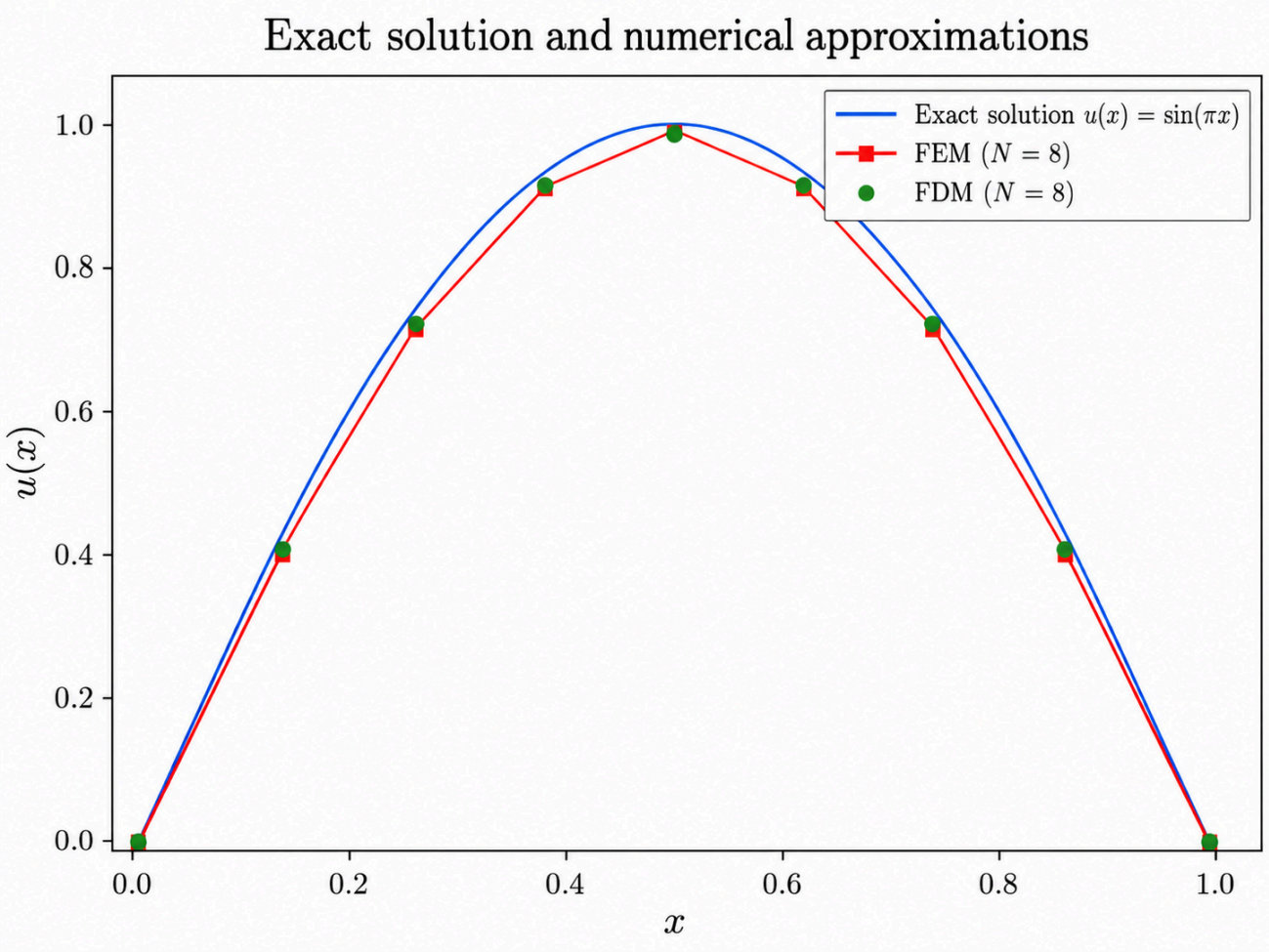


图 3.3-5　N = 8 时一维 Poisson 问题的精确解、分片线性有限元解和中心差分节点值。FEM 在相邻节点间以直线连接，FDM 以离散节点标记显示。

由图 3.3-5 可见，两种方法都能较好地描述解的变化趋势，但它们产生的近似对象不同。以中点 $x=1/2$ 为例，精确解为 $u(1/2)=1$；本实验中四点 Gauss 求积的 FEM 中点值约为 1，而中心差分法得到 $W_{N/2}\approx 1.012951$。即使 FEM 在节点处十分准确，它的单元内部仍为直线段，与精确的正弦曲线并不重合。图像的视觉接近不足以说明收敛速度，需要进一步计算误差。

（2）误差随网格加密的变化

表 3.3-1 给出了不同网格下的 FEM 整体误差和 FDM 节点最大误差。FEM 的节点最大误差另在下文讨论，避免把接近浮点舍入量级的数字用于不可靠的收敛阶计算。

表 3.3-1　网格加密下的数值误差（FEM 载荷采用四点 Gauss 求积）

N	h	$E_{L^2}^{\mathrm{FEM}}$	$E_{H^1}^{\mathrm{FEM}}$	$E_{\infty}^{\mathrm{FDM}}$
8	0.125000	9.920920×10^{-3}	2.511818×10^{-1}	1.295075×10^{-2}
16	0.062500	2.486501×10^{-3}	1.258332×10^{-1}	3.218964×10^{-3}
32	0.031250	6.220178×10^{-4}	6.294691×10^{-2}	8.035777×10^{-4}
64	0.015625	1.555290×10^{-4}	3.147724×10^{-2}	2.008218×10^{-4}
128	0.0078125	3.888378×10^{-5}	1.573910×10^{-2}	5.020092×10^{-5}

随着 N 增大、h 减小，表中三种误差都逐步下降。为了判断下降规律，将相邻两级网格的误差代入前述经验收敛阶公式，得到表 3.3-2。

表 3.3-2　网格加密时的经验收敛阶

细网格单元数 N	FEM L^2	FEM H^1 半范数	FDM 节点最大误差
16	1.9964	0.9972	2.0084
32	1.9991	0.9993	2.0021
64	1.9998	0.9998	2.0005
128	1.9999	1.0000	2.0001

例如，根据 $N=8$ 与 $N=16$ 的 FEM L^2 误差，有

$$p_h=\frac{\log\left(\frac{9.920920\times10^{-3}}{2.486501\times10^{-3}}\right)}{\log 2}\approx 1.9964,$$

接近二阶。图 3.3-6 使用双对数坐标展示误差随网格尺寸变化的趋势。

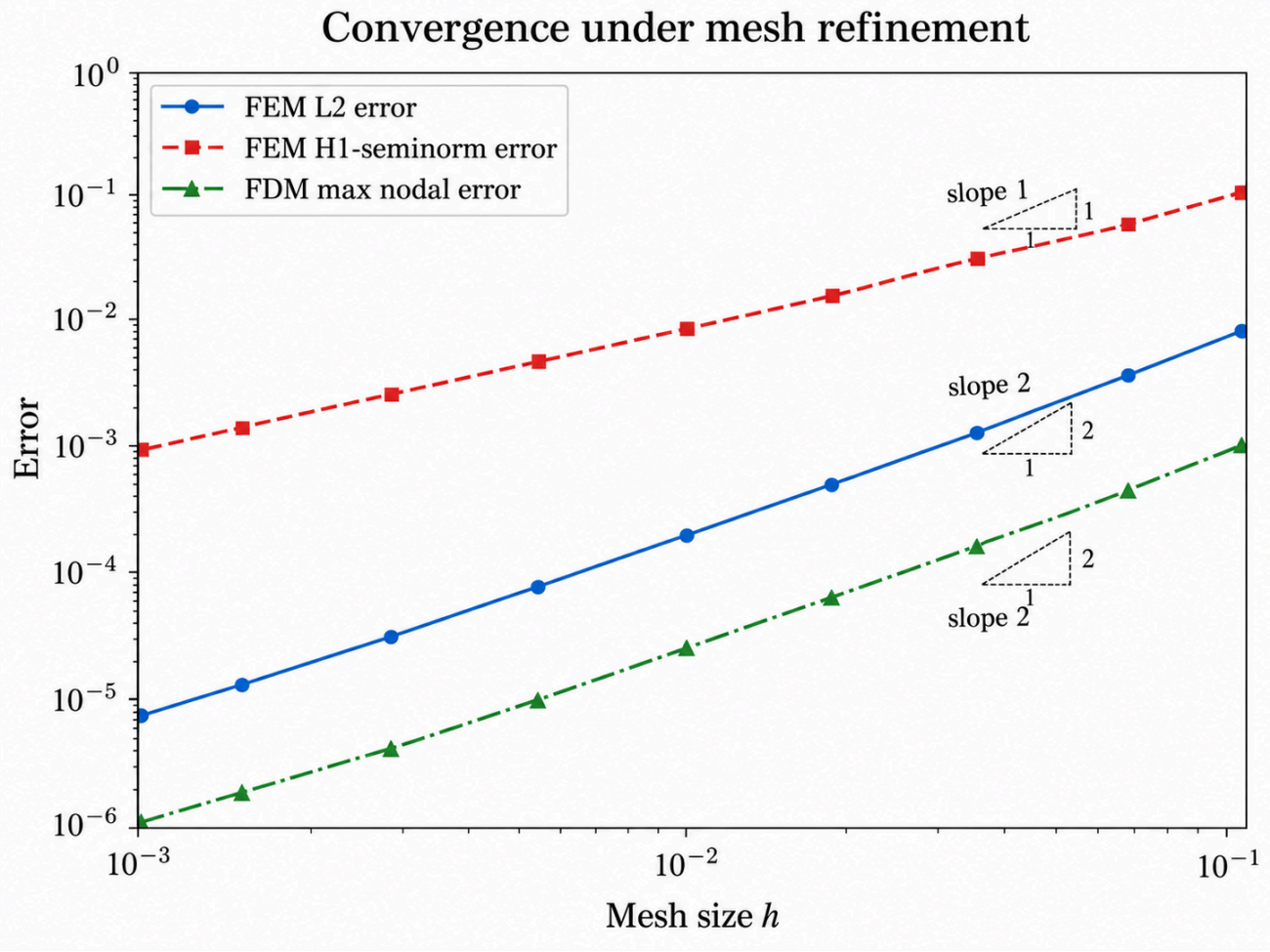


图 3.3-6　网格加密下的误差收敛曲线。横轴为网格尺寸 h，纵轴为误差，均采用对数坐标；三条曲线对应不同的误差指标。

若 $E_h \approx Ch^p$，则

$$\log E_h \approx \log C + p \log h,$$

双对数曲线的斜率近似为 p。从表 3.3-2 与图 3.3-6 可以看出，本实验中 FEM 的 L^2 误差表现为约二阶收敛、H^1 半范数误差约为一阶收敛，FDM 节点最大误差约为二阶收敛，与运行前预测一致。这些结论来自同一种误差指标在不同网格上的变化，而不是对不同指标绝对数值大小的直接比较。

（3）有限元节点值与精确解的关系

本实验采用四点 Gauss 求积时，$N = 8$ 的 FEM 节点最大误差约为 2.24×10^{-12}，加密后接近浮点舍入量级，因此不适合用这组节点误差计算收敛阶。这是当前一维常系数 Poisson 问题在精确载荷积分时的一项特殊性质，而不是一般有限元方法都具有的结论。

设 $I_h u$ 为精确解的分片线性节点插值函数，则

$$(u - I_h u)(x_i) = 0.$$

由于任意 $v_h \in V_h$ 在每个单元 K_e 上的导数是常数，有

$$\int_{K_e} (u - I_h u)' v_h' \, \mathrm{d}x = v_h'|_{K_e} \left[(u - I_h u)(x_e) - (u - I_h u)(x_{e-1}) \right] = 0.$$

对所有单元求和，结合精确弱形式和离散问题解的唯一性，可得

$$u_h = I_h u.$$

也就是说，在精确积分的理想情形下，FEM 节点值与精确解的节点值一致；但在单元内部，正弦曲线与线性插值一般不同，因此 L^2 与 H^1 半范数误差仍然不为零。这个现象也解释了3.3.1 节引例中的两单元折线为什么能够准确得到中间节点值。

3.3.7 方法比较与边界项讨论

3.3.7.1 Galerkin 有限元与中心差分法

在一维均匀网格 Poisson 问题中，有限差分和有限元最终得到的矩阵可能很像。

中心差分从 $-u''(x_i)$ 出发，用

$$-u''(x_i) \approx \frac{-W_{i-1} + 2W_i - W_{i+1}}{h^2}$$

直接近似微分算子。

而有限元从弱形式

$$\int_0^1 u'v' \,\mathrm{d}x = \int_0^1 fv \,\mathrm{d}x$$

出发，再通过基函数得到

$$K_{ij} = \int_0^1 \phi_j' \phi_i' \,\mathrm{d}x.$$

因此，两者在某些模型中具有相似的系数矩阵，但离散原理并不相同：有限差分法近似微分算子，Galerkin 有限元法近似变分问题。

3.3.7.2 积分分部与 Neumann 边界条件

分部积分得到

$$\int_0^1 u'v' \,\mathrm{d}x - [u'v]_0^1 = \int_0^1 fv \,\mathrm{d}x.$$

在本节中，因为 $v(0) = v(1) = 0$，所以边界项才消失。

如果右端采用 Neumann 条件，例如

$$u'(1) = g_N,$$

而左端仍为 Dirichlet 条件，则检验函数通常只要求 $v(0) = 0$。这时

$$[u'v]_0^1 = u'(1)v(1) = g_N v(1),$$

因此弱形式变为

$$\int_0^1 u'v' \,\mathrm{d}x = \int_0^1 fv \,\mathrm{d}x + g_N v(1).$$

因此，边界积分项是否保留取决于具体的边界条件；对 Neumann 条件，边界数据通常作为右端线性泛函的一部分进入弱形式。

3.3.8 本节小结

本节以一维 Poisson 边值问题为例，建立了从强形式到弱形式、从有限元空间到 Galerkin 离散、从局部单元计算到整体装配与边界处理的完整联系。分部积分降低了方程对未知函数经典导数的要求，使连续分片线性函数能够用于有限元近似；帽函数则把求解未知函数转化为求解有限个节点值。

通过单元刚度矩阵与载荷向量的计算、整体装配以及 Dirichlet 边界条件处理，原边值问题最终转化为可以由计算机求解的线性代数系统。实验又表明，在本节所选的光滑一维模型上，FEM 的 H^1 半范数误差约按一阶减小、L^2 误差约按二阶减小，中心差分法的节点最大误差约为二阶。

需要特别注意：节点值接近精确解，并不意味着整个分片线性近似函数处处准确。 应根据研究目标选取适当的误差指标，并通过网格加密检验数值解的收敛规律。有限元方法的核心也不只是得到一个与差分法形式相似的矩阵，而是先建立弱形式，再在有限维函数空间中进行 Galerkin 近似，最终形成可计算的代数系统。

3.3.9 习题

习题 3.3-1（节点误差与整体误差） 为什么 FEM 在本实验中几乎准确地求出了全部节点值，H^1 半范数误差却仍不为零？

习题 3.3-2（数值求积的影响） 将 FEM 载荷积分由四点 Gauss 求积改为一点求积，观察节点误差如何变化。此时还能用节点误差计算稳定的经验收敛阶吗？

习题 3.3-3（非齐次 Dirichlet 条件） 若将一个端点改为非零 Dirichlet 边界值，求解器应该在哪一步引入已知边界值？为什么不能只在解出内部节点后覆盖边界节点值？

习题 3.3-4（两单元有限元计算） 考虑下列一维 Poisson 问题：

$$\begin{cases} -u''(x) = 1, & 0 < x < 1, \\ u(0) = u(1) = 0. \end{cases}$$

将区间划分为两个单元

$$K_1 = [0, 0.5], \qquad K_2 = [0.5, 1],$$

对应三个节点

$$x_0 = 0, \qquad x_1 = \frac{1}{2}, \qquad x_2 = 1.$$

完成以下计算，并与 3.3.5 节的两单元示例对照：

1. 写出两个单元的局部刚度矩阵；
2. 写出两个单元的局部载荷向量；
3. 完成整体矩阵和整体载荷向量的装配；
4. 说明 Dirichlet 边界条件如何进入计算；
5. 求内部节点值 U_1；
6. 用代码核对手算结果。

参考文献

[1] Larson M G, Bengzon F. The Finite Element Method: Theory, Implementation, and Applications. Springer, 2013. 参见第 2.1 节。

[2] Chandrupatla T R, Belegundu A D. Introduction to Finite Elements in Engineering. 5th ed. 参见第 1.6 节和第 4 章。

3.4 AI关联：Neural ODE与PINN（2学时）

导入：从数值积分走向科学机器学习

在第 3.1 节中，我们研究了一个基本问题：当微分方程的右端函数和初始条件已经给定时，怎样利用 Euler 方法、Runge–Kutta 方法等数值算法计算近似解，并通过误差和稳定性分析评价结果。这一研究以微分方程已知为前提。然而，在许多科学与工程问题中，我们面对的往往不只是“怎样求解方程”，还包括“怎样根据有限信息建立方程”。

例如，在某个动态过程的实验中，研究者可能只记录了若干时刻的系统状态，却尚未掌握状态随时间变化的准确规律。如果直接假设一种变化率函数，模型可能无法充分描述观测数据。一个自然的思路是：用带有待定参数的函数来表示未知变化率，根据观测轨迹确定参数，再通过数值积分得到连续时间的模型预测。Neural ODE 将这一思路与神经网络结合起来，使动力学模型的学习和常微分方程的数值求解相互联系。

另一些问题具有不同的特点：微分方程本身已知，但求解区域、边界条件或方程结构使解析求解较为困难。传统方法通过网格离散建立代数方程组；物理信息神经网络（PINN） 则尝试用可微的神经网络表示未知解，并利用微分方程残差、初始条件和边界条件构造训练目标。这种方法把已知数学规律引入函数近似过程，但仍需要对所得解进行数值检验。

因此，神经网络参与微分方程计算后，需要同时关注两个层面：模型是否正确表达了所研究的问题，以及计算过程是否足够准确地求解了该模型。本节将分别介绍 Neural ODE 与 PINN 的基本原理，分析模型误差、数值离散误差和训练误差之间的关系，并通过可复现算例说明独立验证的重要性。

本节讨论神经网络与微分方程的两种典型结合机制。学习后，应能够区分“学习微分方程的右端”与“学习满足微分方程的解”，写出相应的训练目标，理解数值积分器与初边值条件对学习结果的作用，并能设计有独立参照的最小数值实验。所需先修知识包括 3.1 节的初值问题、Euler 方法、RK4 方法、误差与稳定性，以及微积分、最小二乘法和导数的基本概念。

3.4.1 从已知方程求解到数据驱动建模

（一）三类需要区分的计算问题

考虑 3.1 节的一阶常微分方程初值问题

$$\begin{cases} y'(t) = f(t, y(t)), & t \in [t_0, T], \\ y(t_0) = y_0. \end{cases} \tag{3.4-1}$$

这里沿用原有约定：$y(t_n)$ 为精确解在 t_n 处的值，y_n 为数值积分所得的节点近似值，h 表示时间步长。当右端 f 已知时，研究重点是如何以适当的精度和成本求出 y_n；当 f 不完全已知时，问题就转化为从观测数据中识别演化规律。

另一类问题是边值问题，例如

$$\begin{cases} -u''(x) = g(x), & 0 < x < 1, \\ u(0) = 0, \quad u(1) = 0. \end{cases} \tag{3.4-2}$$

其中 x 是空间自变量，$u(x)$ 是待求函数，$g(x)$ 为已知源项。为了区别时间离散与空间离散，本节空间网格间距统一记为 Δx，不与时间步长 h 混用。

问题	已知的信息	主要未知量	典型方法
常规 ODE 数值求解	f 与初值 y_0	节点近似值 y_n	Euler、RK4、自适应积分
动力学辨识	观测轨迹与初值，f 未知或部分未知	变化率模型 f_θ	Neural ODE
方程约束的函数近似	微分算子、源项、初边值条件及可用数据	解的近似函数 u_θ	PINN

这一区分是理解本节的出发点。Neural ODE 与 PINN 不是彼此排斥的技术类别，而是两种典型的建模与训练方式；实际工作可以结合轨迹数据与方程残差。

（二）Neural ODE：学习微分方程右端

神经常微分方程（Neural Ordinary Differential Equation，Neural ODE）通常以含有可训练参数 θ 的函数 f_θ 近似未知动力学：

$$\boxed{\begin{cases} y_\theta'(t) = f_\theta(t, y_\theta(t)), \\ y_\theta(t_0) = y_{\theta,0}. \end{cases}} \tag{3.4-3}$$

其中 $y_\theta(t)$ 为学习到的连续时间模型的解，f_θ 一般由神经网络表示。应当特别注意：网络给出的是变化率，ODE 数值积分器据此生成状态轨迹。这正是它与直接把时间 t 输入网络并输出 $y(t)$ 的普通回归模型的区别。Chen 等人在 2018 年系统研究了这一连续深度建模方式，见参考文献 [3]。

对于给定的时间网格 $t_n = t_0 + nh$，用数值积分方法 S 求解式（3.4-3），记所得近似为 $y_{\theta,n}^{S,h}$。若选用向前 Euler，更新形式是

$$y_{\theta,n+1}^{\mathrm{FE},h} = y_{\theta,n}^{\mathrm{FE},h} + h f_\theta\left(t_n, y_{\theta,n}^{\mathrm{FE},h}\right). \tag{3.4-4}$$

若使用 3.1 节的 RK4，则把原公式中的 f 换成 f_θ 即可。由此可见，神经网络参数 θ、数值积分方法 S 以及步长 h 会共同影响训练时观察到的轨迹。

设存在 M 条观测轨迹，第 m 条轨迹在 t_j 处的观测为 $y_{m,j}^{\mathrm{obs}}$，并且观测时刻为 $t_0, \ldots, t_K$。对相应初值分别积分后，可以构造观测拟合目标

$$\boxed{\mathcal{L}_{\mathrm{obs}}(\theta; S, h) = \frac{1}{M(K+1)} \sum_{m=1}^{M} \sum_{j=0}^{K} \left| y_{\theta,m,j}^{S,h} - y_{m,j}^{\mathrm{obs}} \right|^2.} \tag{3.4-5}$$

其中 $y_{\theta,m,j}^{S,h}$ 是数值积分器输出的预测值，θ 由优化算法调整，使目标函数尽可能小。该目标只保证对指定的观测点和指定的求解器拟合得好；它并不直接保证学习到的 f_θ 就是真实的右端函数。

这里的参数优化可通过对离散积分步骤反向传播、伴随灵敏度法等途径实现。不同求导策略及数值求解器的误差应分别分析，不能认为自动微分会自动消除时间离散误差。有关这一点的后续研究见 [5]、[8]。

（三）PINN：学习满足方程与条件的解

物理信息神经网络（Physics-Informed Neural Network，PINN）通常令一个可微网络 $u_\theta(x)$ 直接表示未知解的近似，并利用已知微分方程构造训练约束。对于一般微分方程

$$\mathcal{N}[u](x) = g(x), \qquad x \in \Omega, \tag{3.4-6}$$

设 $\mathcal{N}$ 为已知微分算子、Ω 为定义域，定义网络的方程残差

$$r_\theta(x) = \mathcal{N}[u_\theta](x) - g(x). \tag{3.4-7}$$

选取 N_r 个内部配点 $x_i^{(r)}$，令

$$\mathcal{L}_r(\theta) = \frac{1}{N_r}\sum_{i=1}^{N_r} \left| r_\theta\left(x_i^{(r)}\right) \right|^2. \tag{3.4-8}$$

若边界条件写成 $\mathcal{B}[u](x) = b(x)$，在 N_b 个边界点 $x_j^{(b)}$ 上定义

$$\mathcal{L}_b(\theta) = \frac{1}{N_b}\sum_{j=1}^{N_b} \left| \mathcal{B}[u_\theta]\left(x_j^{(b)}\right) - b\left(x_j^{(b)}\right) \right|^2. \tag{3.4-9}$$

若另有观测数据，可增加数据损失 $\mathcal{L}_d$。常见的综合目标为

$$\boxed{\mathcal{L}_{\mathrm{PINN}}(\theta) = \lambda_r \mathcal{L}_r(\theta) + \lambda_b \mathcal{L}_b(\theta) + \lambda_d \mathcal{L}_d(\theta),} \tag{3.4-10}$$

其中 $\lambda_r, \lambda_b, \lambda_d \geqslant 0$ 为预先确定或在训练中调整的权重；没有观测数据时可取 $\lambda_d = 0$。在采用二阶微分方程时，可以通过自动微分获得网络关于 x 的二阶导数；但自动微分计算的是网络函数的导数，而非网络近似误差的证明。

式（3.4-10）采用边界条件的软约束形式。有时也可以通过结构设计使边界条件被精确满足。例如对于 $u(0) = u(1) = 0$，令

$$u_\theta(x) = x(1-x)v_\theta(x), \tag{3.4-11}$$

其中 v_θ 为可训练网络，则不论参数如何变化，都有 $u_\theta(0) = u_\theta(1) = 0$。这称为硬边界约束。硬约束可以消除本例的边界值误差，但不会自动保证内部微分方程成立。

PINN 的代表性工作由 Raissi、Perdikaris 和 Karniadakis 发表在 Journal of Computational Physics（2019），见 [4]。对于 PINN 的近似误差、训练误差、泛化误差及稳定性的系统数值分析，可参见 De Ryck 和 Mishra（2024）[7]。

（四）两种方法的本质比较

比较维度	Neural ODE 的典型做法	PINN 的典型做法
网络主要表示	未知变化率 $f_\theta(t,y)$	未知解 $u_\theta(x)$
解如何获得	用数值 ODE 积分器推进状态	直接由网络输出近似解

比较维度	Neural ODE 的典型做法	PINN 的典型做法
主要训练信号	观测轨迹误差	方程残差、边界条件残差及可选数据误差
主要数值风险	积分步长与求解器偏差、初值敏感性	配点覆盖不足、权重失衡、优化误差、边界处理
本节关注的验证方式	更换积分器和步长，用未见初值评估	检查完整方程与边界，用独立网格或解析解评估

因此，Neural ODE 典型地学习“状态怎样变化”，PINN 典型地学习“哪个函数满足给定方程与条件”。这一说法只概括两种常用架构，而不是对所有混合方法的限制。

3.4.2 为什么训练损失不等于方程求解误差

（一）Neural ODE 中的三类误差

设真实轨迹满足式（3.4-1），而学习到的连续时间轨迹满足式（3.4-3）。在比较时，为简化记号，记真实初值为 y_0、模型初值为 $y_{\theta,0}$。定义连续模型误差 $e(t)=y_\theta(t)-y(t)$。

假设在两条轨迹经过的区域，$f_\theta(t,\cdot)$ 对状态变量满足 Lipschitz 条件

$$|f_\theta(t,a)-f_\theta(t,b)|\leqslant L|a-b|, \tag{3.4-12}$$

并且沿真实轨迹存在统一的模型误差界

$$|f_\theta(t,y(t))-f(t,y(t))|\leqslant \varepsilon. \tag{3.4-13}$$

由积分形式、三角不等式和 Grönwall 不等式，可得当 $L>0$ 时

$$\boxed{|e(t)|\leqslant e^{L(t-t_0)}|e(t_0)|+\frac{\varepsilon}{L}\left(e^{L(t-t_0)}-1\right).} \tag{3.4-14}$$

当 $L=0$ 时，第二项取极限 $\varepsilon(t-t_0)$。这说明，即便数值积分完全准确，初值偏差和右端模型偏差也可能传播到预测轨迹中。

进一步设数值方法 S 在固定时间区间上，对学习到的微分方程是稳定、收敛且全局精度为 p 阶的方法，并满足所需光滑性条件，那么还可得到相应的数值积分误差估计

$$\max_n|y_{\theta,n}^{S,h}-y_\theta(t_n)|\leqslant Ch^p. \tag{3.4-15}$$

因此，利用三角不等式可得到一个有条件的总体误差上界：真实轨迹与实际积分输出之间的偏差，不超过连续模型误差界与数值积分误差界之和。此处的 C 取决于方程、计算区间及数值方法等条件，不能仅凭神经网络训练损失较小就假设 C 或 ε 很小。经典误差理论参见 [1]、[2]。

这也提醒我们区分：①模型辨识误差；②数值积分误差；③优化与观测误差。三者可能相互作用；观察到的误差通常不能未经设计实验就全部归结为神经网络。

（二）Poisson 问题中残差与误差的关系

以一维 Poisson 边值问题为例，考虑

$$\boxed{\begin{cases}-u''(x)=\pi^2\sin(\pi x), & 0<x<1,\\ u(0)=0, \quad u(1)=0.\end{cases}} \tag{3.4-16}$$

直接求导可检验解析解为

$$u(x)=\sin(\pi x). \tag{3.4-17}$$

若用 PINN 近似该解，其残差为

$$r_\theta(x)=-u_\theta''(x)-\pi^2\sin(\pi x). \tag{3.4-18}$$

边界遗漏的反例。 取 $\tilde{u}(x)=\sin(\pi x)+x$，则在整个开区间内都满足 $-\tilde{u}''(x)=\pi^2\sin(\pi x)$，所以内部残差恒为零；但是 $\tilde{u}(1)=1\neq 0$，因此它不是式（3.4-16）的解。这一反例说明：内部方程完全正确，仍不能代替边界条件。

还可以更严格地解释残差和误差之间的联系。设 $u_\theta\in C^2([0,1])$，令 $e(x)=u_\theta(x)-u(x)$、$b_0=e(0)$、$b_1=e(1)$。由于 $-e''=r_\theta$，该问题的 Green 函数表示为

$$e(x)=(1-x)b_0+xb_1+\int_0^1 G(x,s)r_\theta(s)\,\mathrm{d}s, \tag{3.4-19}$$

其中

$$G(x,s)=\begin{cases} s(1-x), & 0\leqslant s\leqslant x,\\ x(1-s), & x\leqslant s\leqslant 1. \end{cases} \tag{3.4-20}$$

因为 $G(x,s)\geqslant 0$ 且 $\int_0^1 G(x,s)\,\mathrm{d}s=x(1-x)/2\leqslant 1/8$，于是有

$$\boxed{\|u_\theta-u\|_\infty\leqslant \max\{|b_0|,|b_1|\}+\frac{1}{8}\|r_\theta\|_\infty.} \tag{3.4-21}$$

这一估计只针对当前一维 Poisson 问题，并要求知道全区间上的一致残差界。它清楚表明，误差控制既需要边界误差受控，也需要方程残差受控。不同 PDE 是否能建立类似估计、常数如何依赖问题，则应由相应的稳定性理论判断，不能机械套用式（3.4-21）。

有限配点的反例。 如果内部配点恰好为 $x_i=i/N$（$i=1,\dots,N-1$），取

$$\tilde{u}_N(x)=\sin(\pi x)+A\sin(N\pi x),\quad A\neq 0. \tag{3.4-22}$$

这时 $\tilde{u}_N(0)=\tilde{u}_N(1)=0$，而每一个内部配点上也有 $-\tilde{u}_N''(x_i)-\pi^2\sin(\pi x_i)=0$；然而它与真实解的最大偏差是 $|A|$。这说明有限训练配点上的残差为零，不代表连续区域内的残差为零，更不能直接使用式（3.4-21）中的全区间残差界。这是构造性的教学反例，不表示实际 PINN 必然出现这一特定振荡。

3.4.3 一维 Poisson 边值问题的有限差分参照解

为了有一个与 PINN 相互独立的数值参照，先用有限差分法求解式（3.4-16）。取整数 $N\geqslant 2$，将 $[0,1]$ 均分为 N 个子区间，令

$$\Delta x=\frac{1}{N},\qquad x_i=i\Delta x,\quad i=0,1,\dots,N. \tag{3.4-23}$$

记 U_i 为 $u(x_i)$ 的数值近似（使用大写 U_i，避免同 3.1 节的时间推进量 y_n 混淆）。在内部节点应用中心差分公式，得到

$$\boxed{\frac{-U_{i-1}+2U_i-U_{i+1}}{(\Delta x)^2}=\pi^2\sin(\pi x_i),\quad i=1,\dots,N-1,} \tag{3.4-24}$$

同时把两端边界显式放入离散系统：

$$U_0=0,\qquad U_N=0. \tag{3.4-25}$$

对足够光滑的解，中心差分对二阶导数的截断误差为 $O((\Delta x)^2)$；结合该 Dirichlet 问题的离散稳定性，得到其节点误差的二阶收敛性。注意这是有关数值差分解的结论，与 PINN 仅在有限配点上最小化残差并非同一命题。有限差分方法及其稳定性理论可参见 [2]。

当前例子还可以解析地检验差分结果。设 $U_i = C_{\Delta x}\sin(\pi x_i)$，将其代入式（3.4-24），利用三角恒等式可得

$$C_{\Delta x} = \frac{\pi^2(\Delta x)^2}{4\sin^2\left(\frac{\pi\Delta x}{2}\right)}. \tag{3.4-26}$$

由于 $C_{\Delta x} = 1 + O((\Delta x)^2)$，这为下节的网格收敛实验提供了独立核查途径。因此，这个离散系统不仅具有明确的数学定义，也具备可用于数值实验的独立核查表达式。

3.4.4 数值实验一：Neural ODE 的训练误差与方程误差

（一）实验目标、已知条件与运行前预测

本实验使用一个已知解析解的线性衰减模型生成观测数据，再分别借助向前 Euler 和 RK4 训练最小 Neural ODE，检验训练轨迹拟合得非常好，是否就意味着恢复了真实微分方程。

真实系统为

$$\begin{cases} y'(t) = -a_* y(t), & 0 \leqslant t \leqslant 2, \\ y(0) = y_0, \end{cases} \qquad a_* = 1.6, \tag{3.4-27}$$

其解析解为 $y(t) = y_0 e^{-a_* t}$。为了与 3.1 节的线性衰减算例衔接，这里仍采用相同的方程形式，但把真实衰减系数由 1 改成了 1.6；本实验中的时间 t 与状态 y 均视为无量纲量，故 a_* 也采用无量纲化后的数值。

为尽量减少网络结构等因素的干扰，使用只有一个可训练权重的线性右端网络

$$\boxed{f_\theta(y) = -a_\theta y, \qquad a_\theta > 0.} \tag{3.4-28}$$

这是 Neural ODE 的最小可训练实例，不代表复杂多层神经网络的全部性质。我们在运行前预期：当训练积分器的离散误差足够小时，a_θ 应接近 $a_* = 1.6$；若使用较粗步长的向前 Euler，网络也可能通过改变 a_θ 来补偿离散误差，从而得到很小的训练损失，却学到不准确的连续动力学。

实验输入与评价方案见表 3.4-1。

表 3.4-1　Neural ODE 最小实验的统一设定

项目	实验设定与选择依据
要回答的问题	Euler 与 RK4 用相同步长训练时，训练误差和独立求解器评估误差是否一致
真实方程及数据来源	$y' = -1.6y$，解析解 $y_0 e^{-1.6t}$ 生成无噪声模拟数据
自变量与参数	$t \in [0, 2]$，时间与状态均无量纲；真实系数 $a_* = 1.6$，待学习参数 $a_\theta > 0$
训练轨迹与时间网格	初值 $0.8, 1.0, 1.2$；时刻 $0, 0.2, \ldots, 2$，共 3×11 个状态值
独立评估设置	未参与训练的初值 $0.9, 1.4, 2.0$；评估间隔 0.02，共 3×101 个状态值

项目	实验设定与选择依据
模型与训练积分器	$f_\theta(y)=-a_\theta y$；分别使用 Euler、RK4；训练步长均为 $h=0.2$
参数优化	在 $a_\theta\in[0.5,2.5]$ 上对式（3.4-5）进行有界一维最小化；无随机初始化
独立参考与基线	真实解析解为首要参考；另用高精度 DOP853 对已学得的连续右端模型重新积分
误差指标	训练均方误差 MSE、独立评估 RMSE、最大绝对误差、$\lvert a_\theta-a_*\rvert$
自设验收标准	独立评估 RMSE $\leqslant 10^{-3}$；仅为本教学实验的事前比较阈值，并非普遍精度保证
数值环境	Python 3.13.5、NumPy 2.3.5、SciPy 1.17.0；float64、CPU；目标函数确定性，无随机种子需求
运行预算	两个模型各进行一次一维参数优化；本次 Euler、RK4 优化分别使用 12、13 次目标函数求值

将训练与独立评估的时间网格、初值及积分器分别记录，是因为如果同一积分器同时负责训练与“验证”，可能遮蔽它自身的系统性偏差。评估初值 2.0 也检验了训练初值范围之外的预测情况。

（二）两种训练积分器的解析关系

在模型（3.4-28）下，向前 Euler 的更新为

$$y^{\mathrm{FE},h}_{\theta,n+1}=(1-a_\theta h)y^{\mathrm{FE},h}_{\theta,n}. \tag{3.4-29}$$

真实系统每经过一个长度为 h 的时间段，状态恰好乘以 e^{-a_*h}。为了在无噪声训练节点上完全匹配该真实演化，Euler 参数必须满足

$$1-a_\theta h=e^{-a_*h},\qquad \boxed{a^{\mathrm{FE}}_\theta=\frac{1-e^{-a_*h}}{h}}. \tag{3.4-30}$$

当 $h=0.2$、$a_*=1.6$ 时，代入得 $a^{\mathrm{FE}}_\theta\approx 1.3692548$，显著低于 1.6。而由 Taylor 公式

$$a^{\mathrm{FE}}_\theta=a_*-\frac{a_*^2}{2}h+O(h^2),\qquad h\to 0, \tag{3.4-31}$$

可见这是固定非零训练步长引起的系统性参数偏差，并不表示 Euler 方法在 $h\to 0$ 时失去一致性。

RK4 对于 $y'=-a_\theta y$ 的一步放大因子为

$$R_{\mathrm{RK4}}(-a_\theta h)=1-a_\theta h+\frac{(a_\theta h)^2}{2}-\frac{(a_\theta h)^3}{6}+\frac{(a_\theta h)^4}{24}. \tag{3.4-32}$$

因此，在本实验所取的参数搜索区间内，RK4 训练的最优系数满足 $R_{\mathrm{RK4}}(-0.2a_\theta)=e^{-0.32}$，给出 $a^{\mathrm{RK4}}_\theta\approx 1.6001828$。对这一简单模型，可以由放大因子精确解释训练参数与真实参数的差别；对于复杂非线性系统，还需要研究更一般的离散动力学与模型辨识偏差，参见 [5]。

（三）实际数值结果与独立核查

按照表 3.4-1 的设定运行确定性参数优化，并对学得的模型使用 DOP853（相对容差 10^{-12}，绝对容差 10^{-14}）在独立初值和细时间网格上积分，得到表 3.4-2。表中结果由实际数值运算所得，并非仅凭理论预期填入。

表 3.4-2　训练结果与独立评估结果

指标	Euler 训练模型	RK4 训练模型
训练步长 h	0.2	0.2
学得参数 a_θ	1.369254812	1.600182836
参数绝对误差 $\lvert a_\theta - a_* \rvert$	0.230745188	0.000182836
训练集 MSE	4.77×10^{-19}	2.68×10^{-18}
独立评估 RMSE	0.065389496	0.000046647
独立评估最大绝对误差	0.114465984	0.000084069
是否达到自设 RMSE 阈值	否	是

这里，独立评估 RMSE 定义为

$$\mathrm{RMSE} = \sqrt{\frac{1}{3 \times 101} \sum_{m=1}^{3} \sum_{j=0}^{100} \left| y_{\theta,m}^{\mathrm{DOP853}}(t_j) - y_m(t_j) \right|^2}. \tag{3.4-33}$$

两组模型在训练节点上都几乎完美重合，但 Euler 训练得到的右端系数相对真实系数有明显偏差；换用独立高精度求解器后，误差立即显现。相比之下，RK4 训练模型恢复的系数更接近真实值，独立评估误差也显著较小。这个例子构成了一个明确的反例：损失函数在训练节点上几乎为零，不蕴含真实动力学辨识准确。

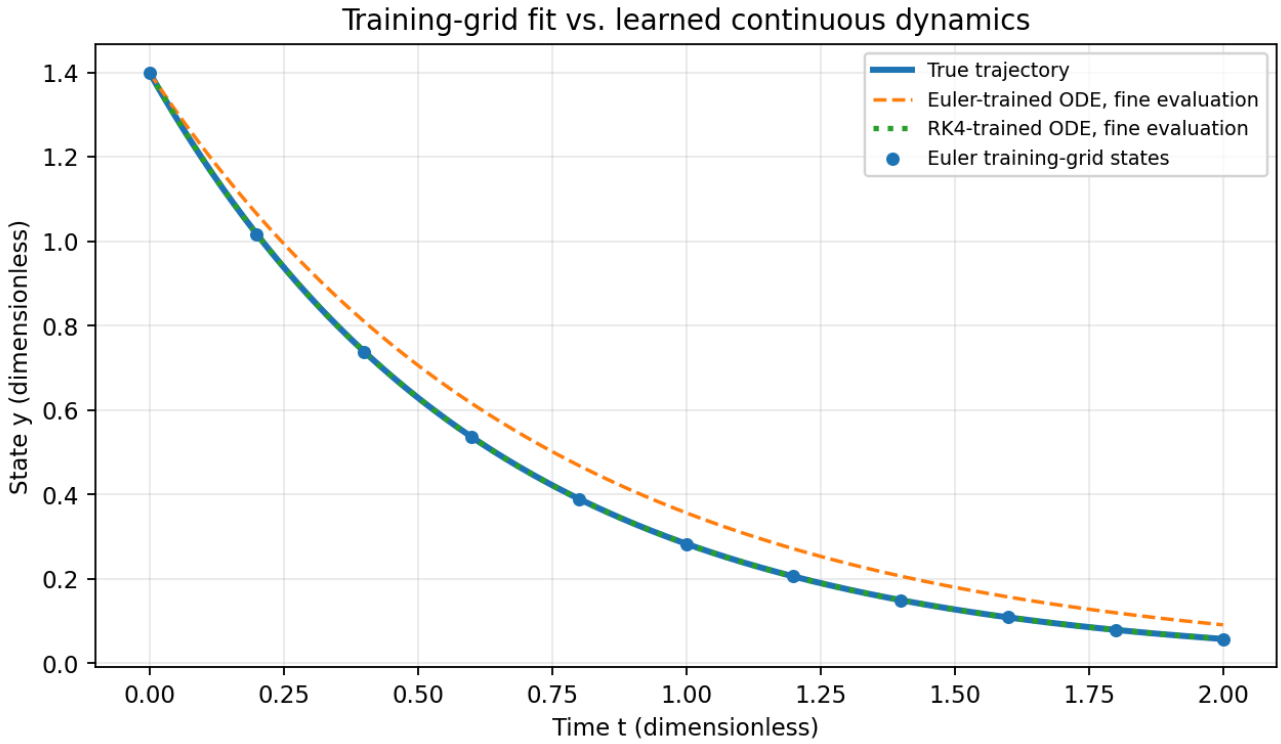


图 3.4-1　初值 $y_0 = 1.4$ 下的独立评估轨迹。两模型均在训练步长 $h = 0.2$ 下拟合生成数据，图中通过高精度 DOP853 重新积分已学习的右端，并与解析解对比；时间和状态均无量纲。图像由配套复现代码生成。

（四）改变评估步长后的反常现象

固定未参与训练的初值 $y_0 = 1.4$，仍用各模型对应的训练积分方法，只改变评估步长，观察终点 $t = 2$ 的绝对误差。

表 3.4-3　更换步长后的终点绝对误差

评估方式	Euler 训练模型	RK4 训练模型
原方法，$h = 0.2$	4.76×10^{-10}	1.13×10^{-9}
原方法，$h = 0.1$	0.01656955	1.97×10^{-5}
原方法，$h = 0.05$	0.02499400	2.08×10^{-5}
DOP853 独立积分	0.033466（约）	2.09×10^{-5}（约）

第一行的极小但非零值来自浮点参数优化的终止精度；在解析关系（3.4-30）精确成立时，其离散终点误差为零。Euler 训练模型的终点误差在缩小评估步长后反而变大，这是因为更细步长逐渐逼近的是学得参数为 1.3692548 的连续模型，而非真实衰减系数为 1.6 的模型。数值积分精度提高，不会自动修正错误的微分方程。

这一结论只针对所设定的简单线性模型和训练方式。它不能被解释为“Euler 永远不适合 Neural ODE”或“RK4 在任何情况下都更好”。选择数值求解器仍须综合考虑稳定性、问题刚性、所需精度和计算成本。

3.4.5 数值实验二：Poisson 边值问题的独立差分基线

为与前一实验形成互补，本实验采用第 3.4.3 节的中心差分法，研究空间网格加密对一维 Poisson 边值问题求解误差的影响。这里的实验对象是传统有限差分基线，不涉及完整 PDE PINN 的网络训练；后者可在后续章节结合自动微分、配点采样和参数优化进一步展开。

对式（3.4-16）分别取 $N = 5, 10, 20, 40$，由式（3.4-24）—（3.4-25）求 U_i。将每个内部节点与解析解 $u(x_i) = \sin(\pi x_i)$ 比较，定义

$$E_\infty(\Delta x) = \max_{0 \leqslant i \leqslant N} |U_i - u(x_i)|. \tag{3.4-34}$$

通过式（3.4-26）可以直接核查计算结果。所得数据如下。

表 3.4-4　Poisson 中心差分的节点最大误差

子区间数 N	网格间距 Δx	$E_\infty(\Delta x)$	相邻两组误差比
5	0.2000	0.031915927	—
10	0.1000	0.008265417	3.86
20	0.0500	0.002058707	4.01
40	0.0250	0.000514200	4.00

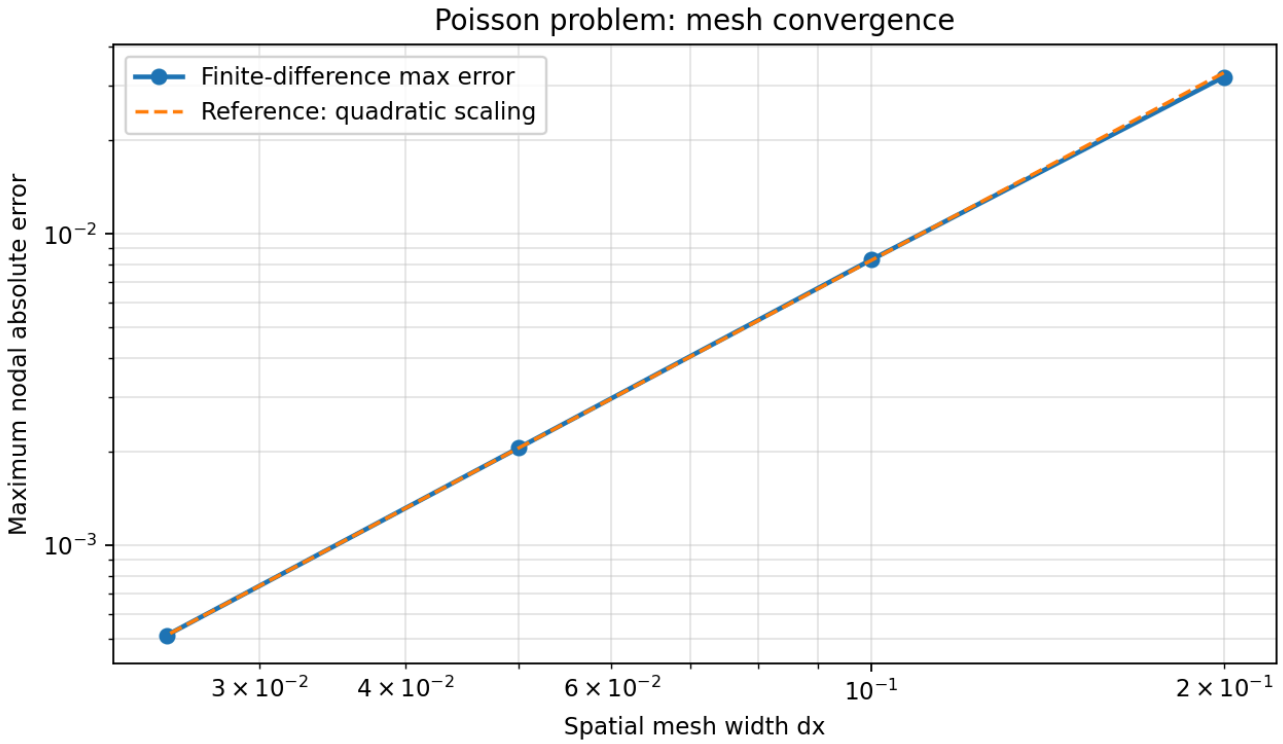


图 3.4-2　对数坐标下的最大节点误差随空间网格间距的变化。曲线接近斜率为 2 的趋势，与本题中心差分二阶收敛的理论相符。

当 Δx 减半时，误差约缩小到四分之一。这一结果由明确的边界条件、独立解析解和数值误差指标支持，且与网格加密的二阶收敛规律相符。

不过，差分结果仅验证了当前问题上的传统数值基线。不能据此直接得出“任何 PINN 都是二阶收敛的”或“任何足够密的配点都能保证 PINN 正确”。PINN 的精度仍取决于网络表达能力、优化、配点采样、边界处理和方程稳定性等因素 [6,7]。

3.4.6 数值结果的可靠性检验与适用范围

（一）区分模型误差、离散误差与训练误差

在传统数值求解中，方程已经确定，误差分析通常集中于离散格式、初值或边界数据以及计算过程。引入神经网络后，函数表示和参数优化也成为误差来源，因此不能把所有预测偏差统称为“神经网络误差”。

对于 Neural ODE，首先应区分学习到的右端函数 f_θ 与真实右端函数 f 之间的模型辨识误差，其次是数值积分器及其步长引入的离散误差，最后是有限观测、测量噪声和参数优化产生的误差。式（3.4-14）给出了初值误差与右端模型误差的传播界，式（3.4-15）则描述在相应假设下的积分误差。训练损失只是特定观测节点和求解器下的拟合指标，不能直接代替上述误差界。

对于 PINN，主要应区分网络的函数逼近误差、优化目标未充分最小化带来的训练误差、有限配点对连续残差的代表性不足，以及初边值条件没有得到准确满足所造成的偏差。式（3.4-21）在一维 Poisson 问题上建立了全区间残差、边界误差与解误差之间的联系，同时说明仅凭有限配点上的残差无法直接推出全区间精度。

（二）建立可复核的验证依据

评价学习结果时，应尽量采用独立于训练过程的数值证据，而不是只展示最终损失函数值。不同计算任务的重点如下。

检验对象	主要检验方式	能够支持的判断
Neural ODE 的连续动力学	固定已学得参数，使用独立积分器与不同步长评估轨迹	判断训练时的积分离散是否掩盖了模型偏差
Neural ODE 的预测能力	使用未参与训练的初值和观测时刻，并与解析解或可信参考轨迹比较	评价模型在指定测试范围内的泛化表现
PINN 的方程与条件	在独立测试配点计算方程残差，并分别核查初始条件与边界条件	检验近似解是否满足所给微分方程及其约束
空间离散的传统参照解	在相同模型与边界条件下进行网格加密，计算误差与观测收敛阶	判断结果是否符合相应差分格式的预期精度

实验报告还应明确记录求解区间、初值或边界值、参数与单位、训练和评估数据的划分、误差指标及容差、算法设置，以及用于重现计算的代码。对于无量纲化问题，应明确无量纲化约定；对于有量纲问题，应检查方程各项的单位是否一致。

（三）正确界定实验结论

第 3.4.4 节的一参数线性衰减实验表明，Euler 和 RK4 训练模型都能够获得极小的节点拟合误差，但其学得的连续方程并不相同。独立积分与步长变化实验进一步表明，更准确地积分一个存在辨识偏差的模型，并不等于更准确地恢复真实动力学。这一现象具有清晰的解析解释，但不意味着任意 Neural ODE 都会表现出同样的参数偏差。

第 3.4.5 节给出的 Poisson 中心差分实验，则在边界条件完整施加的前提下，通过解析解与网格加密验证了当前问题的二阶收敛规律。它是一项传统数值方法的独立基线实验，并不是对完整 PINN 网络训练的直接测试；因此不能据此推断 PINN 具有同样的收敛阶。

对于复杂非线性动力系统、刚性方程、含噪声观测和高维偏微分方程，还需进一步研究采样覆盖、参数可辨识性、方程稳定性、网络表示能力和优化算法的影响。相关问题可参阅文献 [5]—[9]。

3.4.7 本节小结

本节在经典常微分方程数值积分与有限差分的基础上，介绍了神经网络参与微分方程建模和求解的两种典型方式，并说明了评价计算结果时必须保留的数值分析原则。关键结论归纳如下。

1. Neural ODE 与 PINN 的学习对象不同。 Neural ODE 通常以神经网络表示微分方程的右端函数，再利用数值积分器生成轨迹；PINN 通常直接以神经网络表示未知解，并使用方程残差以及初边值条件构造训练目标。两者可以结合，但不能混淆其基本作用机制。
2. 训练损失不能直接代替真实解误差。 Neural ODE 的预测同时受初值、右端模型、积分方法及其步长影响；PINN 的结果则还取决于网络表示、配点分布、边界处理与优化过程。误差估计必须陈述相应的数学假设和适用范围。
3. 微分方程与初边值条件共同定义求解问题。 一维 Poisson 算例表明，内部残差即使为零，也不能忽略边界条件；有限配点处残差很小，也不等于连续区间内残差处处很小。正确处理约束是获得可信近似解的必要环节。
4. 数值结论需要独立验证。 对 Neural ODE，应开展跨求解器、跨步长和测试初值的评估；对 PINN，应核查独立配点上的方程残差、初边值条件以及可获得的参考解；对传统离散基线，应关注网格加密与收敛阶。任何单一指标都不能替代完整的误差分析。

本节的两个教学实验分别揭示了模型辨识偏差可能被训练积分器掩盖，以及正确边界处理和网格收敛检验能够为数值结果提供独立证据。这些结论构成继续学习数据驱动动力学、物理约束学习和偏微分方程数值求解的基础。

3.4.8 可复现代码与进一步阅读

以下 Python 代码不依赖深度学习框架，用一参数线性网络作为最小 Neural ODE，独立复现表 3.4-2 至表 3.4-4。它用于辨别数值积分与参数学习之间的关系；需要学习一般多层网络的自动微分与 PINN 训练时，可在后续章节中使用 PyTorch 或 JAX 扩展。

```
import numpy as np
from scipy.integrate import solve_ivp
from scipy.optimize import minimize_scalar

A_TRUE, T, H = 1.6, 2.0, 0.2
TRAIN_Y0 = np.array([0.8, 1.0, 1.2])
TEST_Y0 = np.array([0.9, 1.4, 2.0])
T_TRAIN = np.linspace(0, T, 11)
T_TEST = np.linspace(0, T, 101)

def amplification(a, h, solver):
    z = -a * h
    if solver == "Euler":
        return 1 + z
```

```
    return 1 + z + z**2/2 + z**3/6 + z**4/24

def train_mse(a, solver):
    factors = amplification(a, H, solver) ** np.arange(len(T_TRAIN))
    pred = TRAIN_Y0[:, None] * factors[None, :]
    exact = TRAIN_Y0[:, None] * np.exp(-A_TRUE * T_TRAIN)[None, :]
    return np.mean((pred - exact)**2)

for solver in ["Euler", "RK4"]:
    result = minimize_scalar(
        lambda a: train_mse(a, solver),
        bounds=(0.5, 2.5), method="bounded",
        options={"xatol": 1e-14},
    )
    a_fit = result.x
    pred = np.stack([
        solve_ivp(
            lambda t, y: -a_fit*y, (0, T), [y0],
            t_eval=T_TEST, method="DOP853",
            rtol=1e-12, atol=1e-14,
        ).y[0]
        for y0 in TEST_Y0
    ])
    exact = TEST_Y0[:, None] * np.exp(-A_TRUE * T_TEST)[None, :]
    diff = pred - exact
    print(solver, "a=", a_fit,
          "train MSE=", train_mse(a_fit, solver),
          "test RMSE=", np.sqrt(np.mean(diff**2)),
          "test max error=", np.max(np.abs(diff)))
    print("Final-point errors (y0=1.4):")
    for h in [0.2, 0.1, 0.05]:
        n = round(T / h)
        yn = 1.4 * amplification(a_fit, h, solver)**n
        print(" h=", h, " error=", abs(yn-1.4*np.exp(-A_TRUE*T)))

print("Poisson finite-difference errors:")
for N in [5, 10, 20, 40]:
    dx = 1.0/N
    x = np.arange(N+1)*dx
    coefficient = np.pi**2*dx**2/(4*np.sin(np.pi*dx/2)**2)
    approximate = coefficient*np.sin(np.pi*x)
    error = np.max(np.abs(approximate-np.sin(np.pi*x)))
    print("N=", N, "dx=", dx, "max error=", error)
```

代码的输入数据由解析模型直接生成，既不存在随机噪声，也不存在需要设置随机种子的优化初始化。表中极小的训练损失会随平台的舍入及优化终止精度略有变化，不应过度解读其最后几位数字；关键结论由式（3.4-30）及独立评估结果共同支持。

参考文献（阅读顺序建议：经典数值方法 [1]、[2]；方法起源 [3]、[4]；数值与训练误差研究 [5]—[9]。）

[1] Hairer E, Nørsett S P, Wanner G. Solving Ordinary Differential Equations I: Nonstiff Problems. 2nd ed. Berlin: Springer, 1993. 有关初值问题的稳定性与误差估计，可参考第 I、II 章。https://doi.org/10.1007/978-3-540-78862-1

[2] LeVeque R J. Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems. Philadelphia: SIAM, 2007. 关于有限差分、边值问题和误差分析，参见相关章节。https://doi.org/10.1137/1.9780898717839

[3] Chen R T Q, Rubanova Y, Bettencourt J, Duvenaud D K. Neural Ordinary Differential Equations. Advances in Neural Information Processing Systems, 31, 2018. https://proceedings.neurips.cc/paper/2018/hash/69386f6bb1dfed68692a24c8686939b9-Abstract.html

[4] Raissi M, Perdikaris P, Karniadakis G E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. Journal of Computational Physics, 378: 686–707, 2019. https://doi.org/10.1016/j.jcp.2018.10.045

[5] Zhu A, Jin P, Zhu B, Tang Y. On Numerical Integration in Neural Ordinary Differential Equations. Proceedings of the 39th International Conference on Machine Learning, PMLR 162: 27527–27547, 2022. https://proceedings.mlr.press/v162/zhu22f.html

[6] Wang S, Teng Y, Perdikaris P. Understanding and Mitigating Gradient Flow Pathologies in Physics-Informed Neural Networks. SIAM Journal on Scientific Computing, 43(5): A3055–A3081, 2021. https://doi.org/10.1137/20M1318043

[7] De Ryck T, Mishra S. Numerical analysis of physics-informed neural networks and related models in physics-informed machine learning. Acta Numerica, 33: 633–713, 2024. https://doi.org/10.1017/S0962492923000089

[8] Xu Y, Chen S, Li Q. Correcting Autodifferentiation in Neural ODE Training. SIAM Journal on Applied Mathematics, 86(5): 2360–2378, 2026. https://doi.org/10.1137/25M172673X

[9] Celledoni E, Owren B, Ruthotto L, Yang N T. Mixed Precision Training of Neural ODEs. SIAM Journal on Scientific Computing, 2026. https://doi.org/10.1137/25M1821405

本章习题与参考解答

第03章 离散的世界：课后习题模板

题1：对应3.1 ODE初值问题：Euler与RK4

习题 3.1-1（初值问题的解析解与单步计算） 考虑下列非自治常微分方程初值问题：

$$\begin{cases} y'(t) = -2y(t) + 2t + 1, & 0 \le t \le 2, \\ y(0) = 1. \end{cases}$$

记右端函数为 $f(t,y) = -2y + 2t + 1$，设 $t_n = nh$、$y_n \approx y(t_n)$，其中 $h > 0$ 为时间步长，$y_0 = 1$。本题的时间和状态均采用无量纲表示。

（1）通过积分因子法或适当的变量代换求该初值问题的解析解，并验证所得结果满足微分方程和初始条件。说明解析解为什么能够作为检验数值误差的独立参考。

（2）分别写出向前 Euler 和向后 Euler 方法的单步更新公式。将 $f(t,y)$ 代入向后 Euler 公式，整理得到关于 y_{n+1} 的代数方程，并求出本题的一步显式计算表达式。说明两种 Euler 方法在求解下一步状态时有何不同。

（3）取 $h=0.5$，从 $(t_0,y_0)=(0,1)$ 出发，分别手算向前 Euler、向后 Euler 和经典四阶 Runge–Kutta（RK4）方法的一步近似值。对于 RK4，须列出四个阶段斜率 k_1,k_2,k_3,k_4 的计算过程以及最终加权更新式。将三种近似值与解析解在 $t=0.5$ 处的值进行比较。

（4）从单步斜率的使用方式出发，说明向前 Euler 为什么可以看作一阶 Runge–Kutta 方法；简述典型二阶 Runge–Kutta 方法与经典 RK4 在阶段数及斜率组合方式上的区别。本题不要求单独实现二阶方法。

习题 3.1-2（步长加密与收敛阶） 仍考虑习题 3.1-1 的初值问题，在固定终止时刻 $T=2$ 下，取

$$t_n=nh,\qquad h=\frac{T}{N}=\frac{2}{N},\qquad N=4,8,16,32,64.$$

分别采用向前 Euler、向后 Euler 和经典 RK4 方法计算数值解，并完成以下问题。

（1）自行编写三种方法的逐步推进程序，至少能够正确求解本题所给方程。程序应保存时间节点和对应的数值解，不得以现成 ODE 求解器代替三种方法的核心更新。对于向后 Euler，允许使用习题 3.1-1 推得的代数方程闭式解，但须说明其适用范围。

（2）以解析解为参考，列表给出五组步数下的终点数值解 y_N 和终点绝对误差

$$E_T(h)=|y_N-y(T)|.$$

当步长减半时，计算观测收敛阶

$$p_{\mathrm{obs}}(h)=\log_2\!\left(\frac{E_T(h)}{E_T(h/2)}\right),$$

比较其与三种方法理论收敛阶的关系，并解释有限步长下观测阶数为何可能偏离整数值。

（3）绘制 $h=0.5$ 时三种数值解与解析解在区间 $[0,2]$ 上的轨迹对比图；再绘制终点绝对误差随步长变化的双对数图。图中应明确标注方法、参数、英文坐标名称以及坐标量的无量纲性质。

（4）本节将从精确起点推进一步所得的差值（不除以步长 h）称为局部截断误差。分别写出向前 Euler、向后 Euler 和经典 RK4 方法的局部截断误差阶与全局误差阶，并说明这些阶数结论通常要求哪些光滑性与误差传播条件。为什么不能直接把局部截断误差的阶数当作全局误差的阶数？

习题 3.1-3（非自治方程的稳定性辨析） 将习题 3.1-1 的微分方程延长到 $0\le t\le 5$，初始条件保持不变。取 $T=5$、$N=4$，即 $h=1.25$。

（1）引入变量变换 $w(t)=y(t)-t$，推导 w 所满足的初值问题，并写出其解析解。解释为什么考察 w 的衰减特性，比仅观察原变量 $y(t)$ 是否随时间增大更适合分析本题的数值稳定性。

（2）记 $w_n=y_n-t_n$。将三种数值方法分别应用于原方程，推导 w_n 的一步递推关系，并写成 $w_{n+1}=R(-2h)w_n$ 的形式。由此求出向前 Euler、向后 Euler 和经典 RK4 在 $h=1.25$ 时的放大因子，结合 $|R(-2h)|$ 判断衰减分量的数值稳定性。

（3）计算三种方法在 $t=0,1.25,2.5,3.75,5$ 处的数值解，绘制它们与解析解的比较图，并计算 $t=5$ 处的终点绝对误差。说明“数值方法保持衰减稳定”是否意味着“数值近似已经足够准确”。图中同样应给出英文坐标名称、无量纲说明、图例与步长等参数。

（4）判断以下三个说法是否正确，并根据理论推导或本题的数值反例说明理由：①“只要原变量 y_n 随时间增大，算法就一定不稳定”；②“向前 Euler 与向后 Euler 都是一阶方法，因此二者的绝对稳定区域相同”；③“经典 RK4 具有四阶精度，因此任意步长下均能得到稳定且准确的数值解”。上述说法均为本题专门设计的辨析命题，并非对某个真实 AI 回答的引用。

习题 3.1-4（精度要求、计算成本与程序核对） 继续考虑习题 3.1-1 的方程与终止时刻 $T=2$，规定终点绝对误差须满足

$$E_T(h)\le 10^{-4}.$$

只考虑步数 $N=4,8,16,32,\dots$ 构成的二分加密候选网格。

（1）分别找出三种方法在上述候选网格中首次满足误差要求的步数与步长，并记录相应的终点误差。比较向前 Euler 与 RK4 达到相同误差目标所需的右端函数求值总次数。

（2）对向后 Euler，写出本题每一步隐式代数方程的残差

$$r_{n+1}=y_{n+1}-y_n-hf(t_{n+1},y_{n+1}),$$

计算并报告各步残差的最大绝对值，建议以 10^{-10} 为代数方程求解的残差检验阈值。若采用 Newton 迭代求解，另外记录右端函数求值次数、所用导数或雅可比求值次数及迭代次数；若采用本题的闭式更新公式，应明确指出这一点。说明代数残差与 ODE 的时间离散误差有何区别。

（3）结合以上结果，讨论为什么比较不同方法的计算成本时，应先固定相同的精度要求；并说明向后 Euler 的代数求解开销为什么不能简单等同于显式方法的每步函数求值次数。

（4）将本题的程序、计算结果与三张图保存在独立于正文课堂实验的习题 Notebook 中，并注明主要参数、Python 及所用库的版本。允许使用 Python 标准库、NumPy 和 Matplotlib，不得调用现成 ODE 求解器替代核心算法。程序数值结果可按 10^{-8} 的绝对容差与独立计算结果复核；这一复核容差与本题规定的 10^{-4} 终点误差要求含义不同。未实际执行的代码与结果应明确注明“未运行”。

题2：对应3.2 PDE有限差分：热传导与CFL条件

出题方向： 在两端不同恒温的热传导问题中，利用均匀网格构造 FTCS 显式差分格式，选择合适的时间步长；通过多模态初值的解析解、稳定性对比实验和网格收敛实验，辨析“稳定”和“精确”的不同含义。

习题 3.2-1（非齐次 Dirichlet 边界下的 FTCS 稳定性与网格收敛）

情境： 一根长 $L=1.2\,\mathrm{m}$ 的细长均匀金属导热条，左端与 $20\,^\circ\mathrm{C}$ 的恒温装置接触，右端与 $10\,^\circ\mathrm{C}$ 的恒温装置接触。忽略横向温差及内热源，材料热扩散系数取 $\kappa=0.08\,\mathrm{m^2/s}$。温度 $u(x,t)$ 满足

$$\begin{cases} u_t=\kappa u_{xx}, & 0<x<L,\quad 0<t\le T,\\ u(0,t)=20\,^\circ\mathrm{C},\quad u(L,t)=10\,^\circ\mathrm{C}, & 0<t\le T,\\ u(x,0)=20-\dfrac{10x}{L}+5\sin\!\left(\dfrac{\pi x}{L}\right)+2\sin\!\left(\dfrac{2\pi x}{L}\right), & 0\le x\le L, \end{cases}$$

其中，温度表达式中的常数 20、10、5 和 2 均以 $^\circ\mathrm{C}$ 计。终止时间设为 $T=0.45\,\mathrm{s}$。

将区间 $[0, L]$ 划分成 $J = 60$ 个等长小区间，采用“空间二阶中心差分＋时间前向 Euler”的 FTCS 显式格式。计划采用三种备选时间步长：

方案	时间步长 Δt（s）
A	**0.001875**
B	**0.002500**
C	**0.003750**

请先根据理论预测，再开展数值实验。回答哪些时间步长满足本格式的标准扩散型 CFL 稳定性条件，并解释为什么即使满足条件，也仍需用误差和网格加密检验数值精度。

（1）从连续模型到差分格式。设 $x_j = j\Delta x$、$t_n = n\Delta t$、$U_j^n \approx u(x_j, t_n)$。用 Taylor 展开说明空间中心差分和时间前向差分的精度阶，逐步推导 FTCS 显式递推式，解释无量纲参数 $r = \kappa\Delta t/(\Delta x)^2$ 的物理和数值含义。写明初始层赋值与两端温度的处理方式。

（2）稳定性与步长选择。通过差分系数非负且和为 1 的性质，说明标准条件 $0 \le r \le 1/2$ 为什么能够保证离散最大值原理。计算本题的 Δx、$\Delta t_{\max}$ 和三个方案的 r，据此逐个判断。特别说明：本题的边界温度不为零，为什么仍可使用这个稳定性条件？

（3）独立解析解。先求稳态温度分布 $u_s(x)$（提示：稳态满足 $u_s'' = 0$），再令 $v(x,t) = u(x,t) - u_s(x)$。利用正弦模态的指数衰减，写出并验证该题的解析解，解释它与教材中单一正弦初值的区别。

（4）稳定不等于精确。若 J 增大为原来的两倍，为保持相同的 r，Δt 应如何调整？结合 FTCS 局部截断误差 $\mathcal{O}(\Delta t + (\Delta x)^2)$，说明稳定性、截断误差、全局误差和网格收敛性的关系。

（5）稳定性对照（先预测后计算）。使用 Python、NumPy、Matplotlib，自行实现 FTCS 显式格式，不得调用现成 PDE 求解器。固定 $J = 60$，比较方案 A 和 C，在代码运行前记录对它们稳定性的预测。为避免特殊低频初值掩盖高频不稳定现象，在这两组的初值中加入相同的、与边界相容的微小高频扰动：

$$u_\varepsilon(x, 0; J) = u(x, 0) + \varepsilon \sin\!\left(\frac{(J-1)\pi x}{L}\right), \qquad \varepsilon = 10^{-10}\,^\circ\mathrm{C}.$$

要求将解析解中的高频衰减项也加入独立参考，逐步记录

$$E_\infty^n = \max_{1 \le j \le J-1} \left| U_j^n - u_{\varepsilon,\mathrm{ref}}(x_j, t_n) \right|$$

及相对稳态分布的偏差幅值

$$M_v^n = \max_{0 \le j \le J} |U_j^n - u_s(x_j)|.$$

分别绘制：① 两组 E_∞^n 的误差—时间曲线（误差纵轴使用对数刻度）；② $t = T$ 时稳定组、不稳定组各自与解析解对照的空间温度剖面（建议左右双图，因幅值差异很大而使用独立纵轴）。图中明确标注坐标、单位、J、r、Δt 和图例。不允许删除或隐瞒不稳定结果。

（6）网格加密与精度。为使不同网格都逼近同一个连续初边值问题，恢复不含人为高频扰动的原始初值。取 $J = 30, 60, 120, 240$，保持 r 与方案 A 相同，均计算到 $T = 0.45\,\mathrm{s}$，并用解析解计算终点最大节点误差。提交包含 J、Δx、Δt、时间步数、$E_\infty(T)$ 的表格，以及误差随 Δx 变化的双对数图；计算相邻两网格的观测收敛阶

$$p = \log_2 \frac{E_\infty(\Delta x)}{E_\infty(\Delta x/2)}.$$

（7）结果分析。解释不稳定组何以可能先看似正常、随后突然出现大幅振荡；说明方案 B 在标准 CFL 判据中的地位。概括“满足 CFL”“具有较小误差”“加密后误差收敛”分别能支持什么结论。

（8）CFL 条件的适用范围与错误辨析。下面是一段专为教学构造的错误判断：

> “教材的 $r \le 1/2$ 只适用于两端为零的金属杆；本题两端温度不同，所以不必考虑这一限制。方案 C 即使有问题，只要把空间网格加密、时间步长保持不变，就会自动变稳定。另外，$r \le 1/2$ 是所有 PDE 的通用 CFL 公式，满足它便能确保结果准确。”

请至少指出其中四处错误或不严谨之处。解释为什么非齐次 Dirichlet 边界可以通过稳态变量变换处理；为什么在 Δt 不变时加密空间网格可能反而使 FTCS 更不稳定；为什么不能把该限制套用于所有 PDE 或所有差分格式；为什么稳定性不等于精度。并回答：若某次运行方案 C 的数值解暂时没有爆炸，能否据此认定格式稳定？

提交与验收要求

理论部分： 必须写出正确的 FTCS 格式及非零边界处理，列出稳态分解和两模态解析参考解；正确计算 Δx、稳定时间步上界和三种方案的 r，给出理由而非只写判断结果。

计算部分： A、C 必须用相同物理终止时间和相同扰动初值比较；两端温度在每一步固定为题目给定值。数值结果应与独立解析解作节点比较，误差曲线和空间剖面均注明参数。实验中的失稳应如实展示。

收敛部分： 加密实验不得使用随 J 改变频率的人为高频扰动；保持同一个连续模型并固定 r，用观测阶说明联合空间—时间收敛。若算得 p 接近 2，可接受的教学判断区间为 $1.8 \le p \le 2.2$。

评价重点： 推导过程、方法假设、图表可复现性及“稳定 $\neq$ 精确”的解释；允许合理的浮点舍入差异，但不允许改变题目参数、边界条件或终止时间。

难度、预计课后用时与来源

难度：中等偏上（迁移应用与实验综合题）。 建议课后用时 80～110 分钟：手工推导及解析解约 30 分钟，代码实验约 35～55 分钟，图表讨论约 15～25 分钟。

来源： 根据本教材 3.2.1 一维热方程与初边值条件、3.2.2 空间差分与显式时间推进、3.2.3 稳定性限制与网格实验 的方法与知识点原创编制。采用新的非齐次边界、多模态初值、网格和时间参数，不是教材例题的数值换算或复述。

题3：对应3.3 有限元：弱形式与Galerkin方法

习题 3.3-1（节点误差与整体误差） 对于 3.3.6 节中的边值问题

$$-u''(x) = \pi^2 \sin(\pi x), \qquad u(0) = u(1) = 0,$$

精确解为 $u(x) = \sin(\pi x)$。采用连续分片线性 Galerkin 有限元，并假定单元积分精确。

（1）设 $I_h u$ 为精确解的分片线性节点插值函数。利用各单元上检验函数导数为常数这一性质，说明在本问题中 $u_h = I_h u$。

（2）由（1）说明为什么节点误差为零，L^2 误差和 H^1 半范数误差却不为零。

（3）结合表 3.3-2，说明网格尺寸减半时上述两种积分误差分别如何变化。

习题 3.3-2（数值求积对节点值的影响） 仍考虑习题 3.3-1 的问题。将 3.3.6 节计算载荷向量时使用的四点 Gauss－Legendre 求积改为单元中点的一点 Gauss 求积，刚度矩阵保持不变。

（1）设 $m_e=(x_{e-1}+x_e)/2$，写出一点求积下的局部载荷向量近似式。

（2）分别取均匀网格 $N=8,16,32,64$，计算一点求积与四点求积下的节点最大误差

$$E_\infty=\max_{0\le i\le N}|u(x_i)-U_i|,$$

列表比较两者的结果。

（3）根据一点求积的节点误差估计经验收敛阶；说明为什么不宜利用接近数值舍入量级的四点求积节点误差作相同计算。

习题 3.3-3（非齐次 Dirichlet 边界条件） 考虑

$$-u''(x)=f(x),\quad 0<x<1,\qquad u(0)=0,\quad u(1)=\beta,$$

其中 β 为已知常数。仍采用连续分片线性 Galerkin 有限元。

（1）分别写出试探函数与检验函数应满足的端点条件，并说明两者为何不同。

（2）设 I 为内部节点集合，B 为边界节点集合。从装配后的整体矩阵出发，推导内部自由度所满足的方程

$$K_{II}^{\mathrm{all}}U_I=F_I^{\mathrm{all}}-K_{IB}^{\mathrm{all}}U_B.$$

（3）说明为什么不能先令两端节点值均为零，求出内部节点值后再把右端节点值改为 β。

习题 3.3-4（两单元有限元的计算与验证） 考虑下列边值问题：

$$\begin{cases}-u''(x)=1, & 0<x<1,\\ u(0)=0,\quad u(1)=\dfrac{1}{2}.\end{cases}$$

将区间 $[0,1]$ 等分为两个单元，

$$K_1=\left[0,\frac{1}{2}\right],\qquad K_2=\left[\frac{1}{2},1\right],$$

三个节点依次记为 $x_0=0$、$x_1=\frac{1}{2}$、$x_2=1$。采用连续分片线性 Galerkin 有限元，单元积分按精确积分处理。

（1）写出本问题的弱形式。说明积分分部后边界项消失的原因。

（2）写出一般单元上的两个线性形函数，并计算 K_1、K_2 的局部刚度矩阵 $K^{(1)},K^{(2)}$ 和局部载荷向量 $F^{(1)},F^{(2)}$。

（3）按照局部节点与全局节点的对应关系，装配完整矩阵 K^{all} 和向量 F^{all}。施加给定的 Dirichlet 边界条件，求内部节点值 U_1，并写出有限元解 $u_h(x)$ 的分段表达式。

（4）求出精确解 $u(x)$，分别比较 $x=\frac{1}{2}$ 和 $x=\frac{1}{4}$ 处的精确值与有限元近似值。

（5）编写程序核对（2）、（3）的手算结果。再取均匀网格 $N=2,4,8,16$，计算节点最大误差、L^2 误差和 H^1 半范数误差，按 3.3.6.4 节的定义求相邻网格的经验收敛阶。程序可使用 Python 3 和 NumPy，单元积分可采用四点 Gauss–Legendre 求积。提交程序及误差表，并给出内部线性方程组的残差。

（6）判断下列说法是否正确，并简述理由：① 积分分部后的边界项消失，是因为未知解在左端点等于零；② 求得内部节点值以后，再赋予右端点非零的边界值，不影响计算结果；③ 有限元与中心差分所得矩阵均为三对角矩阵，因此两种方法的离散原理相同；④ 若全部节点值与精确解一致，则有限元解在整个区间上均无误差。

题4：对应3.4 AI关联：Neural ODE与PINN

出题方向： 设计改变初值或评估步长后重新验证的题。

- 完整题面： 【待填写】对象、已知条件、参数、需要回答的具体问题。
- 推导或概念小问： 【待填写】要求学生亲自解释的关键步骤。
- 代码或结果分析小问： 【待填写】输入、允许使用的库、要求提交的图表或误差。
- 陷阱辨析小问： 【待填写】可从“将求解器产生的误差全归给网络，或将拟合一条轨迹称作已学会所有初值问题。”改编；标明是教学构造，不能伪造AI原话。
- 验收依据： 【待填写】答案应满足的条件、参考依据与合理容差。
- 难度、预计课后用时与来源： 【待填写】。

第03章 离散的世界：习题参考解答模板

题1：对应3.1 ODE初值问题：Euler与RK4

以下解答对应教材式习题 3.1-1—3.1-4。数值验证以三种方法的独立学生习题算例为对象，未采用正文课堂实验的 $y'=-y$ 代替题设；所用代码与输出收录于 习题3.1教师验证 Notebook。

习题 3.1-1　初值问题的解析解与单步计算

（1）解析解及独立参考的构造

原初值问题为

$$\begin{cases} y'(t)=-2y(t)+2t+1, & 0\le t\le 2,\\ y(0)=1. \end{cases}$$

记 $f(t,y)=-2y+2t+1$。由于右端显含时间 t，该方程为非自治常微分方程。令

$$w(t)=y(t)-t,$$

则 $y'(t)=w'(t)+1$，代入原方程得

$$w'(t)+1=-2(w(t)+t)+2t+1,$$

即

$$\begin{cases} w'(t)=-2w(t),\\ w(0)=1. \end{cases}$$

因此 $w(t)=e^{-2t}$，原问题的解析解为

$$\boxed{y(t)=t+e^{-2t}.}$$

验证：$y'(t)=1-2e^{-2t}$，而

$$-2y(t)+2t+1=-2\left(t+e^{-2t}\right)+2t+1=1-2e^{-2t},$$

且 $y(0)=0+1=1$，故解析解满足原初值问题。解析解由微分方程直接推得，不依赖任一种数值推进算法，因而可作为误差计算的独立参考。

（2）向前 Euler 与向后 Euler 的更新公式

设 $t_n=nh$，$y_n\approx y(t_n)$，且 $y_0=1$。向前 Euler 为

$$\begin{aligned}y_{n+1}&=y_n+hf(t_n,y_n)\\&=(1-2h)y_n+h(2t_n+1).\end{aligned}$$

向后 Euler 则为

$$\begin{aligned}y_{n+1}&=y_n+hf(t_{n+1},y_{n+1})\\&=y_n+h(-2y_{n+1}+2t_{n+1}+1).\end{aligned}$$

收集含 y_{n+1} 的项，得到

$$(1+2h)y_{n+1}=y_n+h(2t_{n+1}+1).$$

由于本题 $h>0$，有 $1+2h\neq 0$，故一步更新可显式求解为

$$\boxed{y_{n+1}=\frac{y_n+h(2t_{n+1}+1)}{1+2h}.}$$

向前 Euler 直接利用已知的 (t_n,y_n) 计算下一步；向后 Euler 的原始公式包含未知 y_{n+1}，需先求解隐式代数方程。本题因右端关于 y 是线性的才有上述闭式形式；对一般非线性方程，通常需要 Newton 迭代等方法求解。

（3）$h=0.5$ 时的一步手算

初始条件为 $t_0=0,\ y_0=1$，故

$$f(0,1)=-2+1=-1.$$

向前 Euler：

$$\boxed{y_1^{\mathrm{FE}}=1+0.5(-1)=0.5.}$$

向后 Euler：此时 $t_1=0.5$，由闭式更新式得

$$\boxed{y_1^{\mathrm{BE}}=\frac{1+0.5(2\times 0.5+1)}{1+2\times 0.5}=1.}$$

对于经典四阶 Runge–Kutta 方法，本解答采用 k_i 表示斜率，而非已乘以 h 的增量。四个阶段为

$$\begin{cases}k_1=f(t_0,y_0)=-1,\\[2pt]k_2=f\left(t_0+\dfrac{h}{2},y_0+\dfrac{h}{2}k_1\right)=f(0.25,0.75)=0,\\[2pt]k_3=f\left(t_0+\dfrac{h}{2},y_0+\dfrac{h}{2}k_2\right)=f(0.25,1)=-0.5,\\[2pt]k_4=f(t_0+h,y_0+hk_3)=f(0.5,0.75)=0.5.\end{cases}$$

按权重 $\mathbf{1:2:2:1}$ 组合，得到

$$\begin{aligned} y_1^{\mathrm{RK4}} &= y_0 + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \\ &= 1 + \frac{0.5}{6}\left[-1 + 2(0) + 2(-0.5) + 0.5\right] \\ &= \boxed{0.875}. \end{aligned}$$

精确值为

$$y(0.5) = 0.5 + e^{-1} \approx 0.867879441171.$$

方法	一步数值值	有符号误差 $y_1 - y(0.5)$	绝对误差
向前 Euler	0.500000000000	−0.367879441171	0.367879441171
向后 Euler	1.000000000000	+0.132120558829	0.132120558829
经典 RK4	0.875000000000	+0.007120558829	0.007120558829

在本题当前的一步计算中，RK4 最接近精确值；这一结论并不意味着它对任意方程或任意大步长都优于所有其他方法。

（4）一阶、二阶与四阶 Runge–Kutta 方法的联系

向前 Euler 可写为 $k_1 = f(t_n, y_n)$、$y_{n+1} = y_n + hk_1$，只有一个阶段，是最简单的显式一阶 Runge–Kutta 方法。一种典型的二阶显式方法可写为

$$\begin{cases} k_1 = f(t_n, y_n), \\ k_2 = f(t_n + h, y_n + hk_1), \\ y_{n+1} = y_n + \dfrac{h}{2}(k_1 + k_2). \end{cases}$$

它在一步内综合两个阶段的斜率；经典 RK4 则使用四个阶段及满足四阶精度条件的加权组合。阶段数增加不自动保证阶数提高，还必须合理选择阶段位置、预测状态及权重。这里的二阶方法仅作概念比较，不作为本题另外编程的对象。

习题 3.1-2　步长加密与收敛阶

（1）数值算法及独立核对公式

固定终止时刻 $T = 2$，步长 $h = 2/N$，依次选取 $N = 4, 8, 16, 32, 64$。三种方法的程序按各自一步更新公式实现；完整代码及已保存的运行输出位于独立的教师验证 Notebook中。

为独立核对逐步推进结果，令 $w_n = y_n - t_n$。由于 $t_{n+1} - t_n = h$，向前、向后 Euler 均可转换为对 w_n 的线性递推。对经典 RK4，在各阶段同时考虑对应的时间节点，可得到相同的变量变换关系。因此，

$$\begin{cases} y_N^{\mathrm{FE}} = T + (1 - 2h)^N, \\ y_N^{\mathrm{BE}} = T + (1 + 2h)^{-N}, \\ y_N^{\mathrm{RK4}} = T + [R_{\mathrm{RK4}}(-2h)]^N, \end{cases}$$

其中

$$R_{\mathrm{RK4}}(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}.$$

这些表达式来自针对本题模型的代数递推，不依赖数值推进程序，可用来检查其实现正确性。代码核验时，逐步计算值与独立递推值的差别处于浮点计算允许范围内。

（2）终点误差及观测收敛阶

解析参考值为

$$y(2) = 2 + e^{-4} \approx 2.018315638889.$$

取终点绝对误差

$$E_T(h) = |y_N - y(2)|,$$

得到以下计算结果。

表 解3.1-1　不同步长下的终点数值解（$T = 2$）

N	h	向前 Euler	向后 Euler	经典 RK4
4	0.5	2.0000000000	2.0625000000	2.0197753906
8	0.25	2.0039062500	2.0390184423	2.0183737403
16	0.125	2.0100225958	2.0281474977	2.0183185781
32	0.0625	2.0139398370	2.0230738084	2.0183158043
64	0.03125	2.0160753964	2.0206513250	2.0183156487

表 解3.1-2　不同步长下的终点绝对误差（$T = 2$）

N	h	向前 Euler	向后 Euler	经典 RK4
4	0.5	1.831564×10^{-2}	4.418436×10^{-2}	1.459752×10^{-3}
8	0.25	1.440939×10^{-2}	2.070280×10^{-2}	5.810140×10^{-5}
16	0.125	8.293043×10^{-3}	9.831859×10^{-3}	2.939254×10^{-6}
32	0.0625	4.375802×10^{-3}	4.758169×10^{-3}	1.654431×10^{-7}
64	0.03125	2.240243×10^{-3}	2.335686×10^{-3}	9.814253×10^{-9}

由

$$p_{\mathrm{obs}}(h) = \log_2 \frac{E_T(h)}{E_T(h/2)}$$

计算观测收敛阶，结果如下。

表 解3.1-3　步长减半时的观测收敛阶

$N \to 2N$	向前 Euler	向后 Euler	经典 RK4
4 → 8	0.3461	1.0937	4.6510
8 → 16	0.7970	1.0743	4.3051

	向前 Euler	向后 Euler	经典 RK4
16 → 32	0.9224	1.0471	4.1510
32 → 64	0.9659	1.0266	4.0753

在给定的网格范围内，两种 Euler 方法的观测阶数逐渐接近 $\mathbf{1}$，经典 RK4 的观测阶数逐渐接近 $\mathbf{4}$。其中，向前 Euler 在 $\boldsymbol{N=4}$ 时对应的放大因子恰为零，属于较粗步长的特殊情形，因此最初两组误差的比值并不能代表其渐近阶数。一般而言，有限步长下仍有高阶误差项，观测阶数可能偏离理论值；若误差已接近浮点舍入水平，阶数估计也可能失去可靠性。

（3）数值轨迹与误差曲线

在 $\boldsymbol{T=2,\ h=0.5}$ 时，数值轨迹如下图所示。

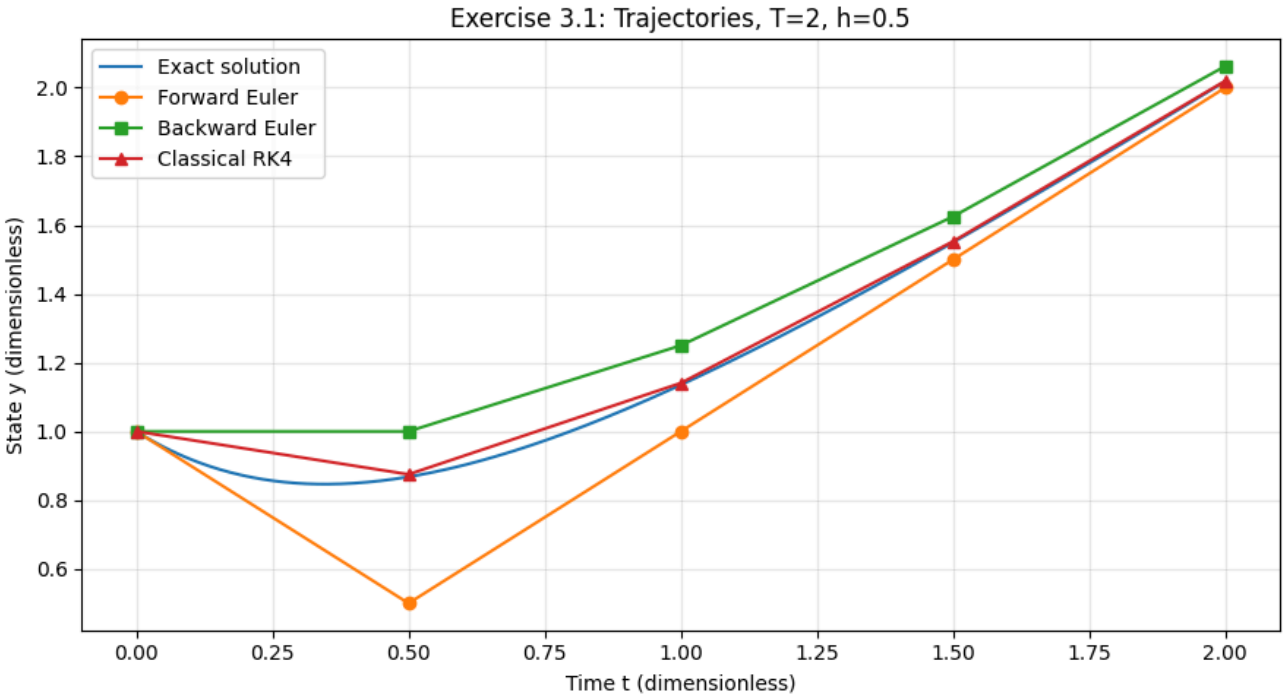


图 解3.1-1　向前 Euler、向后 Euler 与经典 RK4 的节点解和解析解；$\boldsymbol{T=2}$，$\boldsymbol{h=0.5}$，时间与状态均无量纲。

终点误差随步长变化的双对数图如下。

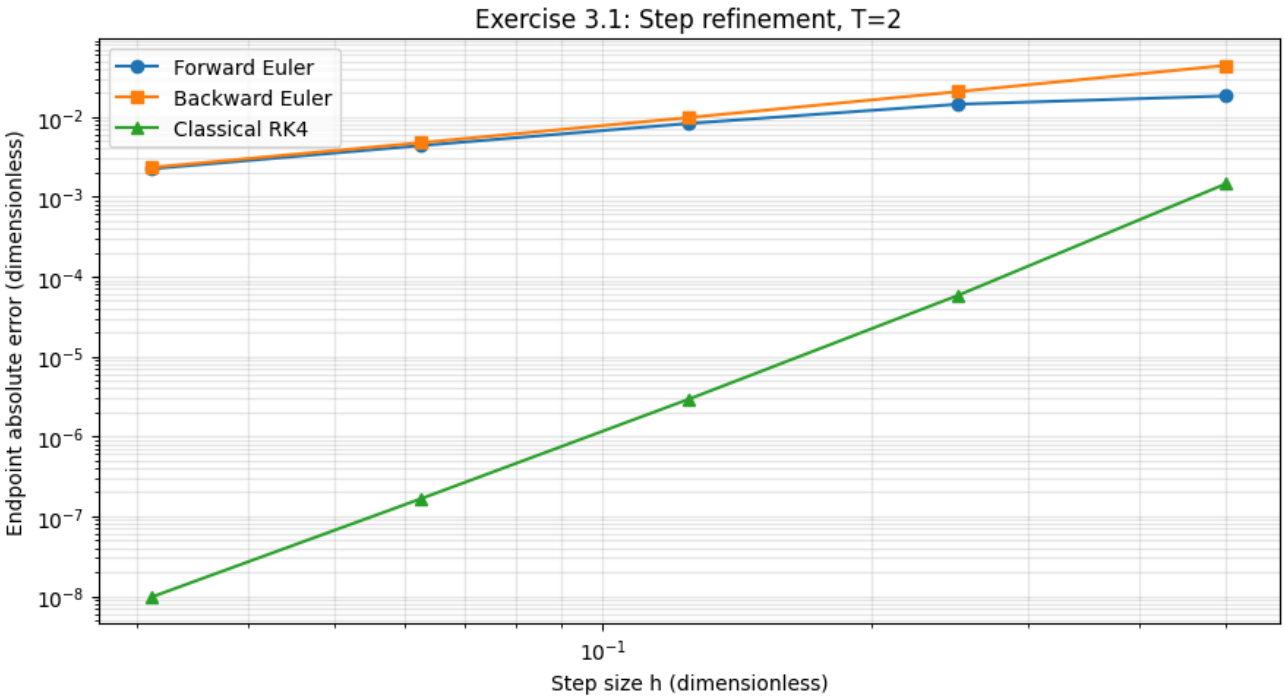


图 解3.1-2　终点绝对误差关于步长的双对数图；$T=2$，$N=4,8,16,32,64$。横轴为无量纲步长，纵轴为无量纲终点绝对误差。

曲线与误差表共同显示：随着步长加密，RK4 误差下降的速度明显快于两种 Euler 方法。

（4）局部截断误差与全局误差

按题目约定，局部截断误差是从精确起点推进一步的差值，且不除以 h。在精确解足够光滑、固定有限区间内误差传播可控等通常条件下：

方法	一步局部截断误差	全局误差	方法阶数
向前 Euler	$O(h^2)$	$O(h)$	一阶
向后 Euler	$O(h^2)$	$O(h)$	一阶
经典 RK4	$O(h^5)$	$O(h^4)$	四阶

Euler 方法的局部误差阶可由二阶 Taylor 展开说明；RK4 的四阶结论要求各阶段和权重满足四阶 Runge－Kutta 条件，并需要相应阶数的光滑性。全局误差还涉及多次推进时的误差累积与传播，因此不能直接把局部差值的阶数当作全局误差阶数。常见充分条件包括右端函数对状态具有适当的 Lipschitz 性质、精确解具有足够连续导数，以及数值递推满足所需的稳定性与求解条件。

习题 3.1-3　非自治方程的稳定性辨析

（1）衰减分量的提取

仍令 $w(t)=y(t)-t$，则

$$\boxed{\begin{cases} w'(t)=-2w(t), \\ w(0)=1, \end{cases} \qquad w(t)=e^{-2t}.}$$

原变量 $y(t)=t+w(t)$ 含有随时间增长的项 t。因此，即使 $w(t)$ 按指数规律衰减，$y(t)$ 也未必随时间下降。研究本题的衰减稳定性，应把增长的已知趋势与衰减分量分开，重点考察 w_n 的传播因子。

（2）三个方法的放大因子

记 $w_n=y_n-t_n$，令 $z=-2h$。由向前 Euler 更新式，得到

$$w_{n+1}=(1-2h)w_n=(1+z)w_n,$$

故 $R_{\mathrm{FE}}(z)=1+z$。由向后 Euler 更新式，得到

$$(1+2h)w_{n+1}=w_n,$$

故 $R_{\mathrm{BE}}(z)=(1-z)^{-1}$。

对于 RK4，若将各阶段对原变量 y 的斜率记为 k_i，定义 $s_i=k_i-1$，则由于线性变换 $w=y-t$，其四阶段可以写为

$$\begin{cases} s_1=-2w_n, \\ s_2=-2\left(w_n+\dfrac{h}{2}s_1\right), \\ s_3=-2\left(w_n+\dfrac{h}{2}s_2\right), \\ s_4=-2(w_n+hs_3), \end{cases}$$

并且

$$w_{n+1}=w_n+\frac{h}{6}(s_1+2s_2+2s_3+s_4)=R_{\mathrm{RK4}}(z)w_n.$$

综上，

$$\boxed{\begin{cases} R_{\mathrm{FE}}(z)=1+z, \\ R_{\mathrm{BE}}(z)=\dfrac{1}{1-z}, \\ R_{\mathrm{RK4}}(z)=1+z+\dfrac{z^2}{2}+\dfrac{z^3}{6}+\dfrac{z^4}{24}. \end{cases}}$$

取 $h=1.25$，即 $z=-2.5$，得到

方法	$R(-2.5)$	是否满足 $\lvert R(-2.5)\rvert<1$
向前 Euler	-1.5000000000	否
向后 Euler	0.2857142857	是
经典 RK4	0.6484375000	是

故在此测试方程及此步长下，向前 Euler 对衰减分量不稳定；向后 Euler 与 RK4 能保持该分量的离散衰减。

（3）大步长数值结果与误差解释

由 $w_0=1$ 和上述放大因子可知，三种数值解在网格点上满足

$$\boxed{y_n=t_n+[R(-2.5)]^n}, \qquad t_n=1.25n, \quad n=0,1,2,3,4.$$

逐步计算得到：

表 解3.1-4　大步长稳定性实验的节点数值（$T=5$，$h=1.25$）

t_n	解析解 $y(t_n)$	向前 Euler	向后 Euler	经典 RK4
0.00	1.0000000000	1.0000000000	1.0000000000	1.0000000000
1.25	1.3320849986	-0.2500000000	1.5357142857	1.8984375000
2.50	2.5067379470	4.7500000000	2.5816326531	2.9204711914
3.75	3.7505530844	0.3750000000	3.7733236152	4.0226492882
5.00	5.0000453999	10.0625000000	5.0066638900	5.1767960228

由于 $y(5)=5+e^{-10}\approx 5.000045399930$，终点绝对误差为

方法	y_4	$\lvert y_4-y(5)\rvert$
向前 Euler	**10.062500000000**	**5.062454600070**
向后 Euler	**5.006663890046**	**0.006618490116**
经典 RK4	**5.176796022803**	**0.176750622873**

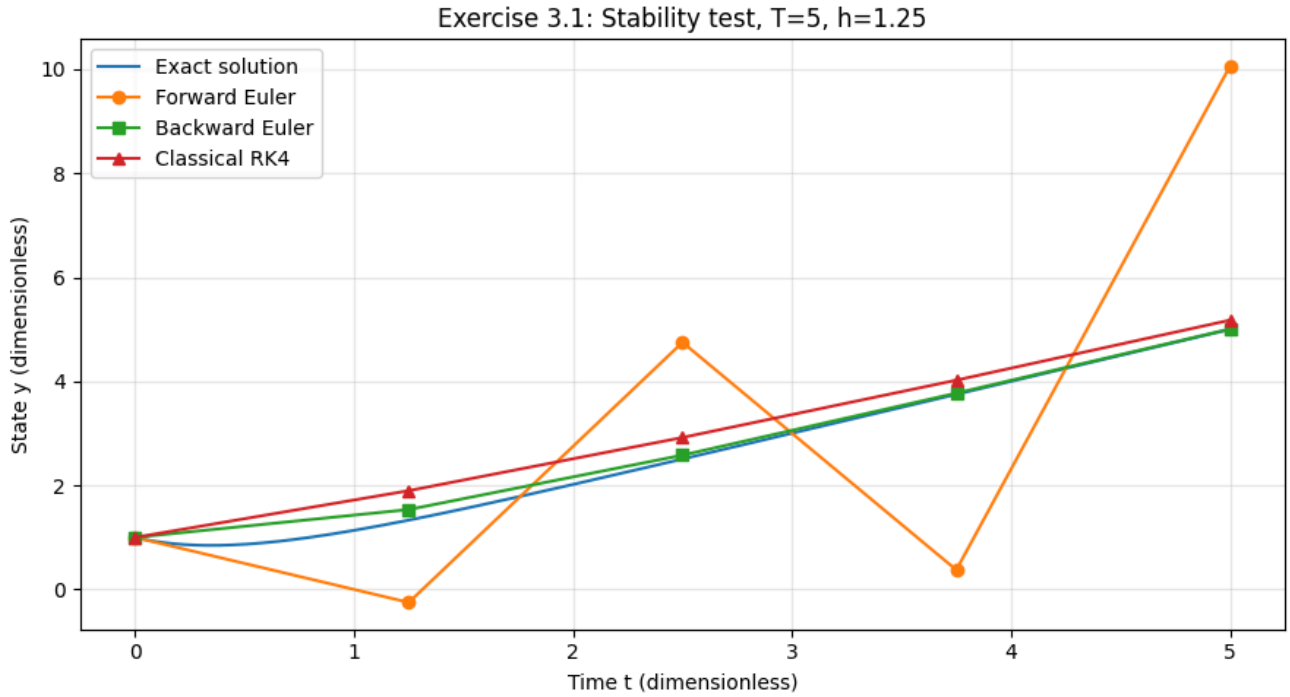


图 解3.1-3　解析解与三种数值轨迹的比较；$T=5$，$h=1.25$，横、纵坐标均为无量纲量。

本例中，向后 Euler 和 RK4 的衰减分量均保持稳定，但误差并不相同；特别是 RK4 虽具有四阶理论精度，却在该较大步长下出现了不可忽略的误差。因此，稳定不等于精确，方法阶数也不能决定所有粗步长下的精度高低。

（4）辨析题判断

① 错误。 原变量 $y(t)=t+e^{-2t}$ 包含增长项 t，即便衰减分量 $w(t)=e^{-2t}$ 正常衰减，$y(t)$ 也可能增大。因此，不能仅凭 y_n 的上升趋势判断数值方法不稳定，应结合方程结构及放大因子分析。

② 错误。 向前 Euler 和向后 Euler 同属一阶方法，但稳定函数分别为 $1+z$ 和 $(1-z)^{-1}$，绝对稳定区域不同。特别是对于 $\mathrm{Re}(z)<0$，向后 Euler 满足 $|1/(1-z)|<1$；向前 Euler 则不具有这一性质。

③ 错误。 RK4 的四阶精度是步长趋于零时的渐近收敛性质，并不保证任意步长下稳定、准确。例如取 $z=-4$ 时，经典 RK4 的稳定函数值为 $R_{\mathrm{RK4}}(-4)=5$，其绝对值大于 1，会放大衰减模态的数值振幅。本题 $z=-2.5$ 时虽然 $|R_{\mathrm{RK4}}|<1$，终点误差仍然明显。

以上三句话均为习题构造的讨论命题，不是某一真实 AI 回答的转述。

习题 3.1-4　精度要求、计算成本与程序核对

（1）首次达到终点误差要求的网格

仍取 $T=2$，要求 $E_T(h)\le 10^{-4}$，且仅在 $N=4,8,16,32,\dots$ 的二分加密候选网格中寻找首次达标结果。得到：

表 解3.1-5　相同精度目标下的数值结果与计算量

方法	首次达标步数 N	$h=2/N$	终点绝对误差	计算成本
向前 Euler	2048	0.0009765625	7.149884×10^{-5}	2048 次右端函数求值
向后 Euler	2048	0.0009765625	7.159200×10^{-5}	2048 次闭式隐式更新，另有残差核对
经典 RK4	8	0.25	5.810140×10^{-5}	32 次右端函数求值

在这一误差要求及规定的候选网格上，向前 Euler 的右端函数求值总次数为 2048，RK4 为 $8\times4=32$，比值为 64。这一比例仅反映本题及该误差目标，不能当作所有初值问题的通用加速比。

（2）向后 Euler 的代数残差

向后 Euler 的隐式方程残差定义为

$$r_{n+1}=y_{n+1}-y_n-hf(t_{n+1},y_{n+1}).$$

代入 $f(t,y)=-2y+2t+1$，整理可得

$$\boxed{r_{n+1}=(1+2h)y_{n+1}-y_n-h(2t_{n+1}+1).}$$

在本题使用的闭式更新式下，数学上应有 $r_{n+1}=0$；由于浮点运算存在舍入误差，实际计算得到的各步残差不一定严格为零。在首次达标的 $N=2048$ 网格上，已执行的教师 Notebook 记录到

$$\boxed{\max_{1\le n\le 2048}|r_n|\approx 2.212\times10^{-16}<10^{-10}.}$$

这表明隐式代数方程已得到充分精确的数值求解，却不能据此推断时间离散误差为零。事实上，此时向后 Euler 的终点误差仍约为 7.1592×10^{-5}。

本实现使用闭式公式，因此没有执行 Newton 迭代，也没有产生 Newton 导数求值与迭代次数的记录。若改用 Newton 法，须另行统计右端函数求值、导数求值和迭代更新次数，不应虚构这些数据。

（3）成本比较及容差的不同含义

本题中向前 Euler 每步计算一次 f，经典 RK4 每步计算四次 f。向后 Euler 需要求解隐式代数方程，本模型可直接求得闭式结果，其实际开销主要为算术运算及必要的残差验证；在一般非线性问题中，这一求解成本还可能包含 Newton 迭代、雅可比计算与线性系统求解，不能简单计为“每步一次函数求值”。

本题中三种容差的用途需要严格区分：

容差或指标	用途
10^{-4}	本题指定的 ODE 终点误差目标，必须与解析解比较
10^{-10}	向后 Euler 代数方程残差的建议验收阈值
10^{-8}	学生程序数值结果与独立计算结果之间的复核容差

三者评价的对象不同；只有代数残差足够小，不能代替数值方法收敛性和终点误差的检验。

（4）实现位置、运行环境与提交证据

教师验证 Notebook 已实际执行，并与正文课堂实验 Notebook 分离保存，文件为：

下载 / 查看：习题3.1教师解答与数值核验.ipynb

该文件依次给出方法实现、习题 3.1-1 一步核查、习题 3.1-2 误差与观测阶、习题 3.1-3 稳定性节点表及图像、习题 3.1-4 精度目标与代数残差。生成的三张图片在同一配套目录的 `images/` 子目录中，且已经在本节各处引用。

本次运行记录为：Python 3.10.13、NumPy 2.2.6、Matplotlib 3.9.0、Darwin arm64，双精度浮点运算。模型与参数均为确定性的，不涉及随机数和随机种子。执行的 8 个代码单元全部成功，包含逐步更新结果与独立递推的交叉校验；图像及运行输出均保存在 Notebook 中。以上为实际程序自检，不冒充 B 组与 C 组的人工审核。

复核状态及理论依据

- 数值公式、单步手算与计算代码核查： 已完成独立程序检查，并保存可复现输出。
- 数学表达与数值误差： 与题设非自治方程及解析解 $y(t) = t + e^{-2t}$ 对照。
- B 组独立核验、C 组人工复核： 待有关人员实际开展并记录，不预先填写“通过”。
- Git 提交记录与最终排版复核： 待项目仓库实际操作后填写。

理论依据可参见 Burden、Faires、Burden 的 Numerical Analysis（第 10 版，第 5 章）及 Sauer 的 Numerical Analysis（第 3 版，第 6 章）。本题中的受迫模型、计算参数与讨论命题均以对应的教材式习题为准。

题2：对应3.2 PDE有限差分：热传导与CFL条件

- 条件与符号： 【待填写】。
- 手动推导或概念解释： 【待填写】关键步骤与依据。
- 实现位置与独立参考： 【待填写】Notebook节号/代码单元及参考来源。

- 实际输出、误差和容差理由： 【待填写】，未运行写未运行。
- 陷阱的正确解释及适用范围： 【待填写】。
- B核验、C复核及版本： 待核验。

题3：对应3.3 有限元：弱形式与Galerkin方法

习题 3.3-1

（1）设 V_h 为满足齐次 Dirichlet 条件的连续分片线性空间。对任意 $v_h \in V_h$，由于 v_h' 在每个单元 $K_e = [x_{e-1}, x_e]$ 上为常数，

$$\int_{K_e} (u - I_h u)' v_h' \,\mathrm{d}x = v_h'|_{K_e} \big[(u - I_h u)(x_e) - (u - I_h u)(x_{e-1})\big] = 0.$$

各单元求和得

$$\int_0^1 (u - I_h u)' v_h' \,\mathrm{d}x = 0.$$

结合精确弱形式可知，$I_h u$ 满足 Galerkin 离散方程。由离散解的唯一性，有

$$u_h = I_h u.$$

（2）由插值函数定义，$U_i = u_h(x_i) = u(x_i)$，故精确积分时节点误差为零。但 $u(x) = \sin(\pi x)$ 在单元内部不是一次函数，因而 $u - u_h$ 一般不为零，其 L^2 误差与 H^1 半范数误差也不为零。

（3）根据正文表 3.3-2，L^2 误差的经验阶约为 2，网格尺寸减半后误差约变为原来的 $1/4$；H^1 半范数误差的经验阶约为 1，减半后误差约变为原来的 $1/2$。

习题 3.3-2

（1）令单元中点 $m_e = (x_{e-1} + x_e)/2$。一点 Gauss–Legendre 求积给出

$$F^{(e)} \approx \frac{h_e}{2} f(m_e) \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

刚度矩阵仍为

$$K^{(e)} = \frac{1}{h_e} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}.$$

（2）采用相同的均匀网格与载荷函数 $f(x) = \pi^2 \sin(\pi x)$，计算结果如下。表内数字为示例程序使用双精度浮点数所得结果，最后几位会随计算环境略有差异。

N	一点求积 E_∞	四点求积 E_∞	一点求积经验阶
8	6.512818×10^{-3}	2.242206×10^{-12}	—
16	1.611809×10^{-3}	8.659740×10^{-15}	2.0146
32	4.019341×10^{-4}	3.108624×10^{-15}	2.0037
64	1.004200×10^{-4}	9.103829×10^{-15}	2.0009

（3）一点求积产生有规律的、明显高于舍入误差的节点误差。其经验阶

$$p_h = \frac{\log(E_h/E_{h/2})}{\log 2}$$

逐渐接近 2。相反，四点求积下的节点误差很快接近浮点舍入量级，此时误差比受求积残差和舍入影响，不能据此判断可信的渐近收敛阶。

还可以解析验证上述结果：对偶数 N，令 $t = \pi h/2$，一点求积所得内部节点值具有形式

$$U_i = A_h \sin(\pi x_i), \qquad A_h = \frac{t^2 \cos t}{\sin^2 t}.$$

因 $A_h = 1 - \frac{t^2}{6} + O(t^4)$，故节点最大误差为 $O(h^2)$。

习题 3.3-3

（1）试探函数满足 $u(0)=0,\ u(1)=\beta$，而检验函数须满足 $v(0)=v(1)=0$。记

$$V_\beta = \{w \in H^1(0,1) : w(0)=0,\ w(1)=\beta\}, \quad V_0 = H_0^1(0,1).$$

弱形式为：求 $u \in V_\beta$，使

$$\int_0^1 u'v'\,\mathrm{d}x = \int_0^1 fv\,\mathrm{d}x, \quad \forall v \in V_0.$$

右端边界值非零并不影响齐次检验空间的选取。

（2）将全部节点自由度按内部和边界重排，内部节点方程为

$$K_{II}^{\mathrm{all}} U_I + K_{IB}^{\mathrm{all}} U_B = F_I^{\mathrm{all}}.$$

故

$$K_{II}^{\mathrm{all}} U_I = F_I^{\mathrm{all}} - K_{IB}^{\mathrm{all}} U_B.$$

（3）若解方程时错将 U_B 设为零，就遗漏了右端修正项 $-K_{IB}^{\mathrm{all}} U_B$，从而一般会得到错误的内部节点值。正确做法是在求解内部自由度以前引入所有已知边界值。

习题 3.3-4

（1）设

$$V_g = \{w \in H^1(0,1) : w(0)=0,\ w(1)=\tfrac{1}{2}\}, \qquad V_0 = H_0^1(0,1).$$

对 $-u''=1$ 乘以 $v \in V_0$，积分分部可得

$$\int_0^1 u'v'\,\mathrm{d}x - [u'v]_0^1 = \int_0^1 v\,\mathrm{d}x.$$

由于 检验函数 在两端取零，边界项消失。因此弱形式为

$$\int_0^1 u'v'\,\mathrm{d}x = \int_0^1 v\,\mathrm{d}x, \quad \forall v \in V_0.$$

（2）对于 $K_e = [x_{e-1}, x_e]$，$h_e = x_e - x_{e-1}$，取

$$N_1(x) = \frac{x_e - x}{h_e}, \qquad N_2(x) = \frac{x - x_{e-1}}{h_e}.$$

计算可得

$$K^{(e)}=\frac{1}{h_e}\begin{bmatrix}1 & -1\\ -1 & 1\end{bmatrix},\qquad F^{(e)}=\frac{h_e}{2}\begin{bmatrix}1\\ 1\end{bmatrix}.$$

因两单元长度均为 $\frac{1}{2}$，

$$K^{(1)}=K^{(2)}=\begin{bmatrix}2 & -2\\ -2 & 2\end{bmatrix},\quad F^{(1)}=F^{(2)}=\begin{bmatrix}\frac{1}{4}\\ \frac{1}{4}\end{bmatrix}.$$

（3）单元 K_1 的局部节点对应全局节点 $(0,1)$，单元 K_2 对应 $(1,2)$。在公共节点累加贡献，得到

$$K^{\text{all}}=\begin{bmatrix}2 & -2 & 0\\ -2 & 4 & -2\\ 0 & -2 & 2\end{bmatrix},\quad F^{\text{all}}=\begin{bmatrix}\frac{1}{4}\\ \frac{1}{2}\\ \frac{1}{4}\end{bmatrix}.$$

已知 $U_0=0$、$U_2=\frac{1}{2}$，内部节点方程为

$$-2U_0+4U_1-2U_2=\frac{1}{2},\quad\Longrightarrow\quad 4U_1=\frac{3}{2},U_1=\frac{3}{8}.$$

因此

$$u_h(x)=\begin{cases}\dfrac{3}{4}x, & 0\le x\le\frac{1}{2},\\[2mm] \dfrac{1}{4}x+\dfrac{1}{4}, & \frac{1}{2}\le x\le 1.\end{cases}$$

（4）精确解为

$$u(x)=x-\frac{x^2}{2}.$$

中点处 $u(\frac{1}{2})=u_h(\frac{1}{2})=\frac{3}{8}$；而

$$u(\tfrac{1}{4})=\frac{7}{32},\qquad u_h(\tfrac{1}{4})=\frac{3}{16}=\frac{6}{32},$$

故 $x=\frac{1}{4}$ 处绝对误差为 $\frac{1}{32}$。节点正确并不意味着单元内部的折线与精确抛物线重合。

（5）由精确积分可知，本常系数一维问题中的有限元解是精确解的分片线性节点插值。对任一长度为 h 的单元 $[a,b]$，有

$$u(x)-u_h(x)=\frac{(x-a)(b-x)}{2}.$$

因此在均匀网格上

$$E_{L^2}^2=N\int_0^h\left[\frac{t(h-t)}{2}\right]^2\mathrm{d}t=\frac{h^4}{120},$$
$$E_{H^1}^2=N\int_0^h\left(\frac{h}{2}-t\right)^2\mathrm{d}t=\frac{h^2}{12}.$$

即

$$E_{L^2}=\frac{h^2}{\sqrt{120}},\qquad E_{H^1}=\frac{h}{\sqrt{12}}.$$

数值核验结果为

N	h	E_∞（理论值）	E_{L^2}	E_{H^1}	p_{L^2}	p_{H^1}
2	0.50000	0	0.022821773	0.144337567	—	—
4	0.25000	0	0.005705443	0.072168784	2	1
8	0.12500	0	0.001426361	0.036084392	2	1
16	0.06250	0	0.000356590	0.018042196	2	1

程序使用双精度数值计算时，节点误差可能是约 10^{-16} 的舍入值，而非严格的零。由于这一误差太小，不宜用它计算经验收敛阶。内部方程的无穷范数残差应接近数值舍入误差；两单元例题的参考程序结果为 0。

（6）① 错误。边界项消失是因为检验函数在两端取零，与未知解在左端的数值无直接关系。② 错误。应在求内部节点值前引入非零边界值，并修正方程右端。③ 错误。有限差分近似微分算子，Galerkin 有限元从弱形式与基函数积分出发，矩阵结构相似不等于原理相同。④ 错误。此题 $x=\frac{1}{4}$ 处仍有非零误差，L^2 与 H^1 半范数误差也非零。

题4：对应3.4 AI关联：Neural ODE与PINN

- 条件与符号： 【待填写】。
- 手动推导或概念解释： 【待填写】关键步骤与依据。
- 实现位置与独立参考： 【待填写】Notebook节号/代码单元及参考来源。
- 实际输出、误差和容差理由： 【待填写】，未运行写未运行。
- 陷阱的正确解释及适用范围： 【待填写】。
- B核验、C复核及版本： 待核验。

第03章 离散的世界：习题参考解答模板

题2：对应3.2 PDE有限差分：热传导与CFL条件

习题 3.2-1（非齐次 Dirichlet 边界下的 FTCS 稳定性与网格收敛）参考解答

条件与符号

本题采用不同于教材例题的非齐次 Dirichlet 边界与双正弦模态初值，不能直接使用教材中“两端为零、单模态初温”的解析解。

$$\begin{cases} u_t=\kappa u_{xx}, & 0<x<L,\quad 0<t\le T,\\ u(0,t)=20\,^\circ\mathrm{C},\quad u(L,t)=10\,^\circ\mathrm{C},\\ u(x,0)=20-\dfrac{10x}{L}+5\sin\!\left(\dfrac{\pi x}{L}\right)+2\sin\!\left(\dfrac{2\pi x}{L}\right). \end{cases}$$

其中 $L=1.2\,\mathrm{m}$，$\kappa=0.08\,\mathrm{m^2/s}$，$T=0.45\,\mathrm{s}$；初温表达式中的数值系数以 $^\circ\mathrm{C}$ 为单位。

符号	含义
J、$\Delta x=L/J$	空间分段数及网格间距，基准取 $J=60$

符号	含义
$x_j = j\Delta x$	空间节点，$j = 0, \dots, J$
$t_n = n\Delta t$	时间层
U_j^n	温度近似值
$r = \kappa\Delta t/(\Delta x)^2$	网格 Fourier 数（无量纲）
$u_s(x)$	连续稳态温度分布
$V_j^n = U_j^n - u_s(x_j)$	去掉稳态分布后的离散温度偏差
$\varepsilon = 10^{-10}\,^\circ\mathrm{C}$	仅用于稳定性对照实验的人工高频扰动幅度

候选时间步长 A、B、C 分别为 $0.001875\,\mathrm{s}$、$0.002500\,\mathrm{s}$、$0.003750\,\mathrm{s}$。稳定性实验固定 $J = 60$ 并加入同一高频扰动；网格加密实验采用原始（无扰动）初值。

（1）FTCS 差分格式的推导

在均匀网格 $x_j = j\Delta x$、$t_n = n\Delta t$ 上，Taylor 展开给出

$$\frac{u(x_j, t_{n+1}) - u(x_j, t_n)}{\Delta t} = u_t + \frac{\Delta t}{2}u_{tt} + \mathcal{O}((\Delta t)^2),$$
$$\frac{u(x_{j+1}, t_n) - 2u(x_j, t_n) + u(x_{j-1}, t_n)}{(\Delta x)^2} = u_{xx} + \frac{(\Delta x)^2}{12}u_{xxxx} + \mathcal{O}((\Delta x)^4).$$

用上述两个差商代替原方程的偏导数，舍去截断余项，得到

$$\frac{U_j^{n+1} - U_j^n}{\Delta t} = \kappa\frac{U_{j+1}^n - 2U_j^n + U_{j-1}^n}{(\Delta x)^2}.$$

两边乘 Δt，令 $r = \kappa\Delta t/(\Delta x)^2$ 并整理：

$$\boxed{U_j^{n+1} = rU_{j-1}^n + (1-2r)U_j^n + rU_{j+1}^n \quad (j = 1, \dots, J-1).}$$

边界与初值不参与未知量联立求解，直接设置

$$U_0^n = 20, \quad U_J^n = 10,$$
$$U_j^0 = 20 - \frac{10x_j}{L} + 5\sin(\pi x_j/L) + 2\sin(2\pi x_j/L).$$

空间差分二阶、时间差分一阶；并且 $[\kappa\Delta t/(\Delta x)^2] = (\mathrm{m}^2/\mathrm{s})\,\mathrm{s}/\mathrm{m}^2 = 1$，因此 r 无量纲。

（2）CFL 稳定性条件与步长选择

在非齐次边界下，先用符号 u_s 表示与两端恒温相容的线性稳态函数；它的具体表达式将在第（3）问给出。由于该函数的离散二阶差分为零，去稳态后的扰动与原热方程使用同一个 FTCS 递推算子。

对 $0 \le r \le 1/2$，系数 $r, 1-2r, r$ 均非负且和为 1，因而 U_j^{n+1} 为上一层三个相邻值的凸组合。更严格地，定义 $b_j = u_s(x_j)$。由于 $b(x)$ 是线性函数，离散二阶差分严格为零，即 $b_{j-1} - 2b_j + b_{j+1} = 0$。所以去稳态偏差 $V_j^n = U_j^n - b_j$ 满足

$$V_j^{n+1} = rV_{j-1}^n + (1-2r)V_j^n + rV_{j+1}^n, \qquad V_0^n = V_J^n = 0.$$

利用三系数非负且和为 1，

$$|V_j^{n+1}| \le r|V_{j-1}^n| + (1-2r)|V_j^n| + r|V_{j+1}^n| \le \|V^n\|_\infty .$$

对全部节点取最大值得

$$\boxed{\|V^{n+1}\|_\infty \le \|V^n\|_\infty .}$$

可见两端不为零不妨碍使用该标准稳定性结论，因为把已知稳态减去后仍是相同的 FTCS 齐次误差递推。

本题 $\Delta x = 1.2/60 = 0.020000\,\mathrm{m}$，由 $r \le 1/2$ 得

$$\boxed{\Delta t_{\max} = \frac{(0.02)^2}{2(0.08)} = 0.002500\,\mathrm{s}.}$$

于是：

方案	Δt (s)	$r = 0.08\Delta t/(0.02)^2$	标准 CFL 判定	$T/\Delta t$ (步)
A	0.001875	0.375	满足	240
B	0.002500	0.500	满足，处于临界值	180
C	0.003750	0.750	不满足	120

（3）稳态分解与独立解析解

稳态温度 $u_s(x)$ 不随时间变化，应满足

$$0 = \kappa u_s''(x), \qquad u_s(0) = 20, \quad u_s(L) = 10.$$

因此 $u_s'' = 0$，设 $u_s(x) = a + bx$，由边界条件得 $a = 20$、$b = -10/L$，所以

$$\boxed{u_s(x) = 20 - \frac{10x}{L}.}$$

令 $v(x,t) = u(x,t) - u_s(x)$。由于 $u_s'' = 0$，原方程化为

$$v_t = \kappa v_{xx}, \qquad v(0,t) = v(L,t) = 0,$$

$$v(x,0) = 5\sin(\pi x/L) + 2\sin(2\pi x/L).$$

对第 m 个模态 $\sin(m\pi x/L)$，有

$$\frac{\partial^2}{\partial x^2}\sin(m\pi x/L) = -\left(\frac{m\pi}{L}\right)^2 \sin(m\pi x/L).$$

因此该模态随时间乘以 $e^{-\kappa(m\pi/L)^2 t}$，利用方程的线性性得到独立解析参考解

$$\boxed{\begin{aligned} u_{\mathrm{ref}}(x,t) = 20 - \frac{10x}{L} \\ + 5\sin\!\left(\frac{\pi x}{L}\right) e^{-\kappa(\pi/L)^2 t} \\ + 2\sin\!\left(\frac{2\pi x}{L}\right) e^{-\kappa(2\pi/L)^2 t}. \end{aligned}}$$

逐项对 t 微分、对 x 求二阶导数，可验证 $u_t = \kappa u_{xx}$；由于两项正弦在 $x = 0, L$ 为零，端点一直分别为 $20\,^\circ\mathrm{C}$ 和 $10\,^\circ\mathrm{C}$；取 $t = 0$ 恰好恢复题目初值。第二个模态的衰减指数是第一个模态的 4 倍，所以温度剖面的形状会随时间变化，不能照搬教材单模态“形状不变、只衰减振幅”的结论。随着时间延长，解趋向非

零的线性稳态 u_s。

（4）截断误差、稳定性与网格加密

将精确解代入差分方程，得到局部截断误差

$$\tau_j^n = \frac{u(x_j,t_{n+1})-u(x_j,t_n)}{\Delta t} - \kappa\frac{u(x_{j+1},t_n)-2u(x_j,t_n)+u(x_{j-1},t_n)}{(\Delta x)^2}$$
$$= \frac{\Delta t}{2}u_{tt} - \frac{\kappa(\Delta x)^2}{12}u_{xxxx} + \mathcal{O}\big((\Delta t)^2+(\Delta x)^4\big).$$

连续方程中 κ 为常数，且精确解足够光滑，故 $u_{tt}=\kappa^2 u_{xxxx}$，得到

$$\tau_j^n = \left(\frac{\kappa^2\Delta t}{2} - \frac{\kappa(\Delta x)^2}{12}\right)u_{xxxx} + \mathcal{O}\big((\Delta t)^2+(\Delta x)^4\big).$$

所以格式相容阶为 $\mathcal{O}(\Delta t+(\Delta x)^2)$，在标准稳定性条件成立时，固定终止时间上的全局误差通常也具有该量级。稳定性只限制误差被放大的方式，不意味着数值解与解析解相同。

如果空间网格加密一倍，即 $\Delta x' = \Delta x/2$，为保持 r 不变，

$$r = \frac{\kappa\Delta t'}{(\Delta x')^2} = \frac{\kappa\Delta t}{(\Delta x)^2} \quad\Longrightarrow\quad \boxed{\Delta t' = \Delta t/4}.$$

保持 r 不变的连续网格加密中，$\Delta t \propto (\Delta x)^2$，因此可预期总体误差按 $\mathcal{O}((\Delta x)^2)$ 减小；这只能验证联合加密的收敛阶，不能单凭这一个实验分别测出时间与空间的独立误差阶数。

（5）稳定性对照实验及计算实现

实现位置： 建议将参考实现放置在“第 03 章 Notebook → 3.2.3 稳定性限制与网格实验 → 课后题2（非齐次边界与多模态）”的独立单元中。这只是建议的安排，未声称已修改课程仓库中的 Notebook。

独立参考依据： 使用第（3）问由连续热方程、稳态分解和正弦模态叠加推导的解析解。稳定性对照实验还需将人为高频模态的连续解析衰减项包含进去：

$$u_{\varepsilon,\mathrm{ref}}(x,t;J) = u_{\mathrm{ref}}(x,t) + \varepsilon\sin\left(\frac{(J-1)\pi x}{L}\right)e^{-\kappa((J-1)\pi/L)^2 t}.$$

其参数依赖 J，因此只用于固定 $J=60$ 的稳定性对照。网格加密时应改用不含扰动的 u_{ref}，以确保所有空间网格逼近同一个连续问题。

以下代码完整实现非零边界的 FTCS 更新，绘制误差增长、A/C 末态双剖面和网格收敛图，并打印结果。只需 NumPy、Matplotlib 与 Python 标准库，不使用现成 PDE 求解器：

```
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt

L, kappa, T = 1.2, 0.08, 0.45
left, right = 20.0, 10.0
epsilon = 1e-10
outdir = Path("outputs_3_2_exercise2")
outdir.mkdir(exist_ok=True)

def steady(x):
    return left + (right-left)*x/L
```

```
def reference(x, t, J, noise=0.0):
    u = (
        steady(x)
        + 5.0*np.sin(np.pi*x/L)*np.exp(-kappa*(np.pi/L)**2*t)
        + 2.0*np.sin(2*np.pi*x/L)*np.exp(-kappa*(2*np.pi/L)**2*t)
        + noise*np.sin((J-1)*np.pi*x/L)
          * np.exp(-kappa*((J-1)*np.pi/L)**2*t)
    )
    u[0], u[-1] = left, right
    return u

def ftcs(J, r, noise=0.0):
    dx = L/J
    dt = r*dx**2/kappa
    N = int(round(T/dt))
    if not np.isclose(N*dt, T, atol=1e-12, rtol=0):
        raise ValueError("T must be an integer multiple of dt")
    x = np.linspace(0.0, L, J+1)
    u = reference(x, 0.0, J, noise).copy()
    t_values, errors, deviations = [], [], []
    for n in range(N+1):
        t = n*dt
        t_values.append(t)
        errors.append(float(np.max(np.abs(
            u[1:-1] - reference(x, t, J, noise)[1:-1]
        ))))
        deviations.append(float(np.max(np.abs(u-steady(x)))))
        if n < N:
            v = u.copy()
            v[1:-1] = u[1:-1] + r*(u[:-2]-2*u[1:-1]+u[2:])
            v[0], v[-1] = left, right
            u = v
    ref = reference(x, T, J, noise)
    diff = u[1:-1]-ref[1:-1]
    return {
        "J": J, "r": r, "dx": dx, "dt": dt, "N": N,
        "x": x, "U": u, "ref": ref,
        "times": np.asarray(t_values),
        "errors": np.asarray(errors),
        "deviations": np.asarray(deviations),
        "Einf": float(np.max(np.abs(diff))),
        "Erms": float(np.sqrt(np.mean(diff**2))),
    }

cases = {
    "A": ftcs(60, 0.375, noise=epsilon),
    "B": ftcs(60, 0.5, noise=epsilon),  # 临界情况，供复核
    "C": ftcs(60, 0.75, noise=epsilon),
}
refined = [ftcs(J, 0.375) for J in (30, 60, 120, 240)]

for name, v in cases.items():
    print(name, v["r"], v["dt"], v["N"],
          v["Einf"], v["Erms"])

old_error = None
for v in refined:
    order = (None if old_error is None else
             np.log2(old_error/v["Einf"]))
    print(v["J"], v["dx"], v["dt"], v["N"], v["Einf"], order)
```

```
    old_error = v["Einf"]

## 图1：误差随时间变化（对数纵轴）
fig, ax = plt.subplots(figsize=(8, 4.5))
for tag in ("A", "C"):
    v = cases[tag]
    ax.semilogy(v["times"], np.maximum(v["errors"], 1e-16),
                label=f"{tag}: r={v['r']}, dt={v['dt']:.6f} s")
ax.set(xlabel="Time (s)", ylabel="Maximum error (deg C)",
       title="Error growth: J=60, dx=0.02 m, T=0.45 s")
ax.legend()
ax.grid(True, which="both", alpha=0.25)
fig.tight_layout()
fig.savefig(outdir/"exercise2_error_growth.png", dpi=160)
plt.close(fig)

## 图2：稳定/非稳定终点空间剖面；左右纵轴单独缩放
fig, axes = plt.subplots(1, 2, figsize=(12, 4.6), sharex=True)
for ax, tag in zip(axes, ("A", "C")):
    v = cases[tag]
    ax.plot(v["x"], v["ref"], "--", label="Exact solution")
    ax.plot(v["x"], v["U"], "o-", ms=3, label="FTCS")
    ax.set(xlabel="Position x (m)",
           ylabel="Temperature (deg C)",
           title=f"Case {tag}, r={v['r']}, dt={v['dt']:.6f} s")
    ax.legend()
    ax.grid(True, alpha=0.25)
fig.suptitle("Spatial profiles: J=60, dx=0.02 m, T=0.45 s (different y scales)")
fig.tight_layout()
fig.savefig(outdir/"exercise2_profiles.png", dpi=160)
plt.close(fig)

## 图3：固定 r 的联合网格收敛
hs = np.array([v["dx"] for v in refined])
Es = np.array([v["Einf"] for v in refined])
fig, ax = plt.subplots(figsize=(8, 4.5))
ax.loglog(hs, Es, "o-", label="Measured E_inf")
ax.loglog(hs, Es[0]*(hs/hs[0])**2, "--", label="Reference O(dx^2)")
ax.set(xlabel="Spatial step dx (m)",
       ylabel="Maximum error at T=0.45 s (deg C)",
       title="Grid refinement, r=0.375")
ax.legend()
ax.grid(True, which="both", alpha=0.25)
fig.tight_layout()
fig.savefig(outdir/"exercise2_refinement.png", dpi=160)
plt.close(fig)
```

如需追踪单个高频模态的增长，可将 $U_j^n - u_s(x_j)$ 投影到 $\sin(59\pi j/60)$ 上。该步骤是对实验机制的进一步验证，而不是取代全空间误差的计算。 代码实际运行输出（原始控制台记录）： 前三行为方案 A、B、C 的 r、dt、N、E_inf、E_rms；后四行为网格数 J、dx、dt、N、E_inf 及观测收敛阶。首行收敛阶为 None，表示没有更粗网格可供比较。

```
A 0.375 0.0018750000000000001 240 0.001041609223864981 0.0006314067206144387
B 0.5 0.0025 180 0.0016675198232114496 0.0010108664074988402
C 0.75 0.0037500000000000003 120 1.1749052918673314e+26 8.377949347293786e+25
30 0.04 0.007500000000000001 60 0.0041814391357135605 None
60 0.02 0.0018750000000000001 240 0.001041609223864981 2.005185434195741
```

```
120 0.01 0.00046875000000000004 960 0.00026016933572847734 2.0012912921189745
240 0.005 0.00011718750000000001 3840 6.502779562111982e-05 2.0003225083167253
```

运行状态：已运行。

三种时间步长的终点结果。（固定 $J=60$，$T=0.45\,\mathrm{s}$，均加入相同高频扰动 $\varepsilon=10^{-10}\,^\circ\mathrm{C}$）：

方案	r	更新步数	终点最大误差 E_∞（°C）	均方根误差 E_{RMS}（°C）
A	0.375	240	$\mathbf{1.04160922\times10^{-3}}$	$\mathbf{6.31406721\times10^{-4}}$
B	0.500	180	$\mathbf{1.66751982\times10^{-3}}$	$\mathbf{1.01086641\times10^{-3}}$
C	0.750	120	$\mathbf{1.17490529\times10^{26}}$	$\mathbf{8.37794935\times10^{25}}$

方案 A、B 在本终止时间内稳定，但都有非零误差；方案 C 则发生数值爆炸，数值温度同时出现巨大正负值。这些巨幅温度不是实际金属导热现象，而是高频离散模态不稳定的结果。

误差随时间的演化，比较题目要求的 A 与 C：

时间 t（s）	方案 A：E_∞（°C）	方案 C：E_∞（°C）
0.00	$\mathbf{0}$	$\mathbf{0}$
0.15	$\mathbf{6.19302227\times10^{-4}}$	$\mathbf{1.05519683\times10^{2}}$
0.30	$\mathbf{9.23609725\times10^{-4}}$	$\mathbf{1.11344204\times10^{14}}$
0.45	$\mathbf{1.04160922\times10^{-3}}$	$\mathbf{1.17490529\times10^{26}}$

画出双对数（或仅纵轴对数）的误差增长图后，方案 A 的误差始终较小，而 C 的误差快速跨越许多数量级。空间剖面对比图应同时绘制每个方案的数值解与其解析解：A 的数值曲线与解析曲线近乎重合；C 则出现相邻节点正负交替且振幅极大的振荡。两组必须使用不同的纵轴范围才能把这一差异解释清楚。

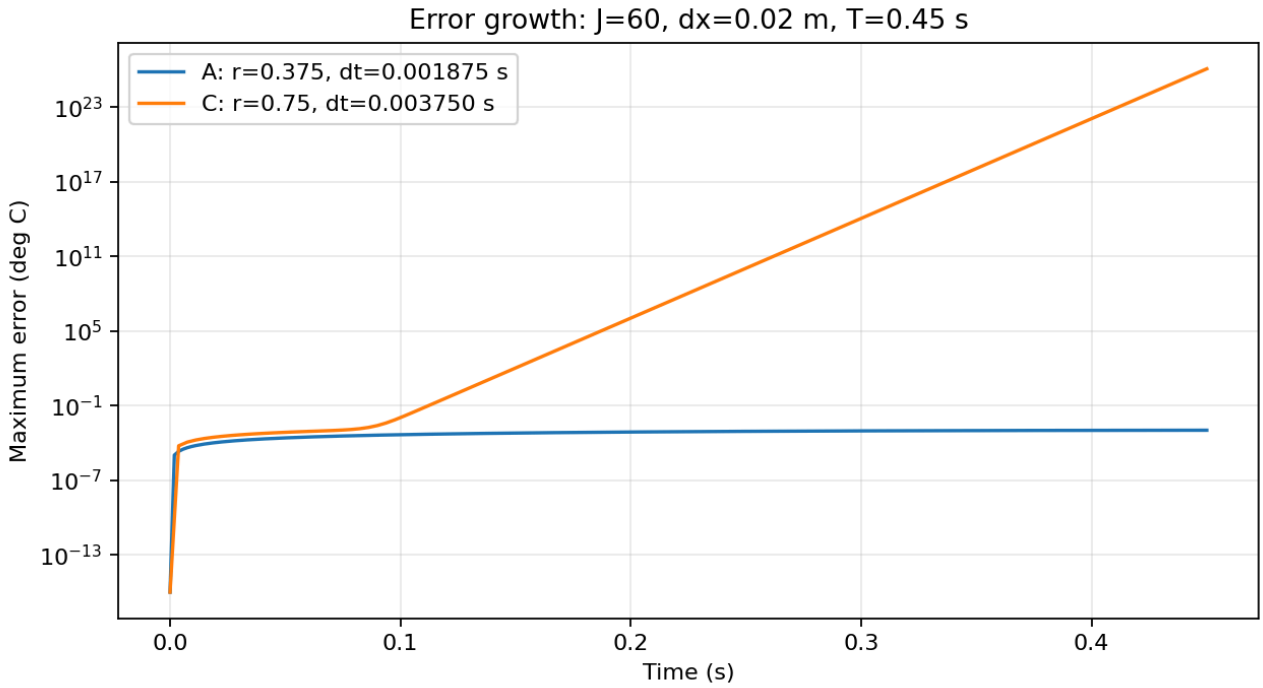


图 题2-1　两种时间步长下的最大节点误差—时间曲线。横轴为时间（s），纵轴为最大节点误差（° C，对数刻度）；$J = 60$，$\Delta x = 0.02\,\mathrm{m}$，$T = 0.45\,\mathrm{s}$。

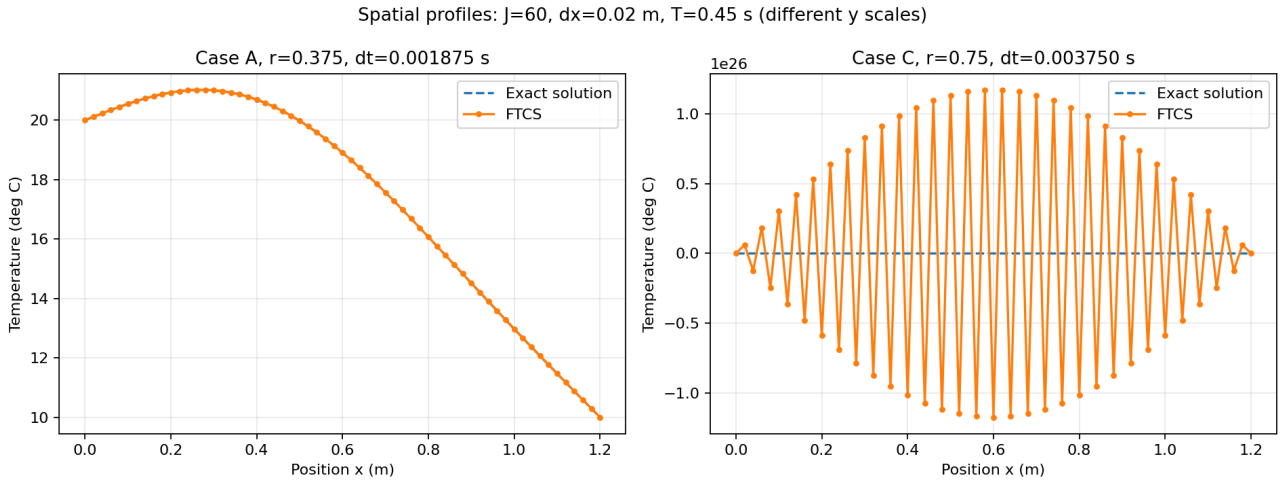


图 题2-2　$t = 0.45\,\mathrm{s}$ 时的稳定组与不稳定组空间温度剖面。左图为方案 A（$r = 0.375$），右图为方案 C（$r = 0.75$）；两组均与各自解析解比较。横轴为位置（m），纵轴为温度（° C），左右子图使用独立纵轴刻度。

高频模态放大的定量依据。 对固定 $J = 60$、齐次偏差 $V_j^n = U_j^n - u_s(x_j)$，离散模态 $\sin(m\pi j/J)$ 每步乘以

$$G_m(r) = 1 - 4r\sin^2\left(\frac{m\pi}{2J}\right).$$

其中最高阶模态 $m = 59$ 的放大因子为

$$G_{59}(r) = 1 - 4r\cos^2\left(\frac{\pi}{120}\right).$$

分别有

$$G_{59}(0.375) \approx -0.498972,$$
$$G_{59}(0.500) \approx -0.998630,$$
$$G_{59}(0.750) \approx -1.997944.$$

因此，虽然 B 符合标准稳定条件，但它的最高频模态阻尼较弱；C 中的高频微扰在 120 步之后理论上达到的振幅约为

$$10^{-10}|G_{59}(0.750)|^{120} \approx 1.174912 \times 10^{26}\ ^\circ\mathrm{C},$$

与实际计算误差的数量级完全吻合。因为 $G_{59} < 0$，高频分量每推进一步就变号；因为 $|G_{59}| > 1$，其幅值指数增长。

（6）网格加密与误差收敛

固定 $r = 0.375$ 的网格加密结果。 恢复无扰动初值，比较同一个连续问题的数值近似：

J	Δx (m)	Δt (s)	更新步数	终点最大误差（°C）	观测阶 p
30	0.040000	0.007500000	60	$4.18143914 \times 10^{-3}$	—
60	0.020000	0.001875000	240	$1.04160922 \times 10^{-3}$	2.0052
120	0.010000	0.000468750	960	$2.60169336 \times 10^{-4}$	2.0013
240	0.005000	0.0001171875	3840	$6.50277956 \times 10^{-5}$	2.0003

观测阶按 $p = \log_2(E_\infty(\Delta x)/E_\infty(\Delta x/2))$ 计算。每次将 Δx 减半，Δt 随之减少为四分之一，终点误差大致减少为原来的四分之一，符合理论估计 $\mathcal{O}((\Delta x)^2)$。注意这仍是时间与空间同时变化的联合网格收敛实验。

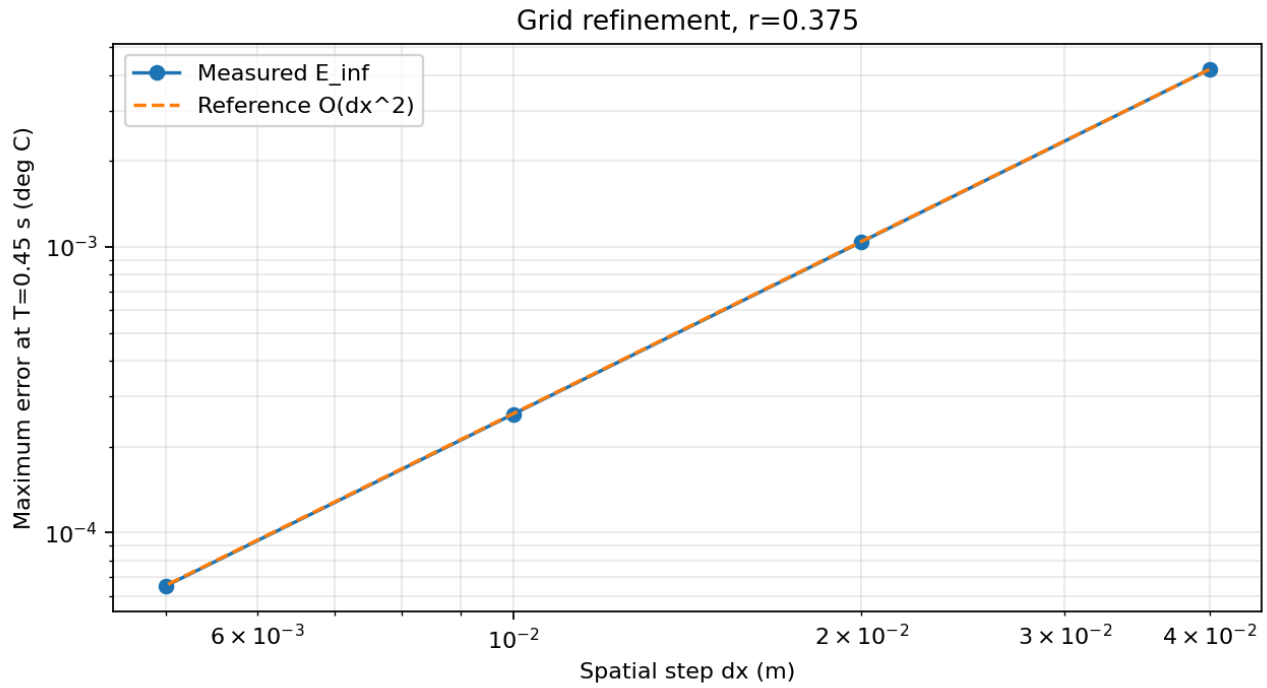


图 题2-3　保持 $r = 0.375$ 时最大节点误差与空间网格间距的双对数关系。横轴为 Δx（m），纵轴为 $T = 0.45\,\mathrm{s}$ 时的最大误差（° C）；实测曲线与二阶参考曲线接近。

（7）结果解释与教学验收

综合上述计算，方案 A 满足标准 CFL 条件，并在所给网格上具有较小的终点误差；方案 B 处于标准稳定性条件的边界，其高频模态虽不放大，但不代表误差为零；方案 C 违反稳定性条件，在高频扰动作用下发生显著数值振荡。这些现象分别说明：满足 CFL 条件是本显式格式的稳定性保障，误差大小需要借助独立解析解判断，而网格收敛趋势需要通过一系列加密计算验证。

仅凭某个低频初值的一次运行没有出现爆炸，不足以证明差分格式对所有误差模态稳定。

教学验收容差。 对采用相同参数、双精度浮点及题给扰动的实现，可使用以下分层标准：A 的终点 E_∞ 接近 $1.04\times10^{-3}\,^\circ\mathrm{C}$（约 $\pm5\%$）；B 接近 $1.67\times10^{-3}\,^\circ\mathrm{C}$（约 $\pm5\%$）；C 至少观察到明显高频振荡与迅速增长，其终点误差应达到 $10^{25}\sim10^{27}\,^\circ\mathrm{C}$ 的量级，允许对浮点细节更宽松的数值差异。网格实验的相邻观测阶在 $[1.8,2.2]$ 内可以视为支持二阶联合收敛。以上仅是本题教学验收尺度，不构成其他网格或 PDE 的普适误差界。

（8）CFL 条件的适用范围与错误辨析

以下依次辨析习题第（8）问中的错误判断。

错误一：“非零边界就不需要 CFL 条件。” 边界温度是否为零，不决定 FTCS 的时间步长能否任意选。这里的非零边界是两个固定常数，减去线性稳态 $u_s(x)$ 后，就得到零边界的齐次热方程；离散稳态 $u_s(x_j)$ 的中心二阶差分也恰好为零，所以偏差 V_j^n 满足相同的 FTCS 递推。相应的标准 $r\le1/2$ 稳定性条件依然成立。

错误二：“空间网格变细、时间步长不变会自动更稳定。” 本格式的 $r=\kappa\Delta t/(\Delta x)^2$。若把空间步长减半、$\Delta t$ 保持不变，则 r 会增大到原来的 4 倍。例如方案 A 原为 $r=0.375$，只将空间间距减半而保留 $\Delta t=0.001875\,\mathrm{s}$，则 $r'=1.5$，显然违反标准条件。细化空间网格并不自动带来数值稳定，显式扩散计算通常必须同步减小时间步长。

错误三：“$r\le1/2$ 是所有 PDE 的通用 CFL 公式。” 该界限针对一维常系数扩散方程在均匀网格上的二阶中心空间差分＋前向 Euler 时间推进。改变时间方法，例如把同一热方程改为后向 Euler，模态放大因子变为

$$G_m^{\mathrm{BE}}=\frac{1}{1+4r\sin^2(m\pi/(2J))},$$

对 $r\ge0$ 始终不超过 1（在这里的线性模态稳定性意义下无条件稳定）；改变 PDE 为一维平流方程 $u_t+au_x=0$ 并用 $a>0$ 的前向时间、一阶迎风空间差分，标准 CFL 条件则是 $0\le a\Delta t/\Delta x\le1$。因此不能离开方程类型和具体格式谈通用 CFL 公式。

错误四：“稳定就说明解精确。” 稳定性控制误差在递推中的增长；精度还要看截断误差、边界实现、空间步长、时间步长与问题光滑性。本题方案 A 和 B 都满足标准稳定条件，却有不同的非零终点误差；只有结合解析解与网格加密才能判断是否达到精度要求。

错误五：“单次运行方案 C 没看到爆炸就能认定稳定。” 对单个低频模态，方案 C 的放大因子 $G_1(0.75)=1-4(0.75)\sin^2(\pi/120)$ 仍处于 $(-1,1)$，所以低频解有可能在短期内衰减；然而最高频 $m=59$ 模态的放大因子约为 -1.997944，其模长远大于 1。稳定性必须对所有允许的误差模态进行检查。主动加入小幅高频扰动是为了让这一潜在不稳定更容易被观察到，不是改变热方程的稳定性判据。

更精细的限定： 对固定有限 J 且两端 Dirichlet 的离散热方程，谱稳定性的精确上界为

$$0 \le r \le \frac{1}{2\cos^2(\pi/(2J))}.$$

本题 $J=60$ 时，该界限约为 0.500343，略高于标准的 0.5；当空间网格细化时，它趋近于 0.5。因此不应把“任何固定有限网格中，刚超过 0.5 就一定增长”写成严格命题。不过方案 C 的 $r=0.75$ 已明确超出两种界限，实验中高频失稳的判断不受这一细节影响。

总评： 先确定初边值问题，再明确差分格式；用 CFL 判据检查稳定性，最后通过独立解析解与网格收敛检验精度。 这才是本题与教材 3.2 节的共同学习目标。

第03章 离散的世界：习题参考解答模板

题3：对应3.3 有限元：弱形式与Galerkin方法

习题 3.3-1

（1）设 V_h 为满足齐次 Dirichlet 条件的连续分片线性空间。对任意 $v_h \in V_h$，由于 v_h' 在每个单元 $K_e=[x_{e-1},x_e]$ 上为常数，

$$\int_{K_e} (u-I_hu)'v_h' \,\mathrm{d}x = v_h'|_{K_e}\big[(u-I_hu)(x_e)-(u-I_hu)(x_{e-1})\big]=0.$$

各单元求和得

$$\int_0^1 (u-I_hu)'v_h' \,\mathrm{d}x = 0.$$

结合精确弱形式可知，I_hu 满足 Galerkin 离散方程。由离散解的唯一性，有

$$u_h = I_hu.$$

（2）由插值函数定义，$U_i=u_h(x_i)=u(x_i)$，故精确积分时节点误差为零。但 $u(x)=\sin(\pi x)$ 在单元内部不是一次函数，因而 $u-u_h$ 一般不为零，其 L^2 误差与 H^1 半范数误差也不为零。

（3）根据正文表 3.3-2，L^2 误差的经验阶约为 2，网格尺寸减半后误差约变为原来的 $1/4$；H^1 半范数误差的经验阶约为 1，减半后误差约变为原来的 $1/2$。

习题 3.3-2

（1）令单元中点 $m_e=(x_{e-1}+x_e)/2$。一点 Gauss–Legendre 求积给出

$$F^{(e)} \approx \frac{h_e}{2} f(m_e)\begin{bmatrix}1\\1\end{bmatrix}.$$

刚度矩阵仍为

$$K^{(e)}=\frac{1}{h_e}\begin{bmatrix}1 & -1\\ -1 & 1\end{bmatrix}.$$

（2）采用相同的均匀网格与载荷函数 $f(x)=\pi^2\sin(\pi x)$，计算结果如下。表内数字为示例程序使用双精度浮点数所得结果，最后几位会随计算环境略有差异。

N	一点求积 E_∞	四点求积 E_∞	一点求积经验阶
8	6.512818×10^{-3}	2.242206×10^{-12}	—
16	1.611809×10^{-3}	8.659740×10^{-15}	2.0146
32	4.019341×10^{-4}	3.108624×10^{-15}	2.0037
64	1.004200×10^{-4}	9.103829×10^{-15}	2.0009

（3）一点求积产生有规律的、明显高于舍入误差的节点误差。其经验阶

$$p_h=\frac{\log(E_h/E_{h/2})}{\log 2}$$

逐渐接近 2。相反，四点求积下的节点误差很快接近浮点舍入量级，此时误差比受求积残差和舍入影响，不能据此判断可信的渐近收敛阶。

还可以解析验证上述结果：对偶数 N，令 $t=\pi h/2$，一点求积所得内部节点值具有形式

$$U_i=A_h\sin(\pi x_i),\qquad A_h=\frac{t^2\cos t}{\sin^2 t}.$$

因 $A_h=1-\frac{t^2}{6}+O(t^4)$，故节点最大误差为 $O(h^2)$。

习题 3.3-3

（1）试探函数满足 $u(0)=0,\,u(1)=\beta$，而检验函数须满足 $v(0)=v(1)=0$。记

$$V_\beta=\{w\in H^1(0,1):w(0)=0,\,w(1)=\beta\},\quad V_0=H_0^1(0,1).$$

弱形式为：求 $u\in V_\beta$，使

$$\int_0^1 u'v'\,\mathrm{d}x=\int_0^1 fv\,\mathrm{d}x,\quad \forall v\in V_0.$$

右端边界值非零并不影响齐次检验空间的选取。

（2）将全部节点自由度按内部和边界重排，内部节点方程为

$$K_{II}^{\mathrm{all}}U_I+K_{IB}^{\mathrm{all}}U_B=F_I^{\mathrm{all}}.$$

故

$$K_{II}^{\mathrm{all}}U_I=F_I^{\mathrm{all}}-K_{IB}^{\mathrm{all}}U_B.$$

（3）若解方程时错将 U_B 设为零，就遗漏了右端修正项 $-K_{IB}^{\mathrm{all}}U_B$，从而一般会得到错误的内部节点值。正确做法是在求解内部自由度以前引入所有已知边界值。

习题 3.3-4

（1）设

$$V_g=\{w\in H^1(0,1):w(0)=0,\,w(1)=\tfrac12\},\qquad V_0=H_0^1(0,1).$$

对 $-u''=1$ 乘以 $v\in V_0$，积分分部可得

$$\int_0^1 u'v'\,\mathrm{d}x-[u'v]_0^1=\int_0^1 v\,\mathrm{d}x.$$

由于 检验函数 在两端取零，边界项消失。因此弱形式为

$$\int_0^1 u'v'\,\mathrm{d}x = \int_0^1 v\,\mathrm{d}x, \quad \forall v \in V_0.$$

（2）对于 $K_e = [x_{e-1}, x_e]$，$h_e = x_e - x_{e-1}$，取

$$N_1(x) = \frac{x_e - x}{h_e}, \qquad N_2(x) = \frac{x - x_{e-1}}{h_e}.$$

计算可得

$$K^{(e)} = \frac{1}{h_e}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \qquad F^{(e)} = \frac{h_e}{2}\begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

因两单元长度均为 $\frac{1}{2}$，

$$K^{(1)} = K^{(2)} = \begin{bmatrix} 2 & -2 \\ -2 & 2 \end{bmatrix}, \quad F^{(1)} = F^{(2)} = \begin{bmatrix} \frac{1}{4} \\ \frac{1}{4} \end{bmatrix}.$$

（3）单元 K_1 的局部节点对应全局节点 $(0,1)$，单元 K_2 对应 $(1,2)$。在公共节点累加贡献，得到

$$K^{\text{all}} = \begin{bmatrix} 2 & -2 & 0 \\ -2 & 4 & -2 \\ 0 & -2 & 2 \end{bmatrix}, \quad F^{\text{all}} = \begin{bmatrix} \frac{1}{4} \\ \frac{1}{2} \\ \frac{1}{4} \end{bmatrix}.$$

已知 $U_0 = 0$、$U_2 = \frac{1}{2}$，内部节点方程为

$$-2U_0 + 4U_1 - 2U_2 = \frac{1}{2}, \quad \Longrightarrow \quad 4U_1 = \frac{3}{2}, U_1 = \frac{3}{8}.$$

因此

$$u_h(x) = \begin{cases} \dfrac{3}{4}x, & 0 \le x \le \frac{1}{2}, \\ \dfrac{1}{4}x + \dfrac{1}{4}, & \frac{1}{2} \le x \le 1. \end{cases}$$

（4）精确解为

$$u(x) = x - \frac{x^2}{2}.$$

中点处 $u(\frac{1}{2}) = u_h(\frac{1}{2}) = \frac{3}{8}$；而

$$u(\tfrac{1}{4}) = \frac{7}{32}, \qquad u_h(\tfrac{1}{4}) = \frac{3}{16} = \frac{6}{32},$$

故 $x = \frac{1}{4}$ 处绝对误差为 $\frac{1}{32}$。节点正确并不意味着单元内部的折线与精确抛物线重合。

（5）由精确积分可知，本常系数一维问题中的有限元解是精确解的分片线性节点插值。对任一长度为 h 的单元 $[a,b]$，有

$$u(x) - u_h(x) = \frac{(x-a)(b-x)}{2}.$$

因此在均匀网格上

$$E_{L^2}^2 = N\int_0^h \left[\frac{t(h-t)}{2}\right]^2 \mathrm{d}t = \frac{h^4}{120},$$
$$E_{H^1}^2 = N\int_0^h \left(\frac{h}{2} - t\right)^2 \mathrm{d}t = \frac{h^2}{12}.$$

即

$$E_{L^2}=\frac{h^2}{\sqrt{120}},\qquad E_{H^1}=\frac{h}{\sqrt{12}}.$$

数值核验结果为

N	h	E_∞（理论值）	E_{L^2}	E_{H^1}	p_{L^2}	p_{H^1}
2	0.50000	0	0.022821773	0.144337567	—	—
4	0.25000	0	0.005705443	0.072168784	2	1
8	0.12500	0	0.001426361	0.036084392	2	1
16	0.06250	0	0.000356590	0.018042196	2	1

程序使用双精度数值计算时，节点误差可能是约 10^{-16} 的舍入值，而非严格的零。由于这一误差太小，不宜用它计算经验收敛阶。内部方程的无穷范数残差应接近数值舍入误差；两单元例题的参考程序结果为 0。

（6）① 错误。边界项消失是因为检验函数在两端取零，与未知解在左端的数值无直接关系。② 错误。应在求内部节点值前引入非零边界值，并修正方程右端。③ 错误。有限差分近似微分算子，Galerkin 有限元从弱形式与基函数积分出发，矩阵结构相似不等于原理相同。④ 错误。此题 $x=\frac{1}{4}$ 处仍有非零误差，L^2 与 H^1 半范数误差也非零。

题4：对应3.4 AI关联：Neural ODE与PINN

- 条件与符号： 【待填写】。
- 手动推导或概念解释： 【待填写】关键步骤与依据。
- 实现位置与独立参考： 【待填写】Notebook节号/代码单元及参考来源。
- 实际输出、误差和容差理由： 【待填写】，未运行写未运行。
- 陷阱的正确解释及适用范围： 【待填写】。
- B核验、C复核及版本： 待核验。

第04章 矩阵的解剖

> 编写组：A2（G18）；来源版本：`230873df074c`，main已合并稿。本文件按原稿收录正文、独立习题与参考解答，保留原稿待完成项。

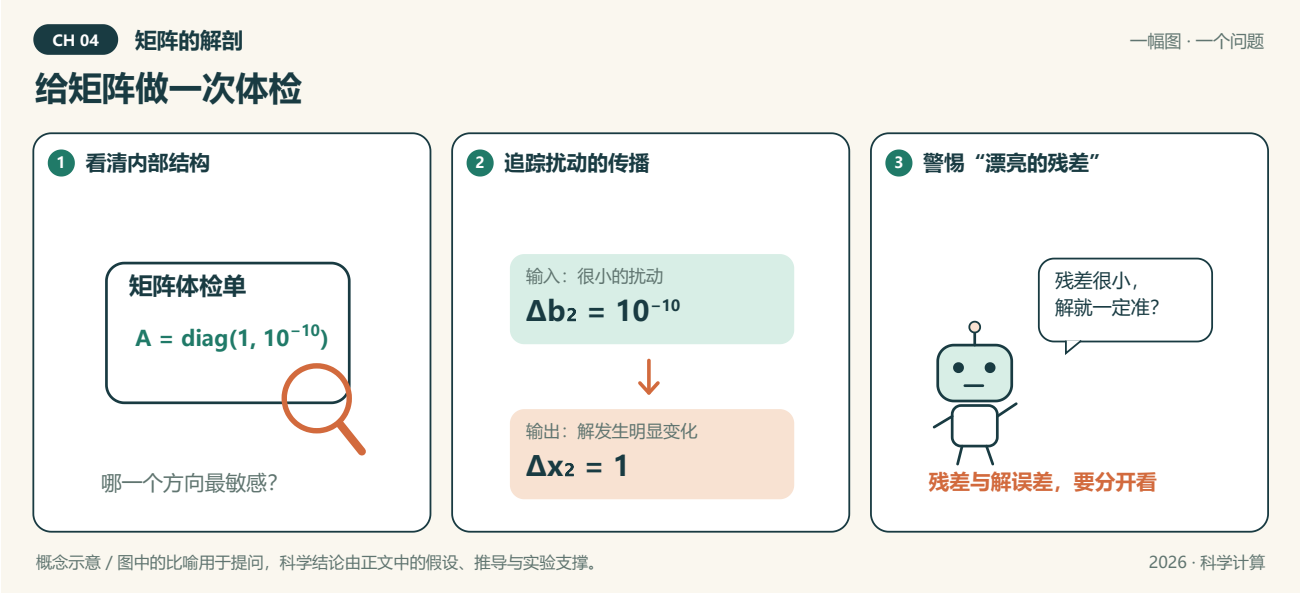


图04-1 • 给矩阵做一次体检。初始化概念示意；适用条件见下文，不作为实测结果。

先看一幕：给矩阵做一次体检

求解器递来一份漂亮的报告：“残差几乎为零。”矩阵医生却拿起放大镜：如果输入轻轻动一下，解会不会跳很远？

先猜，再验证： 设 $A = \mathrm{diag}(1, 10^{-10})$。仅把右端第二个分量增加 10^{-10}，解的第二个分量会改变多少？这与求解器报告的小残差是否矛盾？

把故事翻译成数学： 在这个对角例子中，解的第二分量增加1，二范数条件数为 10^{10}。图中体检对应对矩阵结构与扰动敏感性的分析；小残差和小解误差是不同主张，还要说明残差针对哪个右端计算。

知识主题： 线性方程组、特征值与SVD

本章核心问题： Ax=b如何高效求解？矩阵的本质结构是什么？

只聚焦三个核心概念： LU、CG、SVD

先修知识： 矩阵运算、内积、正定性与向量/矩阵范数

课时： 4节 × 2学时 = 8学时；原八章共64学时；第09章另列8学时。

本章四节安排

节	内容	学时
4.1	直接法：LU与Cholesky	2
4.2	迭代法：CG与预条件	2
4.3	特征值与SVD	2
4.4	AI关联：可微求解器、PCA、LoRA	2

先修概念与符号约定

先修概念

- 线性方程组：$Ax=b$，其中 $A\in\mathbb{R}^{n\times n}$，$x,b\in\mathbb{R}^n$。
- 矩阵运算：转置 A^T、逆 A^{-1}、行列式 $\det(A)$、迹 $\mathrm{tr}(A)$。
- 内积与范数：$\langle x,y\rangle=x^Ty$；向量 2-范数 $\|x\|_2=\sqrt{x^Tx}$；矩阵范数 $\|A\|_2=\sigma_{\max}(A)$，且满足相容性 $\|Ax\|\leqslant\|A\|\|x\|$。
- 正定性：$A=A^T$ 且对任意非零 x 有 $x^TAx>0$；等价于所有顺序主子式为正。
- 特征值与特征向量：$Av=\lambda v$（$v\neq0$）；对称矩阵可正交对角化 $A=V\Lambda V^T$。
- SVD：任意实矩阵 $A=U\Sigma V^T$，奇异值 $\sigma_1\geqslant\sigma_2\geqslant\cdots\geqslant0$。
- 隐函数定理：若 $F(x,\theta)=0$ 且 $\frac{\partial F}{\partial x}$ 可逆，则 $\frac{\partial x}{\partial\theta}=-(\frac{\partial F}{\partial x})^{-1}\frac{\partial F}{\partial\theta}$。

符号约定

符号	含义
$A\in\mathbb{R}^{n\times n}$	系数矩阵（默认方阵、非奇异，除非另述）
$x,b\in\mathbb{R}^n$	未知向量与右端向量
$r=b-Ax$	残差（$r_k=b-Ax_k$ 为第 k 步残差）
$\delta b,\delta x,\delta A$	右端、解、系数矩阵的微小扰动
$\|\cdot\|$	某种相容矩阵范数；默认 2-范数，需注明时写 $\|\cdot\|_2$
$\kappa(A),\mathrm{cond}(A)$	条件数，$\kappa(A)=\mathrm{cond}(A)=\|A^{-1}\|\,\|A\|$； 默认为 $\kappa_2(A)=\mathrm{cond}_2(A)=\frac{\sigma_{\max}}{\sigma_{\min}}$
σ_i	奇异值，按 $\sigma_1\geqslant\sigma_2\geqslant\cdots\geqslant0$ 排列
λ_i	特征值
$\rho(A)$	谱半径
$W_0,\Delta W$	预训练权重与微调增量

4.1 直接法：LU 与 Cholesky（2 学时）

本节目标： 能按矩阵条件选择分解，并解释主元和三角求解。

概念讲解：把问题说明白

线性方程组 $\boldsymbol{Ax=b}$ 是科学计算最基本的问题之一。尽管在学习线性代数时，我们就学习了手动计算的方法，但假如直接让机器按照手动计算的方法进行运算，会遇到一些现实问题：矩阵规模一旦变大，计算量增长会很快；数值的量级差距过大时，误差可能会被急剧放大。机器需要的不是“能算”，而是“算得稳、算得快、还能反复用”。那么，计算机在解决这类问题时，采用的会是什么方法呢？

一种思想是，想办法将矩阵通过等价变形，化简为容易求解的形式，比如将矩阵分解成若干个易于计算的矩阵。本节会讨论对于不同的系数矩阵提出的各种分解方案，并尝试说明“计算过程稳定”与“问题本身敏感”是两件不同的事。

4.1.1 从 Gauss 消元法到矩阵的 LU 分解

对我们来说，求解线性方程组 $\boldsymbol{Ax=b}$ 最基础的求解思想，便是通用的手动求解方法—— Gauss 消元法。Gauss 消元法的基本思路是：通过对矩阵 $\boldsymbol{A}$ 进行一系列的行变换处理，将矩阵变成一个上三角矩阵，即可从下到上求解每一个未知数。若要将这一思想迁移到机器运算，并设计一种算法的话，我们希望这种算法的流程更加固定、计算更加稳定、能尽量避免误差，还能尽可能地减小所需计算量的规模。

回顾 Gauss 消元法的过程：对 $\boldsymbol{A}$ 进行的一系列操作可以看作对矩阵的一系列线性变换，复合之后便得到一个能代表整个消元过程的线性变换 $\boldsymbol{T}$，满足 $\boldsymbol{U=TA}$ 为上三角矩阵，之后求 $\boldsymbol{Ux=Tb}$ 的解即可。对于上三角矩阵 $\boldsymbol{U}$，这一过程只需从最下面开始求解，然后逐次带入到上一个方程中即可得到方程的解。

注意到：操作过程保证了 $\boldsymbol{T}$ 其实是一个下三角矩阵，显然可逆。现在，假如我们进行一个简单的变形，就得到了 $\boldsymbol{A=T^{-1}U}$。换句话说，可以看作将 $\boldsymbol{A}$ 分解为一个下三角矩阵 $\boldsymbol{L=T^{-1}}$ 和一个上三角矩阵 $\boldsymbol{U}$ 的乘积：

$$\boxed{\boldsymbol{A=LU}} \tag{4.1}$$

上述从矩阵分解角度的思考带给我们很多启发，我们能进一步从中总结出直接求解线性方程组的基本概念与核心思想：

> 定义 4.1（直接法）
> 通过有限步算术运算直接求方程的解的方法，称为直接法。其核心思想是将系数矩阵 $\boldsymbol{A}$ 分解为若干个结构简单的矩阵的乘积（如下三角/上三角矩阵，或对角矩阵、正交矩阵等），从而把原方程化简为更易求解的形式。

最基础的分解即式 (4.1) 中的形式。下面我们给出这种分解的定义：

> 定义 4.2（LU 分解）
> 将 $\boldsymbol{A}$ 写成 $\boldsymbol{A=LU}$ 的形式，其中 $\boldsymbol{L}$ 为单位下三角矩阵（即对角元全为 $\boldsymbol{1}$，这一要求是为了保证分解唯一），$\boldsymbol{U}$ 为上三角矩阵。这种分解被称为 LU 分解。

进行 LU 分解后，可以把 $Ax=b$ 的求解过程分为两步：

1. 解 $Ly=b$，得到 y;
2. 解 $Ux=y$，得到 x。

事实上，Gauss 消元法与 LU 分解本质上是同一件事。消元过程通过行初等变换把 A 变形为上三角矩阵，从矩阵角度来看这个过程，其中每一步变换都能“翻译”为对系数矩阵 A 左乘一个下三角矩阵。具体地说，Gauss 消元法第 k 步相当于左乘一个单位下三角矩阵 L_k，这里

$$L_k=\begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -m_{k+1,k} & 1 & \\ & & \vdots & & \ddots \\ & & -m_{n,k} & & 1 \end{pmatrix},$$

其中 $m_{ik}=\frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}$（这里 $a_{ij}^{(k)}$ 表示第 k 步中的 a_{ij}）。消元与分解的关系可以画成下面这张推导图：

$$\begin{aligned} &A \xrightarrow{L_1} L_1A \xrightarrow{L_2} \cdots \xrightarrow{L_{n-1}} U \\ &A \xrightarrow{\text{反过来}} (L_{n-1}\cdots L_1)^{-1}U=L_1^{-1}\cdots L_{n-1}^{-1}U=LU \end{aligned}$$

也就是说，消元把 A 变成 U，而把每一步消元的“逆”按顺序乘起来，就得到 L。因此消元的最终结果为 $L_{n-1}\cdots L_2L_1A=U$，从而有

$$A=(L_{n-1}\cdots L_1)^{-1}U=L_1^{-1}L_2^{-1}\cdots L_{n-1}^{-1}U. \tag{4.2}$$

直接计算可得，L_k^{-1} 恰为 L_k 的非对角元取相反数，也都是单位下三角矩阵。故 $L=L_1^{-1}\cdots L_{n-1}^{-1}$ 也是单位下三角矩阵，且其第 i 行第 j 列（$i>j$）的元素恰为消元过程中使用的乘数 m_{ij}。因此式 (4.2) 本质上和 (4.1) 的分解形式相同，这样就说明了 Gauss 消元法与 LU 分解的等价关系。

直接三角分解法（Doolittle 分解）是 LU 分解的另一种实现方式。既然已知分解目标，那么便可以绕过 Gauss 消元法中间的计算步骤，直接比对 (4.1) 两边对应位置的元素，即可得递推公式：

$$\begin{cases} u_{1i}=a_{1i}, & i=1,2,\cdots,n, \\ l_{i1}=\dfrac{a_{i1}}{u_{11}}, & i=2,3,\cdots,n, \end{cases}$$

对 $r=2,3,\cdots,n$：

$$\begin{cases} u_{ri}=a_{ri}-\displaystyle\sum_{k=1}^{r-1}l_{rk}u_{ki}, & i=r,r+1,\cdots,n, \\ l_{ir}=\dfrac{1}{u_{rr}}\left(a_{ir}-\displaystyle\sum_{k=1}^{r-1}l_{ik}u_{kr}\right), & i=r+1,\cdots,n. \end{cases}$$

Doolittle 分解的计算顺序是“先 U 行、后 L 列”：第 r 步先用已得的 L 前 $r-1$ 列算 u_{ri}，再用 u_{rr} 算 l_{ir}。这样每一步只用已算出的元素，很适合按行编程。

下面我们来估计 Gauss 消元法和 LU 分解的运算复杂度：

Gauss 消元法先做前向消元，第 k 步约需进行 $(n-k)^2+(n-k)$ 次乘除法 $(k=1,2,\cdots,n-1)$，求和得 $\sum_{k=1}^{n-1}[(n-k)^2+(n-k)]=\frac{n^3-n}{3}=O(n^3)$，而回代求解过程中的计算次数约为 $\frac{n(n+1)}{2}=O(n^2)$，故总复杂度为 $O(n^3)$；

LU 分解中第 k 步约需 $(2n-2k+1)(k-1)$ 次乘除法，求和得 $\sum_{k=1}^{n}(2n-2k+1)(k-1)=\frac{n(n-1)(2n-1)}{6}=O(n^3)$，而分解后求解过程中还需要约 $n^2=O(n^2)$ 次运算，因此总复杂度也是 $O(n^3)$。

既然复杂度差别不大，和 Gauss 消元法相比，LU 分解是不是就没有什么计算层面的优势了呢？其实并不是这样的，因为对于同一个矩阵 A 来说，LU 分解完成后，假如只改变 b 再次求解，那么复杂度仅为 $O(n^2)$。因此，虽然 LU 分解在单次计算中的复杂度与 Gauss 消元法差别不大，但是在实际问题中，当需要求解多个具有相同系数矩阵 A 而 b 不相同的方程组时，矩阵分解的结果即可反复使用，从而在多次运算中大大节省计算的复杂度。

4.1.2 LU 分解的性质与改进

LU 分解的存在性依赖于 A 的顺序主子式：若 A 的所有顺序主子式均不为零，则存在唯一的单位下三角矩阵 L 和非奇异上三角矩阵 U，使得 $A=LU$. 下面我们用归纳法证明：

设 A 的所有顺序主子式均不为零。当 $n=1$ 时结论显然。假设结论对 $n-1$ 阶矩阵成立，将 A 分块为

$$A=\begin{pmatrix}A_{11} & A_{12}\\ A_{21} & A_{22}\end{pmatrix},$$

其中 A_{11} 为 $n-1$ 阶顺序主子矩阵。由归纳假设，$A_{11}=L_{11}U_{11}$。令

$$L_{21}=A_{21}U_{11}^{-1},\qquad U_{12}=L_{11}^{-1}A_{12},\qquad U_{22}=A_{22}-L_{21}U_{12},$$

则

$$A=\begin{pmatrix}L_{11} & 0\\ L_{21} & I\end{pmatrix}\begin{pmatrix}U_{11} & U_{12}\\ 0 & U_{22}\end{pmatrix}.$$

由 A 非奇异可知 U_{22} 非奇异，这就得到了矩阵 A 的 LU 分解。

而 LU 分解的唯一性也很容易说明：若 $A=LU=\bar{L}\bar{U}$，变形后得到 $L^{-1}\bar{L}=U\bar{U}^{-1}$。注意此时左边是单位下三角矩阵，右边是上三角矩阵，因此两边都只能是单位矩阵，从而 $L=\bar{L}$，$U=\bar{U}$。

理论上，只要顺序主子式都不为零，LU 分解就存在且唯一。而当某个顺序主子式为零时，消元过程就会出现零主元，使得分解无法进行（事实上，Gauss 消元法也会遇到类似的问题，因为 Gauss 消元法能顺利完成的充要条件就是 A 的所有顺序主子式都不为零）。

为了说明这一点，考虑一个简单但极端的例子：取 $A=\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix}$，尽管 A 非奇异，但 $a_{11}=0$，第一步消元就无法进行。

更隐蔽的情况是主元非零却又十分接近零的情况。当绝对值极小时，作除数会导致舍入误差被急剧放大。因此虽然此时消元过程可以继续，但计算结果将可能严重失真。

比方说，我们取三位有效数字计算

$$\begin{pmatrix} 0.001 & 2.000 \\ 1.000 & 3.000 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 2.001 \\ 4.000 \end{pmatrix}.$$

精确解为 $x_1 = 1$，$x_2 = 1$。但若用 0.001 作主元消元，乘数应为 1000，第二个方程的系数变为 $3.000 - 2000 = -1997$，可以看出舍入到三位有效数字后精度损失极大，三位有效数字根本存不下这个跨度，因此计算出的 x_1 可能完全错误。所以，主元并不是“不为零就行”，它的大小决定了误差会被放大多少倍。

为了避免放大舍入误差，改进方法是预先做行交换，把每一列中绝对值最大的元素交换到主元位置（对角线上），再按普通消元进行。第 k 步的流程可以写成：

选：第 k 列中 $|a_{ik}|$ 最大者
↓
换：与第 k 行交换，记入 P
↓
消：用 m_{ik} 消去第 k 列下方元素

三步循环 $k = 1, \dots, n-1$，就得到 $PA = LU$。

> 列主元法（部分选主元法）：消元前先做行交换，把每一列中绝对值最大的元素换到主元位置，再按普通消元进行；对应的矩阵形式为 $PA = LU$，其中 P 记录行交换。

列主元 LU 分解的矩阵形式为

$$\boxed{PA = LU} \tag{4.3}$$

其中 P 是置换矩阵，记录行交换的信息。L 仍然是单位下三角矩阵，但它的元素在交换行之后会相应改变，且满足 $|l_{ij}| \leqslant 1$。这一性质保证了消元过程中乘数的绝对值不超过 1，从而避免舍入误差的过度放大。对于一般非奇异矩阵，列主元 LU 分解总是可行的。

另外，在实际编程中，L 和 U 的元素可以直接存放在 A 的相应位置，不需要额外存储空间。(4.3) 对应的分解法大约需要 $\frac{n^3}{3}$ 次乘除法，与 Gauss 消元法的计算量量级相同。

需要注意的是，列主元 LU 分解必须明确置换约定：$PA = LU$ 中 P 左乘 A 表示行交换，而 $AP = LU$ 表示列交换，$P \neq P^T$ 时不能混用。每次行交换后，L 中已存好的对应行乘子也要同步交换，否则 $PA = LU$ 不成立。

4.1.3 对称正定矩阵的 Cholesky 分解

下面我们考虑一类特殊矩阵——对称正定矩阵。在最小二乘问题、偏微分方程的数值求解，以及函数的 Hessian 矩阵等实际问题中，都经常能见到这类矩阵的影子，因此针对这类矩阵进行专门研究也十分必要。

对称正定矩阵的分解比较特殊：若 A 对称且正定，进行 LU 分解后得到的上下三角矩阵也应该是互相对称的，因此可以仿照式 (4.1)，将分解写成

$$\boxed{A = LL^T} \tag{4.4}$$

式 (4.4) 就是对称正定矩阵的 Cholesky 分解，下面给出定义：

> 定义 4.3（Cholesky 分解）
> 设 A 对称正定。若 $A=LL^T$，其中 L 为对角元全为正的下三角矩阵，则称该分解为 Cholesky 分解。

定义中要求 L 对角元全为正，是为了保证分解唯一性。A 对称正定时，所有顺序主子式都为正，故上述 L 存在且唯一。因为这种分解就像给矩阵开平方：这里 L 相当于 A 的平方根，故也称 平方根法。

下面通过比较 (4.4) 两边的元素来推导 Cholesky 分解的递推公式：

设 $A=LL^T$，对 $j\leqslant i$，有

$$a_{ij}=\sum_{k=1}^{j}l_{ik}l_{jk}=\sum_{k=1}^{j-1}l_{ik}l_{jk}+l_{ij}l_{jj}.$$

当 $i=j$ 时，

$$a_{jj}=\sum_{k=1}^{j-1}l_{jk}^2+l_{jj}^2\quad\Longrightarrow\quad l_{jj}=\sqrt{a_{jj}-\sum_{k=1}^{j-1}l_{jk}^2}.$$

这里由 A 的正定性，$a_{jj}-\sum\limits_{k=1}^{j-1}l_{jk}^2>0$，故 l_{jj} 有定义，不会出现零主元。开平方时只取正平方根，保证 L 的对角元素全为正数。当 $i>j$ 时，

$$l_{ij}=\frac{1}{l_{jj}}\left(a_{ij}-\sum_{k=1}^{j-1}l_{ik}l_{jk}\right).$$

从计算过程中，我们还可以推导出 Cholesky 分解不需要选主元，因为对称正定矩阵的消元过程中不会出现小主元：由 $a_{jj}=\sum\limits_{k=1}^{j}l_{jk}^2$ 可得 $l_{jk}^2\leqslant a_{jj}\leqslant\max\limits_{1\leqslant j\leqslant n}\{a_{jj}\}$，因此 $\max\limits_{1\leqslant k\leqslant j\leqslant n}\{l_{jk}^2\}\leqslant\max\limits_{1\leqslant j\leqslant n}\{a_{jj}\}$。这说明分解过程中 l_{jk} 不会无界增长。因此，Cholesky 分解是数值稳定的方法。另外，也能发现定义中要求 L 的对角元素全为正数的必要性：开根号运算会得到正负两个平方根，因此不妨只取其中的正平方根。

与 LU 分解相比，Cholesky 分解的计算量大约减半（约 $\frac{n^3}{6}$ 次乘除法），存储量也减半（只需存储下三角矩阵 L）。它的代价是需要 n 次开平方运算。为避免开方，可使用改进的平方根法：

> LDL^T 分解（改进的平方根法）：将对称正定矩阵写成 $A=LDL^T$，其中 L 为单位下三角矩阵，D 为对角矩阵。它与 Cholesky 分解计算量相当，但不需要开平方。

$$\boxed{A=LDL^T}\tag{4.5}$$

其中 L 为单位下三角矩阵，D 为对角矩阵。按行计算 L 和 D 的公式为：$d_1=a_{11}$，对 $i=2,3,\cdots,n$，有：

$$\begin{cases}t_{ij}=a_{ij}-\sum\limits_{k=1}^{j-1}t_{ik}l_{jk}, & j=1,2,\cdots,i-1,\\ l_{ij}=\dfrac{t_{ij}}{d_j}, & j=1,2,\cdots,i-1,\\ d_i=a_{ii}-\sum\limits_{k=1}^{i-1}t_{ik}l_{ik}.\end{cases}$$

该方法与 Cholesky 分解的计算量差不多，但不再需要进行开平方计算。不过，其适用条件并不比 Cholesky 更宽，仍要求 A 对称正定（或对称且顺序主子式非零）；且不开方不等于可用于非正定矩阵，否则 D 的对角元可能为零或负数。

4.1.4 三对角矩阵的追赶法

前几节介绍的分解方法都是面向一般的稠密矩阵的方法，并没有针对某些稀疏矩阵的特殊结构进行专门的优化。下面以一种具有特殊结构的矩阵为例，介绍一种针对性强的简化的分解方案。

三对角矩阵是只有三条对角线上的元素非零的矩阵，它经常出现在常微分方程边值问题、三次样条插值和热传导方程的差分格式中。针对三对角矩阵设计的 LU 分解，乘除法运算量仅为 $O(n)$，而不是一般的 $O(n^3)$。

> 追赶法（Thomas 算法）：针对三对角矩阵设计的 LU 分解与三角求解方法，乘除法运算量仅 $O(n)$；分解为 $A=LU$，L 单位下双对角、U 上双对角，求解分“追”（前代）与“赶”（回代）两步。

下面设三对角矩阵为

$$A=\begin{pmatrix} b_1 & c_1 & & & \\ a_2 & b_2 & c_2 & & \\ & \ddots & \ddots & \ddots & \\ & & a_{n-1} & b_{n-1} & c_{n-1} \\ & & & a_n & b_n \end{pmatrix},$$

用角标对应行数，即下对角线为 $a_i\ (i=2,\cdots,n)$，主对角线为 $b_i\ (i=1,\cdots,n)$，上对角线为 $c_i\ (i=1,\cdots,n-1)$。

追赶法的矩阵分解形式仍为 $A=LU$，这里 L 是单位下双对角矩阵（次对角线为 l_i，对角元为 1），U 是上双对角矩阵（对角元为 u_i，上次对角线为 c_i）：

$$L=\begin{pmatrix} 1 & & & \\ l_2 & 1 & & \\ & \ddots & \ddots & \\ & & l_n & 1 \end{pmatrix},\qquad U=\begin{pmatrix} u_1 & c_1 & & \\ & u_2 & c_2 & \\ & & \ddots & \ddots \\ & & & u_n \end{pmatrix}.$$

这里 U 的上次对角线与 A 的上对角线 c_i 保持一致，这是追赶法能够简化的关键：消去过程中上对角线不变。

将 $A=LU$ 按元素展开，比较第一行得 $b_1=u_1, c_1=c_1$。对 $i=2,\cdots,n$，比较第 i 行的次对角元与主对角元，得

$$a_i=l_iu_{i-1},\qquad b_i=l_ic_{i-1}+u_i.$$

由此直接解出递推公式：

$$u_1=b_1,\qquad l_i=\frac{a_i}{u_{i-1}},\qquad u_i=b_i-l_ic_{i-1}\quad (i=2,\cdots,n).$$

计算顺序是：先由 b_1 得 u_1；然后对 $i=2,3,\cdots,n$ 依次算 $l_i=\frac{a_i}{u_{i-1}}, u_i=b_i-l_ic_{i-1}$。这里每步只用到前一步的 u_{i-1}，因此只需 $O(n)$ 次运算。

得到 L,U 后，求解 $Ax=d$ 分为“追”和“赶”两步：

追——前代求解 $Ly=d$，

$$y_1=d_1,\qquad y_i=d_i-l_iy_{i-1}\quad (i=2,\cdots,n).$$

赶——回代求解 $Ux=y$，

$$x_n=\frac{y_n}{u_n},\qquad x_i=\frac{y_i-c_ix_{i+1}}{u_i}\quad (i=n-1,\cdots,1).$$

前代自上而下、回代自下而上，故称追赶法。可以把“追”和“赶”当成两句话来记：

> 追——从上往下前代，$y_i=d_i-l_iy_{i-1}$；
> 赶——从下往上回代，$x_i=\frac{y_i-c_ix_{i+1}}{u_i}$。
> 一追一赶，整个求解只用 $5n-4$ 次乘除法、约 $3n$ 次加减法，复杂度 $O(n)$。

追赶法能够顺利进行的条件是三对角矩阵满足对角占优条件：

$$|b_1|>|c_1|,\qquad |b_i|\geqslant |a_i|+|c_i|\ (i=2,\cdots,n-1),\qquad |b_n|>|a_n|.$$

下面用归纳法证明：$u_i\neq 0(i=1,2,\cdots,n)$，且若设 $\beta_i=\frac{c_i}{u_i}$，则可证明 $|\beta_i|<1\ (i=1,\cdots,n-1)$，从而计算过程稳定。

$i=1$ 时，由 $|b_1|>|c_1|$ 及 $u_1=b_1$，得

$$|\beta_1|=\left|\frac{c_1}{u_1}\right|=\left|\frac{c_1}{b_1}\right|<1.$$

$i>1$ 时，若 $|\beta_{i-1}|<1$，则由 $u_i=b_i-l_ic_{i-1}=b_i-a_i\beta_{i-1}$（因为 $l_i=\frac{a_i}{u_{i-1}}$、$\beta_{i-1}=\frac{c_{i-1}}{u_{i-1}}$，故 $l_ic_{i-1}=a_i\beta_{i-1}$），有

$$|u_i|=|b_i-a_i\beta_{i-1}|\geqslant |b_i|-|a_i|\,|\beta_{i-1}|>|b_i|-|a_i|\geqslant |c_i|\geqslant 0.$$

其中第二个不等号用了 $|\beta_{i-1}|<1$，第三个不等号用了对角占优 $|b_i|\geqslant |a_i|+|c_i|$。于是 $u_i\neq 0$，且 $|\beta_i|=\left|\frac{c_i}{u_i}\right|<1$。

由归纳法知 $|\beta_i|<1$ 对 $i=1,\cdots,n-1$ 成立。

又对 $i=n$，由 $|b_n|>|a_n|$ 及 $|\beta_{n-1}|<1$ 得

$$|u_n|=|b_n-a_n\beta_{n-1}|\geqslant |b_n|-|a_n|\,|\beta_{n-1}|>|b_n|-|a_n|>0,$$

故 $u_n\neq 0$，上述结论成立。因此在回代过程中，误差不会被放大，追赶法是稳定的。

4.1.5 误差分析

直接法在不考虑舍入误差时给出的是精确解，且计算量基本固定。它的优势在于稳定、可靠，适用于中小规模稠密矩阵，也适用于某些具有特殊结构的大型稀疏矩阵。不过它的局限在于当矩阵规模很大，分解过程会产生大量“填充”（原本为零的位置在分解后变成非零），存储和计算代价都迅速增长。

当然，上面的讨论都建立在精确算术的前提下。实际计算中，浮点舍入误差不可避免，因此需要考虑一个更基本的问题：输入端的微小扰动，会在解的误差中被放大多少倍？ 直觉上，这取决于矩阵“把某些方向压得多扁”。压得越扁，反推回来时放大得越厉害。

我们通过计算来分析。设 A 非奇异，考虑方程 $Ax=b$，先只讨论右端扰动：假定 A 精确，右端有微小扰动 δb，实际求解的是扰动方程组 $A\tilde{x}=b+\delta b$。记真解为 $x=A^{-1}b$，解的扰动为 $\delta x=\tilde{x}-x$。两式相减得 $A\delta x=\delta b$，故

$$\delta x=A^{-1}\delta b.$$

两边取范数，得

$$\|\delta x\|\leqslant\|A^{-1}\|\,\|\delta b\|.$$

又由 $b=Ax$ 得 $\|b\|\leqslant\|A\|\,\|x\|$，即 $\dfrac{1}{\|x\|}\leqslant\dfrac{\|A\|}{\|b\|}$。两式相乘，得到相对误差上界。推导路线如下：

$$\frac{\|\delta x\|}{\|x\|}\leqslant\|A^{-1}\|\,\|A\|\,\frac{\|\delta b\|}{\|b\|}.$$

这里右端出现的因子记为

$$\boxed{\kappa(A)\triangleq\|A^{-1}\|\,\|A\|} \tag{4.6}$$

> 定义 4.4（条件数）
> 设 A 非奇异，$\|\cdot\|$ 为相容矩阵范数，称 $\kappa(A)\triangleq\|A^{-1}\|\,\|A\|$ 为矩阵 A 的条件数。它给出右端相对扰动对解相对误差的放大上界：$\dfrac{\|\delta x\|}{\|x\|}\leqslant\kappa(A)\dfrac{\|\delta b\|}{\|b\|}$。

上式说明，右端相对扰动对解的误差影响由条件数决定，放大倍数不超过 $\kappa(A)$：条件数越大，解对扰动越敏感，问题越病态。条件数是问题本身的属性，与算法无关。选主元、Cholesky、追赶法解决的是“算法不引入额外不稳定”的问题，但不能把病态问题变成良态问题。

本章默认条件数使用 2-范数：$\kappa_2(A)=\|A^{-1}\|_2\|A\|_2$，也称为谱条件数；后续若无特殊说明，条件数 $\kappa(A)$ 均指 $\kappa_2(A)$。关于条件数的更多性质，留到后续章节进一步介绍。

关键推导：让结论有依据

例 1：手算列主元 LU 的消元与回代，明确置换矩阵约定。取

$$A=\begin{pmatrix}0&2&1\\1&0&3\\2&1&1\end{pmatrix},\quad x^{*}=\begin{pmatrix}1\\-2\\3\end{pmatrix},\quad b=\begin{pmatrix}-1\\10\\3\end{pmatrix}.$$

约定置换矩阵 P 满足 $PA=LU$，即 P 左乘 A 表示对 A 做行交换；P 由单位矩阵经同样的行交换得到（列主元 LU 的形式见式 (4.3)）。

关键步骤

第一步：选主元并消元。 原位第一个主元 $a_{11}=0$，不能直接相除。第一列绝对值最大元素为 $a_{31}=2$，交换第 1、3 行，得

$$\begin{pmatrix} 2 & 1 & 1 \\ 1 & 0 & 3 \\ 0 & 2 & 1 \end{pmatrix}.$$

用乘子 $m_{21}=\frac{1}{2}$ 消去第二行首项：

$$\begin{pmatrix} 2 & 1 & 1 \\ 0 & -\frac{1}{2} & \frac{5}{2} \\ 0 & 2 & 1 \end{pmatrix}.$$

第二列主元为 $-\frac{1}{2}$，第三行第二列元素为 2，$|2|>|-\frac{1}{2}|$，需再次换行。

第二步：第二次换行并消元。 交换第 2、3 行，得

$$\begin{pmatrix} 2 & 1 & 1 \\ 0 & 2 & 1 \\ 0 & -\frac{1}{2} & \frac{5}{2} \end{pmatrix}.$$

用乘子 $m_{32}=-\frac{1}{4}$ 消去第三行第二列元素：

$$\begin{pmatrix} 2 & 1 & 1 \\ 0 & 2 & 1 \\ 0 & 0 & \frac{11}{4} \end{pmatrix}.$$

第三步：写出 P、L、U。 两次行交换对应

$$P=\begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix},\qquad U=\begin{pmatrix} 2 & 1 & 1 \\ 0 & 2 & 1 \\ 0 & 0 & \frac{11}{4} \end{pmatrix}.$$

L 的单位对角元为 1，非对角元为消元乘子；第二次换行发生在消元之后，L 中已存好的第 2、3 行乘子也要同步交换，得

$$L=\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ \frac{1}{2} & -\frac{1}{4} & 1 \end{pmatrix}.$$

第四步：核验 $PA=LU$。 直接计算

$$PA=\begin{pmatrix} 2 & 1 & 1 \\ 0 & 2 & 1 \\ 1 & 0 & 3 \end{pmatrix},$$

而

$$LU=\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ \frac{1}{2} & -\frac{1}{4} & 1 \end{pmatrix}\begin{pmatrix} 2 & 1 & 1 \\ 0 & 2 & 1 \\ 0 & 0 & \frac{11}{4} \end{pmatrix}=\begin{pmatrix} 2 & 1 & 1 \\ 0 & 2 & 1 \\ 1 & 0 & 3 \end{pmatrix}.$$

两边相等，验证 $PA=LU$ 成立。本题 $P\neq P^{T}$，因此 $PA=LU$ 与 $AP=LU$ 不同，必须按约定的左乘形式核验。

第五步：前代与回代。 原方程 $Ax=b$ 等价于 $LUx=Pb$。先算

$$Pb=\begin{pmatrix} 3 \\ -1 \\ 10 \end{pmatrix}.$$

解 $Ly=Pb$：

$$y_1 = 3, \qquad y_2 = -1, \qquad \frac{1}{2}y_1 - \frac{1}{4}y_2 + y_3 = 10,$$

得 $y = (3, -1, \frac{33}{4})^T$。再解 $Ux = y$：

$$\frac{11}{4}x_3 = \frac{33}{4} \;\Rightarrow\; x_3 = 3,$$

$$2x_2 + x_3 = -1 \;\Rightarrow\; x_2 = -2,$$

$$2x_1 + x_2 + x_3 = 3 \;\Rightarrow\; x_1 = 1.$$

结论

列主元 LU 分解 $PA = LU$ 成立，置换约定为“P 左乘 A 做行交换”；回代得 $x = (1, -2, 3)^T = x^*$，手算消元与回代完成。

条件不满足时的限制

本题 $a_{11} = 0$，一阶顺序主子式为零，无主元 LU 第一步即失败（无主元 LU 的存在性条件见 4.1.2 节）；列主元 LU 通过换行绕过，对一般非奇异矩阵总是可行。但列主元 LU 只保证乘数有界，不改善病态性。若 $P \neq P^T$，不能把 $PA = LU$ 与 $AP = LU$ 混用，否则核验不成立。

推导 2：检验 Cholesky 适用条件。

已知条件。 给定四个小矩阵

$$A_1 = \begin{pmatrix} 4 & 2 \\ 2 & 5 \end{pmatrix}, \quad A_2 = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}, \quad A_3 = \begin{pmatrix} 4 & 2 \\ 1 & 5 \end{pmatrix}, \quad A_4 = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}.$$

Cholesky 分解 $A = LL^T$（L 对角元为正）存在且唯一，当且仅当 A 对称正定；等价地，A 对称且所有顺序主子式为正。逐一对四个矩阵检验。

关键步骤。

对 A_1： 对称。顺序主子式 $\Delta_1 = 4 > 0$，$\Delta_2 = 4 \cdot 5 - 2 \cdot 2 = 16 > 0$，正定，可 Cholesky。按递推公式计算：

$$l_{11} = \sqrt{a_{11}} = \sqrt{4} = 2,$$

$$l_{21} = \frac{a_{21}}{l_{11}} = \frac{2}{2} = 1,$$

$$l_{22} = \sqrt{a_{22} - l_{21}^2} = \sqrt{5 - 1} = 2.$$

得

$$L = \begin{pmatrix} 2 & 0 \\ 1 & 2 \end{pmatrix}, \qquad LL^T = \begin{pmatrix} 2 & 0 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 4 & 2 \\ 2 & 5 \end{pmatrix} = A_1.$$

对 A_2： 对称。$\Delta_1 = 1 > 0$，$\Delta_2 = 1 \cdot 1 - 2 \cdot 2 = -3 < 0$，非正定。按公式算到第二步：

$$l_{11} = 1, \qquad l_{21} = 2, \qquad l_{22} = \sqrt{a_{22} - l_{21}^2} = \sqrt{1 - 4} = \sqrt{-3},$$

根号下为负数，分解失败。

对 A_3： $a_{12} = 2 \neq a_{21} = 1$，不对称。若强行按公式计算，$l_{11} = 2$，$l_{21} = \frac{a_{21}}{l_{11}} = \frac{1}{2}$，但 LL^T 的 $(1,2)$ 元为 $l_{11} l_{21} = 1$，与 $a_{12} = 2$ 矛盾，故 A_3 不能写成 LL^T。

对 A_4： 对称。$\Delta_1 = 1 > 0$，$\Delta_2 = 1\cdot 1 - 1\cdot 1 = 0$，半正定但非正定。按公式：

$$l_{11} = 1, \qquad l_{21} = 1, \qquad l_{22} = \sqrt{1-1} = 0.$$

L 的对角元出现零，分解退化，不满足“L 对角元全为正”的唯一性要求。

结论

四个矩阵中只有 A_1 可 Cholesky 分解，$L = \begin{pmatrix} 2 & 0 \\ 1 & 2 \end{pmatrix}$；$A_2$ 非正定，根号下出现负数；A_3 不对称，LL^T 必对称不可能等于 A_3；A_4 半正定非正定，对角元出现零，分解退化。

条件不满足时的限制

非正定矩阵根号下可能出现负数，分解失败；不对称矩阵不能写成 LL^T，因为 LL^T 必对称；半正定矩阵对角元可能为零，分解不唯一或退化；接近奇异的对称正定矩阵虽满足理论条件，舍入可能破坏数值正定性。因此 Cholesky 不能无条件用于仅半正定、非正定或不对称矩阵；对对称正定矩阵，Cholesky 无需选主元，计算量约为 LU 的一半。

实验设计：先预测，再运行

实验任务：用小型一般矩阵及正定矩阵求解，与可信库函数比较；加入需交换主元的例子。

要填写的项目	本组内容
要回答的问题与运行前预测	研究部分选主元 LU 能否处理首个主元为 0、但矩阵仍可逆的情况；验证 Cholesky 分解对称正定矩阵的适用性，并与 SciPy 参考实现比较。预测：LU 通过行交换后满足 $PA = LU$；正定矩阵可得到对角元为正的下三角 Cholesky 因子；分解误差、相对残差和已知真解误差应接近双精度机器精度，并低于设定容差。
输入、参数、单位、数据来源与划分	全部为人工构造的实数稠密矩阵，`float64`，无物理单位。LU 主例：$A_{41} = \begin{pmatrix} 0 & 2 & 1 \\ 1 & 0 & 3 \\ 2 & 1 & 1 \end{pmatrix}$，已知真解 $x^* = (1,-2,3)^T$，右端 $b = (-1,10,3)^T$；首个对角主元为 0，需要交换主元。Cholesky 例：$H = \begin{pmatrix} 4 & 2 & 2 \\ 2 & 5 & 1 \\ 2 & 1 & 3 \end{pmatrix}$，以 X^* 的三列（$(1,-2,3)^T$、$(1,1,1)^T$、$(-2,1,0)^T$）构造 $B = HX^*$。另构造独立的 128×128 稠密矩阵 A_{exp41}，并用解析公式生成 x_{direct}、$b_{\text{direct}} = A_{\text{exp41}} x_{\text{direct}}$，用于规模核验。没有训练集与测试集划分。
独立参考解/基线及选择理由	LU 与 Cholesky 小矩阵均使用事先指定的制造真解 x^*、X^*，避免把待测算法自身输出当作真值；另以 `scipy.linalg.lu`、`scipy.linalg.cho_factor` 和 `scipy.linalg.cho_solve` 作为独立库函数对照。128×128 例同样使用预先构造的真解作为参考。
误差指标、容差及其依据	分解误差：$\frac{\|LU-PA\|_F}{\|PA\|_F}$（Cholesky 为 $\frac{\|LL^T-H\|_F}{\|H\|_F}$）；相对残差：$\frac{\|A\hat{x}-b\|_2}{\|b\|_2}$；相对解误差：$\frac{\|\hat{x}-x^*\|_2}{\|x^*\|_2}$。小矩阵检查采用 10^{-12} 量级容差，128×128 LU 分解与解误差检查采用 2×10^{-12} 量级容差；同时记录 2-范数条件数。

要填写的项目	本组内容
环境、种子、设备与运行预算	Notebook 实际记录：Python `3.12.10`、NumPy `1.26.4`、SciPy `1.14.1`；使用 Matplotlib 绘图、`threadpoolctl` 限制线程（这两项的具体版本未在可见输出中记录）。CPU，`float64`，随机种子 `2026`；设置 `OMP_NUM_THREADS`、`OPENBLAS_NUM_THREADS`、`MKL_NUM_THREADS` 为 `1`。本节矩阵最大规模为 128×128；没有单独记录本节运行时间预算。

结果解释与本节结论

方法	分解误差	相对残差	相对解误差	$\mathrm{cond}_2(A)$
部分选主元 LU（3×3）	0	0	0	**2.66345**
Cholesky（3×3，3个右端）	5.38537×10^{-17}	0	1.0847×10^{-16}	**5.51823**
部分选主元 LU（128×128）	3.06096×10^{-16}	5.41247×10^{-16}	3.8816×10^{-16}	**2.44612**

实际结果与预测一致。3×3 LU 例成功完成主元交换，输出置换矩阵 P，并满足 $PA = LU$；计算得到 $\hat{x} = (1, -2, 3)^T$。Cholesky 例得到下三角因子

$$L = \begin{pmatrix} 2 & 0 & 0 \\ 1 & 2 & 0 \\ 1 & 0 & \sqrt{2} \end{pmatrix},$$

且 $LL^T = H$，三个右端的解与制造真解及 SciPy 结果一致。128×128 例的分解误差、相对残差和相对解误差均约为 10^{-16}，明显低于设定容差。

程序还验证了异常输入处理：非正定矩阵和非对称矩阵被 Cholesky 检查拒绝；奇异、含非有限值和非方阵输入被 LU 检查拒绝。以上五项负例检查均按预期触发异常。

计算量说明：稠密 LU 分解的理论复杂度为 $O(n^3)$，Cholesky 分解约为 $O\left(n^3\right)$，每个右端的三角回代约为 $O(n^2)$；本节没有单独记录分解与回代的计时，因此不把理论复杂度当作实测耗时。

易错点与讨论

1. 并非任意矩阵都可以进行无主元的 LU 分解；顺序主子式为零只说明无主元 LU 无法进行，不说明 A 奇异（如 $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ 非奇异但 $a_{11} = 0$），此时应改用列主元 LU。对非正定矩阵不能无条件使用 Cholesky 分解。
2. 列主元 LU 必须明确置换约定：$PA = LU$ 中 P 左乘 A 表示行交换，$AP = LU$ 表示列交换；$P \neq P^T$ 时不能混用。每次行交换后，L 中已存好的对应行乘子也要同步交换，否则 $PA = LU$ 不成立。
3. 列主元消去法虽然能解决小主元问题，但它不能改善矩阵本身的病态性；对于病态矩阵，选主元只能避免因算法引起的额外误差，而无法消除问题固有的敏感性。病态矩阵仍可逆，只是解对扰动敏感；这与奇异矩阵不可逆是两件事。

4. 追赶法的对角占优条件只是充分条件，不是必要条件；某些非对角占优的三对角矩阵也可能稳定求解，但需要额外验证：可检查分解得到的 $u_i \neq 0$ 且 $|\beta_i| < 1$（β_i 的定义见 4.1.4 ），二者成立则回代不放大误差。U 的上次对角线就是 A 的 c_i，消去过程中保持不变，无需重新计算。

5. 残差小不等于解误差小，讨论残差时必须说明它针对哪个右端计算；条件数依赖所选范数，$\kappa_2(A)$、$\kappa_\infty(A)$、$\kappa_1(A)$ 一般不同，比较或引用数值时必须注明范数。

6. Cholesky 分解要求对称正定，接近奇异时舍入可能破坏数值正定性；标准无主元 Cholesky 不能无条件用于仅半正定、非正定或不对称矩阵。

7. LDL^T 分解避免了开方，但适用条件并不比 Cholesky 宽：仍要求 A 对称正定（或对称且顺序主子式非零）；不开方不等于可以用于非正定矩阵，D 的对角元可能为零或负。

8. 条件数是问题本身的属性，与算法无关；选主元、Cholesky、追赶法解决的是“算法不引入额外不稳定”，不能把病态问题变成良态问题。若右端已被扰动为 $b+\delta b$，对扰动后右端的残差可以为零，但这不意味着对原右端的解误差小。

随堂提问： 给定矩阵

$$A = \begin{pmatrix} 0 & 2 & 1 \\ 1 & 0 & 3 \\ 2 & 1 & 1 \end{pmatrix}, \qquad b = \begin{pmatrix} -1 \\ 10 \\ 3 \end{pmatrix}.$$

要求：

(1) 不做行交换，直接按 $a_{11} = 0$ 开始消元，说明第一步为何失败；

(2) 改用列主元法，写出第一次行交换后的矩阵、置换矩阵 P，并完成 $PA = LU$；

(3) 判断 A 是否对称正定，说明能否对 A 使用 Cholesky 分解；若不能，指出是哪一条适用条件不满足。

可能的回答：

- 一部分同学只看到 $a_{11} = 0$，回答“主元为零，换行即可”，但没有写出 P 与 $PA = LU$ 的对应关系，也未核验换行后 L 中乘子是否随之交换；
- 一部分同学看到 A 的元素非负、对角元非零，误以为“看起来正定”，直接尝试 Cholesky；实际上 A 不对称（$a_{12} = 2 \neq a_{21} = 1$），且一阶顺序主子式 $a_{11} = 0$ 不满足正定性要求；
- 还有同学把“列主元 LU 可行”与“Cholesky 可行”混为一谈，认为既然列主元能分解，Cholesky 也一定能用。

解释：

1. 无主元 LU 要求所有顺序主子式非零；本题 $a_{11} = 0$，一阶顺序主子式为零，第一步消元即无法进行，但这不说明 A 奇异。

2. 列主元法在第一列选绝对值最大的 $a_{31} = 2$，交换第 1、3 行，得到 $P = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$，再完成消元得 $PA = LU$。换行后 L 中已存乘子也要相应交换，核验 $PA = LU$ 两边相等。

3. Cholesky 要求 A 对称正定。本题 A 不对称（$a_{12}=2\neq a_{21}=1$），且 $a_{11}=0$ 已使一阶顺序主子式不为正，正定性不成立，因此不能使用 Cholesky。列主元 LU 对一般非奇异矩阵总是可行，但它解决的是算法稳定性，不是正定性；两者适用条件不同，不能互相替代。

小结

直接法的核心是矩阵分解，而分解的选择取决于矩阵结构与条件数。4.1 节从 Gauss 消元法出发，把直接法统一到 $A=LU$ 的框架：LU 分解把 $Ax=b$ 化为两次三角求解；列主元 LU 用置换矩阵保证 $PA=LU$ 中乘数有界；Cholesky 分解利用对称正定性把计算量和存储量减半；追赶法利用三对角结构把复杂度降到 $O(n)$。选择过程可以为一棵决策树：

对称正定？ $\xrightarrow{\text{是}}$ **Cholesky** 或 LDL^T

↓ 否

三对角？ $\xrightarrow{\text{是}}$ 追赶法

↓ 否

列主元 **LU**

方法选择可以归纳为以下几步：

1. 先看结构。 一般非奇异矩阵用列主元 LU；对称正定矩阵用 Cholesky 或 LDL^T；三对角矩阵用追赶法。结构越特殊，可用的分解越省计算：LU 与 Gauss 消元法同为 $O(n^3)$，但分解结果可复用于多个右端，每个右端只增加 $O(n^2)$；Cholesky 乘除法约 $\frac{n^3}{6}$，约为 LU 的一半；追赶法乘除法约 $5n-4$，降至 $O(n)$。
2. 再看算法稳定性。 无主元 LU 要求所有顺序主子式非零；顺序主子式非零但主元极小时，乘子可能很大，舍入误差被急剧放大，列主元 LU 使 $|l_{ij}|\leqslant 1$，避免乘子无界增长。Cholesky 由 $a_{jj}=\sum_{k=1}^{j} l_{jk}^2$ 得 $l_{jk}^2\leqslant a_{jj}$，元素不会无界增长，无需选主元；但接近奇异时舍入可能破坏数值正定性。追赶法在对角占优条件下由 $|\beta_i|<1$ 保证回代不放大误差；对角占优是充分条件，不是必要条件。
3. 最后看问题本身。 条件数 $\kappa(A)=\|A^{-1}\|\|A\|$ 是矩阵的固有属性，与算法无关。选主元、Cholesky、追赶法解决的是“算法不引入额外不稳定”，不能把病态问题变成良态问题。对病态矩阵，即使残差 $r=b-A\hat{x}$ 很小，解误差仍可能很大：

$$\frac{\|\hat{x}-x\|}{\|x\|}\leqslant \kappa(A)\frac{\|r\|}{\|b\|}.$$

这一上界是本章贯穿线索“小残差是否等于小解误差”的定量表达：残差是输出端误差，解误差是输入端误差，条件数是二者之间的放大因子。讨论“残差小”时必须说明残差针对哪个右端计算；若右端已被扰动为 $b+\delta b$，对扰动后右端的残差可以为零，但这不意味着对原右端的解误差小。

适用条件与稳定性要点： 分三层看。

第一层，能不能做。 无主元 LU 要求顺序主子式非零；Cholesky 要求对称正定；追赶法要求三对角结构。

第二层，做得稳不稳。 主元极小时乘子可能很大，列主元 LU 把乘子压到 $|l_{ij}|\leqslant 1$；Cholesky 由 $l_{jk}^2\leqslant a_{jj}$ 保证元素不无界增长，但接近奇异时舍入可能破坏数值正定性；追赶法在对角占优下由 $|\beta_i|<1$ 保证回代不放大误差，对角占优只是充分条件。

第三层，问题本身敏不敏感。 病态性是矩阵的固有性质；选主元、Cholesky、追赶法提高的是算法稳定性，不能把病态问题变成良态问题。

4.2 迭代法：CG 与预条件（2 学时）

本节目标： 能说明 CG 的适用条件和预条件改善的对象。

概念讲解：把问题说明白

4.1 节所介绍的直接法运算量为 $O(n^3)$，存储需要 $O(n^2)$，对中小规模稠密矩阵足够可靠。但一旦目标换成大规模稀疏矩阵，情况就不同了。

以偏微分方程离散化得到的线性方程组 $Ax=b$ 为例。阶数 n 往往能达到 10^6，且 A 中只有少数几个非零元（一般为 $O(n)$ 量级）。直接法在这种情境下会遇到两个难题：

第一个麻烦是填充：消元过程中，原本为零的位置被非零元填入，稀疏结构被破坏，存储量从 $O(n)$ 膨胀到 $O(n^2)$，矩阵越消元反而越稠密。

第二个麻烦是计算量：直接法分解需要 $n^3=10^{18}$ 次运算，即使每秒进行 10^{12} 次浮点运算，也要约 10^6 秒（超过 11 天）；而一次矩阵与向量的乘积只需 $O(n)$ 次运算。

因此，对于较大规模的稀疏矩阵来说，直接法通常不是最佳选择。既然直接走不通，不如找路绕过去。我们可以试着改变一下思路——能否不直接求解，只利用“A 作用在向量上”这一操作，想办法逼近精确解，最终得到一个误差在可接受范围内的近似解？

4.2.1 迭代法的定义与线性定常迭代

迭代法的出发点很朴素：不分解矩阵，也不求一步到位，而是先随便猜一个初始值，然后按照某种固定规则反复修正，让每一步都比上一步更接近真解。就像蒙着眼站在山坡上，看不见谷底，但能感觉哪边更低，于是朝低的方向一步一步走，直到走不动为止。每次迭代的主体运算是矩阵与向量的乘积，不需要像直接法中进行矩阵的显式分解，因此这种方法特别适合大型稀疏矩阵。

上面用自然语言描述了迭代法的基本思路。下面给出它的正式定义。

> 定义 4.5（迭代法）
> 从一个初始值 $x^{(0)}$ 出发，按某个迭代公式产生序列 $\{x^{(k)}\}$，若该序列收敛到 $Ax=b$ 的真解 x^*，则称这种求解方法为迭代法。

迭代公式的形式多种多样。特别地，我们先来看一种最简单的迭代方法：

> 定义 4.6（线性定常迭代）
> 若迭代公式为 $x^{(k+1)}=Bx^{(k)}+f$，其中 B 为固定矩阵、f 为固定向量，则称其为线性定常迭代，B 称为迭代矩阵。

线性定常迭代是迭代法中最基本、最常用的一类，下面先以这种迭代法为例，展示迭代公式的构造思路。

如何由 $Ax=b$ 构造线性定常迭代？先将矩阵 A 变形为 $A=M-N$，其中 M 非奇异且容易求逆（通常取 A 的某一部分，如对角部分或三角部分）。因此 $Ax=b$ 变为 $Mx=Nx+b$，也即 $x=M^{-1}Nx+M^{-1}b$。

从而我们可以根据这个形式构造一个迭代公式：记 $B=M^{-1}N=M^{-1}(M-A)=I-M^{-1}A$，$f=M^{-1}b$，得到

$$x^{(k+1)}=Bx^{(k)}+f,\quad k=0,1,2,\cdots \tag{4.7}$$

这里 B 被称为迭代矩阵。构造流程如下：

$$\begin{aligned} Ax=b &\xrightarrow{A=M-N} Mx=Nx+b \\ &\xrightarrow{M^{-1}} x=M^{-1}Nx+M^{-1}b \\ &\xrightarrow{B=M^{-1}N,\ f=M^{-1}b} x=Bx+f \\ &\xrightarrow{\text{加迭代步}} x^{(k+1)}=Bx^{(k)}+f. \end{aligned}$$

M 的不同取法对应不同的迭代格式：取 M 为 A 的对角部分得到 Jacobi 迭代，取 M 为 A 的下三角部分得到 Gauss-Seidel 迭代，加权版本得到 SOR 等。这些格式本质都是 (4.7) 的变形，区别只在于 M 的选择。

接下来我们要回答两个问题：线性定常迭代什么时候收敛，收敛多快。这两个问题都与迭代矩阵 B 的特征值有关。

先看收敛性。设 x^* 为真解，它满足 $x^*=Bx^*+f$。将它与式 (4.7) 相减，得到误差的递推关系

$$e^{(k+1)}=Be^{(k)},\qquad e^{(k)}=x^{(k)}-x^*,$$

于是 $e^{(k)}=B^ke^{(0)}$。因此，$x^{(k)}\to x^*$ 对任意初值成立，当且仅当 $B^k\to 0$。而 $B^k\to 0$ 的充要条件是 B 的所有特征值的模都小于 1。记 B 的特征值中绝对值最大者为 $\rho(B)$（称为谱半径），这一条件即 $\rho(B)<1$。

> 收敛判据：线性定常迭代 $x^{(k+1)}=Bx^{(k)}+f$ 对任意初值收敛的充要条件是 $\rho(B)<1$。

收敛速度同样由谱半径决定：$\rho(B)$ 越小，B^k 衰减越快，收敛越快。概括地说，线性定常迭代的收敛与快慢由迭代矩阵的谱半径决定。

4.2.2 对称正定矩阵的迭代法：共轭梯度法

4.1 节中，我们已经介绍过对称正定矩阵的重要性。对大规模稀疏的对称正定矩阵，我们仍希望选用一种迭代方法，运算时只有矩阵与向量的乘积。

那么，对称正定性又能给迭代法带来什么额外的优势？答案是，它可以把“解方程”变成“求最小值”。而求最小值时，每一步只要让函数值下降，就能逐步逼近最小值点，因此，迭代公式正好可以按照这样的原则进行构造！

我们知道，当 A 对称正定时，函数

$$\phi(x)=\frac{1}{2}x^TAx-b^Tx \tag{4.8}$$

是严格凸的二次函数。它的梯度为 $\nabla\phi(x)=Ax-b$，Hessian 矩阵为 A（正定），因此 ϕ 有唯一最小值点，且易知该最小值点恰好是 $Ax=b$ 的解。于是求解 $Ax=b$ 等价于求 $\phi(x)$ 的最小值点。

这个等价关系为什么有用？因为最小化问题有几何直观：ϕ 的等高线是一族同心椭圆（高维时为椭球面），最小值点位于中心。从任意初始点出发，沿某个方向逐步移动，让函数值下降，就能逼近中心。这就把“解方程”变成了“找最低点”。

最直接的做法是沿负梯度方向搜索。负梯度 $-\nabla\phi(x)=b-Ax$ 正是残差 $r=b-Ax$，因此每一步沿当前残差方向移动，步长则可以取使 ϕ 在该方向上最小的值。

> 定义 4.7（最速下降法）
> 从初始值 x_0 出发，按

$$x_{k+1}=x_k+\alpha_k r_k,\qquad k=0,1,2,\cdots \tag{4.9}$$

> 迭代，其中 $r_k=b-Ax_k$ 为第 k 步残差，步长 α_k 取使 $\phi(x_k+\alpha r_k)$ 最小的值。称该迭代法为最速下降法。

步长的显式表达可以直接求出：令 $\frac{d}{d\alpha}\phi(x_k+\alpha r_k)=0$，得 $\alpha_k=\frac{r_k^Tr_k}{r_k^TAr_k}$。从“解方程”到“找最低点”的推导路线如下：

$$\begin{aligned}
Ax=b &\xrightarrow{A\text{ 对称正定}} \min_x\ \phi(x)=\tfrac12 x^TAx-b^Tx\\
&\xrightarrow{\nabla\phi=Ax-b} \text{负梯度}=-\nabla\phi=b-Ax=r\\
&\xrightarrow{\text{沿 } r_k \text{ 搜索}} x_{k+1}=x_k+\alpha_k r_k \quad\text{（最速下降）}\\
&\xrightarrow{\text{改用 } A\text{-共轭方向}} \mathbf{CG}.
\end{aligned}$$

最速下降法在每一步都沿负梯度方向走，而负梯度垂直于等高线。当 A 的条件数很大时，等高线就被拉成一个又扁又长的椭球。负梯度垂直于等高线，几乎总指向短轴。于是，迭代路径就会在短轴两侧来回横跳，像在一条窄走廊里左右撞墙，明明离谷底并不远，却迟迟走不到。这就是最速下降法“每一步都最陡，合起来却很慢”的原因。

> 最速下降：每一步都走最陡的方向，但下一步又折回来了。
> CG：选一组共轭方向，走过的不再回头。

为了定量说明这一点，需要引入一种新的范数。

> 定义 4.8（A-范数，能量范数）
> 当 A 对称正定时，$\|v\|_A=\sqrt{v^TAv}$ 满足范数的三条公理，称为 A-范数（或能量范数）。

最速下降法的误差在 A-范数下满足

$$\|x^*-x_k\|_A\leqslant\left(\frac{\kappa(A)-1}{\kappa(A)+1}\right)^k\|x^*-x_0\|_A,$$

其中 $\kappa(A)$ 即为条件数。这个估计表明：$\kappa(A)$ 越接近 1，收敛越快；$\kappa(A)$ 很大时，收敛可能极慢。

既然折返的根源是每一步都只用了当前残差方向，没有利用此前搜索过的方向，那么针对该缺点所提出的改进思路是：选一组关于 A 共轭的搜索方向，使每一步都在新的方向上取得进展，不会退回已搜索过的方向。

> 定义 4.9（A-共轭与共轭梯度法）
> 设 A 对称正定。若非零向量 p_i, p_j 满足 $p_i^T A p_j = 0$（$i \neq j$），则称 p_i, p_j 关于 A 共轭。若迭代法每一步沿一组关于 A 共轭的方向搜索，则称该迭代法为共轭梯度法（Conjugate Gradient，CG）。

共轭方向的好处是：在精确算术下，一个 n 维问题至多 n 步即可收敛。CG 正是按这一思路构造搜索方向：第 k 步的搜索方向 p_k 由当前残差与上一方向组合而成，使得 p_k 与前 $k-1$ 个方向都关于 A 共轭。

CG 的适用条件也很明确：A 必须对称正定。若 A 非对称或非正定，$p_k^T A p_k$ 可能不为正，步长公式失去意义，标准 CG 不适用，需要改用其他方法。

CG 每次迭代只需计算矩阵与向量的乘积 Ap_k，不需要显式存储 A 或分解 A，因此适合大规模稀疏矩阵。CG 的收敛速度同样由 A 的条件数支配：$\kappa(A)$ 越接近 1，收敛越快。

最后，我们再来解释一下 CG 与二次函数最小化的关系。CG 的第 k 步近似解 x_k 是在一个逐步扩大的子空间中使 A-范数误差最小的解。这个子空间由初始残差与 A 的幂张成，称为 Krylov 子空间；CG 的构造保证它在每一步都在该子空间中取到最优。这一视角解释了 CG 为什么能在 n 步内收敛，也把 CG 与更一般的 Krylov 方法联系起来，这里不再展开。

4.2.3 误差分析

4.2.2 中我们讨论了迭代法收敛的速度。本节我们再来讨论另一个问题：停止迭代时，残差与解误差是什么关系？

CG 的迭代过程自然产生残差 $r_k = b - Ax_k$，常用的停止准则是相对残差 $\frac{\|r_k\|}{\|b\|}$ 小于给定容差。然而，残差小并不意味着解误差小。由 $r_k = b - Ax_k$，得 $A(x - x_k) = r_k$，故误差与残差之间的关系是 $x - x_k = A^{-1} r_k$。因此

$$\|x - x_k\| \leqslant \|A^{-1}\| \, \|r_k\|.$$

又由 $b = Ax$ 得 $\|b\| \leqslant \|A\| \, \|x\|$。两式相乘，可推出在相对误差的形式下，

$$\frac{\|x - x_k\|}{\|x\|} \leqslant \|A^{-1}\| \, \|A\| \, \frac{\|r_k\|}{\|b\|} = \kappa(A) \frac{\|r_k\|}{\|b\|}.$$

这表明残差到解误差的放大倍数，正是条件数。下面用具体例子来演示放大的效果：

取 $A = \mathrm{diag}(1, 10^{-10})$，$b = (1,\, 10^{-10})^T$，真解为 $x^* = (1,1)^T$。设近似解为 $x_k = (1,2)^T$，即第二个分量尚未收敛。此时

$$r_k = b - Ax_k = (0,\, -10^{-10})^T, \qquad \frac{\|r_k\|}{\|b\|} \approx 10^{-10},$$

而解误差为

$$x^* - x_k = (0, -1)^T, \qquad \frac{\|x^* - x_k\|}{\|x^*\|} = \frac{1}{\sqrt{2}} \approx 0.707.$$

相对残差只有 10^{-10}，相对解误差却约为 0.707，两者相差 10^{10} 倍。而这个矩阵的条件数正是 $\kappa_2(A) = 10^{10}$，且上界 $\kappa(A)\frac{\|r_k\|}{\|b\|} \approx 1$。反过来，当 $\kappa(A)$ 接近 1 时，$\kappa(A)\frac{\|r_k\|}{\|b\|}$ 与 $\frac{\|r_k\|}{\|b\|}$ 同量级，相对残差小确实能保证相对解误差小，因此问题只出在 $\kappa(A)$ 很大时。

既然残差不能直接当作误差，实际计算中可以用残差给出后验误差估计：$\|x - x_k\| \leqslant \|A^{-1}\|\,\|r_k\|$。其中 $\|A^{-1}\|$ 可以用条件数和 $\|A\|$ 估计，或者用其他方法近似。但这个上界同样含 $\|A^{-1}\|$，$\kappa(A)$ 很大时上界会很松。因此，实践中也有用绝对残差 $\|r_k\|$ 或 A-范数误差 $\|x^* - x_k\|_A$ 作停止准则的；相对残差对右端尺度不敏感，最常用，但它同样受条件数放大，不能单独作为“解已经足够精确”的依据。

这与 4.1.5 中条件数影响舍入误差放大倍数的现象一致：同一个条件数，在直接法中放大输入扰动，在迭代法中又放大了残差到解误差的倍数。

4.2.4 共轭梯度法的改进

由 4.2.2 的收敛估计可知，当矩阵 A 的条件数 κ 很大时，CG 的收敛速度会很慢。因此我们要进一步改进迭代方法，使得预条件后系统的条件数尽可能小。预条件就是为加速CG收敛而出现的关键方法，选取一个矩阵 M （被称作预条件子。注意这里的 M 与 4.2.1 中分裂矩阵 $A = M - N$ 的含义不同：4.2.1 的 M 用于构造迭代格式，这里的 M 用于改善条件数）将原系统 $Ax = b$ 转化为一个等价且条件数更小的系统：

$$M^{-1}Ax = M^{-1}b, \tag{4.10}$$

其中 M 是对 A 的某种近似，且 M^{-1} 易于应用。这样处理的目标是使 $M^{-1}A$ 的条件数远小于 A 的条件数，从而加速收敛。

> 定义 4.10（预条件共轭梯度法，PCG）
> 对预条件系统 $M^{-1}Ax = M^{-1}b$ 应用共轭梯度法进行迭代，称为预条件共轭梯度法（简称 PCG）。

这里 M 需要满足两个条件：一是对称正定（为了保持对称性）；二是 M^{-1} 容易计算。最简单的选择是对角预条件 $M = \mathrm{diag}(A)$，即取 A 的对角线元素构成对角矩阵。为了保持对称性，有时也会写成

$$M^{-\frac{1}{2}}AM^{-\frac{1}{2}}y = M^{-\frac{1}{2}}b, \qquad y = M^{\frac{1}{2}}x$$

这个矩阵仍然对称正定。对角预条件计算代价极低（只需逐元素除法），但这种方法只利用了原矩阵的对角元素的信息，对病态性不主要体现在对角线上的问题来说，改善的效果相对有限。

除对角预条件外，常见的预条件子还有以下几类：（这里列出是为了说明预条件子的选择不限于对角，由于本章主要介绍对角预条件的相关内容，更多预条件的相关内容留到后续第8章学习。）

- 不完全 Cholesky 分解（IC）：对 A 做近似 Cholesky 分解，保留稀疏性，丢弃填充元。IC 预条件对稀疏矩阵非常有效，但构造代价较高。
- 多重网格预条件：利用问题的多尺度结构，构造层次化的预条件子。对由 PDE 离散化得到的矩阵特别有效。
- 稀疏近似逆（SPAI）：直接构造 M^{-1} 的稀疏近似，避免求解线性方程组。

一个实际运算的例子：取二维对角正定系统

$$A=\begin{pmatrix}1 & 0\\ 0 & 100\end{pmatrix},\qquad b=\begin{pmatrix}1\\ 1\end{pmatrix}.$$

真解为 $x^*=(1,0.01)^T$，条件数 $\kappa_2(A)=100$。从零初值出发，标准CG的第一步为

$$r_0=b=(1,1)^T,\quad p_0=r_0,$$

$$\alpha_0=\frac{r_0^Tr_0}{p_0^TAp_0}=\frac{2}{1+100}=\frac{2}{101},$$

$$x_1=x_0+\alpha_0p_0=\left(\frac{2}{101},\frac{2}{101}\right)^T\approx(0.0198,0.0198)^T.$$

第二个分量已接近真解 0.01 的量级，但第一个分量 0.0198 与真解 1 之间还有很大差距。若改用对角预条件

$$M=\mathrm{diag}(A)=\begin{pmatrix}1 & 0\\ 0 & 100\end{pmatrix},$$

则预条件系统 $M^{-1}Ax=M^{-1}b$ 对应的矩阵为 $M^{-1}A=I$，条件数降为 1。PCG的第一步就等价于直接求解

$$\tilde{x}_1=M^{-1}b=(1,0.01)^T=x^*.$$

这个例子说明：对角预条件的作用正是把各个方向上的尺度差异“拉平”。预条件要做的事，就是把等高线的扁椭球拉圆——x 轴拉长、y 轴压短，长短轴之比降下来，CG 不必再在窄走廊里左右撞墙，几步就能走到中心。当对角尺度差异是条件数的主要来源时，对角预条件的效果非常明显；但当病态来自非对角耦合时，仅靠对角预条件就不够，可能需要更强的预条件子。

要注意的是，预条件改善的是预条件后系统的条件数，从而加快 CG 的收敛，但它不改变原问题的病态性。PCG 的总成本需要同时计入迭代次数的减少和每步迭代中应用预条件子的额外开销。在比较有无预条件的性能时，应报告迭代次数和总计算时间的对比，而不仅仅是迭代次数。

关键推导：让结论有依据

推导 1：从二次函数最小化到 CG 迭代格式。 设 A 对称正定，函数 $\phi(x)=\frac{1}{2}x^TAx-b^Tx$。其梯度为 $\nabla\phi(x)=Ax-b$，Hessian矩阵为 A，故最小值点满足 $Ax=b$。CG在第 k 步沿方向 p_k 搜索：

$$x_{k+1}=x_k+\alpha_kp_k.$$

步长 α_k 通过最小化 $\phi(x_k+\alpha p_k)$ 确定：

$$\frac{d}{d\alpha}\phi(x_k+\alpha p_k)=p_k^T(A(x_k+\alpha p_k)-b)=0,$$

解得 $\alpha_k=\frac{r_k^Tr_k}{p_k^TAp_k}$，其中 $r_k=b-Ax_k$ 为第 k 步的残差。搜索方向 p_k 由当前残差和上一方向组合而成：

$$p_{k+1}=r_{k+1}+\beta_kp_k,$$

其中 $\beta_k=\frac{r_{k+1}^Tr_{k+1}}{r_k^Tr_k}$。初始方向 $p_0=r_0$。残差递推公式为 $r_{k+1}=r_k-\alpha_kAp_k$。每一步只需计算一次矩阵与向量的乘积 Ap_k。

对应基本迭代式：

$$\alpha_k=\frac{r_k^Tr_k}{p_k^TAp_k},\quad x_{k+1}=x_k+\alpha_kp_k,\quad r_{k+1}=r_k-\alpha_kAp_k,$$

$$\beta_k = \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}, \qquad p_{k+1} = r_{k+1} + \beta_k p_k.$$

从上述推导中可以看出，若 A 非对称或非正定，$\phi(x)$ 不一定有最小值点，$p_k^T A p_k$ 也可能不为正，标准 CG 不适用。

设 $\kappa = \kappa_2(A) = \frac{\lambda_{\max}}{\lambda_{\min}}$，其中 $\lambda_{\max}$、$\lambda_{\min}$ 为 A 的最大、最小特征值。CG 的误差在 A-范数意义下满足

$$\|x^* - x_k\|_A \leqslant 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k \|x^* - x_0\|_A,$$

其中 $\|v\|_A = \sqrt{v^T A v}$。这个估计与最速下降法的估计形式类似，但结果更紧。尽管估计式通常偏保守（实际收敛往往更快，尤其当特征值聚簇时），但它清晰地揭示了条件数对收敛速度的支配作用。

推导 2：对角 PCG 的格式。 设 $M = \mathrm{diag}(A)$。预条件系统为 $M^{-1}Ax = M^{-1}b$。定义 $z_k = M^{-1}r_k$。PCG 的迭代格式为：

$$\alpha_k = \frac{r_k^T z_k}{p_k^T A p_k}, \quad x_{k+1} = x_k + \alpha_k p_k, \quad r_{k+1} = r_k - \alpha_k A p_k,$$

$$z_{k+1} = M^{-1} r_{k+1}, \quad \beta_k = \frac{r_{k+1}^T z_{k+1}}{r_k^T z_k}, \quad p_{k+1} = z_{k+1} + \beta_k p_k.$$

注意 $M^{-1}r$ 的计算只需逐元素除以对角元，代价为 $O(n)$。对应预条件的对称变换。

这里要求 M 对称正定，否则 $M^{\frac{1}{2}}$ 与 $M^{-\frac{1}{2}}$ 不一定存在，PCG 的对称性会被破坏。实现时不必显式形成 $M^{\frac{1}{2}}$，只需每步解 $Mz_k = r_k$；对角预条件取 $M = \mathrm{diag}(A)$，逐元素除法即可。$M^{-1}A$ 与 $M^{-\frac{1}{2}}AM^{-\frac{1}{2}}$ 相似，特征值相同。当 M 对称正定时，两者都是对称正定矩阵，2-范数条件数也相同。因此正文中“改善 $M^{-1}A$ 的条件数”与这里“改善 $\widetilde{A}$ 的条件数”是同一件事。

PCG 等价于对变换后的系统 $\widetilde{A} = M^{-\frac{1}{2}}AM^{-\frac{1}{2}}$ 应用标准 CG，其中 $\widetilde{A}$ 仍然对称正定。因此，PCG 的收敛速度由 $\widetilde{A}$ 的条件数 $\kappa(\widetilde{A}) = \kappa(M^{-\frac{1}{2}}AM^{-\frac{1}{2}})$ 支配。若 M 是 A 的良好近似，则 $\kappa(\widetilde{A}) \ll \kappa(A)$，收敛速度显著提高。这一推导说明了为什么预条件子需要对称正定：只有这样才能保持变换后系统的对称性。

实验设计：先预测，再运行

实验任务：对同一正定系统比较 CG 与对角预条件 CG，在相同停止标准下记录迭代次数和实际误差。

要填写的项目	本组内容
要回答的问题与运行前预测	比较普通共轭梯度法（CG）与对角预条件共轭梯度法（对角 PCG）在同一对称正定系统上的收敛速度和实际误差。预测：对角预条件可改善系统的条件数，使达到相同真实相对残差阈值所需的迭代次数减少；但评价总成本时还应计入预条件矩阵的建立和应用成本。

要填写的项目	本组内容
输入、参数、单位、数据来源与划分	人工构造 128×128 实数稠密 SPD 系统。令 $n = 128$、$i = 0, \ldots, 127$、$d_i = 10^{\frac{2i}{127}}$，$T$ 为主对角元 3、上下副对角元 -1 的三对角矩阵，$A_{\mathrm{cg}} = \mathrm{diag}(d)T\,\mathrm{diag}(d)$。制造真解为 $x_i = \sin\left(\frac{\pi(i+1)}{129}\right) + \frac{1}{2}\cos\left(\frac{3\pi(i+1)}{129}\right)$，右端 b 由独立分量公式构造。Jacobi 预条件取 $M = \mathrm{diag}(A_{\mathrm{cg}})$。初值 $x_0 = 0$，$\mathrm{rtol} = 10^{-10}$、$\mathrm{atol} = 0$、$\mathrm{maxiter} = 2000$；每种方法重复计时 9 次，报告中位数。数据为合成数据，无训练集与测试集划分。
独立参考解/基线及选择理由	制造真解 x 是主要参考，可直接计算实际解误差；另用 `numpy.linalg.solve(A_cg,b)` 做稠密直接求解对照。SciPy 的 `scipy.sparse.linalg.cg` 配合 `LinearOperator` 形式的 Jacobi 逆预条件器作独立 PCG 交叉验证。
误差指标、容差及其依据	真实相对残差 $\frac{\|b - A\hat{x}\|_2}{\|b\|_2}$，停止阈值 $\max(\mathrm{rtol} \cdot \|b\|_2, \mathrm{atol})$，其中 $\mathrm{rtol} = 10^{-10}$、$\mathrm{atol} = 0$；相对解误差 $\frac{\|\hat{x} - x^*\|_2}{\|x^*\|_2}$；记录迭代次数、$\mathrm{cond}_2(A)$、对称预条件变换后的条件数及运行时间中位数。程序还检查解误差与条件数—残差误差界的一致性，并要求解误差小于 10^{-6}。
环境、种子、设备与运行预算	Python `3.12.10`、NumPy `1.26.4`、SciPy `1.14.1`；CPU、`float64`、种子 `2026`，线程数限制为 1。规模 $n = 128$，每种方法计时重复 9 次，迭代上限 2000。SciPy `1.14.1` 的 `cg` 接口使用 `tol` 参数而不是 `rtol`；Notebook 已通过检查函数签名自动选择参数名，并显式传入 `atol=0`。

结果解释与本节结论

方法	迭代次数	真实相对残差	相对解误差	求解时间中位数（秒）
CG	517	8.8912×10^{-11}	1.17632×10^{-8}	0.0171246
Jacobi-PCG	24	5.84311×10^{-11}	5.24993×10^{-11}	0.000992

原矩阵条件数 $\mathrm{cond}_2(A_{\mathrm{cg}}) \approx 30888.6621$；对称预条件变换后的条件数约为 4.99644，下降约 6182 倍。Jacobi-PCG 将迭代次数由 517 次降至 24 次，减少约 95.36%。两种方法的真实相对残差均达到 10^{-11} 量级，解误差也均小于 10^{-6}。本次计时中，CG 求解中位数为 $0.0171246\,\mathrm{s}$，Jacobi-PCG 求解中位数为 $0.000992\,\mathrm{s}$；Jacobi 预处理建立时间中位数为 $2.8 \times 10^{-6}\,\mathrm{s}$，计入建立成本后总时间约为 $0.0009948\,\mathrm{s}$。本次运行的含预处理总耗时仍约快 17 倍，但该计时仅代表当前 CPU、矩阵规模和运行环境，不应外推为普遍加速比。

控制实验使用常数对角矩阵 T，CG 与 Jacobi-PCG 均为 23 次迭代，解误差约 4.46×10^{-11}，说明预条件器并非仅因“加入对角矩阵”就必然减少迭代；效果取决于它是否改善了问题的谱性质。残差历史图的横轴为迭代次数，纵轴为真实相对残差（无量纲），并标出 10^{-10} 停止线。

尚未验证不同矩阵规模、不同预条件器（如不完全 Cholesky）、稀疏存储格式和多线程与多设备条件下的性能。

易错点与讨论

1. 残差与解误差的区别：残差是输出端的误差，解误差是输入端的误差。条件数描述了从输出端误差到输入端误差的放大倍数。小残差只在条件数不很大时才能保证小解误差。
2. 预条件改善的对象：预条件改善的是预条件后系统的条件数，从而加快 CG 的收敛，不是解本身，也不是问题的病态性。预条件 CG 的总成本需要同时计入迭代次数的减少和每步迭代中应用预条件子的额外开销。对角预条件每步成本 $O(n)$。
3. CG 的适用条件：A 必须对称正定。若 A 非对称或非正定，标准 CG 不适用，需要其他 Krylov 方法。
4. 最速下降法与 CG 的区别：最速下降法沿负梯度方向搜索，相邻方向正交，收敛慢；CG 选择共轭方向，避免重复搜索，收敛快。

随堂提问： 设 $A=\mathrm{diag}(1,10,100,1000)$，$b=(1,1,1,1)^T$，$n=4$，$\kappa_2(A)=1000$。

(1) 写出对角预条件子 $M=\mathrm{diag}(A)$，计算 $M^{-1}A$ 的条件数。

(2) 用 CG 误差上界 $2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k$ 估算标准 CG 达到容差 10^{-6} 所需迭代次数 k_{CG}（$\kappa=1000$）；PCG 因 $M^{-1}A=I$，理论 1 步收敛。

(3) 设每步矩阵-向量乘法成本为 n，预条件构造成本为 n，预条件应用成本为 n/步。分别写出 CG 和 PCG 的总成本表达式，代入 k_{CG} 比较。

(4) 若预条件构造成本改为 n^2（如不完全 Cholesky），PCG 总成本在什么条件下仍低于 CG？这对选择预条件子有什么启示？

可能的回答：

- 一部分同学只比较迭代次数（230 次 vs 1 次），认为 PCG 一定更快，没有把预条件构造和应用的成本计入；
- 一部分同学能算出 CG 总成本约 $230\times 4=920$，但 PCG 只写“1 步”，漏掉构造 n 和应用 n；
- 对第 (4) 问，一部分同学只会说“构造贵了就不划算”，没有给出具体的成本不等式。

解释：

1. $M=\mathrm{diag}(1,10,100,1000)$，$M^{-1}A=I$，条件数降为 1。
2. $\kappa=1000$，$\sqrt{\kappa}\approx 31.6$，$\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\approx 0.939$；由 $2\times 0.939^k\leqslant 10^{-6}$ 得 $k_{\mathrm{CG}}\approx 230$。PCG 理论 1 步。
3. CG 总成本 $\approx 230\times n=920$；PCG 总成本 $=n$（构造）$+1\times n$（应用）$=8$。PCG 远低于 CG。
4. 若构造成本为 $n^2=16$，PCG 总成本 $=16+4=20$，仍远低于 920。要让 PCG 不划算，需构造成本 $>920-4=916$，即约 $57n^2$，实际中少见。启示：对角预条件构造成本低，通常总成本也低；但不完全 Cholesky 等构造昂贵的预条件子，需要权衡“迭代次数减少”与“构造开销增加”，不能只看迭代次数。

小结

迭代法的核心是收敛性和收敛速度，两者都由迭代矩阵的谱性质决定。4.2 节从线性定常迭代出发，把收敛性归结为谱半径判据，再以 CG 作为对称正定系统的高效迭代法，最后用预条件改善条件数。方法选择可以归纳为以下几步：

1. 先看迭代框架。 线性定常迭代 $x^{(k+1)} = Bx^{(k)} + f$ 的收敛性由谱半径 $\rho(B) < 1$ 决定；$\rho(B)$ 越小，B^k 衰减越快，收敛越快。
2. 再看矩阵结构。 对称正定系统用 CG：它把解方程转化为二次函数最小化，沿 A-共轭方向搜索，在精确算术下至多 n 步收敛；适用条件是 A 对称正定。
3. 最后看条件数。 CG 的收敛速度由 $\kappa(A)$ 支配，$\kappa(A)$ 越接近 1 收敛越快。预条件通过改善预条件后系统的条件数来加速收敛，但改善的是收敛速度，不是问题的病态性；总成本需同时计入迭代次数的减少和预条件子的构造、应用开销。

适用条件与要点： CG 要求 A 对称正定，非对称或非正定时标准 CG 不适用；预条件子 M 需对称正定且 M^{-1} 易计算；残差小不等于解误差小，条件数决定了残差到解误差的放大倍数，讨论“残差小”时必须说明残差针对哪个右端计算。

4.3 特征值与 SVD（2 学时）

本节目标： 能区分两种分解，并用奇异值解释敏感性和低秩近似。

概念讲解：把问题说明白

4.1 和 4.2 分别讨论了求解 $Ax = b$ 的直接法和迭代法。两种方案各有侧重：直接法关心如何稳定、高效地分解矩阵；迭代法关心如何加快并保持收敛速度。但在分析误差时，这两节都有一个概念反复出现——矩阵的条件数。

在直接法中，条件数决定了舍入误差的放大倍数；在迭代法中，条件数支配了 CG 的收敛速度。那么，条件数究竟由矩阵的什么结构决定？为什么同样是 n 阶方阵，有的“轻轻一碰就跳”，有的“怎么扰动都稳”？只看 A 的元素，很难回答；得看 A 在各个方向上能够让空间如何伸缩。这正是特征值和奇异值要刻画的事。

4.3.1 特征值与幂迭代算法

特征值与特征向量是理解矩阵结构的第一把钥匙。回顾基础定义：

> 定义 4.11（特征值与特征向量）
> 对方阵 A，若存在数 λ 和非零向量 v，使得 $Av = \lambda v$，则称 λ 为 A 的特征值，v 称为 λ 的一个特征向量。

如果 n 阶方阵 A 有 n 个线性无关的特征向量，那么 A 可以在这组基下对角化：

$$A = V\Lambda V^{-1} \tag{4.11}$$

特征值具有明确的几何意义：将方阵 A 看作对向量的线性变换时，若作用在 A 的一个特征向量 v 上，则不改变向量的方向，只改变其长度，且伸缩因子为对应的特征值 λ。若 A 可对角化，那么 A 的作用完全可以看作由它在各个特征方向上的伸缩作用的复合。因此，研究方阵 A 对一般向量的作用时，可以把 A 的所有特征向量作为一组基，这样研究矩阵的作用时会更加方便。

特征值在矩阵的相关计算方面最常见的应用是求方阵的幂次。由对角化分解 $A=V\Lambda V^{-1}$ ，即得 $A^n=V\Lambda^nV^{-1}$。我们可以利用这个结果更便捷地计算 A^n，在求解关于 A 的微分方程，以及一些迭代问题的计算中，这种方法非常重要（不可对角化的方阵可以利用 Jordan 标准型计算幂次，但不是本章重点，不再展开讲解）。特征值还可以用于判断矩阵是否奇异（有零特征值则奇异），以及用来分析矩阵的稳定性（谱半径 $\rho(A)=\max\limits_{1\leqslant i\leqslant n}|\lambda_i|$ 和迭代法的收敛性有关）。此外，特征值还有以下两个基本性质：

$$\det(A)=\lambda_1\lambda_2\cdots\lambda_n,\qquad \operatorname{tr}(A)=\lambda_1+\lambda_2+\cdots+\lambda_n. \tag{4.12}$$

这两个式子分别联系了特征值与行列式、迹这些矩阵中的重要概念，在理论分析和实际计算中都十分有用。

进一步地，我们希望研究：矩阵作用对向量的影响主要取决于什么？我们通过研究矩阵对向量反复作用，放大其影响效果的方法来说明这一点。为了方便研究，假设 A 的特征值按模长大小排列为：$|\lambda_1|>|\lambda_2|\geqslant\cdots\geqslant|\lambda_n|$，其中 λ_i 对应特征向量 $v_i(i=1,2,\cdots,n)$，$|\lambda_1|$ 为严格最大特征值。那么对任意初值 x_0，先用 A 的特征向量表示为

$$x_0=c_1v_1+c_2v_2+\cdots+c_nv_n,$$

假设 $c_1\neq0$，反复左乘 A，得到：

$$x_k=A^kx_0=c_1\lambda_1^kv_1+c_2\lambda_2^kv_2+\cdots+c_n\lambda_n^kv_n.$$

k 充分大时，λ_1^k 相对来说远大于其他项，归一化之后，方向会越来越由 v_1 主导。每左乘一次 A，各个特征方向上的分量就按各自的特征值被放大一次。$|\lambda_1|$ 最大，它的分量就涨得最快；乘的次数越来越多，其他方向的分量相对就成了零头。归一化只改长度、不改方向，所以 x_k 的方向会越来越贴近 v_1——就像生物繁衍，增长率最快的种群最终便会压倒其余。

这就是幂迭代的思想：如果反复用矩阵左乘向量，最大特征值对应的向量会逐渐占据主导地位，因此也被称为主特征向量方向。

> 定义 4.12（幂迭代）
> 给定初值 x_0，按 $y_k=Ax_{k-1}$、$x_k=\dfrac{y_k}{\|y_k\|}$ 迭代，用 Rayleigh 商 $\lambda_1^{(k)}\approx\dfrac{x_k^TAx_k}{x_k^Tx_k}$ 估计最大特征值。

$$y_k=Ax_{k-1},\qquad x_k=\frac{y_k}{\|y_k\|},$$

则最大特征值 λ_1 可用 Rayleigh 商估计，第 k 步得到的估计为：

$$\lambda_1^{(k)}\approx\frac{x_k^TAx_k}{x_k^Tx_k}. \tag{4.13}$$

需要强调这里的条件：最大模特征值 λ_1 的模必须严格最大；初值 x_0 在主特征向量方向上的展开系数 c_1 不能为零，可以通过合理选取初值来保证这一点；矩阵最好可对角化，否则不能保证收敛。上述三条中任一条不满足，简单幂迭代就可能失效。

4.3.2 从特征值到奇异值

尽管特征值是一个对于研究矩阵非常重要的概念，特征值分解也还是有两个重要局限：其一，它只适用于方阵；其二，即使对于方阵，也并非所有矩阵都能对角化。因此，我们需要在一般的矩阵上寻找一个具有类似概念和作用的值，并找到一种分解，它对任意矩阵都存在，且像特征值分解一样，能揭示矩阵在若干方向上的伸缩作用的效果。奇异值分解（Singular Value Decomposition，SVD）克服了特征值的上述局限：

> 定义 4.13（奇异值分解，SVD）
> 对任意 $m \times n$ 实矩阵 A，都存在分解

$$\boxed{A = U\Sigma V^T} \tag{4.14}$$

> 其中 U 为 m 阶正交矩阵，V 为 n 阶正交矩阵，Σ 为 $m \times n$ 矩形对角矩阵，对角元 $\sigma_1 \geqslant \sigma_2 \geqslant \cdots \geqslant \sigma_{\min(m,n)} \geqslant 0$ 称为奇异值。

这一分解对任意矩阵都存在，不要求方阵、可逆或对称。正交矩阵 U 和 V 的列向量分别称为左奇异向量和右奇异向量。

SVD 的几何含义是：任意矩阵 A 的作用可以分解为三步——先用 V^T 旋转（或反射）输入空间，再用 Σ 沿坐标轴伸缩，最后用 U 旋转输出空间。因此，若 A 把单位球映射为一个椭球，则椭球的半轴长度就对应着矩阵的奇异值。

奇异值与特征值之间有明确的联系。对矩阵 $A = U\Sigma V^T$，把 SVD 代入 A^TA 和 AA^T，直接展开就得到了右、左奇异向量：

$$\begin{aligned} A = U\Sigma V^T \Longrightarrow\ & A^TA = V\Sigma^T\Sigma V^T \ \Longrightarrow\ \text{特征值 } \sigma_i^2\text{, 特征向量 } V \text{ 的列;} \\ A = U\Sigma V^T \Longrightarrow\ & AA^T = U\Sigma\Sigma^T U^T \ \Longrightarrow\ \text{特征值 } \sigma_i^2\text{, 特征向量 } U \text{ 的列.} \end{aligned}$$

因此在理论上，奇异值可以通过求出 A^TA 的特征值再开方得到。

不过，数值计算中通常不直接计算 A^TA 的特征值，因为这样会很容易损失精度：A^TA 的条件数是 A 的条件数的平方，舍入误差会被放大。实际使用的是专门的 SVD 算法，一般是直接对 A 做正交变换，避免形成 A^TA。但 A^TA 的特征值分解在理论分析中仍然重要，它建立了 SVD 与对称矩阵特征值分解的联系。这一联系为 SVD 的数值计算提供了理论基础。更重要的是，奇异值总为非负实数，且左、右奇异向量各自构成正交系，这比特征向量（可能复数、可能不正交）具有更良好的性质。

幂迭代方法的目标是寻找方阵最大特征值的方向，而 SVD 可看作把“找主方向”的思想推广到任意矩阵：右奇异向量 v_i 是 A^TA 的特征向量，左奇异向量 u_i 是 AA^T 的特征向量，奇异值 σ_i 是 A^TA 特征值的平方根。因此 SVD 同时给出输入空间和输出空间的一对主方向，比特征值分解更通用。

秩与奇异值之间有直接的关系。设矩阵 A 的秩 $r = \mathrm{rank}(A)$，则 A 非零奇异值的个数即为 r。若从数值计算角度出发，当某些奇异值极小时，数值运算中会被视为 0，因此矩阵的“数值秩”可能小于理论秩。

条件数与奇异值之间也有紧密的关系。设 $A = U\Sigma V^T$ 可逆，则 $A^{-1} = V\Sigma^{-1}U^T$，奇异值为 $\dfrac{1}{\sigma_i}(i = 1, 2, \cdots, n)$。因此

$$\|A\|_2 = \sigma_1, \qquad \|A^{-1}\|_2 = \frac{1}{\sigma_{\min}}, \qquad \kappa_2(A) = \|A\|_2\|A^{-1}\|_2 = \frac{\sigma_1}{\sigma_{\min}}.$$

特别地，当 A 为对称方阵时，

$$\kappa_2(A) = \frac{\max\limits_{1\leqslant i\leqslant n} |\lambda_i|}{\min\limits_{1\leqslant i\leqslant n} |\lambda_i|}.$$

可以由上式推出条件数的一些基本性质，在处理相关问题时很有用：

1. 对任何非奇异矩阵 A，都有 $\kappa(A) \geqslant 1$；
2. 对任何非零常数 c，$\kappa(cA) = \kappa(A)$；
3. 若 Q 为正交矩阵，则 $\kappa(QA) = \kappa(AQ) = \kappa(A)$。

性质 3 说明，正交变换不改变条件数，这也是 SVD 数值稳定的原因之一：SVD 只使用正交变换。

需要特别强调：非对称矩阵的特征值绝对值一般不等于奇异值。比如取 $J = \begin{pmatrix} 1 & 10 \\ 0 & 1 \end{pmatrix}$，则 J 是上三角矩阵，特征值为主对角元，因此两个特征值均为 1。而 $J^TJ = \begin{pmatrix} 1 & 10 \\ 10 & 101 \end{pmatrix}$，其特征值为 $51 \pm 10\sqrt{26}$，开方得 J 的奇异值约为 $\sigma_1 \approx 10.09902, \sigma_2 \approx 0.09902$。

显然 J 的奇异值不等于其特征值的绝对值。实对称矩阵确有“奇异值等于特征值绝对值”的性质，更一般的正规矩阵也有相应结论，但不能推广到任意非对称矩阵。这个反例说明：SVD 给出的奇异值不是特征值的简单推广，而是通过 A^TA 的特征值来定义的另一套谱信息。非对称矩阵的特征值绝对值不能直接刻画矩阵在所有方向上的伸缩，这正是需要 SVD 的原因。

4.3.3 从奇异值角度重新看矩阵

奇异值分解与敏感性的联系是本节的核心。考虑 $Ax = b$，A 可逆。若右端有扰动 δb，解的变化满足 $A\delta x = \delta b$，即 $\delta x = A^{-1}\delta b$。将 SVD 代入，$A^{-1} = V\Sigma^{-1}U^T$。扰动量 δb 若在左奇异向量 u_i 方向上有分量，则解中该分量被放大 $\dfrac{1}{\sigma_i}$ 倍。特别地，沿最小奇异值 $\sigma_{\min}$ 对应的左奇异向量方向上的扰动会被放大最多 $\dfrac{1}{\sigma_{\min}}$ 倍，这是最坏情况下的放大。矩阵在 $\sigma_{\min}$ 对应的方向上压缩效果最明显，几乎把向量压成零；而反推回来时，这个方向上的微小扰动就被放得最大。换句话说，$\sigma_{\min}$ 指向矩阵“最薄弱的环节”：沿这个方向轻轻一推，解就晃得最厉害。因此，最小的奇异值对应着最敏感的方向。扰动放大的推导图如下：

$$\begin{aligned} A\delta x = \delta b &\Longrightarrow \delta x = A^{-1}\delta b \\ &\Longrightarrow \delta x = V\Sigma^{-1}U^T\delta b \\ &\Longrightarrow \delta b \text{ 在 } u_i \text{ 方向的分量被放大 } \frac{1}{\sigma_i} \text{ 倍} \\ &\Longrightarrow \text{最坏情况沿 } u_{\min}\text{，放大 } \frac{1}{\sigma_{\min}} \text{ 倍.} \end{aligned}$$

当 $\sigma_{\min}$ 非常小时，A 接近奇异，矩阵的作用在某个方向上几乎把向量压缩为零，因此逆运算在该方向上的放大极大。下面用手算核验这一放大关系。取

$$A = \operatorname{diag}(1, 10^{-10}).$$

特征值：对角矩阵的特征值就是对角元，因此

$$\lambda_1 = 1, \qquad \lambda_2 = 10^{-10}.$$

奇异值：对角矩阵的奇异值等于对角元的绝对值，因此

$$\sigma_1 = 1, \qquad \sigma_2 = 10^{-10}.$$

对这个对角矩阵，特征值绝对值等于奇异值。条件数

$$\kappa_2(A) = \frac{\sigma_1}{\sigma_2} = 10^{10}.$$

右端第二分量增加 10^{-10} 后，精确解为 $\tilde{x} = (1, 2)^T$，而原真解为 $x^* = (1, 1)^T$，因此

$$\|\tilde{x} - x^*\|_2 = 1, \qquad \frac{\|\tilde{x} - x^*\|_2}{\|x^*\|_2} = \frac{1}{\sqrt{2}} \approx 0.707107.$$

对原右端的相对残差为 10^{-10}，对扰动后右端的残差为 0。因此，小残差与小解误差是不同主张，残差针对哪个右端必须说明。

设 A 可对角化，$A = V \Lambda V^{-1}$。若 A 有扰动 δA，则特征值扰动满足

$$|\lambda_i(A + \delta A) - \lambda_i(A)| \leqslant \|V\| \, \|V^{-1}\| \, \|\delta A\|_2.$$

这里 $\|V\| \, \|V^{-1}\|$ 是特征向量矩阵的条件数。对对称矩阵，V 正交，$\|V\| \, \|V^{-1}\| = 1$，特征值对扰动不敏感；对非对称矩阵，特征向量矩阵可能病态，特征值对扰动敏感。这进一步说明 SVD 的优势：奇异值对扰动不敏感，因为 SVD 只使用正交变换。

类似地，若系数矩阵本身有扰动 δA，解的相对误差上界为

$$\boxed{\frac{\|\delta x\|}{\|x\|} \leqslant \kappa_2(A) \frac{\|\delta A\|}{\|A\|}} \tag{4.15}$$

这一估计式与 4.2 节中残差与解误差的关系式形式一致，条件数总是出现在右端作为放大因子。因此，条件数不仅决定迭代法的收敛速度，也决定问题对输入扰动的敏感程度。它是矩阵结构的一个内在量，与具体的算法无关。

4.3.4 矩阵的低秩近似

截断奇异值分解与低秩近似是 SVD 的另一个核心应用。将 SVD 按列写开，写成外积形式：

$$A = \sum_{i=1}^{r} \sigma_i u_i v_i^T,$$

其中 $r = \operatorname{rank}(A)$。每一项是一个秩为 1 的矩阵，σ_i 给出该项的“权重”。保留前 k 个最大的奇异值及其对应的奇异向量，得到

$$A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T.$$

> 定义 4.14（截断 SVD，低秩近似）
> 保留前 k 个最大的奇异值及其对应的奇异向量，得到 $A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T$，其秩不超过 k，称为截断 SVD（或低秩近似）。

> 定理（Eckart – Young – Mirsky）[3]：在 Frobenius 范数和谱范数下，截断 SVD A_k 是所有秩不超过 k 的矩阵中与 A 最接近的，误差分别为

$$\|A-A_k\|_F^2=\sum_{i=k+1}^{r}\sigma_i^2,\qquad \|A-A_k\|_2=\sigma_{k+1}.$$

这一结论说明，奇异值的衰减速度决定了低秩近似的效果：若奇异值迅速衰减，则较小的 k 就能捕捉 A 的主要结构；若奇异值衰减缓慢，则需要较大的 k 才能达到相同的误差。这为 4.4 节中的 PCA 和 LoRA 提供了理论基础：PCA 用截断 SVD 对中心化数据降维，LoRA 用低秩增量近似参数更新，二者的有效性都依赖于奇异值（或相应结构）的衰减假设。

Eckart-Young-Mirsky 定理的证明思路。 设

$$A_k=\sum_{i=1}^{k}\sigma_i u_i v_i^T.$$

则

$$A-A_k=\sum_{i=k+1}^{r}\sigma_i u_i v_i^T.$$

由于 $\{u_i\}$ 和 $\{v_i\}$ 均为正交系，

$$\|A-A_k\|_F^2=\sum_{i=k+1}^{r}\sigma_i^2,\qquad \|A-A_k\|_2=\sigma_{k+1}.$$

在秩不超过 k 的矩阵集合中，A_k 使上述两个范数同时达到最小。

谱范数的证明思路是：对任意秩不超过 k 的矩阵 B，子空间 $\mathrm{span}\{v_1,\cdots,v_{k+1}\}$ 的维数为 $k+1$，故 B 在该子空间上的零空间维数至少为 $(k+1)-k=1$，即存在非零向量 $z\in\mathrm{span}\{v_1,\cdots,v_{k+1}\}$ 使 $Bz=0$；将其归一化，即得单位向量 z。于是

$$\|A-B\|_2\geqslant\|(A-B)z\|_2=\|Az\|_2\geqslant\sigma_{k+1}.$$

而 A_k 恰好达到该下界。Frobenius 范数的证明类似，利用正交投影和尾部平方和。另一种代数证明的思路是令 $C=U^TBV$，则 $\|A-B\|_2=\|\Sigma-C\|_2\geqslant\sigma_{k+1}$，当 $C=\mathrm{diag}(\sigma_1,\ldots,\sigma_k,0,\ldots,0)$ 时取等（对应 $B=A_k$），这里不再详细展开。

低秩近似的现实意义 截断 SVD 把矩阵拆成按重要性排序的若干模式，大奇异值对应主要结构，小奇异值则对应细节或噪声。图像压缩只保留前 k 个奇异值 $\sigma_1,\ldots,\sigma_k$ 及对应的奇异向量 u_i,v_i，用远少于原始像素的数据近似原图；奇异值衰减越快，压缩效果越好。数据降噪则丢掉分散噪声的小奇异值再重构，保留主要信号。这两类用途都依赖奇异值快速衰减，4.4 节将说明低秩思想在数据降维等场景中的应用。秩与条件数已在 4.3.2 和 4.3.3 涉及。

关键推导：让结论有依据

推导 1：奇异值分解的计算实例 取

$$A=\begin{pmatrix}3&0\\4&5\end{pmatrix},$$

对它做奇异值分解 $A=U\Sigma V^T$。

第一步：计算 A^TA。

$$A^TA=\begin{pmatrix}3&4\\0&5\end{pmatrix}\begin{pmatrix}3&0\\4&5\end{pmatrix}=\begin{pmatrix}25&20\\20&25\end{pmatrix}.$$

第二步：求特征值和奇异值。 解

$$\det(A^TA-\lambda I)=(25-\lambda)^2-400=0,$$

得到 $\lambda_1=45$，$\lambda_2=5$，因此奇异值为

$$\sigma_1=\sqrt{45}=3\sqrt{5},\qquad \sigma_2=\sqrt{5},$$

按从大到小排列为 $\Sigma=\operatorname{diag}(3\sqrt{5},\sqrt{5})$。

第三步：求右奇异向量。 对应 $\lambda_1=45$ 的单位特征向量为

$$v_1=\frac{1}{\sqrt{2}}\begin{pmatrix}1\\1\end{pmatrix},$$

对应 $\lambda_2=5$ 的单位特征向量为

$$v_2=\frac{1}{\sqrt{2}}\begin{pmatrix}1\\-1\end{pmatrix},$$

因此

$$V=\begin{pmatrix}\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}}\\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}}\end{pmatrix}.$$

第四步：求左奇异向量。 利用 $u_i=\dfrac{Av_i}{\sigma_i}$：

$$u_1=\frac{Av_1}{\sigma_1}=\begin{pmatrix}\frac{1}{\sqrt{10}}\\ \frac{3}{\sqrt{10}}\end{pmatrix},\qquad u_2=\frac{Av_2}{\sigma_2}=\begin{pmatrix}\frac{3}{\sqrt{10}}\\ -\frac{1}{\sqrt{10}}\end{pmatrix},$$

所以

$$U=\begin{pmatrix}\frac{1}{\sqrt{10}} & \frac{3}{\sqrt{10}}\\ \frac{3}{\sqrt{10}} & -\frac{1}{\sqrt{10}}\end{pmatrix}.$$

第五步：写出 SVD 结果并核验。

$$A=\begin{pmatrix}\frac{1}{\sqrt{10}} & \frac{3}{\sqrt{10}}\\ \frac{3}{\sqrt{10}} & -\frac{1}{\sqrt{10}}\end{pmatrix}\begin{pmatrix}3\sqrt{5} & 0\\ 0 & \sqrt{5}\end{pmatrix}\begin{pmatrix}\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}}\\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}}\end{pmatrix}=\begin{pmatrix}3 & 0\\ 4 & 5\end{pmatrix}.$$

第六步：秩 1 近似与误差。 只保留最大奇异值，得到

$$A_1=\sigma_1u_1v_1^T=3\sqrt{5}\begin{pmatrix}\frac{1}{\sqrt{10}}\\ \frac{3}{\sqrt{10}}\end{pmatrix}\begin{pmatrix}\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}}\end{pmatrix}=\begin{pmatrix}1.5 & 1.5\\ 4.5 & 4.5\end{pmatrix}.$$

原始矩阵与近似矩阵之差为

$$A-A_1=\begin{pmatrix}1.5 & -1.5\\ -0.5 & 0.5\end{pmatrix},$$

其 Frobenius 范数误差为

$$\|A-A_1\|_F=\sqrt{1.5^2+(-1.5)^2+(-0.5)^2+0.5^2}=\sqrt{5}\approx 2.236,$$

这正好等于第二个奇异值 σ_2。原始矩阵的总能量为 $\sigma_1^2+\sigma_2^2=45+5=50$：只保留第一个奇异值后，保留的能量为 45，能量保留比例为 $\frac{45}{50}=90\%$。也就是说，只用一个主要模式，就保留了原矩阵约 90% 的信息。这一结果与本节截断 SVD 的误差公式 $\|A-A_1\|_F=\sigma_2$ 一致，也说明奇异值衰减速度决定低秩近似的效果。

实验设计：先预测，再运行

实验任务：运行现有敏感性例子，再对小矩阵做不同秩截断，比较保留奇异值与重构误差。

要填写的项目	本组内容
要回答的问题与运行前预测	研究矩阵条件数如何放大右端扰动对解的影响，并区分残差小与解误差小；验证非对称剪切矩阵的特征值与奇异值并不相同；考察截断 SVD 的秩、重构误差和保留能量之间的关系。预测：条件数越大，固定右端扰动造成的解误差越大；秩越高，截断误差越小、保留能量越高。
输入、参数、单位、数据来源与划分	敏感性例：$A_{43}=\mathrm{diag}(1,10^{-10})$、真解 $x^*=(1,1)^T$、原右端 $b=A_{43}x^*$，扰动 $\delta b=(0,10^{-10})^T$；另将小对角元依次设为 $10^{-2},10^{-4},\dots,10^{-14}$ 做条件数扫描。特征值例：剪切矩阵 $J=\begin{pmatrix}1&10\\0&1\end{pmatrix}$；幂迭代例 $K=\mathrm{diag}(5,2,-1)$。SVD 主例为 8×6 合成矩阵，预设奇异值 $(8,4,2,1,0.5,0.1)$，正交因子由种子 `2026` 的随机矩阵 QR 分解构造。全部为无量纲合成数据，无数据集划分。
独立参考解/基线及选择理由	敏感性扫描使用解析解误差公式 $\frac{10^{-10}}{\varepsilon\sqrt{2}}$ 作独立预测；剪切矩阵使用解析特征值 $(1,1)$ 和解析奇异值作基准；SVD 使用预先指定的奇异值谱及截断尾部奇异值公式作为参考，不仅依赖数值分解本身。
误差指标、容差及其依据	敏感性例记录 $\mathrm{cond}_2(A)$、相对于原右端的相对残差、相对于扰动右端的相对残差及相对解误差；扫描解析公式比较采用 $\mathrm{rtol}=10^{-7}$、$\mathrm{atol}=10^{-14}$。SVD 记录谱范数误差、Frobenius 范数误差和保留能量；截断误差与解析尾部比较的绝对容差为 2×10^{-12}。幂迭代记录特征值残差 $\|Kv-\lambda v\|_2$。
环境、种子、设备与运行预算	Python `3.12.10`、NumPy `1.26.4`、SciPy `1.14.1`；CPU、`float64`、种子 `2026`，线程数限制为 1。矩阵规模最大为 8×6（敏感性扫描为 2×2）；SVD 截断秩从 0 到 6。

结果解释与本节结论

（1）病态问题与扰动放大

对 $A_{43}=\mathrm{diag}(1,10^{-10})$，$\mathrm{cond}_2(A)=10^{10}$。原真解为 $(1,1)^T$，右端增加 $\delta b=(0,10^{-10})^T$ 后得到解 $(1,2)^T$，相对解误差为 0.70710678。相对于原右端的相对残差为 10^{-10}，但相对于扰动后的右端残差为 0。这说明“对当前输入方程残差为零”并不代表“相对于原始真解误差很小”。

固定 $\delta b_2=10^{-10}$ 时，条件数扫描结果如下：

$\mathrm{cond}_2(A)$	相对解误差（实测）	解析预测
1×10^{2}	7.07107×10^{-9}	7.07107×10^{-9}
1×10^{4}	7.07107×10^{-7}	7.07107×10^{-7}
1×10^{6}	7.07107×10^{-5}	7.07107×10^{-5}
1×10^{8}	0.00707107	0.00707107
1×10^{10}	0.707107	0.707107
1×10^{12}	70.7107	70.7107
1×10^{14}	7071.07	7071.07

实测值与解析预测一致，验证了病态性会放大输入扰动对解的影响。

（2）特征值与奇异值、幂迭代

矩阵 $J=\begin{pmatrix}1 & 10\\ 0 & 1\end{pmatrix}$ 的特征值为 $(1,1)$，奇异值约为 $(10.09901951, 0.09901951)$，二者明显不同。对 $K=\mathrm{diag}(5,2,-1)$，从具有最大特征向量非零投影的初值进行 30 次幂迭代，得到主特征值 5.0，特征残差约 3.46×10^{-12}；若初值为 $(0,1,0)^T$，其始终落在第二特征方向，得到 2.0，说明幂迭代的结果受初始向量是否包含主特征方向分量影响。

（3）截断 SVD

保留秩 r	谱范数重构误差	Frobenius 重构误差	保留能量比例
0	8.0	9.23363	0
1	4.0	4.61086	0.750645
2	2.0	2.29347	0.938306
3	1.0	1.12250	0.985222
4	0.5	0.509902	0.996951
5	0.1	0.1	0.999883
6	6.84056×10^{-15}	7.17281×10^{-15}	1.0

随着保留秩增加，重构误差单调下降、保留能量上升；秩 2 已保留约 93.83% 能量，秩 3 约 98.52%，秩 4 约 99.70%。满秩时理论截断误差为 0，实测约 10^{-15} 的残差来自浮点舍入。结果与截断 SVD 理论一致。尚未验证真实图像与文本数据上的降维效果，也未对不同噪声水平作比较。

易错点与讨论

1. 特征值与奇异值的区别：特征值只适用于方阵，奇异值适用于任意矩阵；非对称矩阵的特征值绝对值一般不等于奇异值；特征向量可能不正交，奇异向量总是正交的。反例：$J=\begin{pmatrix}1 & 10\\ 0 & 1\end{pmatrix}$，特征值全为 1，奇异值约为 10.09902 和 0.09902。

2. 低秩近似与预测的区别：低秩近似是在 Frobenius 或谱范数意义下的最佳逼近，但不代表能准确预测未知数据或泛化到分布之外。低秩假设的有效性依赖于数据的低秩结构。
3. 数值计算中的 SVD：不应直接计算 A^TA 的特征值来求奇异值，因为这会损失精度；A^TA 的条件数是 A 的条件数的平方。实际使用的 SVD 算法直接对 A 做正交变换。
4. 条件数与敏感性的关系：最小奇异值对应最敏感方向，条件数是扰动放大倍数的上界。病态问题对扰动敏感，这是问题本身的固有性质，与算法无关。
5. 幂迭代的适用条件：幂迭代要求主导模特征值严格最大、初值在主特征方向展开系数不为零、矩阵最好可对角化。若主导模并列、矩阵有缺陷或初值投影为零，简单幂迭代不适用。

随堂提问： 判断：“对任意方阵，奇异值等于特征值的绝对值。”若正确，说明理由；若错误，举一个反例，并在同一矩阵上分别算出特征值与奇异值。

可能的回答：

- 一部分同学举对称矩阵为例，得到“奇异值等于特征值绝对值”，误以为命题正确；实际上对称矩阵恰好是两者相等的情形，不能用来判断一般命题。
- 一部分同学举出非对称矩阵，但只写特征值，没有算奇异值，无法说明“不等于”。
- 还有同学把“特征值只适用于方阵”当作反例本身，但这只说明适用范围不同，没有给出同一矩阵上两个量不相等的具体数值。

解释： 命题错误，反例要选非对称矩阵。例如 $A=\begin{pmatrix}0&1\\0&0\end{pmatrix}$，它是上三角矩阵，特征值为主对角元，故两个特征值均为 0；而 $A^TA=\begin{pmatrix}0&0\\0&1\end{pmatrix}$，特征值为 0 和 1，开方得奇异值为 0 和 1，与特征值 $0,0$ 不同。这个反例说明“特征值全为 0”不意味着“奇异值全为 0”。对称矩阵不能作反例，因为对称矩阵的奇异值等于特征值绝对值；只写“特征值只适用于方阵”也不够，因为那不涉及同一矩阵上两个量的数值比较。

小结

特征值分解和 SVD 是理解矩阵结构的两种工具。4.3 节从特征值出发，用幂迭代求主特征方向，再把“找主方向”的思想推广到任意矩阵得到 SVD，最后用奇异值解释扰动敏感性和低秩近似。方法选择可以归纳为以下几步：

1. 先看特征值。 特征值适用于方阵，刻画矩阵在特征方向上的伸缩；可对角化时 $A=V\Lambda V^{-1}$。幂迭代可求主特征方向，条件是主导模特征值严格最大、初值在主特征方向投影非零、矩阵最好可对角化。
2. 再看 SVD。 SVD 对任意实矩阵存在，$A=U\Sigma V^T$，奇异值 $\sigma_i\geqslant 0$；与特征值的关系是 A^TA 的特征值为 σ_i^2、特征向量为 V 的列。数值上不直接计算 A^TA，而用正交变换，避免条件数平方。
3. 再看敏感性。 最小奇异值对应最敏感方向，$\kappa_2(A)=\frac{\sigma_1}{\sigma_{\min}}$；右端扰动或系数矩阵扰动的放大上界都含 $\kappa_2(A)$。条件数是问题固有属性，与算法无关。
4. 最后看低秩近似。 截断 SVD $A_k=\sum_{i=1}^{k}\sigma_iu_iv_i^T$ 是秩不超过 k 的最佳逼近，误差 $\|A-A_k\|_F^2=\sum_{i=k+1}^{r}\sigma_i^2$、$\|A-A_k\|_2=\sigma_{k+1}$；奇异值衰减速度决定低秩近似效果，为 4.4 的 PCA 和 LoRA 做铺垫。

适用条件与要点： 特征值分解要求方阵且可对角化；SVD 对任意实矩阵存在；幂迭代要求主导模严格最大、初值投影非零、最好可对角化；条件数依赖所选范数，本章默认 2-范数 $\kappa_2(A)=\frac{\sigma_1}{\sigma_{\min}}$；非对称矩阵的特征值绝对值一般不等于奇异值，不能混用；截断 SVD 的最优性只在 Frobenius 或谱范数意义下成立，不代表能准确预测未知数据。

4.4 AI关联：可微求解器、PCA、LoRA（2学时）

本节目标： 能说明矩阵方法与学习模型的连接及其边界。

概念讲解：把问题说明白

4.1 节，4.2 节以及 4.3 节分别介绍了矩阵分解、迭代法和谱结构，它们回答的是“如何高效求解 $Ax=b$ ”和“如何理解矩阵结构”。但这些工具不只用于解方程：在机器学习中，同样的矩阵方法以三种方式出现——可微线性求解把“解方程”嵌入训练流程，PCA 用 SVD 做无监督降维，LoRA 用低秩近似做参数高效微调。

本节依次介绍这三者，并在 4.4.4 说明它们如何在同一框架下连接：PCA 发现数据的主方向，LoRA 在这些主方向上做低秩截断，可微线性求解提供高效且可微的数值手段。三者的共同基础，正是 4.3 节的 SVD 与低秩近似。

适用范围

- 可微线性求解：适用于需要把线性方程组求解嵌入神经网络、做端到端训练的场景。
- PCA：适用于中心化数据、关心方差结构、想做无监督降维的场景。
- LoRA：适用于大模型微调、参数预算有限、任务增量近似低秩的场景。

4.4.1 可微线性求解

传统数值求解器（如解 $Ax=b$）只关心“算得快不快、准不准”，不关心“梯度能不能传回去”。可微求解器把求解过程变成一个可微函数，使得损失函数的梯度可以穿过整个求解过程，回流到上游的网络参数或输入中。两者的区别可以落在“输出什么”上：

> 传统求解器：只返回解 x，不提供 x 对参数 θ 的依赖关系。
> 可微求解器：返回解 x，同时返回梯度 $\frac{\partial L}{\partial \theta}$，使求解过程可嵌入端到端训练。

有了这份梯度，θ 才能被下游损失推动，进而改变 A 和 b，让最终的 x 在下游任务上表现更好。

> 定义 4.15（可微线性求解）
> 设线性系统的系数矩阵和右端依赖参数 θ，即 $A(\theta)x=b(\theta)$。前向求解 x 后，利用隐函数定理把损失 $L(x)$ 对 θ 的梯度传回上游，使整个“求解—传梯度”过程可嵌入端到端训练；这一过程称为可微线性求解。

可微求解器：前向求解与反向伴随

下面给出可微线性求解的核心公式。

给定线性系统 $A(\theta)x = b(\theta)$，前向解出 x。前向结束后，把 x 送入损失 $L(x)$。反向时，我们需要 $\frac{\partial L}{\partial \theta} = \frac{\partial L}{\partial x} \cdot \frac{\partial x}{\partial \theta}$，才能把损失梯度传回 θ。

由 $F(x,\theta) = A(\theta)x - b(\theta) = 0$，解 x 处用隐函数定理：

$$\frac{\partial x}{\partial \theta} = -A^{-1}\frac{\partial F}{\partial \theta} = -A^{-1}\left(\frac{\partial A}{\partial \theta}x - \frac{\partial b}{\partial \theta}\right)$$

令 $g = \frac{\partial L}{\partial x}$，于是

$$\frac{\partial L}{\partial \theta} = g^T\frac{\partial x}{\partial \theta} = -g^TA^{-1}\left(\frac{\partial A}{\partial \theta}x - \frac{\partial b}{\partial \theta}\right)$$

为了避免显式求 A^{-1}，令 $A^Tz = g$

解出伴随变量 z。推导路线如下：

$$\begin{aligned} F(x,\theta) = A(\theta)x - b(\theta) = 0 &\xrightarrow{\text{隐函数定理}} \frac{\partial x}{\partial \theta} = -A^{-1}\frac{\partial F}{\partial \theta} \\ &\xrightarrow{g=\partial L/\partial x} \frac{\partial L}{\partial \theta} = -g^TA^{-1}\frac{\partial F}{\partial \theta} \\ &\xrightarrow{A^Tz=g} \frac{\partial L}{\partial \theta} = -z^T\frac{\partial F}{\partial \theta}. \end{aligned}$$

则

$$\boxed{\frac{\partial L}{\partial \theta} = -z^T\left(\frac{\partial A}{\partial \theta}x - \frac{\partial b}{\partial \theta}\right)} \tag{4.16}$$

这就是可微线性求解的核心公式。

反向只需要再解一次线性系统 $A^Tz = g$，系数矩阵是原来的转置。前向解一次，反向解一次，计算代价大约是前向的两倍。

实际实现时，$\frac{\partial A}{\partial \theta}$ 和 $\frac{\partial b}{\partial \theta}$ 通常不显式构造，而是用自动微分的向量-雅可比积（VJP）计算 $z^T\frac{\partial F}{\partial \theta}$。

下面给出可微线性求解的一个具体例子

> 例　设 $A(\theta) = \begin{pmatrix}\theta & 0\\ 0 & 1\end{pmatrix}$，$b = \begin{pmatrix}1\\ 2\end{pmatrix}$，取损失 $L(x) = x_1^2$，求损失 $L(x)$ 对 θ 的梯度。

解　由 $A(\theta)=\begin{pmatrix}\theta & 0\\ 0 & 1\end{pmatrix}$，$b=\begin{pmatrix}1\\ 2\end{pmatrix}$，得到 $x=\begin{pmatrix}\frac{1}{\theta}\\ 2\end{pmatrix}$

取损失 $L(x)=x_1^2$，于是

$$g=\frac{\partial L}{\partial x}=\begin{pmatrix}2x_1\\ 0\end{pmatrix}=\begin{pmatrix}\frac{2}{\theta}\\ 0\end{pmatrix}$$

反向解 $A^Tz=g$，因为 A 对称，所以

$$z=\begin{pmatrix}\frac{2}{\theta^2}\\ 0\end{pmatrix}$$

再算

$$\frac{\partial A}{\partial \theta}=\begin{pmatrix}1 & 0\\ 0 & 0\end{pmatrix}$$

因此

$$\frac{\partial L}{\partial \theta}=-z^T\left(\frac{\partial A}{\partial \theta}x\right)=-\begin{pmatrix}\frac{2}{\theta^2} & 0\end{pmatrix}\begin{pmatrix}1 & 0\\ 0 & 0\end{pmatrix}\begin{pmatrix}\frac{1}{\theta}\\ 2\end{pmatrix}=-\frac{2}{\theta^3}$$

直接对 $L=\frac{1}{\theta^2}$ 求导，也得到 $-\frac{2}{\theta^3}$，一致。这个例子说明：可微线性求解通过解一次转置系统，就得到了对 θ 的梯度。

可微线性求解常用于：

- 可微物理模拟：解 $K(\theta)u=f(\theta)$，训练材料参数；
- 可微优化：KKT 系统线性化后求解；
- 隐式神经网络、Deep Equilibrium Models；
- 可微渲染、可微控制、元学习；
- 任何“先解线性系统，再算损失”的端到端流程。

前提是 A 可逆或至少可稳定求解。若 A 奇异或病态，反向系统会不稳定，需要正则化、伪逆或特殊处理。

综合来讲，可微求解器的流程如下所示。

$$\text{前向：}\quad \theta\ \to\ A(\theta),b(\theta)\ \xrightarrow{\text{解 } Ax=b}\ x\ \to\ \text{Loss}$$

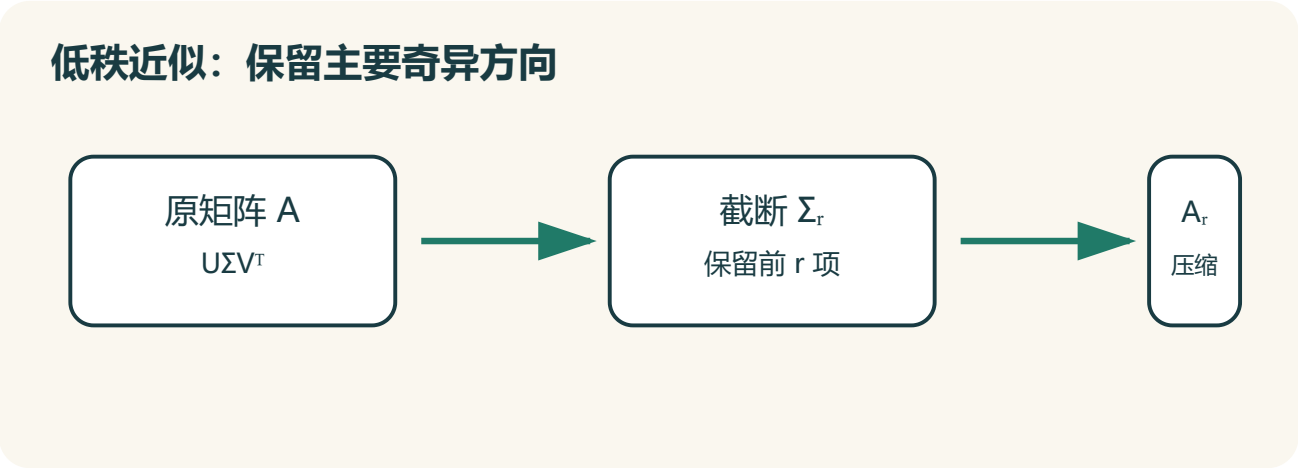
$$\text{反向：}\quad \frac{\partial \text{Loss}}{\partial x}\ \xrightarrow{\text{解 } A^Tz=g}\ z,\qquad \frac{\partial \text{Loss}}{\partial \theta}=-z^T\frac{\partial(Ax-b)}{\partial \theta}$$

下面给出一张对比表格，以便读者更好地理解可微线性求解。

方面	普通线性求解	可微线性求解
前向	解 $Ax=b$	解 $Ax=b$
反向	不提供	解 $A^Tz=g$，算 $\frac{\partial L}{\partial \theta}$
目标	数值精度	端到端训练
迭代展开	不关心	通常不展开，用隐函数定理
内存	小	只存解和伴随变量，不存所有迭代

4.4.2 中心化数据的 PCA

降维是指把高维数据映射到低维空间，同时尽量保留重要信息。它是文件压缩、图像处理等场景中的常用处理手段。



降维通常分为有监督降维和无监督降维两种，其中有标签降维保留对预测标签 y 有用的信息；无标签降维没有对目标的要求，只保留数据自身的结构信息。

> 定义 4.16（中心化数据矩阵）
> 把原始数据矩阵的每一列（或每一维特征）减去该列的均值后得到的矩阵，称为中心化数据矩阵。

> 定义 4.17（主成分分析，PCA）
> 给定中心化数据矩阵 $\tilde{X}$，主成分分析寻找一组正交方向 $v_1, v_2, \ldots$，使得数据投影到这些方向上的方差依次最大；保留前 k 个主成分，即把数据投影到 $v_1, \ldots, v_k$ 张成的子空间，实现从高维到低维的降维。

下面将逐步对 PCA 方法进行介绍。

假设有 n 个样本，每个样本有 d 个特征。通常把数据排成 $X \in \mathbb{R}^{d \times n}$，每一列是一个样本，每一行是一个特征维度：

$$X = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & & \vdots \\ x_{d1} & x_{d2} & \cdots & x_{dn} \end{pmatrix}$$

第 i 行是第 i 个特征在所有样本上的取值，则第 i 个特征的均值：

$$\mu_i = \frac{1}{n} \sum_{j=1}^{n} x_{ij}$$

把所有特征的均值排成向量：

$$\mu = \begin{pmatrix} \mu_1 \\ \mu_2 \\ \vdots \\ \mu_d \end{pmatrix} \in \mathbb{R}^d$$

对每个样本、每个特征，减去对应特征的均值：

$$\tilde{x}_{ij} = x_{ij} - \mu_i$$

写成矩阵形式：

$$\tilde{X} = X - \mu \mathbf{1}^T$$

其中 $\mathbf{1} \in \mathbb{R}^n$ 是全 1 向量。$\tilde{X}$ 就是中心化数据矩阵。它满足：每一行的均值都是 0，即 $\frac{1}{n}\sum_{j=1}^{n} \tilde{x}_{ij} = 0, i = 1, \ldots, d$

在中心化之后，数据点群的“胖瘦”就由方差来量。方差最大的方向，是这群点散得最开的朝向。沿这个方向看，点与点之间的差别最明显。降维时先保留这个方向，再保留第二散的方向，就能用最少的维度留住最多的差别。因此，PCA 寻找的是方差最大的方向。方差定义为：

$$\mathrm{Var}(v^T x) = \frac{1}{n}\sum_{j=1}^{n} \left(v^T x_j - \overline{v^T x}\right)^2$$

如果数据中心不在原点，PCA 会混入均值信息，第一主成分可能只是指向“均值方向”，即从坐标原点到数据均值点的方向，而不是数据波动最大的方向。简单来说，第一主成分可能反映“数据整体离原点有多远”，而不是“数据内部差异有多大”。

中心化后，$\overline{\tilde{x}} = 0$，协方差矩阵变成简单的：

$$C = \frac{1}{n}\tilde{X}\tilde{X}^T$$

PCA 就是对这个 C 做特征分解，保留前 r 个主成分，得到降维表示。对 C 做特征分解，有：

$$C = V \Lambda V^T, \qquad \lambda_1 \geqslant \lambda_2 \geqslant \cdots \geqslant \lambda_d \geqslant 0,$$

其中，V 的列 $v_1, \ldots, v_d$ 就是主成分方向，λ_i 是对应方差。

保留前 r 个主成分，得到降维表示：

$$Z_r = V_r^T \tilde{X} \in \mathbb{R}^{r \times n}.$$

既然可以对 C 做特征分解，那么我们就不免会想到，如果对 C 做 SVD 分解，会得到怎么样的结果？是否会与特征分解得到的结果产生对应？下面，给出中心化数据的 SVD 与主成分的对应关系。

对中心化矩阵做 SVD：

$$\tilde{X} = U \Sigma V^T$$

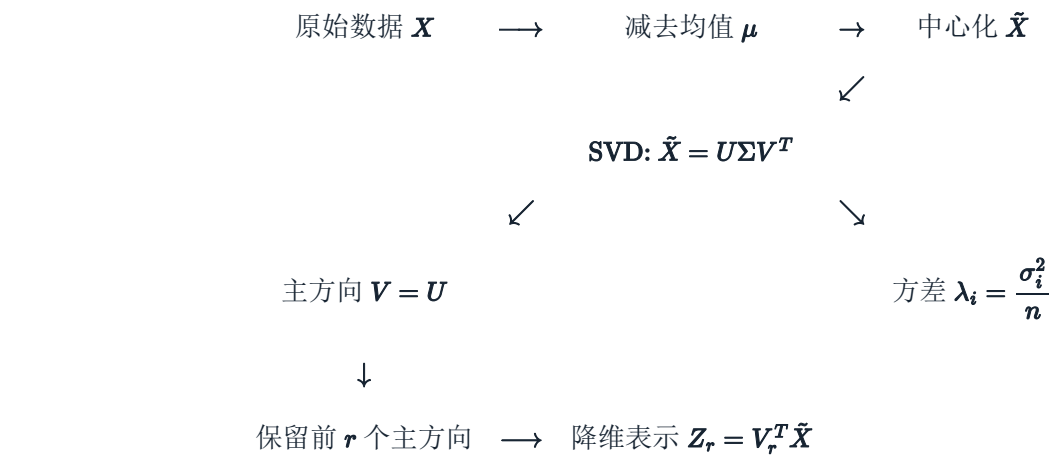
则：

$$\boxed{C = \frac{1}{n}\tilde{X}\tilde{X}^T = U\frac{\Sigma\Sigma^T}{n}U^T} \tag{4.17}$$

所以：

- U 的列是主成分方向；
- $\frac{\sigma_i^2}{n}$ 是对应方差；
- $V^T\tilde{X}$ 或 $U^T\tilde{X}$ 是主成分得分。

可见，PCA 是对这个 C 做特征分解，也等价于对 $\tilde{X}$ 做 SVD，PCA 就是中心化数据矩阵的 SVD。有 $V_{\text{PCA}} = U$，$\lambda_i = \frac{\sigma_i^2}{n}$. 数值上更推荐直接对 $\tilde{X}$ 做 SVD，避免构造 C 带来的条件数平方。



注意：PCA 必须用中心化后的 $\tilde{X}$，不是原始 X。直接对 X 做 SVD 得到的是“未中心化 PCA”，含义不同。

4.4.3 LoRA 的低秩参数增量

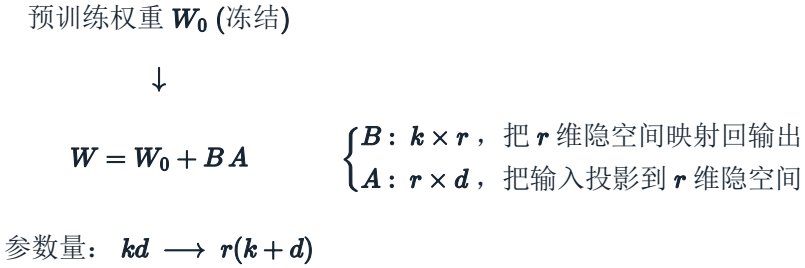
> 定义 4.18（LoRA 低秩适配）
> 设预训练权重为 $W_0 \in \mathbb{R}^{k\times d}$。LoRA 冻结 W_0，只学习低秩增量 $\Delta W = BA$，其中 $B \in \mathbb{R}^{k\times r}$、$A \in \mathbb{R}^{r\times d}$、$r \ll \min(k,d)$；微调后的权重为

$$\boxed{W = W_0 + \Delta W = W_0 + BA} \tag{4.18}$$

> 训练时只更新 A, B，参数量从 kd 降到 $r(k+d)$。

LoRA 的低秩增量之所以有效，核心在于一个被广泛验证的假设[8]：大模型在针对特定任务微调时，其权重更新矩阵 ΔW 虽然维度很高，但真正承载任务信息的“有效秩”（effective rank）非常低。换句话说，从奇异值分解（SVD）的角度看，对ΔW 做低秩近似，等价于保留其最大的几个奇异值所对应的奇异向量方向。ΔW 的主要能量只集中在少数几个奇异值方向上。绝大多数方向上的更新只是噪声或冗余。就像一个物品可以有很多特征，但想要顺利识别并区分不同的物品，往往只需要抓住几个特征即可。

这个假设的理论根基就是 SVD 的低秩近似和 Eckart – Young – Mirsky 定理。在Frobenius范数意义下，截断SVD是原矩阵的最优低秩近似。因此，LoRA的低秩约束，实际上是在引导优化过程去发现并利用 ΔW 中方差最大、信息最集中的那几个变化模式，而忽略次要的、可能包含噪声的方向。



4.4.4 一个把三者串起来的连接

下面把可微线性求解、中心化数据的 PCA、LoRA 的低秩参数增量这三者放在同一个数学框架里，详细解释它们如何连接。

> 核心结论（在线性层、平方损失、正则化最小二乘的近似下）：微调的最优权重增量 ΔW^* 的主要能量集中在数据协方差矩阵的主方向张成的低维子空间里；PCA 负责找出这些主方向；LoRA 的低秩约束正好是在这个子空间里做截断；可微线性求解则提供了高效且可微的数值手段来计算这个增量。

下面逐步展开。

先把“微调”转化成一个标准的正则化最小二乘问题。考虑一个线性层：

$$z = Wx, \qquad W \in \mathbb{R}^{k\times d},\ x \in \mathbb{R}^d.$$

给定训练数据：$X = [x_1, \ldots, x_n] \in \mathbb{R}^{d\times n}, \qquad Y = [y_1, \ldots, y_n] \in \mathbb{R}^{k\times n}$. 预训练权重 W_0，定义残差：$R = Y - W_0 X \in \mathbb{R}^{k\times n}$.

微调时，我们寻找增量 ΔW，最小化带正则化的重构误差：

$$L(\Delta W) = \frac{1}{2n}\|R - \Delta W X\|_F^2 + \frac{\gamma}{2}\|\Delta W\|_F^2.$$

这里：

- 第一项要求 $\Delta W X$ 尽量解释残差 R；
- 第二项防止 ΔW 过大，控制过拟合。

对 ΔW 求梯度：

$$\nabla_{\Delta W} L = -\frac{1}{n}(R - \Delta W X)X^T + \gamma \Delta W.$$

令其为零：

$$\frac{1}{n} R X^T = \Delta W \left(\frac{1}{n} X X^T + \gamma I\right).$$

定义

$$C = \frac{1}{n} X X^T \in \mathbb{R}^{d\times d}.$$

注意：如果 X 已经中心化，则 C 就是协方差矩阵；如果未中心化，C 是二阶矩矩阵。下面假设 X 已中心化，或至少 C 是我们关心的数据矩阵的协方差。

于是：

$$\boxed{\Delta W^* = \frac{1}{n} R X^T (C + \gamma I)^{-1}} \tag{4.19}$$

这是微调最优增量的闭式解。它把 ΔW^* 表示为：

- 残差与输入的互相关 RX^T；
- 数据协方差 C 的正则化逆 $(C + \gamma I)^{-1}$。

PCA 马上要登场了：$(C+\gamma I)^{-1}$ 的结构由 C 的特征分解决定。

对 C 做特征分解：

$$C = V\Lambda V^T = \sum_{i=1}^{d} \lambda_i v_i v_i^T.$$

于是：

$$(C+\gamma I)^{-1} = V(\Lambda+\gamma I)^{-1}V^T = \sum_{i=1}^{d} \frac{1}{\lambda_i+\gamma} v_i v_i^T.$$

代入 ΔW^*：

$$\Delta W^* = \frac{1}{n} R X^T \sum_{i=1}^{d} \frac{1}{\lambda_i+\gamma} v_i v_i^T.$$

定义 $g_i = \frac{1}{n} R X^T v_i \in \mathbb{R}^k$，则：

$$\boxed{\Delta W^* = \sum_{i=1}^{d} \frac{g_i}{\lambda_i+\gamma} v_i^T} \tag{4.20}$$

这个式子是整个连接的核心。逐项看：

- v_i：数据协方差 C 的第 i 个主方向；
- λ_i：该方向上的方差；
- g_i：残差在该方向上的投影，衡量“这个方向需要多少更新”；
- $\frac{1}{\lambda_i+\gamma}$：正则化系数，压制小方差方向。

因此，ΔW^* 是若干秩 1 矩阵 $\frac{g_i}{\lambda_i+\gamma} v_i^T$ 的叠加，每一项对应一个 PCA 主方向。

观察系数：$\frac{1}{\lambda_i+\gamma}$.

- 当 λ_i 很大时，系数接近 $\frac{1}{\lambda_i}$，v_i 方向上的更新被保留；
- 当 λ_i 很小时，系数接近 $\frac{1}{\gamma}$，但 g_i 通常也小，且正则化会压制这些方向；
- 当 $\lambda_i \approx 0$ 时，这些方向主要是噪声或冗余，ΔW^* 在这些方向上的分量很弱。

所以，在 C 的谱快速衰减的前提下，ΔW^* 的主要能量集中在 C 的前几个主方向 $v_1, \ldots, v_r$ 上。这就是“有效低秩”的数学来源。

更精确地说，如果 C 的谱快速衰减：

$$\lambda_1 \gg \lambda_2 \gg \cdots \gg \lambda_r \gg \lambda_{r+1} \approx \cdots \approx 0,$$

那么：

$$\Delta W^* \approx \sum_{i=1}^{r} \frac{g_i}{\lambda_i+\gamma} v_i^T.$$

这是一个秩不超过 r 的矩阵。

LoRA 用 $\Delta W = BA$ 近似 ΔW^*，其中 $B \in \mathbb{R}^{k\times r}, A \in \mathbb{R}^{r\times d}$。这是一个秩不超过 r 的矩阵。

由 Eckart – Young – Mirsky 定理，ΔW^* 的最优秩 r 近似是保留其前 r 个奇异值方向。而 ΔW^* 的主要能量又集中在 C 的前 r 个主方向 $v_1,\ldots,v_r$ 上。

因此：

> LoRA 的低秩约束之所以有效，是因为最优增量 ΔW^* 本身就是“有效低秩”的；而它的主要方向，正是数据协方差矩阵 PCA 的主方向。

进一步，LoRA 的两个因子可以这样理解：

- A 的行空间应该以 $v_1,\ldots,v_r$ 张成的子空间为目标，负责把输入投影到数据协方差的主方向；
- B 的列空间应该以 $g_1,\ldots,g_r$ 张成的子空间为目标，负责把主方向上的更新映射回输出。

这也解释了为什么在特定任务和设置下，用 SVD/PCA 初始化 LoRA 比随机初始化更好[9]：初始化时直接把 A 放在数据主方向上，B 放在残差主方向上，优化起点更接近最优解。

那么，可微线性求解如何介入？

ΔW^* 的闭式解：

$$\Delta W^* = \frac{1}{n} R X^T (C + \gamma I)^{-1}.$$

当 d 很大时（比如大模型隐藏维度几千到几万），显式构造 $C \in \mathbb{R}^{d\times d}$ 并求逆不可行。我们可以把它改写成线性系统。

令

$$Z = (\Delta W^*)^T \in \mathbb{R}^{d\times k}.$$

则：

$$(C + \gamma I) Z = \frac{1}{n} X R^T.$$

这是一个 $d \times d$ 的线性系统，右端有 k 列。可以用共轭梯度法（CG）迭代求解，只需要矩阵-向量乘积，不需要显式构造 C 的逆。

而 CG 可以通过隐函数定理实现可微：

- 前向：迭代解出 Z；
- 反向：解转置系统

$$(C + \gamma I)^T \tilde{Z} = \frac{\partial L}{\partial Z},$$

然后计算对 C 和 R 的梯度，再传回上游参数。

所以：

> 可微线性求解提供了高效计算 ΔW^* 的数值手段，并且保证整个“求增量”的过程可以嵌入神经网络做端到端训练。

三者形成的闭环：

$$\text{数据协方差 } C = \frac{1}{n} X X^T$$

$$\downarrow$$

$$\text{PCA: } C = V \Lambda V^T$$

$$\text{主方向 } v_1, \cdots, v_r$$

$$\text{方差 } \lambda_1, \cdots, \lambda_r$$

$$\downarrow$$

$$\text{最优增量 } \Delta W^* = \sum_i \frac{g_i}{\lambda_i + \gamma} v_i^T$$

能量集中在主方向

↙	↘
$\text{LoRA: } \Delta W = BA$, 秩 r	可微线性求解：
在主方向上做低秩截断	$(C + \gamma I) Z = \frac{1}{n} X R^T$
A 对齐主方向，B 对齐残差	用 CG 迭代 + 隐函数定理反传

用文字表述：

1. PCA 从数据中发现这个低秩结构，告诉你数据的主方向 v_i 和方差 λ_i；
2. 最优增量 ΔW^* 的能量集中在这些主方向上，小方差方向被正则化压制；
3. LoRA 在模型参数中利用这个低秩结构，只保留前 r 个主方向上的增量，用 BA 参数化，大幅减少参数量；
4. 可微线性求解 是在数值上计算这个低秩结构对应的增量，并保持可微，在 d 很大时，用 CG 等迭代法高效计算 ΔW^*，并通过隐函数定理把梯度传回上游。

> 三者分工：PCA 回答“哪些方向重要”；LoRA 回答“怎样用少参数表达这些方向上的更新”；可微线性求解回答“怎样高效且可微地算出这些更新”。

除去上述数学框架，还有一个数能将三者放在一起。条件数是将数值可行性（求解器）、数据结构（PCA）与优化动力学（LoRA） 串联起来的一个基础性诊断指标。

可微求解器的反向传播依赖隐函数定理，其核心是求解一个线性系统（伴随系统）。条件数直接决定了这个过程的成败。反向传播需要解 $A^T z = g$，条件数大的系统会让计算出的梯度“信噪比”极低，可能导致训练不稳定甚至失败。对于 CG 等迭代求解器，条件数直接决定收敛速度。

PCA 中的条件数主要体现在协方差矩阵 C 上，反映了数据在不同主方向上方差的不均衡程度。PCA 通过 SVD 或特征分解求解，其计算精度受 C 的条件数影响。同时，条件数 $\kappa(C) = \frac{\lambda_{\max}}{\lambda_{\min}}$ 是判断数据“有效秩”的指标。谱快速衰减（大 κ）是 PCA 降维能奏效的前提，因为这意味着少数几个主方向就包含了绝大部分信息。

LoRA 的低秩约束与条件数的关系更为微妙，它并非直接最小化条件数，而是通过参数化方式间接影响优化景观和泛化性能。低秩约束 BA 本身构成一种隐式正则化，它限制了权重更新 ΔW 的有效条件数，倾向于寻找低复杂度、谱较集中的解，从而防止过拟合。同时，LoRA 的过参数化（BA 的分解不唯一）导致不同的因子对 (A,B) 会产生不同的条件数，进而影响收敛速度。Balanced LoRA（BaLoRA）[10] 等工作正是通过将因子投影到平衡流形、强制 $B^TB=AA^T$（$r\times r$）来最小化并平衡条件数，从而加速收敛。

三者共享一个核心逻辑：条件数衡量的是“求解或估计过程对扰动的敏感性”。

注意事项

这个连接成立有几个前提：

1. 线性层近似：推导假设被微调的是一个线性层，损失是平方误差。实际大模型是多层非线性网络，但这个分析可以作为局部线性化的理解。
2. 数据协方差谱衰减：LoRA 有效的前提是 C 的奇异值快速衰减，即数据本身具有低维结构。如果数据协方差接近各向同性，低秩约束效果会变差。
3. 正则化强度：γ 控制小方差方向的压制程度。γ 太大，所有方向都被压制；γ 太小，噪声方向可能被放大。
4. 可微线性求解的稳定性：当 $C+\gamma I$ 病态时，CG 收敛慢，反向系统也不稳定。需要预条件、正则化或截断处理。
5. LoRA 的秩选择：r 应该与 C 的有效秩匹配。太小欠拟合，太大则失去参数效率优势。

> 例　　微调一个图像分类头。假设我们有一个预训练好的视觉模型，冻结主干，只微调最后的线性分类头。设定：
>
> - 输入特征维度 $d=512$；
> - 类别数 $k=10$；
> - 训练样本 $n=5000$；
> - 预训练权重 $W_0\in\mathbb{R}^{10\times512}$。
>
> 数据是从某个新任务（比如 10 种鸟分类）中提取的 512 维特征。由于图像特征通常位于低维流形上，协方差矩阵 $C\in\mathbb{R}^{512\times512}$ 只有大约 50 个显著特征值，其余接近 0。

用上面的推导看：

1. PCA：对 C 做特征分解，得到主方向 $v_1,\dots,v_{50}$ 和对应大方差 $\lambda_1,\dots,\lambda_{50}$。
2. 最优增量：$\Delta W^*=\sum_{i=1}^{512}\frac{g_i}{\lambda_i+\gamma}v_i^T$。由于 $\lambda_{51},\dots,\lambda_{512}\approx0$，这些项被 $\frac{1}{\lambda_i+\gamma}$ 压得很小，ΔW^* 的能量几乎全在前 50 个主方向上。
3. LoRA：取 $r=4$ 或 $r=8$，只学前几个主方向上的增量。参数量从 $10\times512=5120$ 降到 $4\times(10+512)=2088$ 或 $8\times(10+512)=4176$，仍然能捕捉主要更新。

4. 可微线性求解：实际训练时，可以用 CG 迭代求解 $(C+\gamma I)Z=\frac{1}{n}XR^T$，并通过隐函数定理反传梯度。这样既不用显式构造 512×512 的逆矩阵，又能端到端训练。

这个例子说明：LoRA 之所以能用极少参数微调大模型，是因为任务相关的权重更新本身就集中在数据协方差的主方向上；PCA 告诉我们这些方向是什么；可微线性求解则提供了高效计算这些增量的数值工具。

以下是其余关系的图解比较

可微线性求解与 LU / CG / SVD

可微线性求解

↙ ↘

前向：解 $Ax=b$ 反向：解 $A^Tz=\frac{\partial L}{\partial x}$

↓ ↓

稠密小矩阵 $\longrightarrow$ **LU** 同样可用 **LU / CG / SVD**

大规模稀疏 $\longrightarrow$ **CG**

需要伪逆/秩分析 $\longrightarrow$ **SVD**

PCA 与 SVD

PCA：对 $C=\frac{1}{n}\tilde{X}\tilde{X}^T$ 做特征分解

SVD：对 $\tilde{X}$ 做奇异值分解

↓

关系：$C=U\left(\frac{\Sigma\Sigma^T}{n}\right)U^T \Longrightarrow V_{\text{PCA}}=U,\quad \lambda_i=\frac{\sigma_i^2}{n}$

↓

结论：**PCA** $\equiv$ 中心化数据的 **SVD**，**SVD** 数值更稳定

LoRA 与 SVD

SVD：$A=U\Sigma V^T$，按奇异值排序

LoRA：$\Delta W=BA$，秩 r

↓

关系：**LoRA** 是 ΔW 的秩 r 近似，**Eckart-Young** 保证最优

↓

结论：**LoRA** 的理论根基是 **SVD** 低秩近似

关键推导：让结论有依据

推导 ：$Ax=b$ 下的微扰关系

在 A 可逆的条件下，由 $Ax=b$ 出发，考虑 A 和 b 的微小扰动 $\delta A,\delta b$，看解 x 会产生怎样的扰动 δx。下面给出推导。

设

$$A\in\mathbb{R}^{n\times n},\qquad b\in\mathbb{R}^n,\qquad x=A^{-1}b$$

给 A,b 一个微小扰动：

$$A\to A+\delta A,\qquad b\to b+\delta b$$

新解记为 $x+\delta x$，它满足

$$(A+\delta A)(x+\delta x)=b+\delta b$$

假设扰动足够小，使得 $A+\delta A$ 仍可逆，且 δx 是一阶小量。

展开左边：

$$Ax+A\delta x+\delta A\,x+\delta A\,\delta x=b+\delta b$$

因为 $Ax=b$，消去后得到

$$A\delta x+\delta A\,x+\delta A\,\delta x=\delta b$$

忽略二阶小量 $\delta A\,\delta x$，得到一阶微扰关系：

$$A\delta x=\delta b-\delta A\,x$$

由于 A 可逆，两边左乘 A^{-1}：

$$\delta x=A^{-1}(\delta b-\delta A\,x)$$

这就是最核心的微扰公式。

当 A,b 依赖参数 θ 时，设

$$A=A(\theta),\qquad b=b(\theta)$$

则

$$\delta A=\frac{\partial A}{\partial\theta}\delta\theta,\qquad \delta b=\frac{\partial b}{\partial\theta}\delta\theta$$

代入微扰公式：

$$\delta x=A^{-1}\left(\frac{\partial b}{\partial\theta}-\frac{\partial A}{\partial\theta}x\right)\delta\theta$$

因此

$$\frac{\partial x}{\partial\theta}=A^{-1}\left(\frac{\partial b}{\partial\theta}-\frac{\partial A}{\partial\theta}x\right)$$

也可以直接对 $Ax=b$ 两边关于 θ 求导得到同样结果：

$$\frac{\partial A}{\partial\theta}x+A\frac{\partial x}{\partial\theta}=\frac{\partial b}{\partial\theta}$$

移项即得上式。上式就是可微线性求解中，把解 x 的梯度传回上游参数 θ 的基础。

说明：

- A 可逆保证 A^{-1} 存在，扰动有唯一的一阶响应；
- 若 A 病态，即条件数 $\kappa(A)=\|A\|\|A^{-1}\|$ 很大，则微小的 $\delta A,\delta b$ 可能被放大成很大的 δx；
- 相对误差大致满足 $\frac{\|\delta x\|}{\|x\|}\lesssim\kappa(A)\left(\frac{\|\delta A\|}{\|A\|}+\frac{\|\delta b\|}{\|b\|}\right)$ 所以可微求解器不仅要能求梯度，还要关注数值稳定性。

实验设计：先预测，再运行

实验任务：用小矩阵核对求解器梯度与有限差分，并画 PCA 重构误差；LoRA 只要求维度与参数量的小例子。

要填写的项目	本组内容
要回答的问题与运行前预测	验证线性方程求解器的伴随梯度是否与中心有限差分一致；研究 PCA 中心化对主方向和重构误差的影响；核对 LoRA 低秩更新的维度、参数量及合并前后输出是否一致；并用病态伴随系统与小方差特征反例检查 4.4.4 连接的适用边界。预测：伴随梯度与适当差分步长下的有限差分接近；PCA 保留主成分越多，重构 MSE 越小；LoRA 参数量随秩线性增长，合并权重与分开计算输出应数值一致；病态伴随系统会把固定上游梯度扰动放大，小方差特征仍可能对应大任务相关更新。
输入、参数、单位、数据来源与划分	可微求解器：$A_{\text{lin}}=\begin{pmatrix}2&1\\1&3\end{pmatrix}$、$b_{\text{lin}}=(1,2)^T$，$x=A_{\text{lin}}^{-1}b_{\text{lin}}$，损失 $L=\frac{1}{2}\|x\|_2^2$；中心差分步长 $h=10^{-1},\dots,10^{-8}$。PCA：列样本矩阵 $X\in\mathbb{R}^{5\times240}$，五个特征由确定性正弦/余弦序列构造，幅度 $(3,1,0.2,0.05,0.01)$，第二个特征另加均值偏移 20。LoRA：基础权重 $W_0\in\mathbb{R}^{256\times128}$，秩 $r=8$、缩放参数 $\alpha=8$，适配器矩阵维度分别为 8×128 和 256×8；另扫描秩 $(1,2,4,8,16)$。均为合成、无量纲数据，无训练集与测试集划分。
独立参考解/基线及选择理由	可微求解器用伴随方程 $A_{\text{lin}}^Tz=\nabla_xL$ 推导 $\nabla_bL=z$、$\nabla_AL=-zx^T$，并用中心有限差分独立核验。PCA 使用构造数据的解析均值和奇异值谱作为参考。LoRA 使用矩阵乘法恒等式比较“分开计算”与“合并权重后计算”的输出，并用参数计数公式 $r(d_{\text{out}}+d_{\text{in}})$ 与全量参数数 $d_{\text{out}}d_{\text{in}}$ 对照。
误差指标、容差及其依据	梯度：记录各差分步长 h 下 $\frac{\partial L}{\partial b_2}$ 的有限差分值及绝对误差，重点检查 $h=10^{-4}$ 时误差是否小于 10^{-7}。PCA：记录每元素重构均方误差（MSE）和解释方差比例，实测 MSE 与解析预测的差要求小于 10^{-12}。LoRA：记录适配器参数量、占全量参数比例、更新矩阵秩及合并前后输出相对误差，合并相对误差阈值为 10^{-12}。适用边界验证：记录 $\varepsilon=1,10^{-4},10^{-8}$ 时固定 10^{-10} 上游梯度扰动造成的伴随解绝对变化，并与解析值 $\frac{10^{-10}}{\varepsilon}$ 对照；记录小方差特征反例中两特征的方差与岭回归系数，检查 $\lvert\Delta W_{0,2}\rvert>1000\lvert\Delta W_{0,1}\rvert$。

要填写的项目	本组内容
环境、种子、设备与运行预算	Python `3.12.10`、NumPy `1.26.4`、SciPy `1.14.1`；CPU、`float64`、种子 `2026`，线程数限制为 `1`。PCA 样本数 240、特征数 5；LoRA 权重规模 256×128。Notebook 将 PyTorch 标注为可选；本次运行中 `torch` 未成功导入，因此自动微分分支未执行，本节梯度结论仅基于伴随法与中心差分两路核对。另新增不依赖 PyTorch 的适用边界验证：病态伴随系统放大与小方差特征反例。

结果解释与本节结论

（1）可微求解器梯度核验

取损失 $L = \frac{1}{2}\|x\|_2^2$，由 $A_{\text{lin}}x = b$ 解得 $x = (0.2, 0.6)^T$，伴随梯度为 $\nabla_b L = A_{\text{lin}}^{-T}x = (0, 0.2)^T$，矩阵梯度为

$$\nabla_A L = -zx^T = \begin{pmatrix} 0 & 0 \\ -0.04 & -0.12 \end{pmatrix}, \qquad z = A_{\text{lin}}^{-T}x.$$

中心差分对 $\frac{\partial L}{\partial b_2}$ 的计算值在各步长下均约为 0.2；当 $h = 10^{-4}$ 时绝对误差约 2.00007×10^{-13}，与伴随梯度高度一致。步长减小到 10^{-8} 时误差增至约 1.28251×10^{-9}，说明差分步长过小会受到浮点舍入影响。

差分步长 h	有限差分 $\frac{\partial L}{\partial b_2}$	绝对误差
10^{-1}	0.2	1.66533×10^{-16}
10^{-2}	0.2	2.60902×10^{-15}
10^{-3}	0.2	2.20379×10^{-14}
10^{-4}	0.2	2.00007×10^{-13}
10^{-5}	0.2	1.18780×10^{-12}
10^{-6}	0.2	5.75112×10^{-12}
10^{-7}	0.2	1.05271×10^{-10}
10^{-8}	0.2	1.28251×10^{-9}

本节结论基于伴随法与中心差分两路独立核对；PyTorch 自动微分交叉验证属可选扩展，本次运行中 `torch` 未成功导入，自动微分分支未执行。

（2）PCA 中心化与重构误差

X 的形状为 5×240，按列作为样本，原始均值向量为 $(0, 20, 0, 0, 0)$，中心化后均为零向量。未中心化时，第二个特征的均值偏移会主导第一主方向。程序核对了中心化后的奇异值谱和协方差特征值。不同保留秩的结果如下：

保留主成分数	每元素重构 MSE	解释方差比例
0	1.00426	0
1	0.10426	0.896182

保留主成分数	每元素重构 MSE	解释方差比例
2	0.00426	0.995758
3	0.00026	0.999741
4	0.00001	0.999990
5	3.89261×10^{-32}	1.0

保留主成分数增加时重构 MSE 单调下降；保留 2 个主成分已解释约 99.58% 的方差，保留全部 5 个主成分时重构误差落在机器精度量级。结果支持“先中心化再做 PCA”的实验预测。

（3）LoRA 维度与参数量

基础权重矩阵形状为 256×128，全量参数数为 32768。LoRA 适配器参数量为 $r(256+128)$：

LoRA 秩 r	全量参数数	适配器参数数	适配器占全量比例
1	32768	384	1.1719%
2	32768	768	2.3438%
4	32768	1536	4.6875%
8	32768	3072	9.3750%
16	32768	6144	18.7500%

当 $r=8$ 时适配器参数为 3072，仅为全量参数的 9.375%。程序还验证了：更新矩阵 $\delta = \frac{\alpha}{r} BA$ 的秩不超过 8；$W_0+\delta$ 仍为满秩（LoRA 不强迫总权重低秩）；零 B 初始化时 $\delta=0$；分开计算与合并计算输出一致，合并相对误差约 2.784×10^{-16}。本实验只验证维度、参数量与代数合并等式；未进行训练，故不构成对 LoRA 训练效果或任务性能的结论。

易错点与讨论

1. PCA 必须中心化：未中心化时，第一主成分会同时捕获均值贡献和波动贡献；当均值远离原点时，主成分方向会被均值“带偏”，误读为数据变异最大的方向。中心化后 PCA 才真正分析数据围绕均值的变异结构。
2. LoRA 与截断 SVD 的区别：标准 LoRA 的 A, B 是通过训练学到的，初始时 $\Delta W = 0$；截断 SVD 是对已有权重的压缩或近似。二者操作不同：前者是“学一个低秩增量”，后者是“截断一个已知矩阵”。
3. LoRA 初始化的影响：$\text{Init}(A)$ 与 $\text{Init}(B)$ 的训练动力学不同，非零初始化也可行。初始化不是“随便选一个为零就行”，它影响训练效率；用 SVD/PCA 初始化 A 比随机初始化更接近最优解。
4. 可微求解器的条件数风险：当 A 病态时，反向系统 $A^T z = g$ 同样病态，伴随求解的误差会被放大。4.4 新增的适用边界验证显示，固定 10^{-10} 的上游梯度扰动在 $\varepsilon = 10^{-8}$（条件数 10^8）时被放大到 10^{-2}；条件数越大，梯度越不稳定。这是 4.3 节条件数概念在训练循环中的延续。

随堂提问： 设数据矩阵 $X \in \mathbb{R}^{d\times n}$，$d=512$，$n=5000$；中心化数据 $\tilde{X}=X-\mu \mathbf{1}^T$；协方差 $C=\frac{1}{n}\tilde{X}\tilde{X}^T \in \mathbb{R}^{512\times 512}$。预训练权重 $W_0 \in \mathbb{R}^{k\times d}$，$k=10$，$d=512$；LoRA 增量 $\Delta W=BA$，$B \in \mathbb{R}^{10\times r}$，$A \in \mathbb{R}^{r\times 512}$。

判断下列说法是否正确，并说明理由：

(1) “PCA 就是对原始数据矩阵 X 做 SVD，取左奇异向量作为主方向。”

(2) “LoRA 的参数量是 $r(k+d)$；当 $r=4$ 时，参数量从 $10\times 512=5120$ 降到 $4\times(10+512)=2088$。”

(3) “LoRA 的 B 和 A 可以通过对 W_0 做截断 SVD 得到。”

(4) “ΔW 的主要能量集中在前 r 个奇异值方向，因此 LoRA 有效。”

可能的回答：

- 一部分同学认为 PCA 可以直接对 X 做 SVD，忽略中心化；或把左、右奇异向量混淆，对 $\tilde{X} \in \mathbb{R}^{512\times 5000}$，$U \in \mathbb{R}^{512\times 512}$、$V \in \mathbb{R}^{5000\times 5000}$，主方向是 U 的列，不是 V 的列。
- 一部分同学能算出参数量，但说不清 kd 与 $r(k+d)$ 分别对应什么。
- 一部分同学把 LoRA 与截断 SVD 混为一谈，认为 B,A 来自对 W_0 的 SVD。
- 一部分同学能说出“低秩”，但说不清低秩来自数据协方差 C 的谱衰减，而非 ΔW 本身的先验低秩。

解释：

1. 错误。PCA 必须用中心化数据 $\tilde{X}$，不是原始 X。对 $\tilde{X}=U\Sigma V^T$ 做 SVD，则 $C=\frac{1}{n}\tilde{X}\tilde{X}^T=U\frac{\Sigma\Sigma^T}{n}U^T$，所以 U 的列是主方向，$\lambda_i=\frac{\sigma_i^2}{n}$。对 X 直接做 SVD 得到的是“未中心化 PCA”，第一主成分可能指向均值方向。注意 $\tilde{X} \in \mathbb{R}^{512\times 5000}$ 时，$U \in \mathbb{R}^{512\times 512}$ 是左奇异向量矩阵，主方向是 U 的列；V 的列不是主方向。
2. 正确。W_0 有 $kd=5120$ 个参数；$\Delta W=BA$ 中 $B \in \mathbb{R}^{10\times r}$、$A \in \mathbb{R}^{r\times 512}$，参数量为 $kr+rd=r(k+d)$。$r=4$ 时 $4\times(10+512)=2088<5120$。
3. 错误。标准 LoRA 的 B,A 是通过训练学到的，初始时 $\Delta W=0$；截断 SVD 是对已知矩阵 W_0 的压缩或近似。用 SVD 初始化 LoRA（如 PiSSA[9]）是一种初始化策略：PiSSA 用 W_0 的主奇异分量初始化适配器 A,B，初始化后仍训练 A,B，不是一次性截断 W_0；因此它与“对已有权重做截断 SVD”不同，二者操作不同。
4. 不准确。$\Delta W^*=\sum_i \frac{g_i}{\lambda_i+\gamma}v_i^T$ 的能量集中在数据协方差 C 的前几个主方向 v_i 上，而不是“ΔW 自身的前 r 个奇异值方向”。LoRA 有效的数据侧前提是 C 的谱快速衰减（$\lambda_1 \gg \lambda_2 \gg \cdots$）；Eckart-Young 定理保证秩 r 近似在 Frobenius 范数下对 ΔW^* 最优，但“最优”针对的是 ΔW^*，不是任意 ΔW。只说“ΔW 的前 r 个奇异值方向”而没说明与 C 主方向的关系，理由不完整。

小结

矩阵方法在 AI 中有三种典型嵌入方式，三者的分工可以归纳为一条链：

1. PCA 回答“哪些方向重要”。 对中心化数据协方差 $C = \frac{1}{n}\tilde{X}\tilde{X}^T$ 做特征分解，得到主方向 v_i 与方差 λ_i；等价地，对 $\tilde{X}$ 做 SVD，U 的列即主方向，$\lambda_i = \frac{\sigma_i^2}{n}$。数值上推荐直接对 $\tilde{X}$ 做 SVD，避免构造 C 带来的条件数平方。

2. LoRA 回答“怎样用少参数表达这些方向上的更新”。 微调最优增量 $\Delta W^* = \sum_i \frac{g_i}{\lambda_i+\gamma} v_i^T$ 的能量集中在 C 的前几个主方向上；LoRA 用 $\Delta W = BA$（秩 r）近似 ΔW^*，参数量从 kd 降到 $r(k+d)$，其理论根基是 SVD 低秩近似与 Eckart-Young 定理。

3. 可微线性求解回答“怎样高效且可微地算出这些更新”。 当 d 很大时，$\Delta W^* = \frac{1}{n}RX^T(C+\gamma I)^{-1}$ 不能显式求逆；改写成 $(C+\gamma I)Z = \frac{1}{n}XR^T$，用 CG 迭代求解，并通过隐函数定理反传梯度，使整个“求增量”过程可嵌入神经网络做端到端训练。

适用条件与要点： 可微线性求解要求 A 可逆或可稳定求解；病态时反向系统 $A^Tz = g$ 同样病态，梯度会被放大，需正则化或伪逆。PCA 要求数据先中心化，未中心化得到的是“未中心化 PCA”，第一主成分可能被均值方向主导，含义不同。LoRA 只保证增量满足 $\mathrm{rank}(\Delta W) = \mathrm{rank}(BA) \le r$，不保证微调后的整体权重 $W = W_0 + \Delta W$ 低秩；其有效性还依赖 C 的谱快速衰减（数据低维结构），若 C 接近各向同性，低秩约束效果变差；r 应与 C 的有效秩匹配，太小欠拟合，太大失去参数效率。低秩近似误差小只说明在 Frobenius 或谱范数下逼近得好，不等于预测或泛化准确。

回到开场：用证据回答

本章核心问题： $Ax = b$ 如何高效求解？矩阵的本质结构是什么？

取 $A = \mathrm{diag}(1, 10^{-10})$，右端第二分量增加 10^{-10}，解的第二分量便从 1 变成 2。核对：扰动前后相对解误差约为 0.707；而把 $\tilde{x} = (1,2)^T$ 作为原右端的近似解，相对残差只有 10^{-10}，对扰动后右端则残差为零。二者并不矛盾——残差是输出端的偏差，解误差是输入端的偏差，二者隔着 A^{-1}。A 在第二个方向上几乎把一切压成零，逆映射在该方向上把微小偏差放大 10^{10} 倍，右端一点扰动就造成解的明显变化，条件数 $\kappa_2(A) = 10^{10}$ 正是这个放大倍数的上界。小残差只说明 Ax 接近某个右端，不说明 x 接近真解；讨论残差，必须先说明它针对哪个右端。

再回到本章核心问题。$Ax = b$ 的高效求解有两条主线：直接法用矩阵分解的方法，把 A 分解为结构更简单的矩阵再进行求解；迭代法不显式分解 A，只用矩阵与向量的乘积实现的迭代算法逐渐逼近解。

矩阵的本质结构决定了选哪种方法、算法是否稳定。

分解层面。 顺序主子式与主元决定无主元 LU 能否进行；对称正定性是 Cholesky 的适用条件；三对角结构和对角占优是追赶法稳定的充分条件。

迭代层面。 对称正定性同样是 CG 的前提；谱半径决定线性定常迭代是否收敛。

谱层面。 特征值和奇异值决定条件数、收敛快慢与扰动敏感程度；奇异值衰减决定低秩近似效果。

全章的主线是：条件数是问题本身的属性，选主元、Cholesky、追赶法、预条件改善的是算法稳定性或收敛速度，不能把病态问题变成良态问题；残差与解误差是不同主张，二者之间隔着条件数，讨论残差必须说明针对哪个右端。

本章小结与课后习题

本章围绕“如何高效求解 $Ax=b$”和“矩阵的本质结构是什么”展开，核心工具有 LU、CG、SVD。三者分别对应直接法、迭代法和谱结构；其他方法大多可以看作这三个概念在特殊结构、特殊条件或特殊应用下的变形。

LU：解决“怎样用有限步分解把方程化为三角求解”。 LU 分解把 A 写成 $A=LU$，先解 $Ly=b$ 再解 $Ux=y$。列主元 LU 用 $PA=LU$ 保证乘数有界；Cholesky 分解 $A=LL^T$ 和 LDL^T 分解是针对对称正定矩阵的特化；追赶法则是三对角矩阵的 $O(n)$ 版 LU。适用条件要分清：无主元 LU 要求所有顺序主子式非零；一般非奇异矩阵用列主元 LU；Cholesky 要求 A 对称正定，LDL^T 仍要求对称正定或对称且顺序主子式非零；追赶法在对角占优条件下稳定，但对角占优只是充分条件。最容易混淆的是：$a_{11}=0$ 只说明无主元 LU 第一步失败，不说明 A 奇异；列主元 LU 只改善算法稳定性，不改善病态性；Cholesky 不能用于非对称、非正定或仅半正定矩阵；残差小不等于解误差小。

CG：解决“大规模稀疏对称正定系统怎样不分解矩阵而迭代求解”。 当 A 对称正定时，解 $Ax=b$ 等价于最小化二次函数 $\phi(x)=\frac{1}{2}x^TAx-b^Tx$；最速下降法沿负梯度走，条件数大时在扁椭球两侧折返，收敛慢；CG 改用一组关于 A 共轭的方向，在精确算术下至多 n 步收敛，每步只需一次矩阵-向量乘积。适用条件是 A 对称正定；预条件 CG 要求预条件子 M 对称正定且 M^{-1} 易计算，对角预条件是最简单的选择。最容易混淆的是：CG 不能无条件用于非对称或非正定矩阵；预条件改善的是预条件后系统的条件数，从而加快收敛，不改变原问题的病态性；预条件的总成本要同时计入迭代次数减少和预条件子构造、应用开销；残差停止准则同样受条件数放大，相对残差小不能单独作为“解已经足够精确”的依据。

SVD：解决“任意矩阵的结构、敏感性和低秩近似怎样统一描述”。 对任意实矩阵 $A=U\Sigma V^T$，奇异值 $\sigma_i\geqslant 0$，$\|A\|_2=\sigma_1$，$\|A^{-1}\|_2=\frac{1}{\sigma_{\min}}$，$\kappa_2(A)=\frac{\sigma_1}{\sigma_{\min}}$；最小奇异值对应最敏感方向，右端扰动和系数矩阵扰动的放大上界都含 $\kappa_2(A)$。截断 SVD $A_k=\sum_{i=1}^{k}\sigma_iu_iv_i^T$ 是秩不超过 k 的最佳逼近，误差满足 $\|A-A_k\|_F^2=\sum_{i=k+1}^{r}\sigma_i^2$、$\|A-A_k\|_2=\sigma_{k+1}$，奇异值衰减速度决定低秩近似效果。最容易混淆的是：非对称矩阵的特征值绝对值一般不等于奇异值；数值上不应直接计算 A^TA 的特征值来求奇异值，因为条件数会被平方；低秩近似是在 Frobenius 或谱范数意义下的最佳逼近，不等于能准确预测未知数据；PCA 必须对中心化数据做 SVD，直接对原始 X 做 SVD 得到的是“未中心化 PCA”，含义不同。

与其他方法的联系可以归纳为一条链：Gauss 消元与 LU 本质相同；Cholesky、LDL^T 和追赶法分别是 LU 在对称正定、对称正定免开方和三对角结构下的特化；最速下降法是 CG 的前身，CG 用共轭方向克服最速下降的折返；预条件 CG 是在 CG 外层改善条件数；PCA 是中心化数据协方差矩阵的特征分解，也等价于中心化数据矩阵的 SVD；LoRA 用低秩增量 BA 近似微调最优增量，其理论根基是 SVD 低秩近似与 Eckart-Young 定理；可微线性求解前向可用 LU、CG 或 SVD，反向解转置系统 $A^Tz=g$，把矩阵求解嵌入端到端训练。4.4 节中的“PCA 发现主方向、LoRA 在主方向上做低秩截断、可微线性求解高效且可微地计算增量”正是这三条主线的综合应用。

课后题按四节对应填写到A2组-习题.md，参考解答单独放在A2组-习题参考解答.md。本节不重复列题，以免与模板文件不一致。本组AI使用记录见 A2组-AI对话记录.md。

选读边界

以下内容非本章学习目标，仅作为进一步的扩展阅读内容，供有兴趣的读者选读。

- 条件数的更多性质与范数依赖
- 盖尔圆盘定理与特征值的分布估计
- Jordan 标准型、不可对角化方阵的幂次
- Krylov 子空间一般理论；GMRES、MINRES 等非对称 Krylov 方法
- 更多预条件子：不完全 Cholesky、多重网格、稀疏近似逆（也可见第 8 章）
- 幂迭代的边界情形：并列主特征值、缺陷矩阵、收敛率精细估计
- 实际 SVD 算法、QR 迭代与特征值全谱计算
- Eckart – Young – Mirsky 定理的完整证明与等号条件
- 稀疏存储细节：CSR/CSC、消元树、填充约简
- 后向误差完整分析
- 可微求解器实现细节：VJP、自动微分机制、病态正则化与伪逆

参考资料

[1] Horn R A, Johnson C R. Matrix Analysis[M]. 2nd ed. Cambridge: Cambridge University Press, 2013. ISBN 978-0-521-83940-2 (hardback), 978-0-521-54823-6 (paperback).

[2] 潘建瑜. 数值分析讲义[Z]. 上海: 华东师范大学数学科学学院, 2024. https://math.ecnu.edu.cn/jypan/Teaching/NA/index.html.

[3] Eckart G, Young G. The approximation of one matrix by another of lower rank[J]. Psychometrika, 1936, 1(3): 211-218.

[4] Jolliffe I T. Principal Component Analysis[M]. 2nd ed. New York: Springer, 2002.

[5] IBM Corporation. IBM SPSS Modeler Subscription: PCA 构建选项[EB/OL]. (2021-03-01). https://www.ibm.com/docs/zh/spss-modeler/18.2.2?topic=pca-data-wh-build-options.

[6] Amos B, Kolter J Z. OptNet: Differentiable optimization as a layer in neural networks[C]//Proceedings of the 34th International Conference on Machine Learning (ICML 2017). 2017: 136-145.

[7] Paszke A, Gross S, Massa F, et al. PyTorch: An imperative style, high-performance deep learning library[C]//Advances in Neural Information Processing Systems 32 (NeurIPS 2019). 2019: 8024-8035.

[8] Hu E J, Shen Y, Wallis P, et al. LoRA: Low-rank adaptation of large language models[EB/OL]. arXiv:2106.09685, 2021.

[9] Meng F, Wang Z, Zhang M. PiSSA: Principal singular values and singular vectors adaptation of large language models[C]//Proceedings of the 38th International Conference on Neural Information Processing Systems (NeurIPS 2024). 2024.

[10] Castin V, Nadjahi K, Ablin P, Peyré G. Balanced LoRA: Removing parameter invariance to accelerate convergence[C]//Proceedings of the 43rd International Conference on Machine Learning (ICML 2026). 2026.

本章习题与参考解答

第04章 矩阵的解剖：课后习题

> 备注：
>
> - 以下辨析题中的说法为教学构造的辨析材料，不代表任何真实结果。
> - 本章习题沿用正文符号约定；未特别说明时，向量范数默认取 2-范数，$\kappa_2(A)$ 表示 2-范数条件数，x^* 表示精确解。

题1（对应 4.1 直接法：LU 与 Cholesky）

题目 设

$$A=\begin{pmatrix}0 & 2 & 1\\ 2 & 1 & 3\\ 1 & 3 & 2\end{pmatrix},\qquad b=\begin{pmatrix}3\\ 6\\ 6\end{pmatrix}.$$

已知按自然顺序进行 Gauss 消元时，第一步会遇到零主元 $a_{11}=0$，无法直接消元。允许进行行交换，置换矩阵记为 P，约定分解形式为

$$PA=LU,$$

其中 L 为单位下三角矩阵，U 为上三角矩阵。

请完成下列各问。

（1）列主元 LU 分解与求解

1. 对 A 做列主元 LU 分解，写出每一步行交换、消元乘数和中间矩阵，计算出最终的矩阵 P,L,U，并验证 $PA=LU$。
2. 用该分解求解 $Ax=b$，写出前代和回代的关键步骤，给出解向量 x，并代回检验。

（2）对称正定化处理

令

$$A'=A+4I.$$

1. 写出 A' 并判断 A' 是否对称。
2. 用顺序主子式判断 A' 是否正定。
3. 讨论 A' 能否使用 Cholesky 分解，并说明依据。
4. 比较并解释 A 与 A' 在分解方法选择上的差异。

（3）扰动处理与条件数

令

$$A'' = A + \delta A, \qquad \delta A = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 10^{-3} \end{pmatrix}.$$

1. 写出 A''。
2. 对 A'' 重新做列主元 LU 分解，观察主元是否变化。
3. 比较 A 与 A'' 的解。要求：
   - 给出 x 与 x'' 的近似值；
   - 本题扰动为 10^{-3} 量级，建议至少保留 4 位小数，先写解析式再取近似，避免过早舍入；
   - 计算或估计解的相对变化 $\frac{\|x''-x\|_2}{\|x\|_2}$。

（4）概念与辨析

1. 解释为什么 A 按自然顺序消元会失败，以及主元交换如何避免这一点。
2. 写出你采用的置换矩阵约定，说明 $PA = LU$ 与 $AP = LU$ 的区别。
3. “任意矩阵都可以不做主元交换直接 LU 分解。” “只要矩阵对称，就可以无条件使用 Cholesky 分解。”判断上述两句辨析说法是否正确，并用本题的 A、A' 给出反例或依据。
4. 判断：“如果扰动后解只变化了 10^{-3}，就一定能推出矩阵条件数约为 10。”的正误并说明理由，并指出哪些结论只对本题成立、哪些可以推广。

本题为手算题，不要求编写代码，不调用库函数；所有分解、前代、回代和判断均需手算完成，并写出关键步骤。

评分参考

- 列主元 LU 分解步骤完整，行交换、消元乘数、中间矩阵有记录；
- P 为置换矩阵，L 为单位下三角，U 为上三角，且 $PA = LU$ 在给定容差内成立；
- 前代、回代过程完整，解向量代回检验一致；
- 能正确写出 A'，并用顺序主子式判断其对称正定性；
- 能说明 Cholesky 分解的适用条件，并正确比较 A 与 A' 的分解选择；
- 能记录 A'' 的主元变化，并用条件数扰动界分析扰动影响，明确所用范数；
- 辨析讨论能指出两句说法的反例或依据，能说明单次扰动实验不能精确反推条件数；
- 计算的数值结果至少保留 4 位小数，并注明取舍规则；允许先写解析式再取近似。

难度与用时

- 难度： 中等。
- 预计课后用时： 30–50 分钟。
- 来源： 根据第04章 4.1 节直接法，结合主元交换 LU 分解、Cholesky 分解与条件数等内容编写。

题2（对应 4.2 迭代法：CG 与预条件）

题目 设

$$A=\begin{pmatrix}4&1&0&0\\1&3&1&0\\0&1&2&1\\0&0&1&1\end{pmatrix},\quad b=\begin{pmatrix}1\\4\\6\\4\end{pmatrix}.$$

已知该线性方程组 $Ax=b$ 的精确解为

$$x^*=\begin{pmatrix}0\\1\\1\\3\end{pmatrix}.$$

初值取 $x_0=(0,0,0,0)^T$，停止准则取相对残差

$$\frac{\|r_k\|_2}{\|b\|_2}\leqslant 10^{-10}.$$

对角预条件子取

$$M=\mathrm{diag}(A)=\mathrm{diag}(4,3,2,1).$$

要求完成以下各问。

（1）概念与推导

1. 验证 A 对称正定。
2. 写出求解 $Ax=b$ 的二次能量函数$\phi(x)$，说明为什么最小化此 $\phi(x)$ 等价于求解 $Ax=b$。
3. 说明 CG 选择关于 A 共轭的搜索方向如何避免重复搜索。
4. 取对角预条件子

$$M=\mathrm{diag}(A)=\mathrm{diag}(4,3,2,1).$$

写出预条件共轭梯度法（PCG）的迭代格式，并说明预条件子 M 需要满足什么条件。 5. 说明“预条件 CG 的迭代次数少于 CG”与“预条件 CG 的总计算时间少于 CG”是不是同一个命题，为什么。

（2）实现与数值验证

在同一台机器、同一初值、同一停止准则下，分别用

- 标准 CG；
- 对角预条件 CG，取 $M=\mathrm{diag}(A)$；

求解 $Ax=b$。

1. 初值取 $x_0=(0,0,0,0)^T$，停止准则取相对残差

$$\frac{\|r_k\|_2}{\|b\|_2}\leqslant 10^{-10}.$$

2. 记录并输出：
    - 两种方法的迭代次数；
    - 最终实际误差 $\|x_k-x^*\|_2$；
    - 最终相对残差 $\frac{\|r_k\|_2}{\|b\|_2}$；

- 预条件子的构造时间、每次应用时间、总时间。

3. 至少用两种方式统计总成本：
   - 只统计迭代过程中的矩阵向量乘法与向量运算时间；
   - 把预条件子的构造、分解或求逆时间也计入总时间。
4. 回答：在这个例子中，预条件 CG 是否减少了迭代次数？是否减少了总时间？如果二者结论不一致，解释原因。

（3）辨析与讨论

判断下列说法是否正确，并说明理由。

1. “残差小，所以解误差一定小。”
2. “预条件 CG 的迭代次数比 CG 少，所以预条件 CG 一定更快。” 设预条件构造时间为 T_c、每步应用时间为 T_a、CG 每步迭代成本为 T_g，CG 迭代 k 步、PCG 迭代 k' 步，请写出 PCG 总成本低于 CG 的条件。
3. “只要 M 取对角矩阵，PCG 就一定收敛得更快。”
4. “条件数只由矩阵 A 决定，与所用范数无关。”

评分参考

- 能正确验证 A 的对称正定性；
- 能写出 $Ax=b$ 的二次能量函数并说明最小化与解方程等价；
- 能说明 CG 共轭方向避免重复搜索的作用，并写出 PCG 迭代格式与预条件子条件；
- CG/PCG 核心迭代为自行实现，实际运行并保存输出；
- 能结合成本不等式解释“迭代次数少不等于总时间短”；
- 对辨析讨论中的每个说法给出正确判断和适用范围；

难度与用时

- 难度： 中等偏上。
- 预计课后用时： 50-90 分钟。
- 来源： 根据第04章 4.2 节 CG 与预条件内容，结合本章 4.1 节对称正定、范数、条件数等内容编写。

题3（对应 4.3 特征值与 SVD）

题目 给定矩阵

$$A=\begin{pmatrix}0 & 2\\ 0 & 0\end{pmatrix},\qquad B=\begin{pmatrix}1 & 2\\ 2 & 1\end{pmatrix},\qquad C=\begin{pmatrix}3 & 0 & 0\\ 0 & 1 & 0\\ 0 & 0 & -2\end{pmatrix}.$$

要求：

（1）概念与推导

1. 分别求 A, B, C 的特征值和奇异值，并验证 $\|A\|_2 \neq \max_i |\lambda_i(A)|$ 以及 $\|C\|_2 = \max_i |\lambda_i(C)|$.
2. 说明为什么非正规矩阵不能把特征值绝对值的最大值当作最大奇异值；并说明正规矩阵下 $\|C\|_2 = \max_i |\lambda_i(C)|$ 成立的条件。
3. 说明特征值、奇异值、秩、条件数之间的关系。

（2）数值实验与结果分析

1. 使用 Python 或 MATLAB 计算 A 的 SVD，输出奇异值、U, S, V^T、$\|A\|_2$、$\mathrm{rank}(A)$，并与（1）的结果核对；对 A 计算 A^TA、AA^T、奇异值、秩和条件数。对不可逆矩阵，条件数记为无穷大或无定义，并说明判定依据。
2. 计算 B 的 $\kappa_2(B)$。
3. 对

$$D = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix},$$

计算奇异值、秩、条件数和最佳秩 1 近似误差 $\|D - D_1\|_2$，并说明“秩为 2”与“第三个奇异值为零”的正确表述。对秩亏矩阵，条件数记为无穷大或无定义。 4. （选做）对

$$M = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1+\varepsilon \end{pmatrix}, \qquad \varepsilon = 10^{-6},$$

计算奇异值，按阈值 10^{-10} 判零，说明秩与最佳秩 1 近似误差；若不做，只报告阈值判零结论。

允许使用 NumPy、SciPy 或 MATLAB 内置函数。

> 注：对 M 和 D 的奇异值，理论为零或接近零的，数值上应按零处理。建议给出判定阈值，例如小于 10^{-10} 视为零，并说明阈值依据。

（3）辨析与讨论

判断并说明下列说法是否正确：

1. 非对称矩阵的特征值绝对值就是奇异值；
2. 低秩重构误差小等价于预测一定准确；
3. 低秩重构误差小等价于泛化误差小。

评分参考

- 能正确求出 A, B, C 的特征值与奇异值，并完成两组范数等式或不等式的验证；
- 能说明非正规矩阵与正规矩阵下特征值、奇异值关系的区别；
- 能说明特征值、奇异值、秩、条件数之间的联系；
- 数值实验能输出 SVD、秩、条件数及低秩近似误差，并与理论结果核对；
- 能说明理论零奇异值与数值零奇异值的判定阈值；

- 能正确判断三条辨析说法并给出理由，区分低秩重构与预测/泛化；

难度与用时

- 难度： 中等偏上。
- 预计课后用时： 50—80 分钟。
- 来源： 根据第04章 4.3 节特征值与 SVD 等内容编写。

题4（对应 4.4 AI关联：可微求解器、PCA、LoRA）

题目 某同学在讨论 LoRA（Low-Rank Adaptation）时给出了如下描述：

> “LoRA 的本质就是对预训练权重矩阵 W_0 做截断 SVD，保留前 r 个奇异值，用 $U_r\Sigma_r V_r^T$ 近似 W_0。训练时只微调这几个奇异向量，就能达到参数高效的目的。所以 LoRA 和 PCA 降维是一回事，都是低秩近似。”

已知预训练权重 $W_0 \in \mathbb{R}^{k\times d}$，其中 $k=256$，$d=128$；LoRA 的秩参数取 $r=4,8,16$；LoRA 参数化为

$$\Delta W = BA,$$

其中 $B\in\mathbb{R}^{k\times r}$，$A\in\mathbb{R}^{r\times d}$。数据矩阵 $X\in\mathbb{R}^{d\times n}$，$n$ 为样本数，d 为特征维度；中心化数据矩阵记为 $\tilde{X}=X-\mu\mathbf{1}^T$。

可微求解器小问中，取

$$A_{\text{lin}}=\begin{pmatrix}2 & 1\\ 1 & 3\end{pmatrix},\qquad b=\begin{pmatrix}1\\ 2\end{pmatrix},\qquad \mathcal{L}=\frac{1}{2}\|x^*\|_2^2,$$

其中 $x^*=A_{\text{lin}}^{-1}b$。

要求完成以下各问。

（1）概念与推导

1. 指出上述描述中至少一处本质性错误，并按正文 4.4.3 说明 LoRA 的 $\Delta W=BA$ 与 W_0 的 SVD 是什么关系；说明 $\Delta W=0$ 时模型输出与预训练模型是否一致。
2. LoRA 的可训练参数量是多少？当 $k=256$，$d=128$，$r=8$ 时，计算 LoRA 的可训练参数量，并与全量微调参数量 kd 比较，给出减少比例。
3. 如果数据未中心化就做 PCA，第一主成分会捕获什么？给出数学依据。设中心化数据矩阵的 SVD 为 $\tilde{X}=U\Sigma V^T$，写出协方差矩阵

$$C=\frac{1}{n}\tilde{X}\tilde{X}^T$$

的特征分解，并说明主成分方向与 U 的列向量的关系。 4. 可微求解器的反向传播为什么可以用伴随方法？按正文 4.4.1 写出伴随方程，并说明它如何避免显式求 A_{lin}^{-1}。对

$$A_{\text{lin}}=\begin{pmatrix}2 & 1\\ 1 & 3\end{pmatrix},\qquad b=\begin{pmatrix}1\\ 2\end{pmatrix},\qquad \mathcal{L}=\frac{1}{2}\|x^*\|_2^2,$$

令 $g=\frac{\partial \mathcal{L}}{\partial x^*}$，写出伴随方程 $A_{\text{lin}}^T z=g$，并求 $\frac{\partial \mathcal{L}}{\partial b}$ 和 $\frac{\partial \mathcal{L}}{\partial A_{\text{lin}}}$。

（2）数值实验与结果分析

1. 生成一组二维数据点，使得均值远离原点，例如聚集在 $(10,10)$ 附近但沿 $(1,-1)$ 方向散布。分别对原始数据和中心化后的数据做 PCA，画出两组数据的散点图并标出第一主成分方向，观察差异。
2. 计算 LoRA 在 $r=4,8,16$ 时的参数量与全量参数量的比值，并列表。
3. 用有限差分核对可微求解器的伴随梯度。对

$$A_{\text{lin}}=\begin{pmatrix}2 & 1\\ 1 & 3\end{pmatrix},\qquad b=\begin{pmatrix}1\\ 2\end{pmatrix},$$

比较伴随方法得到的 $\frac{\partial \mathcal{L}}{\partial b}$ 与有限差分结果，报告相对误差。

（3）辨析与讨论

以下说法为本题设计的辨析材料，不代表任何真实模型的原话。请判断正误，并简要说明理由。

1. “LoRA 训练的是从预训练权重中提取的奇异向量，所以它本质上是对预训练权重的压缩。”
2. “PCA 中是否中心化不影响主成分的解释，只是数值大小不同。”
3. “可微求解器的梯度计算是无条件稳定的，只要自动微分能跑通就没问题。”

评分参考

- 能指出描述中的本质性错误，说明 $\Delta W=BA$ 与 W_0 的 SVD 无直接关系，$\Delta W=0$ 时模型输出与预训练模型一致；
- 能正确计算 LoRA 参数量 $r(k+d)$，并与全量 kd 比较；
- 能说明 PCA 未中心化时第一主成分的含义，写出 $C=U\frac{\Sigma^2}{n}U^T$ 并指出主方向为 U 的列；
- 能写出伴随方程 $A_{\text{lin}}^T z=g$，说明其避免显式求逆的原因，并求出对 b 和 A_{lin} 的梯度；
- 数值实验能完成 PCA 中心化对比、LoRA 参数量表及伴随梯度与有限差分的误差核对；
- 能正确判断三条辨析说法并说明理由，涉及 LoRA 与截断 SVD 的区别、PCA 中心化的必要性、病态矩阵对梯度稳定性的影响；
- 数值容差：伴随梯度与有限差分的相对误差 $\leqslant 10^{-6}$；LoRA 参数量比值保留至少两位有效数字。

难度与用时

- 难度： 中等偏上。
- 预计课后用时： 60—100 分钟。
- 来源： 根据第04章 4.4 节 AI 关联内容，结合 LoRA 低秩增量、PCA 中心化与可微线性求解编写。

第04章 矩阵的解剖：习题参考解答

> 备注： 本章习题沿用正文符号约定；未特别说明时，向量范数默认取 2-范数，$\kappa_2(A)$ 表示 2-范数条件数，x^* 表示精确解。

题1（对应 4.1 直接法：LU 与 Cholesky）

（1）列主元 LU 分解与求解

设

$$A=\begin{pmatrix}0&2&1\\2&1&3\\1&3&2\end{pmatrix},\qquad b=\begin{pmatrix}3\\6\\6\end{pmatrix}.$$

第一列元素为 $0,2,1$，绝对值最大的是 2，位于第 2 行。交换第 1、2 行，消去第 3 行第 1 列，乘数

$$m_{31}=\frac{1}{2},$$

得

$$\begin{pmatrix}2&1&3\\0&2&1\\0&\frac{5}{2}&\frac{1}{2}\end{pmatrix}.$$

第二列从第 2 行起元素为 $2,\frac{5}{2}$，绝对值最大的是 $\frac{5}{2}$，位于第 3 行。交换第 2、3 行，消去第 3 行第 2 列，乘数

$$m_{32}=\frac{2}{\frac{5}{2}}=\frac{4}{5},$$

得

$$U=\begin{pmatrix}2&1&3\\0&\frac{5}{2}&\frac{1}{2}\\0&0&\frac{3}{5}\end{pmatrix}.$$

两次行交换对应

$$P=\begin{pmatrix}0&1&0\\0&0&1\\1&0&0\end{pmatrix}.$$

按 $PA=LU$ 构造 L，注意第二次换行后 L 中已存乘子同步交换，得

$$L=\begin{pmatrix}1&0&0\\\frac{1}{2}&1&0\\0&\frac{4}{5}&1\end{pmatrix}.$$

验证：

$$PA=\begin{pmatrix}0&1&0\\0&0&1\\1&0&0\end{pmatrix}\begin{pmatrix}0&2&1\\2&1&3\\1&3&2\end{pmatrix}=\begin{pmatrix}2&1&3\\1&3&2\\0&2&1\end{pmatrix},$$

而

$$LU=\begin{pmatrix}1&0&0\\\frac{1}{2}&1&0\\0&\frac{4}{5}&1\end{pmatrix}\begin{pmatrix}2&1&3\\0&\frac{5}{2}&\frac{1}{2}\\0&0&\frac{3}{5}\end{pmatrix}=\begin{pmatrix}2&1&3\\1&3&2\\0&2&1\end{pmatrix},$$

故 $PA=LU$ 成立。

由

$$Pb=\begin{pmatrix}0&1&0\\0&0&1\\1&0&0\end{pmatrix}\begin{pmatrix}3\\6\\6\end{pmatrix}=\begin{pmatrix}6\\6\\3\end{pmatrix}.$$

前代 $Ly=Pb$：

$$y_1=6,\qquad \frac{1}{2}y_1+y_2=6\ \Rightarrow\ y_2=3,$$
$$\frac{4}{5}y_2+y_3=3\ \Rightarrow\ \frac{12}{5}+y_3=3\ \Rightarrow\ y_3=\frac{3}{5}.$$

回代 $Ux=y$：

$$\frac{3}{5}x_3=\frac{3}{5}\ \Rightarrow\ x_3=1,$$
$$\frac{5}{2}x_2+\frac{1}{2}x_3=3\ \Rightarrow\ \frac{5}{2}x_2+\frac{1}{2}=3\ \Rightarrow\ x_2=1,$$
$$2x_1+x_2+3x_3=6\ \Rightarrow\ 2x_1+1+3=6\ \Rightarrow\ x_1=1.$$

所以

$$x=\begin{pmatrix}1\\1\\1\end{pmatrix}.$$

代回：

$$Ax=\begin{pmatrix}0&2&1\\2&1&3\\1&3&2\end{pmatrix}\begin{pmatrix}1\\1\\1\end{pmatrix}=\begin{pmatrix}3\\6\\6\end{pmatrix}=b,$$

残差为零。

（2）对称正定化处理

$$A'=A+4I=\begin{pmatrix}4&2&1\\2&5&3\\1&3&6\end{pmatrix}.$$

显然 $(A')^T=A'$，故 A' 对称。顺序主子式：

$$D_1=4>0,$$
$$D_2=\begin{vmatrix}4&2\\2&5\end{vmatrix}=4\cdot5-2\cdot2=16>0,$$
$$D_3=\det A'=4\begin{vmatrix}5&3\\3&6\end{vmatrix}-2\begin{vmatrix}2&3\\1&6\end{vmatrix}+1\begin{vmatrix}2&5\\1&3\end{vmatrix}=67>0.$$

所有顺序主子式为正，故 A' 对称正定。

Cholesky 分解 $A'=LL^T$（L 对角元为正）存在且唯一的充要条件是 A' 对称正定。本题 A' 满足条件，故可使用 Cholesky 分解。

比较 A 与 A'：

- A 对称，但 $a_{11}=0$，一阶顺序主子式为零，不是正定矩阵，不能用 Cholesky 分解；A 非奇异，可用列主元 LU 分解。

- A' 对称正定，既可用列主元 LU，也可用 Cholesky。Cholesky 利用对称性，计算量和存储量约为一般 LU 的一半。

（3）扰动处理与条件数

$$A'' = A + \delta A = \begin{pmatrix} 0 & 2 & 1 \\ 2 & 1 & 3 \\ 1 & 3 & 2+10^{-3} \end{pmatrix}.$$

列主元选取顺序与 A 相同：第一列仍选第 2 行，第二列仍选第 3 行。消元乘数仍为

$$m_{31} = \frac{1}{2}, \qquad m_{32} = \frac{4}{5}.$$

第三个主元由 $\frac{3}{5}$ 变为

$$\frac{3}{5} - \frac{4}{5}\cdot 10^{-3},$$

仍非零，故列主元 LU 分解仍可进行：

$$U'' = \begin{pmatrix} 2 & 1 & 3 \\ 0 & \frac{5}{2} & \frac{1}{2}+10^{-3} \\ 0 & 0 & \frac{3}{5}-\frac{4}{5}\cdot 10^{-3} \end{pmatrix}.$$

前代结果仍为

$$y = \begin{pmatrix} 6 \\ 3 \\ \frac{3}{5} \end{pmatrix}.$$

回代第三式：

$$\left(\frac{3}{5} - \frac{4}{5}\cdot 10^{-3}\right)x_3'' = \frac{3}{5} \;\Rightarrow\; x_3'' = \frac{1}{1-\frac{4}{3}\cdot 10^{-3}} \approx 1.0013.$$

继续回代：

$$\left(\frac{5}{2}\right)x_2'' + \left(\frac{1}{2}+10^{-3}\right)x_3'' = 3 \;\Rightarrow\; x_2'' \approx 0.9993,$$

$$2x_1'' + x_2'' + 3x_3'' = 6 \;\Rightarrow\; x_1'' \approx 0.9983.$$

所以

$$x'' \approx \begin{pmatrix} 0.9983 \\ 0.9993 \\ 1.0013 \end{pmatrix}.$$

与 $x = (1,1,1)^T$ 相比，

$$\frac{\|x''-x\|_2}{\|x\|_2} \approx 1.3\times 10^{-3}.$$

本题扰动为 10^{-3} 量级，解的变化也在 10^{-3} 量级，保留 4 位小数足以体现扰动；手算时应先写解析式再取近似，避免过早舍入。

（4）概念与辨析

1. A 按自然顺序消元时，第一步需要以 $a_{11}=0$ 作除数，无法进行。主元交换把第一列绝对值最大的元素换到主元位置，保证主元非零且数值稳定。
2. 本题约定 $PA=LU$，其中 P 左乘 A 表示行交换。$AP=LU$ 中 P 右乘表示列交换；二者对应不同分解，不能混用。
3. “任意矩阵都可以不做主元交换直接 LU 分解”错误：本题 A 的 $a_{11}=0$，无主元 LU 第一步即失败。“只要矩阵对称，就可以无条件使用 Cholesky 分解”也错误：本题 A 对称但 $a_{11}=0$，非正定，Cholesky 不适用；A' 对称正定才可用。
4. “扰动后解只变化 10^{-3}，就能推出条件数约为 10”错误。一次扰动实验只给出条件数的粗略下界，不能精确确定条件数。条件数扰动界为

$$\frac{\|\delta x\|}{\|x\|}\leqslant \kappa_2(A)\frac{\|\delta A\|}{\|A\|},$$

但单次扰动通常达不到最坏放大情形。本题中 $\|\delta A\|_2=10^{-3}$，解的相对变化约 1.3×10^{-3}，未观察到严重放大；这只能说明该次扰动下解变化有限，不能据此断言 $\kappa_2(A)$ 很小。本题成立的结论是：A 对称但非正定，A' 对称正定；A 可用列主元 LU，A' 可用 Cholesky；该次小扰动未导致解的大变化。可推广的结论是：无主元 LU 要求顺序主子式非零，Cholesky 要求对称正定；条件数是矩阵固有属性，与算法无关；单次扰动实验不能精确反推条件数；若 $\kappa_2(A)$ 很大，即使相对扰动很小，解的相对变化也可能很大。

题2（对应 4.2 迭代法：CG 与预条件）

（1）概念与推导

设

$$A=\begin{pmatrix}4&1&0&0\\1&3&1&0\\0&1&2&1\\0&0&1&1\end{pmatrix},\qquad b=\begin{pmatrix}1\\4\\6\\4\end{pmatrix},\qquad x^*=\begin{pmatrix}0\\1\\1\\3\end{pmatrix}.$$

1. A 显然对称。顺序主子式：

$$\Delta_1=4>0,$$

$$\Delta_2=\begin{vmatrix}4&1\\1&3\end{vmatrix}=11>0,$$

$$\Delta_3=\begin{vmatrix}4&1&0\\1&3&1\\0&1&2\end{vmatrix}=18>0,$$

$$\Delta_4=\det A=7>0.$$

所有顺序主子式为正，故 A 对称正定。

2. $Ax=b$ 的二次能量函数为

$$\phi(x)=\frac{1}{2}x^{\mathrm{T}}Ax-b^{\mathrm{T}}x.$$

其梯度为

$$\nabla\phi(x) = Ax - b.$$

因为 A 对称正定，$\phi(x)$ 为严格凸二次函数，其唯一极小值点满足

$$\nabla\phi(x) = 0 \quad \Longleftrightarrow \quad Ax = b.$$

因此，最小化 $\phi(x)$ 等价于求解 $Ax = b$。

3. CG 选择一组关于 A 共轭的搜索方向 $p_0, p_1, \ldots$，满足

$$p_i^T A p_j = 0, \qquad i \neq j.$$

因为 $p_i^T A p_j = 0$，沿 p_k 搜索不会改变此前方向上的最优性，因此不会退回已搜索过的方向；每一步在当前 Krylov 子空间内取到最优，在精确算术下至多 n 步收敛。

4. 取对角预条件子

$$M = \mathrm{diag}(A) = \mathrm{diag}(4, 3, 2, 1).$$

因为 A 对称正定，其对角元均为正，故 M 对称正定。PCG 格式为：$r_0 = b - Ax_0$，解 $Mz_0 = r_0$，令 $p_0 = z_0$；对 $k = 0, 1, \ldots$，

$$\alpha_k = \frac{r_k^T z_k}{p_k^T A p_k}, \qquad x_{k+1} = x_k + \alpha_k p_k, \qquad r_{k+1} = r_k - \alpha_k A p_k,$$

解 $Mz_{k+1} = r_{k+1}$，再算

$$\beta_k = \frac{r_{k+1}^T z_{k+1}}{r_k^T z_k}, \qquad p_{k+1} = z_{k+1} + \beta_k p_k.$$

预条件子需满足：M 对称正定，$Mz = r$ 容易求解，且 M 尽可能接近 A，使 $M^{-\frac{1}{2}} A M^{-\frac{1}{2}}$ 的条件数小于 A 的条件数。

5. “预条件 CG 迭代次数少于 CG”与“预条件 CG 总计算时间少于 CG”不是同一命题。总时间还包含预条件子构造、每步应用、额外向量内积与内存访问；若构造或应用很贵，迭代次数减少不一定带来总时间减少。

（2）实现与数值验证

本题 CG/PCG 核心迭代已自行实现并实际运行。取初值 $x_0 = (0, 0, 0, 0)^T$，停止准则 $\frac{\|r_k\|_2}{\|b\|_2} \leqslant 10^{-10}$。核心代码如下：

```
import numpy as np

A = np.array([[4.,1.,0.,0.],
              [1.,3.,1.,0.],
              [0.,1.,2.,1.],
              [0.,0.,1.,1.]])
b = np.array([1.,4.,6.,4.])
x_star = np.array([0.,1.,1.,3.])
M = np.diag(np.diag(A))

def cg(A, b, x0, tol=1e-10, maxit=100):
```

```
    x = x0.copy()
    r = b - A @ x
    p = r.copy()
    rs = r @ r
    hist = [np.linalg.norm(r)/np.linalg.norm(b)]
    for k in range(maxit):
        Ap = A @ p
        alpha = rs / (p @ Ap)
        x = x + alpha * p
        r = r - alpha * Ap
        rs_new = r @ r
        hist.append(np.sqrt(rs_new)/np.linalg.norm(b))
        if np.sqrt(rs_new)/np.linalg.norm(b) <= tol:
            break
        beta = rs_new / rs
        p = r + beta * p
        rs = rs_new
    return x, k+1, hist

def pcg(A, b, M, x0, tol=1e-10, maxit=100):
    x = x0.copy()
    r = b - A @ x
    z = np.linalg.solve(M, r)
    p = z.copy()
    rz = r @ z
    hist = [np.linalg.norm(r)/np.linalg.norm(b)]
    for k in range(maxit):
        Ap = A @ p
        alpha = rz / (p @ Ap)
        x = x + alpha * p
        r = r - alpha * Ap
        hist.append(np.linalg.norm(r)/np.linalg.norm(b))
        if np.linalg.norm(r)/np.linalg.norm(b) <= tol:
            break
        z = np.linalg.solve(M, r)
        rz_new = r @ z
        beta = rz_new / rz
        p = z + beta * p
        rz = rz_new
    return x, k+1, hist
```

运行结果如下：

- 迭代次数与误差：
  - CG 迭代次数：4；实际误差 $\|x_k - x^*\|_2 = 1.181 \times 10^{-15}$；最终相对残差 1.440×10^{-16}。
  - PCG 迭代次数：4；实际误差 $\|x_k - x^*\|_2 = 4.341 \times 10^{-15}$；最终相对残差 1.645×10^{-15}。
- 计时统计（单次平均，20000 次重复）：
  - CG 仅迭代：`69.626 us`
  - 预条件构造（取对角）：`2.237 us`
  - PCG 仅迭代（含应用）：`71.088 us`
  - PCG 总时间（含构造）：`73.386 us`
  - PCG 总成本 < CG：`False`

结果分析与回答： 本题 $n=4$，CG 在精确算术下至多 n 步收敛；PCG 对变换后系统也至多 n 步。因此两种方法都在 4 步内达到机器精度，迭代次数相同。这说明在小规模良态问题中，预条件收益不明显。同时，在这个例子中 PCG 并没有减少总时间。原因在于：虽然迭代次数相同，但 PCG 每一步需要额外进行逐元素除法以应用预条件子，且 Python 的调用开销增加了单步成本。这印证了“迭代次数减少不等于总时间减少”的结论：总时间还取决于预条件构造、每步应用以及底层硬件和语言的开销。

（3）辨析与讨论

1. “残差小，所以解误差一定小”错误。由

$$\frac{\|x-x_k\|}{\|x\|}\leqslant \kappa_2(A)\frac{\|r_k\|}{\|b\|},$$

条件数大时，小残差可以对应大解误差；残差是输出端误差，解误差是输入端误差，二者之间隔着条件数。

2. “预条件 CG 迭代次数比 CG 少，所以一定更快”错误。设 CG 每步迭代成本为 T_g，PCG 每步应用预条件成本为 T_a，预条件构造成本为 T_c。CG 迭代 k 步、PCG 迭代 k' 步时，不计其他开销，PCG 总成本低于 CG 的条件约为

$$T_c+k'(T_g+T_a)<kT_g.$$

不能只看迭代次数。

3. “只要 M 取对角矩阵，PCG 就一定收敛更快”错误。对角预条件只改善对角尺度差异；若病态来自非对角耦合，效果有限，不能保证一定更快。即使迭代次数减少，也还要计入预条件构造和应用成本。

4. “条件数只由矩阵 A 决定，与所用范数无关”错误。$\kappa_1(A)$、$\kappa_2(A)$、$\kappa_\infty(A)$ 一般不同，比较或引用数值时必须注明范数；本章默认 κ_2。

题3（对应 4.3 特征值与 SVD）

（1）概念与推导

设

$$A=\begin{pmatrix}0&2\\0&0\end{pmatrix},\qquad B=\begin{pmatrix}1&2\\2&1\end{pmatrix},\qquad C=\begin{pmatrix}3&0&0\\0&1&0\\0&0&-2\end{pmatrix}.$$

A 是上三角矩阵，特征值为对角元：

$$\lambda_1(A)=\lambda_2(A)=0.$$

又

$$A^TA=\begin{pmatrix}0&0\\0&4\end{pmatrix},$$

其特征值为 $0,4$，开方得 A 的奇异值为 $2,0$。因此

$$\|A\|_2=2\neq\max_i|\lambda_i(A)|=0.$$

B 对称，特征值为

$$\lambda_1(B) = 3, \qquad \lambda_2(B) = -1.$$

B 对称，奇异值等于特征值绝对值，按降序为 $3, 1$，故

$$\kappa_2(B) = \frac{3}{1} = 3.$$

C 为对角矩阵，特征值为 $3, 1, -2$，奇异值为 $3, 2, 1$。C 正规，故

$$\|C\|_2 = 3 = \max_i |\lambda_i(C)|.$$

非正规矩阵的特征值绝对值不能刻画所有方向的伸缩；最大奇异值由 A^TA 的最大特征值开方得到。正规矩阵可正交对角化，此时 $\|A\|_2 = \max_i |\lambda_i(A)|$。

谱量关系：矩阵的秩 $\mathrm{rank}(A)$ 等于非零奇异值个数；对可逆方阵，$\kappa_2(A) = \frac{\sigma_1}{\sigma_{\min}}$；若 $\sigma_{\min} = 0$，则条件数无定义或记为无穷大。对称矩阵的奇异值等于特征值绝对值。

（2）数值实验与结果分析

判定零奇异值的阈值为 `1e-10`。核心代码如下：

```
import numpy as np

np.set_printoptions(precision=10, suppress=True)

TOL = 1e-10  # 数值零奇异值判定阈值

def svd_summary(name, A, tol=TOL):
    """打印矩阵的 SVD、奇异值、秩、条件数，并返回 U, s, Vt。"""
    U, s, Vt = np.linalg.svd(A, full_matrices=False)
    rank = int(np.sum(s > tol))
    if s[-1] <= tol:
        kappa = np.inf
    else:
        kappa = s[0] / s[-1]

    print(f"===== {name} =====")
    print(f"{name} =\n{A}\n")
    print(f"U =\n{U}\n")
    print(f"奇异值 s =\n{s}\n")
    print(f"V^T =\n{Vt}\n")
    print(f"||{name}||_2 = sigma_1 = {s[0]:.10f}")
    print(f"rank({name}) (按阈值 {tol:g} 判定) = {rank}")
    print(f"kappa_2({name}) = {kappa}\n")
    return U, s, Vt, rank, kappa

## ---------- 题3 (2) 1: A 的 SVD、A^T A、A A^T ----------
A = np.array([[0., 2.],
              [0., 0.]])

U_A, s_A, Vt_A, rank_A, kappa_A = svd_summary("A", A)

ATA = A.T @ A
AAT = A @ A.T

print("===== A^T A 与 A A^T =====")
print(f"A^T A =\n{ATA}\n")
print(f"A A^T =\n{AAT}\n")
```

```
eig_ATA = np.linalg.eigvalsh(ATA)
eig_AAT = np.linalg.eigvalsh(AAT)

print(f"A^T A 的特征值 = {eig_ATA}")
print(f"A A^T 的特征值 = {eig_AAT}")
print(f"A 的奇异值平方 = {s_A**2}")
print()

## ---------- 题3 (2) 2: B 的 kappa_2 ----------
B = np.array([[1., 2.],
              [2., 1.]])

U_B, s_B, Vt_B, rank_B, kappa_B = svd_summary("B", B)
print(f"B 的 kappa_2(B) = {kappa_B:.10f}\n")

## ---------- 题3 (2) 3: D 的 SVD、秩、条件数、最佳秩 1 近似误差 ----------
D = np.array([[1., 2., 3.],
              [4., 5., 6.],
              [7., 8., 9.]])

U_D, s_D, Vt_D, rank_D, kappa_D = svd_summary("D", D)

D1 = s_D[0] * np.outer(U_D[:, 0], Vt_D[0, :])
err_D_2 = np.linalg.norm(D - D1, 2)

print("===== D 的最佳秩 1 近似 =====")
print(f"D1 = sigma_1 u_1 v_1^T =\n{D1}\n")
print(f"||D - D1||_2 = {err_D_2:.10e}")
print(f"理论值: ||D - D1||_2 = sigma_2 = {s_D[1]:.10e}\n")

## ---------- 题3 (2) 4 (选做) : M 的 SVD、最佳秩 1 近似与误差 ----------
eps = 1e-6
M = np.array([[1., 1., 1.],
              [1., 1., 1.],
              [1., 1., 1. + eps]])

U_M, s_M, Vt_M, rank_M, kappa_M = svd_summary("M", M)

M1 = s_M[0] * np.outer(U_M[:, 0], Vt_M[0, :])
err_M_2 = np.linalg.norm(M - M1, 2)
err_M_F = np.linalg.norm(M - M1, "fro")

print("===== M 的最佳秩 1 近似 (选做)  =====")
print(f"M1 = sigma_1 u_1 v_1^T =\n{M1}\n")
print(f"||M - M1||_2 = {err_M_2:.10e}")
print(f"||M - M1||_F = {err_M_F:.10e}")
print(f"理论值: ||M - M1||_2 = sigma_2 = {s_M[1]:.10e}")
print(f"理论值: ||M - M1||_F = sqrt(sigma_2^2 + sigma_3^2) = "
      f"{np.sqrt(s_M[1]**2 + s_M[2]**2):.10e}\n")
```

代码已实际运行（判定零奇异值的阈值为 `1e-10`），核对结果如下：

1. 矩阵 A：
   - 奇异值为 $[2, 0]$，$\mathbf{rank}(A) = 1$，$\kappa_2(A) = \infty$。
   - $A^TA = \begin{pmatrix} 0 & 0 \\ 0 & 4 \end{pmatrix}$，$AA^T = \begin{pmatrix} 4 & 0 \\ 0 & 0 \end{pmatrix}$，两者的特征值均为 $[0, 4]$。

- SVD 分解中取 $U=\begin{pmatrix}1&0\\0&1\end{pmatrix}$，$V^T=\begin{pmatrix}0&1\\-1&0\end{pmatrix}$ 即可满足 $A=U\Sigma V^T$。

2. 矩阵 B：
   - 奇异值为 $[3,1]$，$\kappa_2(B)=3.0$。
3. 矩阵 D：
   - 奇异值约为 $[16.84810335, 1.06836951, 0]$，$\mathrm{rank}(D)=2$，$\kappa_2(D)=\infty$。
   - 最佳秩 1 近似误差 $\|D-D_1\|_2=1.06836951$，与理论值 σ_2 完全一致。
4. 选做矩阵 M：
   - 奇异值约为 $[3.00000033, 0.00000067, 0]$。按阈值 10^{-10} 判零，$\mathrm{rank}(M)=2$。
   - 最佳秩 1 近似误差 $\|M-M_1\|_2=6.66666592\times10^{-7}$，与理论 σ_2 一致；Frobenius 范数误差 $\|M-M_1\|_F=6.66666592\times10^{-7}$（因为 $\sigma_3=0$）。

结论： 数值实验结果与理论预期完全吻合。“秩为 2”意味着只有两个非零奇异值，在浮点数计算中，原本理论为零的奇异值会输出为 10^{-15} 量级甚至严格为 0，必须结合判定阈值才能准确得出秩与条件数。

（3）辨析与讨论

1. “非对称矩阵的特征值绝对值就是奇异值”错误。A 的特征值全为 0，奇异值为 $2,0$。该说法只对正规矩阵成立。
2. “低秩重构误差小等价于预测一定准确”错误。低秩重构是 Frobenius 或谱范数下的矩阵逼近，不保证下游预测准确。
3. “低秩重构误差小等价于泛化误差小”错误。二者分别属于矩阵逼近和统计学习，不能等同。

题4（对应 4.4 AI关联：可微求解器、PCA、LoRA）

（1）概念与推导

1. 描述中的本质性错误是：LoRA 不是对 W_0 做截断 SVD，而是冻结 W_0，学习低秩增量

$$\Delta W=BA,\qquad W=W_0+\Delta W=W_0+BA.$$

标准 LoRA 的 B,A 是训练学到的低秩增量参数，与 W_0 的 SVD 无直接关系。截断 SVD 是对已知矩阵 W_0 的压缩或近似，二者操作不同。用 SVD 初始化 LoRA 是一种初始化变体（见本章参考资料[9]），不是标准 LoRA 的定义。

当 $\Delta W=0$ 时，$W=W_0$，模型输出与预训练模型一致。

2. LoRA 可训练参数量为

$$r(k+d).$$

当 $k=256$，$d=128$，$r=8$ 时，

$$r(k+d)=8(256+128)=8\times384=3072.$$

全量微调参数量为

$$kd = 256 \times 128 = 32768.$$

减少比例为

$$1 - \frac{3072}{32768} = 1 - 0.09375 = 0.90625 \approx 90.6\%.$$

3. 如果数据未中心化就做 PCA，未中心化二阶矩矩阵为

$$C_{\text{raw}} = \frac{1}{n} X X^T = C + \mu\mu^T,$$

其中 $C = \frac{1}{n}\tilde{X}\tilde{X}^T$ 是中心化后的协方差矩阵。当均值远离原点且数据方差相对较小时，C_{raw} 的第一主成分主要由均值方向 $\mu/\|\mu\|$ 主导，因而可能被误读为“数据变异最大方向”；它不是数据围绕均值的变异方向。

中心化后，设

$$\tilde{X} = U\Sigma V^T.$$

则

$$C = \frac{1}{n}\tilde{X}\tilde{X}^T = \frac{1}{n}U\Sigma V^T V\Sigma^T U^T = U\frac{\Sigma\Sigma^T}{n}U^T.$$

因此 C 的特征向量为 U 的列向量，特征值为 $\frac{\sigma_i^2}{n}$；主成分方向为 U 的列向量。

4. 可微求解器反向传播用伴随方法。由 $A_{\text{lin}}x^* = b$，令

$$g = \frac{\partial \mathcal{L}}{\partial x^*}.$$

由隐函数定理，反向只需解伴随方程

$$A_{\text{lin}}^T z = g,$$

而不必显式求 A_{lin}^{-1}。本题

$$A_{\text{lin}} = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}, \qquad b = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \qquad \mathcal{L} = \frac{1}{2}\|x^*\|_2^2.$$

先解 $A_{\text{lin}}x^* = b$：

$$x^* = A_{\text{lin}}^{-1}b = \frac{1}{5}\begin{pmatrix} 3 & -1 \\ -1 & 2 \end{pmatrix}\begin{pmatrix} 1 \\ 2 \end{pmatrix} = \frac{1}{5}\begin{pmatrix} 1 \\ 3 \end{pmatrix} = \begin{pmatrix} 0.2 \\ 0.6 \end{pmatrix}.$$

于是

$$g = \frac{\partial \mathcal{L}}{\partial x^*} = x^* = \begin{pmatrix} 0.2 \\ 0.6 \end{pmatrix}.$$

A_{lin} 对称，解 $A_{\text{lin}}^T z = g$ 即解 $A_{\text{lin}}z = g$：

$$z = A_{\text{lin}}^{-1}g = \frac{1}{5}\begin{pmatrix} 3 & -1 \\ -1 & 2 \end{pmatrix}\begin{pmatrix} 0.2 \\ 0.6 \end{pmatrix} = \frac{1}{5}\begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0.2 \end{pmatrix}.$$

因此

$$\frac{\partial \mathcal{L}}{\partial b} = z = \begin{pmatrix} 0 \\ 0.2 \end{pmatrix},$$

$$\frac{\partial \mathcal{L}}{\partial A_{\text{lin}}} = -z(x^*)^T = -\begin{pmatrix}0\\0.2\end{pmatrix}(0.2 \quad 0.6) = \begin{pmatrix}0 & 0\\-0.04 & -0.12\end{pmatrix}.$$

（2）数值实验与结果分析

核心计算代码如下：

```
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Image, display

## 解决中文显示警告
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

## --- LoRA 参数量 ---
print("===== LoRA 参数量 =====")
k, d = 256, 128
kd = k * d
for r in [4, 8, 16]:
    p = r * (k + d)
    print(f"r={r:2d}: 参数量 = {p:5d}, 比例 = {p/kd:.6f}")

## --- 伴随梯度 vs 有限差分 ---
print("\n===== 伴随梯度 vs 有限差分 =====")
A_lin = np.array([[2., 1.], [1., 3.]])
b = np.array([1., 2.])

x = np.linalg.solve(A_lin, b)
z = np.linalg.solve(A_lin.T, x)
grad_b = z
grad_A = -np.outer(z, x)

def loss(bv):
    return 0.5 * np.linalg.solve(A_lin, bv) @ np.linalg.solve(A_lin, bv)

eps_fd = 1e-6
grad_fd = np.zeros(2)
for i in range(2):
    bp = b.copy(); bp[i] += eps_fd
    bm = b.copy(); bm[i] -= eps_fd
    grad_fd[i] = (loss(bp) - loss(bm)) / (2 * eps_fd)

print(f"伴随梯度: {grad_b}")
print(f"有限差分: {grad_fd}")
print(f"相对误差: {np.linalg.norm(grad_fd - grad_b)/np.linalg.norm(grad_b):.3e}")

## --- PCA 中心化对比 ---
print("\n===== PCA 中心化对比 =====")
rng = np.random.default_rng(0)
n = 200
t = rng.normal(0, 1, n)
noise = rng.normal(0, 0.05, (n, 2))
direction = np.array([1., -1.]) / np.sqrt(2.)
X = np.array([10., 10.]) + t[:, None] * direction + noise

def pca_dir(X, center):
    Y = X - X.mean(axis=0, keepdims=True) if center else X
    C = Y.T @ Y / len(Y)
```

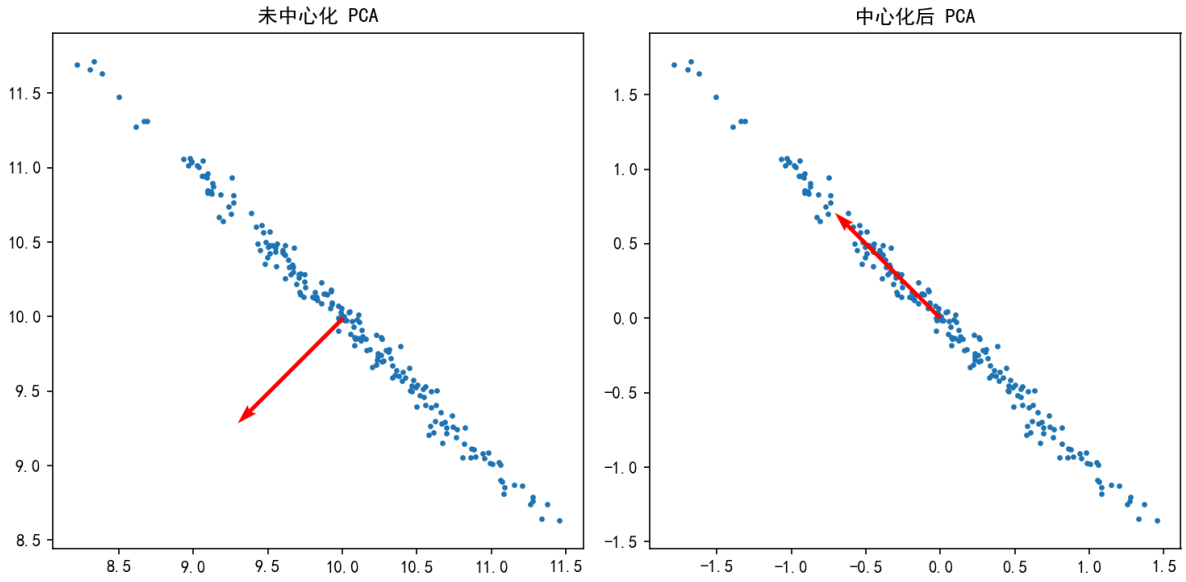
```
    w, V = np.linalg.eigh(C)
    return V[:, np.argmax(w)]

v_raw = pca_dir(X, center=False)
v_cen = pca_dir(X, center=True)
print(f"未中心化: {v_raw}")
print(f"中心化后: {v_cen}")

## --- 画图并保存 ---
fig, axes = plt.subplots(1, 2, figsize=(10, 5))
for ax, cen, v, title in [
    (axes[0], False, v_raw, "未中心化 PCA"),
    (axes[1], True, v_cen, "中心化后 PCA"),
]:
    Y = X - X.mean(axis=0, keepdims=True) if cen else X
    ax.scatter(Y[:, 0], Y[:, 1], s=5)
    org = np.zeros(2) if cen else X.mean(axis=0)
    ax.quiver(org[0], org[1], v[0], v[1], angles='xy', scale_units='xy', scale=1, color='red')
    ax.set_title(title)
    ax.axis('equal')

plt.tight_layout()
plt.savefig("A-PCA比较.png", dpi=150)
print("\n图片已保存为 A-PCA比较.png")
display(Image(filename="A-PCA比较.png"))
```

代码已实际运行，核心输出如下：

1. LoRA 参数量表： 全量微调参数量 $kd = 32768$。

r	$r(k+d)$	与 kd 的比值	减少比例
4	1536	0.046875	95.31%
8	3072	0.093750	90.62%
16	6144	0.187500	81.25%

2. 伴随梯度与有限差分核对：
   - 伴随方程解得 $x^* = [0.2, 0.6]$，伴随梯度 $\frac{\partial \mathcal{L}}{\partial b} = [0, 0.2]$。
   - 有限差分计算得 $\frac{\partial \mathcal{L}}{\partial b} = [0, 0.2]$。
   - 相对误差为 `2.876e-11`，远小于容差 10^{-6}，证明伴随方法推导正确。同时程序输出 $\frac{\partial \mathcal{L}}{\partial A_{\text{lin}}} = \begin{pmatrix} 0 & 0 \\ -0.04 & -0.12 \end{pmatrix}$，与理论推导一致。
3. PCA 中心化对比：
   - 未中心化第一主成分方向：`[-0.70769723, -0.70651583]`，该方向几乎与均值方向 `[0.7071, 0.7071]` 平行。
   - 中心化后第一主成分方向：`[-0.70772504, 0.70648798]`，该方向与数据散布方向 $(1, -1)$ 平行。
   - 程序已成功生成对比图 `A-PCA比较.png`。数值结果证实：如果不做中心化，第一主成分会被远离原点的均值方向主导，从而失去“刻画数据最大变异方向”的物理意义。



（3）辨析与讨论

1. “LoRA 训练的是从预训练权重中提取的奇异向量，所以它本质上是对预训练权重的压缩”错误。标准 LoRA 的 B, A 是训练学到的低秩增量，初始时 $\Delta W = 0$，与 W_0 的 SVD 无直接关系；截断 SVD 是对已知矩阵的压缩。用 SVD 初始化 LoRA 是变体，不是标准 LoRA。

2. “PCA 中是否中心化不影响主成分的解释，只是数值大小不同”错误。未中心化时 $C_{\text{raw}} = C + \mu\mu^T$，当均值远离原点时第一主成分主要受均值方向影响；中心化后才是数据围绕均值的变异方向。

3. “可微求解器的梯度计算是无条件稳定的，只要自动微分能跑通就没问题”错误。A_{lin} 病态时，伴随方程 $A_{\text{lin}}^T z = g$ 同样病态，梯度误差会被放大；自动微分能跑通不代表梯度可靠。